目录

Kubernetes / 云原生面试题库合集

第一篇 • Linux 与容器底层

- Part A · 隔离与限额原理篇（26 题）
- Part B · 容器运行时与经典故障专项篇（14 题）
- Part C · 大规模节点内核 / 系统调优篇（14 题）
- Part D · 反模式清单（可直接用于生产巡检）
- Part E · 可观测性：指标与告警
- Part F · 演练（GameDay）计划表
- Part G · 10 张面试速记卡（临阵磨枪）

第二篇 • APIServer 深度解析

第三篇 • etcd / controller-manager / kube-proxy

- Part B · etcd 篇（12 题）
- Part C · kube-controller-manager 篇（12 题）
- Part D · kube-proxy 专项篇（8 题）
- Part E · 大规模控制面优化篇（12 题）
- Part F · 反模式清单（可直接用于生产巡检）
- Part G · 可观测性：指标与告警（可直接落地）
- Part H · 演练（GameDay）计划表
- Part I · 10 张面试速记卡（临阵磨枪）

第四篇 • Informer 与 client-go

第五篇 • kube-scheduler 调度

- Part A · 原理篇（22 题）
- Part B · 大规模集群篇（16 题）
- Part C · 反模式清单（SRE 视角，可直接用于生产巡检）
- Part D · 可观测性：SLO、指标与告警（可直接落地）
- Part E · 演练（GameDay）计划表
- Part F · 10 张面试速记卡（临阵磨枪）

第六篇 • kubelet 与节点

- Part A · 原理篇（24 题）
- Part B · 大规模集群篇（16 题）
- Part C · 反模式清单（可直接用于生产巡检）
- Part D · 可观测性：SLO、指标与告警（可直接落地）
- Part E · 演练（GameDay）计划表
- Part F · 10 张面试速记卡（临阵磨枪）

第七篇 • 网络与 CNI

- Part A · 原理篇（26 题）
- Part B · 方案 / 模式专项篇（18 题）
- Part C · 大规模集群网络优化篇（14 题）
- Part D · 反模式清单（可直接用于生产巡检）
- Part E · 可观测性：SLO、指标与告警
- Part F · 演练（GameDay）计划表
- Part G · 10 张面试速记卡

第八篇 • 存储与 CSI

- Part A · 原理篇（24 题）
- Part B · 常见存储方案 / 模式专项篇（18 题）
- Part C · 大规模集群存储优化篇（14 题）
- Part D · 反模式清单（可直接用于生产巡检）
- Part E · 可观测性：SLO、指标与告警
- Part F · 演练（GameDay）计划表
- Part G · 10 张面试速记卡（临阵磨枪）

第九篇 • GPU 与大模型基础设施

- Part A · GPU 硬件与软件栈（14 题）
- Part B · Kubernetes GPU 调度机制（16 题）
- Part C · 大模型训练网络（12 题）
- Part D · 大模型场景的存储（10 题）
- Part E · 推理平台与弹性（10 题）
- Part F · 可靠性、容错与成本（10 题）
- Part G · 反模式清单（可直接用于生产巡检）
- Part H · 可观测性：GPU 与大模型指标与告警
- Part I · 演练（GameDay）计划表
- Part J · 面试速记卡（15 条）+ 一道综合设计题

第十篇 • Istio 服务网格

- Part A · 原理篇（26 题）
- Part B · 方案 / 模式专项篇（14 题）
- Part C · 大规模集群网格优化篇（12 题）
- Part D · 反模式清单（可直接用于生产巡检）
- Part E · 可观测性：指标、SLO 与告警
- Part F · 演练（GameDay）计划表
- Part G · 10 张面试速记卡

第十一篇 • 排障实战 60 例

- 第 1 章 · 调度与准入（P01–P08）
- 第 2 章 · Pod 与容器（P09–P25）
- 第 3 章 · 节点（P26–P35）
- 第 4 章 · 网络（P36–P46）
- 第 5 章 · 控制面（P47–P53）
- 第 6 章 · 存储（P54–P57）
- 第 7 章 · GPU 与大模型（P58–P60）
- 附录 A · 出事第一时间要抓的"证据清单"
- 附录 B · 最容易被误判的 10 件事
- 附录 C · 复盘模板（一页纸）
- 附录 D · 最小可用告警集（先有这 20 条再谈治理）

TOCPAGEMARKER

Kubernetes / 云原生面试题库合集

> **Linux 容器 → 控制面 → 调度 → 节点 → 网络 → 存储 → GPU → 服务网格 → 排障实战**
> 由 11 个专题文档合并去重而成，按技术栈自底向上编排。

阅读约定

- **源码/官方默认值**：标注字段，提示现场核对；
- **建议值**：标为 *建议*，必须压测校准；
- **实测值**：本文档不提供任何未经实测的性能数字（IOPS/吞吐/延迟），一律给出**测量方法。**
- 凡涉及默认值、feature gate、GA/Beta 状态、启动参数与 sidecar 版本的地方，一律以**所用版本的官方文档 / Release Notes** 为准。
- 涉及生产变更的内容一律标注「灰度 / 回滚 / 验证」。

全篇结构

篇	主题	覆盖内容
第一篇	Linux 与容器底层	namespace · cgroup · 运行时 · 内核调优
第二篇	APIServer 深度解析	请求链路 · 认证鉴权准入 · watch/etcd · 大规模调优
第三篇	etcd / controller-manager / kube-proxy	控制面存储 · 控制器范式 · 服务转发
第四篇	Informer 与 client-go	Reflector · DeltaFIFO · Indexer · workqueue
第五篇	kube-scheduler 调度	调度框架 · 队列 · 抢占 · 打分 · 性能
第六篇	kubelet 与节点	SyncLoop · PLEG · CRI/CNI/CSI · 探针 · 驱逐
第七篇	网络与 CNI	网络模型 · kube-proxy · DNS · NetworkPolicy · 数据面
第八篇	存储与 CSI	PV/PVC · StorageClass · CSI sidecar · 拓扑 · 快照
第九篇	GPU 与大模型基础设施	GPU 硬件 · 调度 · 训练网络 · 存储 · 推理
第十篇	Istio 服务网格	Envoy · xDS · 流量治理 · 安全 · 可观测性
第十一篇	排障实战 60 例	现象 → 证据 → 止血 → 根治 → 防复发

第一篇 • Linux 与容器底层

> namespace · cgroup · 运行时 · 内核调优

0. 使用说明

0.1 先建立一张"Linux 如何支撑容器"的全景图

```
┌──────────────────────── Linux Kernel ────────────────────────┐
│                                                              │
│  隔离（看得到什么）  限额（能用多少）                          │
│  ├─ namespace ├─ cgroup v2 (unified)                         │
│  │  uts / pid / mount /  │  cpu / cpuset / memory /          │
│  │  net / ipc / user /   │  io / pids / hugetlb /            │
│  │  cgroup / time        │  rdma / misc                      │
│  │  └─ (v1: 每 subsystem 独立层级)                           │
│  ├─ capabilities（能做什么）：CAP_SYS_ADMIN / NET_RAW ...    │
│  ├─ seccomp（能调哪些 syscall）                              │
│  ├─ MAC：SELinux / AppArmor                                  │
│  ├─ dev cgroup + uevent（能看到/操作哪些设备）               │
│  └─ overlayfs / tmpfs / bind mount（文件系统怎么给）         │
│                                                              │
│  网络数据面：netns + veth + bridge + netfilter(nft) + conntrack │
│  + TC/qdisc + XDP/eBPF                                       │
└──────────────────────────────────────────────────────────────┘
```

一句话总纲：容器不是一个"东西"，而是 **namespace（看得见）+ cgroup（用多少）+ capabilities/seccomp（能做什么）+ rootfs（文件系统）** 四组机制的组合。这条主线在面试开场先说出来，后面所有问题都能挂上去。

Part A • 隔离与限额原理篇（26 题）

A1. namespace 有哪几类？各自隔离什么？

Namespace	隔离内容	容器中看得见的证据
Mount (`CLONE_NEWNS`)	挂载点视图	容器里有自己的 `/` 、 `/proc`
PID (`CLONE_NEWPID`)	进程号空间	容器里 `ps` 只看到自己的进程，PID 从 1 开始
Net (`CLONE_NEWNET`)	网卡/IP/路由/iptables/端口	每个 Pod 有自己的 netns 与 IP
UTS (`CLONE_NEWUTS`)	hostname / domainname	容器 hostname = Pod 名
IPC (`CLONE_NEWIPC`)	SystemV IPC / POSIX mq	不同 Pod 的共享内存互不可见
User (`CLONE_NEWUSER`)	uid/gid 映射	容器内 root ≠ 宿主 root（**rootless 的基础**）
Cgroup (`CLONE_NEWCGROUP`)	cgroup 根视图	容器里看 `/proc/self/cgroup` 是 `/`
Time（较新）	部分时钟偏移	用于容器迁移/时钟虚拟化（版本相关）

必答的关键补充：

- 命名空间通过 `/proc/<pid>/ns/*` 以文件描述符形式存在；**只要有一个 fd 或进程持有它，namespace 就不会被销毁**；
- 进入某个容器网络命名空间的排障命令：`nsenter -t <pid> -n <cmd>`（见 A3）。

A2. 三个系统调用：clone / unshare / setns 分别做什么？

调用	作用	例子
clone(flags)	创建新进程的同时创建新 namespace	runc 创建容器进程
unshare(flags)	为**当前进程**创建新 namespace 并加入	`unshare -n bash`（只给自己一个 netns）
setns(fd, nstype)	把当前进程**加入已存在的** namespace	`nsenter` 的实现基础

必答：这就是为什么"进入容器"有两条路：`kubectl exec`（走 CRI Exec，进程还在容器 cgroup/namespace 内）与 `nsenter -t <宿主PID>`（从宿主进）——后者在**容器没有 shell / 网络已断**时是唯一手段，也是面试加分点。

A3. PID namespace 的特殊之处：为什么容器里的 PID 1 这么重要？

PID namespace 是**层级**的：子 namespace 的进程在父 namespace 里也有 PID（双重身份）。

PID 1 的两大职责（超高频考点）：

1. **必须 reap 僵尸进程**：容器孤儿进程会被 reparent 到 PID 1；如果 PID 1 不 `wait()`，僵尸会堆积 ⇒ `ps` 里一堆 `<defunct>`，PID 资源被占。
2. **信号处理的特殊性**：在 PID namespace 中作为 PID 1 的进程，**未处理的终止信号（SIGTERM 等）会被内核忽略**（除非它显式注册 handler）。这就是经典事故：**"容器里执行 `kill -TERM 1` 没反应" / "应用收不到 SIGTERM，滚动更新要等 30 秒才被 SIGKILL"**。

解法（可直接背）：

- 用 **exec 形式** 启动进程（`CMD ["app"]` 而不是 `CMD app "$@"` shell 形式），让业务进程直接当 PID 1；
- 或引入 `dumb-init` / `tini` 这样的小型 init，专门负责转发信号 + 回收僵尸。

```
# Dockerfile 推荐写法
FROM debian
RUN apt-get update && apt-get install -y dumb-init
ENTRYPOINT ["/usr/bin/dumb-init", "--"]
CMD ["python", "app.py"]
```

A4. user namespace 与 rootless：解决了什么？

核心：user namespace 允许把容器内的 uid 0 映射到宿主的**非特权 uid**（如 100000），因此：

- 容器里是 root，宿主上只是一个普通用户 ⇒ **逃逸后的破坏面大幅下降**；
- 可以在没有 root 的情况下跑容器（**rootless container/runc/podman**）。

代价与限制（必答）：

- 部分文件系统（overlayfs 在某些内核/发行版组合下）在 user ns 中受限 ⇒ 常用 fuse-overlayfs；
- 特权操作（如某些设备映射、部分网络能力）不可用；
- Kubernetes 侧 **User Namespace for Pods** 仍在演进阶段（**请以所用版本 Release Notes 确认**），别在面试里说成"生产默认可用"。

A5. mount namespace 与 mount propagation：为什么容器里看不到新挂载？

- Mount namespace 隔离的是**挂载点视图**，不是数据；
- `mountPropagation` 决定挂载事件如何在宿者与容器间传播（详见存储篇 A21）；
- **忘记** `Bidirectional` 是 CSI 节点插件、某些块设备工具失效的根因。

```
# 宿主侧看某个进程的挂载视图差异
cat /proc/<host-pid>/mountinfo | wc -l
cat /proc/<container-pid>/mountinfo | wc -l
nsenter -t <pid> -m mount | head
```

A6. net namespace 与 veth pair 的底层（与网络篇联动）

```
创建 netns → 建 veth pair（两端各一个 MAC）→ 一端放进 netns（改名 eth0）→ 另一端留在 root netns
      → 配 IP + 路由 → 宿主侧按 CNI 决定挂 bridge 或走路由/tunnel
```

关键点：

- **每个 netns 有独立的路由表、iptables 规则、conntrack 条目、socket 端口空间** ⇒ 这也解释了 **hostNetwork 的 Pod 会占用宿主端口 且看不到 Service 的 DNAT 行为差异**；
- 排障手段：`ip netns`（仅对命名 netns 有效）、更通用的是 `nsenter -t <pid> -n ip a`。

```
# 找到某个容器 pause 进程在宿主上的 PID
crictl inspectp <sandbox-id> | jq .info.pid
# 进入它的网络命名空间
nsenter -t <pid> -n ip addr
nsenter -t <pid> -n ss -lntp
nsenter -t <pid> -n tcpdump -i eth0 -nn host 10.0.0.1
```

A7. cgroup v1 与 cgroup v2 的本质区别？为什么社区在迁移？

维度	cgroup v1	cgroup v2
层级	每个 subsystem 一棵独立树，容易混乱	**单一 unified 层级**，所有控制器同一棵树
内存语义	memory 与 memsw、kmem 账目复杂	统一一套 `memory.*`，支持 `memory.min/low/high/max`
委托（delegation）	困难	原生支持（容器内再分层）
PSI	无	有 `/proc/pressure/*`（见 C13）
IO 控制	`blkio` 权重/限流语义混乱	`io.weight` / `io.max` 语义清晰
K8s 支持	传统	新版本逐步成为推荐路径（**具体支持矩阵以所用版本为准**）

面试答法：迁移 v2 的收益是"**统一层级 + 更准确的资源账 + PSI 可观测**"，代价是 host OS/容器运行时/监控组件的兼容性要全部核对（**这是一次高风险的节点级变更，必须按节点池灰度**）。

A8. CPU cgroup 的三个参数，以及最经典的 Throttling 事故

v1：`cpu.shares`（相对权重）、`cpu.cfs_period_us`（周期，常见 100ms）、`cpu.cfs_quota_us`（周期内可用 CPU 时间） **v2**：`cpu.weight`、`cpu.max`（`$quota $period` 或 `max`）

```
cpu.max = 200000 100000 # 等价于 2 个核（200ms/100ms）
```

K8s 的映射：`resources.limits.cpu: 2` ⇒ `cpu.max = 200000 100000`；`requests.cpu: 1` ⇒ 主要转换为 **`cpu.weight`**（v2，范围 1~10000）或 v1 的 `cpu.shares`（1024 基准）。

经典事故（CPUThrottlingHigh，必考）：

```
现象：容器 CPU 使用率并不高，但 P99 延迟很差、RT 抖动
原因：在一个 100ms 周期里，进程前 20ms 就把配额用完 ⇒ 剩下 80ms 被**限流（throttle）**，
      表现为停顿。平均使用率低 ≠ 没有被限流！
```

证据字段：

```
# v2
cat /sys/fs/cgroup/<path>/cpu.stat # nr_throttled, throttled_usec（累计）
# v1
cat /sys/fs/cgroup/cpu/.../cpu.stat # nr_periods, nr_throttled, throttled_time
```

```
# 容器级别 throttling（cAdvisor 暴露，指标名以实际环境为准）
sum by (pod, container) (rate(container_cpu_cfs_throttled_seconds_total[5m]))
# throttled 占比 = throttled_periods / total_periods
sum by (pod) (rate(container_cpu_cfs_throttled_periods_total[5m]))
  / clamp_min(sum by (pod) (rate(container_cpu_cfs_periods_total[5m])), 0.001) > 0.1
```

治理建议（需压测校准）：

1. **延迟敏感服务不要设 CPU limit**（只设 request），避免 quota 造成的毛刺；代价是失去"硬上限保护"⇒ 用节点配额/隔离补；
2. 若必须设：**把 limit 放大到 request 的 2~4 倍**作为"突发上限"，而不是等于 request；
3. 部分版本支持调整 `cpuCFSQuotaPeriod`（**feature gate 阶段，请以版本为准**）来降低抖动频率，但这是演进特性，生产要谨慎；
4. 绑核场景用 `cpuManagerPolicy: static`（见 A10）。

A9. cpuset / CPU manager / NUMA：什么时候该绑核？

概念	作用
`cpuset.cpus`（v1）/ `cpuset.cpus.effective`（v2）	允许进程在哪些 CPU 上运行
Kubernetes `cpuManagerPolicy: static`	满足条件（Guaranteed + 整数 CPU）时给 Pod **独占核**，其余 Pod 被从这些核上剔除
`TopologyManager`	让 CPU/设备/内存在 **同一 NUMA 节点**上分配
NUMA 本地性	跨 NUMA 访问内存延迟与带宽都更差

实践要点：

- 必须配 `reservedSystemCPUs`（否则系统进程与 kubelet 会被挤到共享核上抖动）；
- 独占核**降低调度灵活性**，可能让节点出现"看着有资源但 Pod 放不下" ⇒ 需要对排不上队的事件做监控；

- HPC/低延迟网络类负载才需要，一般 Web 服务不必绑核。

```
numactl -H # 看 NUMA 拓扑
cat /sys/fs/cgroup/cpuset.cpus.effective
taskset -pc <pid> # 看进程允许跑在哪些核
```

A10. 内存 cgroup：为什么"进程没超 limit 却被 OOM"？（超经典）

关键：cgroup 统计的内存包含 page cache。

```
memory.current (v2) / memory.usage_in_bytes (v1)
    = RSS (匿名页，如堆内存)
    + page cache (文件页缓存，容器读写文件时产生)
    + slab / kernel memory (部分版本/配置下)
    + swap
```

典型事故链：

```
容器内大量写/读文件（日志、临时文件、备份导出）
  → page cache 暴涨
  → 计入 cgroup memory.usage
  → 触及 limit
  → cgroup OOM ⇒ 容器被杀，退出码 137
容器内 `free` 看到的 cache 虽然理论上是"可回收的"，但**内核回收不及时就会被判定为超限而触发 cgroup OOM**。
```

排障字段：

```
# v2
cat /sys/fs/cgroup/<path>/memory.stat
# anon / file / slab / sock / shmem / file_dirty / file_writeback / inactive_file / active_file
cat /sys/fs/cgroup/<path>/memory.events # oom_kill 计数！（最直接的证据）
cat /sys/fs/cgroup/<path>/memory.max # 限制值
cat /sys/fs/cgroup/<path>/memory.current
# v1
cat /sys/fs/cgroup/memory/.../memory.usage_in_bytes
cat /sys/fs/cgroup/memory/.../memory.oom_control # oom_kill 计数
```

```
# 容器 OOMKill（依赖 kube-state-metrics / cAdvisor）
increase(kube_pod_container_status_terminated_reason{reason="OOMKilled"}[15m])
# 工作集用量接近 limit
sum by (pod, container) (container_memory_working_set_bytes)
  / sum by (pod, container) (kube_pod_container_resource_limits{resource="memory"}) > 0.9
```

治理：

1. 明确"**cgroup 内存 = 应用内存 + page cache**"，给 Java/有磁盘 IO 的服务留出余量；
2. 大量 IO 的临时文件放 `emptyDir`（**或 memory medium**）并配 `ephemeral-storage` limits，避免污染 page cache；
3. 容器侧 `limit` 要基于**含 cache 的峰值**设定（实测），不能只看 RSS；
4. 用 `memory.events` 的 `max` / `oom` 计数做告警，而不是等业务报 137。

A11. OOM Killer 在 cgroup 场景怎么选进程？

两条通道：

场景	谁动手	依据
容器内超过自身 limit	**cgroup 级 OOM**（杀该 cgroup 内的进程）	选该 cgroup 内 oom_score 最高者
节点整体内存不足	**全局 OOM killer**	**各进程的 `oom_score_adj`**

`oom_score_adj` 由 kubelet 按 QoS 设置（详见 Kubelet 篇 Q14）：

QoS	oom_score_adj 量级
Guaranteed	−997（几乎不会被全局 OOM 选中）
Burstable	依 request/allocatable 计算（通常是负数到正数区间）
BestEffort	1000（最先被杀）

```
# 查看某进程的 oom 倾向
cat /proc/<pid>/oom_score
cat /proc/<pid>/oom_score_adj
# 内核留下的证据
dmesg -T | grep -i "Out of memory"
journalctl -k | grep -i oom
```

> **必答**：**容器 OOM 的现场最可信证据是内核日志（memcg oom: killed + 进程列表）与 `memory.events` 的 oom 计数**，而不是"猜哪个进程占了内存"。

A12. blkio / io cgroup：为什么 K8s 一直没有稳定的"IO 限速" API？

层	v1	v2
权重	`blkio.weight` / `blkio.bfq.weight`	`io.weight`
上限	`blkio.throttle.read_bps_device` / `write_iops_device`	`io.max`（`rbps/wbps/riops/wiops`）
限制	只对某些 I/O 调度器有效；CFQ 时代已过	依赖支持 blkcg 的调度器（bfq/mq-deadline 等）

为什么难标准化（必答）：

1. 需要内核 IO 栈支持，且**不同设备类型（HDD/NVMe）、不同调度器行为不一致**；
2. **buffer IO 的写入可能被"别人"记账**（回写时进程上下文已变）⇒ 归属不清；
3. K8s 目前稳定提供的是 `ephemeral-storage` **的容量限制**，不是 IOPS/带宽限制；
4. 因此生产上 IO 隔离更多依赖 **专用节点池 / 存储侧 QoS / cgroup v2 手工配置**（见 C10）。

A13. pids cgroup：fork bomb 怎么防?

- `pids.max` 限制该 cgroup 内的最大进程/线程数 ⇒ 超限的表现是 `resource temporarily unavailable`（**fork 失败**），而不是 OOM；
- K8s 侧通过 kubelet 的 `--pod-max-pids` 给 Pod 设 pid 限制（结合 `--pod-pids-limit`，请按版本核对 flag 名）；
- Node level: `kernel.pid_max` / `threads-max` 是宿主上限；
- **排障**：`ps -eLf | grep -c <app>` 看线程数；`cat /proc/<pid>/status | grep Threads`。

A14. devices cgroup 与设备直通：GPU 是怎么给到容器的?

- `devices` cgroup 控制 **能否 mknod / 读写某个设备节点**（黑名单式）；
- 容器运行时创建容器时把设备**挂载进容器的** `/dev`，并把 cgroup devices 规则放行；
- GPU 场景的完整链路：device plugin 上报资源 → kubelet 申请 → **运行时** `CreateContainer` **里带上 devices/env/mounts** → 容器里看到 `/dev/nvidia0`。

```
crictl inspect <ctr> | jq '.info.runtimeSpec.linux.devices'
```

A15. Capabilities：为什么容器不该给 CAP_SYS_ADMIN?

Linux 把 root 的特权拆成了几十个 capability。容器**默认只保留一小部分**（如 `CHOWN`、`SETUID`、`NET_BIND_SERVICE` 等），去掉了：`SYS_ADMIN`、`SYS_MODULE`、`SYS_TIME`、`NET_ADMIN`、`SYS_PTRACE`...

必答要点：

场景	说明
`privileged: true`	几乎给全所有 capability + 挂载宿主 `/dev` + 关闭部分隔离 ⇒ **等价于宿主 root**，只在极特殊场景（如 CNI DaemonSet、部分 CSI Node）使用
只想抓包	加 `NET_ADMIN` + `NET_RAW` 即可，不必 privileged
想 ping / 绑低端口	`NET_RAW` / `NET_BIND_SERVICE`（另注意 `net.ipv4.ping_group_range`）
想 mount	`SYS_ADMIN`（**高危**，配合 mount propagation 等于能接触宿主 FS）

```
securityContext:
  capabilities:
    drop: ["ALL"] # 强烈建议：先全丢
    add: ["NET_BIND_SERVICE"]
  privileged: false
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
  runAsNonRoot: true
```

A16. seccomp：容器为什么有时会 "Operation not permitted"?

seccomp 通过 **BPF 规则**限定进程可调用的 syscall 集合。

K8s 侧三种策略：`RuntimeDefault` / `Localhost` / `Unconfined`（默认策略随版本演进，请以所用版本为准；建议显式声明而不要依赖默认）。

经典事故（必考题）：

```
现象：某个新镜像（或升级了 glibc/内核后）在旧节点上跑不起来：
      "unable to start container process: ... clone3: operation not permitted"
原因：较老的 seccomp 实现/容器运行时的 profile 里没有 `clone3` 这个 syscall，
      但新 glibc 的 pthread_create 会调用它。
解决：升级运行时与 seccomp 支持版本；或临时降级 profile（不推荐长期使用）。
```

排查：

```
grep -i seccomp /proc/<pid>/status # Seccomp 字段：0/1/2
cat /proc/<pid>/status | grep -i cap # CapEff / CapPrm
docker/crictl inspect | jq '..|.seccompProfile'
```

> 这道题的价值：它把"内核 / glibc / 运行时 / profile"四者的版本耦合讲清楚了。

A17. SELinux 与 AppArmor 的区别？在容器里怎么起作用?

维度	SELinux	AppArmor
实现	内核 LSM，基于标签（label/type enforcement）	内核 LSM，基于路径的 profile
主流发行版	RHEL/CentOS/Rocky 系	Ubuntu/Debian 系
容器用法	`-Z` /config 给容器打标签；持久化卷要正确 relabel（`:Z` / `:z`）	`--security-opt apparmor=<profile>`
K8s 用法	`securityContext.seLinuxOptions`	`securityContext.appArmorProfile`（较新的 API；老版本用 annotation）

常见坑：**挂载卷后容器报 permission denied**，很可能是 SELinux 标签不对（而很多人第一反应是 chmod 777 —— 这是错的，会掩盖真正的问题）。

A18. overlayfs：容器镜像是怎么"叠"起来的?

```
merged（容器看到的根目录）
   ↑
union mount
   ↑ ↑
upperdir（可写层） lowerdir（镜像各层，只读）
workdir（中间态）
```

写时复制（Copy-up）规则：

- 读文件 → 从 lower 层找；
- 改文件 → **把整个文件复制到 upper 层再改**（大文件一次改动成本高！）；
- 删文件 → 在 upper 层建 **whiteout** 标记，不真删 lower 层；
- 删目录 → **opaque whiteout。**

由此产生的两个大规模坑（必答）：

1. **inode 消耗翻倍/累积**：每个 lower 层的每个文件都会占 inode；镜像层多 + 文件多 ⇒ `df -i` 爆掉（表现为"No space left on device"但 `df -h` 还有空间）；
2. `df` **释放滞后**：容器删了，宿主上被清理 soon, 但若有进程仍持有已删除文件的 fd，**空间不会释放**：

```
lsof -n | grep -i deleted # 找出持有已删除文件的进程
df -i # inode 使用率
du -sh /var/lib/containerd /var/lib/docker
```

A19. /dev/shm 为什么总是不够？（容器经典题）

容器运行时给每个容器默认的 `/dev/shm` **常常只有 64MiB**（默认 `shm-size` 较小，具体默认值随运行时而异）。

典型表现：

- Chrome/Chromium/Selenium：`Failed to create shared memory / bus error`
- PostgreSQL / Oracle / 某些 JVM：`No space left on device`（但 `df` 显示有空间！因为 `df` 看的是根，不是 `/dev/shm`）
- PyTorch DataLoader 多 worker：`unexpected EOF / bus error`

解法：

```
volumes:
  - name: dshm
    emptyDir:
      medium: Memory # tmpfs
      sizeLimit: 2Gi # 建议按业务实测设定；注意它计入容器/Pod 的内存账
volumeMounts:
  - name: dshm
    mountPath: /dev/shm
```

> **注意**：tmpfs 内容计入 **memory cgroup**（anonymous page），所以要一并提高 memory limit，否则会把内存打爆。

A20. ulimit 与文件描述符：三层限制分别是什么?

```
① 系统级：/proc/sys/fs/file-max（全部进程打开 fd 总数上限）
          /proc/sys/fs/nr_open（单个进程 fd 上限的内核侧上限）
② 用户/会话级：/etc/security/limits.conf、systemd DefaultLimitNOFILE
③ 容器/进程级：运行时创建容器时设置的 RLIMIT_NOFILE
```

必答的 K8s 陷阱：

- **Pod spec 里没有稳定的 ulimit 字段**（社区曾多次提案，状态请以版本为准）⇒ 实践中只能在**容器运行时层**配默认值（如 containerd CRI 配置或 `-systemd/open files`），或者依赖 systemd slice；
- "too many open files" 时要分清是 **fd 耗尽** 还是 **inotify watches 耗尽**（后者见 C5），两者报错容易被混淆。

```
cat /proc/<pid>/limits | grep -i "open files"
ls /proc/<pid>/fd | wc -l
cat /proc/sys/fs/file-nr
```

A21. 为什么容器里的 `top` / `free` 看到的是宿主机的数值？（经典）

因为 `/proc/meminfo`、`/proc/stat`、`/proc/cpuinfo` **默认不受 namespace 隔离**（它们属于 procfs，不是 cgroup）。

后果：

- Java/Python 应用在容器里"看到的可用内存/CPU 核数"是宿主的 ⇒ 线程池、堆参数算错 ⇒ OOM/超卖；
- `kubectl top` 与容器内 `free` 数值不一致（**前者来自 cgroup 统计，是对的**）。

治理：

1. **应用侧显式读 cgroup**（现代 JVM/Go/Node 已原生支持 cgroup-aware）：`bash cat /sys/fs/cgroup/memory.max # 可用上限 cat /sys/fs/cgroup/cpu.max # CPU 配额`
2. 或挂 **lxcfs** 提供容器化的 `/proc` 视图（适合遗留应用）；
3. 始终明确：**权威的资源用量来源应该是 cgroup（`cpu.stat`、`memory.stat`），而不是 procfs。**

A22. cgroup driver 为什么要和 systemd 一致？（K8s 经典故障）

问题现象：

```
节点上存在两套 cgroup 路径：
  systemd 管理的一方：/sys/fs/cgroup/system.slice/...  （由 systemd 创建）
  cgroupfs 的一方：  /sys/fs/cgroup/kubepods.slice/... （由 kubelet/runtime 直接创建）
两者互不知情 ⇒ 资源统计错位、limit 不生效、甚至节点内存失控
```

规则：**kubelet 的 `cgroupDriver` 必须与容器运行时保持一致**，通常统一为 `systemd`。

```
# kubelet
apiVersion: kubelet.config.k8s.io/v1beta1
kind: KubeletConfiguration
cgroupDriver: systemd
```

```
# containerd（CRI 配置片段，版本相关）
[plugins."io.containerd.grpc.v1.cri"]
  systemd_cgroup = true
```

检查方式：

```
systemctl status kubelet | grep -i cgroup
systemd-cgls # 看 systemd 视角的层级
cat /sys/fs/cgroup/cgroup.controllers # v2 顶层
```

> **这是"改了不生效"类问题的头号嫌疑对象**，改完必须**重启 kubelet 并驱逐/重建节点上的 Pod** 才彻底生效（属重操作，务必按节点池灰度）。

A23. systemd slice 层级长什么样?

```
-.slice
└── kubepods.slice # 所有 Pod 的总父
    ├── kubepods-burstable.slice
    │   └── kubepods-burstable-pod<uid>.slice
    │   ├── cri-containerd-<cid>.scope # 每个容器一个 scope
    └── kubepods-besteffort.slice
    ...
system.slice # 系统服务（含 kubelet、containerd）
```

```
systemd-cgls -u kubepods.slice | head -40
systemctl status kubepods-burstable-pod<uid>.slice
systemd-cgtop -m # 按 cgroup 看资源占用
systemd-run --scope -p MemoryMax=1G sleep 60 # 排障时临时用 systemd 限个小任务
```

必答：理解这个层级，才能解释"`kubeReserved` 需要指定 cgroup 路径"的原因（见 Kubelet 篇 Q13）——**路径写错，预留就白配。**

A24. eBPF 到底能做什么？在容器场景的典型用法?

它是一个在**内核虚拟机**里安全执行用户程序的机制，通过钩子点挂载：

钩子	用途
`kprobe` / `kretprobe`	追踪内核函数（如定位慢 IO、调度延迟）
`uprobe`	追踪用户态函数
`tracepoint` / raw tracepoint	稳定内核事件点
TC ingress/egress	容器网络转发、策略（Cilium 的数据面）
XDP	网卡驱动层，最高性能（DDoS 防护、负载均衡）
cgroup 钩子	按 cgroup 做 socket 层处理（Cilium socket-LB 的基础）
`LSM` 钩子	安全审计/阻断（Tetragon 等）

工具链：bcc、bpftrace、libbpf/CO-RE、Cilium/Hubble、Tetragon、Inspektor Gadget、Pixie。

```
# 快速定位谁在执行 execve / 谁在制造短连接
execsnoop-bpfcc
tcpconnect-bpfcc
biolatency-bpfcc -m
funclatency-bpfcc 'vfs_*'
```

> **面试加分**：eBPF 让"不重启、不改内核、不侵入应用"的可观测与安全成为可能，也是当前网络/安全方案的主流底座（与网络篇 Cilium 章节呼应）。

A25. 容器里的时间、时区与时钟漂移

- 容器默认**共享宿主内核的时钟**（time namespace 隔离部分于 newer kernels），所以"改容器时间"通常是在改宿主时间（危险）；
- **时区**由 `/etc/localtime` 决定 ⇒ 挂载宿主文件或在镜像里 `TZ=Asia/Shanghai`；
- **时钟同步**由宿主的 chrony/ntpd 保证；**时钟漂移会导致证书校验失败、日志错序、 leases 失效** ⇒ 必须监控。

```
# 节点时钟偏移（node_exporter 的 timex collector 需开启）
abs(node_timex_offset_seconds) > 0.05
```

A26. Linux 网络栈层的容器相关要点（与网络篇联动）

只在 Linux 层需要记住的三件事：

1. **必须开启 IP 转发**：`net.ipv4.ip_forward = 1`（否则跨节点/跨网卡转发全废）；

2. `net.bridge.bridge-nf-call-iptables = 1`：让**经过网桥的流量也走 iptables**（Service 的 DNAT 才能对 Pod 流量生效）——这是 CNI 报错排查里的经典项；
3. **conntrack 机制**：所有 NAT（Service nodePort/masquerade）依赖 conntrack 表（**见网络篇 A18，此处不再展开**）。

顺带两个常被问的参数：

- `net.ipv4.conf.all.rp_filter`：反向路径过滤，某些 CNI（如 Calico 的部分模式）会要求放松 ⇒ **按 CNI 官方要求设，不要求 newbies 别乱改**；
- `net.core.bpf_jit_enable`、`kernel.unprivileged_bpf_disabled`：与 eBPF 安全相关。

Part B • 容器运行时与经典故障专项篇（14 题）

B1. containerd / CRI-O / runc / shim 分别是什么?

```
kubelet ──CRI(gRPC)──► containerd / CRI-O （高层：镜像、容器生命周期、CRI API）
                              │
                              └─► containerd-shim-runc-v2 ──► runc create/start
                                        │ │
                                    持有容器 stdio 与退出状态 真正 clone namespaces + 设 cgroup + exec
                                     （所以 containerd 重启不影响运行中的容器）
```

必答：

- **runc** 是真正"启动容器进程"的 OCI 运行时，**一次性命令**，启动完就退出；
- **shim** 是常驻的"监护人"，负责 stdio 转发、exit code 收集，让 containerd 自身可以重启而不影响业务；
- **shim 泄漏**（成千上万个 shim 进程）会让节点 PID/内存吃紧，是高密度节点的经典问题。

```
ps -ef | grep containerd-shim | wc -l
crictl ps -a | wc -l
# 两者数量应该大致匹配；显著偏多 ⇒ shim 泄漏嫌疑
```

B2. 安全容器（Kata / gVisor）什么时候用？代价是什么?

方案	隔离强度	性能开销	适用
runc（默认）	共享内核（namespace+cgroup）	最小	绝大多数
gVisor (runsc)	用户态内核拦截 syscall	中（syscall 密集场景明显）	多租户 SaaS、不可信代码
Kata Containers	**轻量虚拟机**（独立内核）	启动慢一些、内存有额外开销	强隔离、合规要求

必答：K8s 侧通过 **RuntimeClass** 选择：

```
apiVersion: node.k8s.io/v1
kind: RuntimeClass
metadata:
  name: kata
handler: kata
scheduling:
  nodeSelector:
    runtime: kata # 只在支持该运行时的节点池调度
```

```
spec:
  runtimeClassName: kata
```

B3. 容器逃逸的常见面有多大？（安全必答题）

风险项	说明	缓解
`privileged: true`	几乎等于宿主 root	**禁止**（除极少数系统 DaemonSet）
挂载宿主 `/`（hostPath）	可改宿主文件/crontab	策略引擎禁危险 hostPath
挂载 `/var/run/docker.sock`	直接控制宿主容器	**绝对禁止**
`CAP_SYS_ADMIN` + writable cgroup	可利用 `release_agent` 提权	drop ALL + `allowPrivilegeEscalation: false`
可写 `/proc/sys`、`/sys`	改内核参数影响全节点	只读挂载 + 限制 sysctl
宿主内核漏洞	共享内核是原罪	及时打补丁、必要时上安全容器

K8s 侧的组合防线：

- **Pod Security Admission（baseline / restricted）**；
- `securityContext`：runAsNonRoot + readOnlyRootFilesystem + drop ALL + seccomp RuntimeDefault；
- OPA/Kyverno 做自定义策略；
- 运行时行为监控（Falco / Tetragon）。

```
apiVersion: v1
kind: Pod
spec:
  securityContext:
    runAsNonRoot: true
    seccompProfile:
      type: RuntimeDefault
  containers:
    - name: app
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities: {drop: ["ALL"]}
```

B4. 经典容器"玄学故障"15 例（面试杀手锏，建议直接背）

#	现象	根因	处置
1	容器内 `kill -TERM 1` 无反应	PID 1 的未处理终止信号被忽略	exec 形式入口 / dumb-init
2	Pod 里一堆 `<defunct>`	PID 1 不 reap 僵尸	tini/dumb-init
3	`No space left on device` 但 `df -h` 有空间	**inode 耗尽** 或 `/dev/shm` 太小	`df -i`；扩 `/dev/shm`
4	删了容器/大文件磁盘没释放	有进程持有已删除文件的 fd	`lsof ``| grep deleted`
5	JVM 服务无事 OOM	cgroup memory 含 page cache	`memory.stat`；调 limit
6	CPU 使用率不高但 RT 抖	**CFS throttling**	`cpu.stat nr_throttled`；放大 limit
7	容器内 `free` 看到宿主内存	procfs 不隔离	用 cgroup 数值；必要时 lxcfs
8	`ping: operation not permitted`	缺 `NET_RAW` + `ping_group_range`	加 cap 或调参
9	容器内 ulimit 不对	Pod spec 无稳定 ulimit API	运行时侧默认 + systemd
10	`too many open files`	fd 耗尽 **或** inotify watches 耗尽	`/proc/<pid>/limits` 、 `/proc/sys/fs/inotify/*`
11	滚动更新时应用收不到信号	shell 形式入口 + preStop 未配	exec 形式 + preStop
12	Java 线程数突然失败	**pids cgroup** 超限	`pids.max` / `--pod-max-pids`
13	容器内时区错	`/etc/localtime` 未挂载	挂 hosts path 或设 TZ
14	挂载卷后 permission denied	SELinux 标签/fsGroup 未处理	relabel / 配 fsGroup（**别 chmod 777 糊过去**）
15	`read-only file system` 写报错	`readOnlyRootFilesystem: true`	给需要写的目录挂 emptyDir

B5. 为什么"容器内 top/free/df 不准"会引发级联问题?

见 A21。核心危害是**应用按宿主资源做自我限流决策**（线程池大小、JVM 堆、Go GOMAXPROCS），结果是"超卖 + OOM"。所以：

- JVM 需 `-XX:+UseContainerSupport`（现代 JDK 默认）；
- Go runtime 默认 GOMAXPROCS=CPU limit 相关行为随版本演进，**高并发服务建议显式确认或设置**；
- 中间件（Nginx `worker_processes auto`、MySQL `innodb_buffer_pool_size`）同样要按 cgroup 校准。

B6. cgroup v1 的 kmem accounting 泄漏（深度事故题）

现象：节点运行一段时间后出现莫名其妙的内存占用、`cgroup limit exceeded` 或 `cannot allocate memory`，且账面（`memory.usage_in_bytes`）对不上。

根因：某些内核版本 + cgroup v1 的 **kmem accounting**（对 slab/dentry/inode 记账）在容器删除后**没有释放**（当时的内核 kmem 计费 bug），累积到一定程度会耗尽父 cgroup 甚至占满宿主的账面 memory ⇒ 连带全局 OOM。

处置与预防：

- 升级到修复版本的内核；
- 或在内核启动参数关闭：`cgroup.memory=nokmem`（**需重启节点，属重大变更**）；
- 使用 containerd 时检查相关 CVE/修复版本。

> 这题是"L4 有事故经验"的强证据：**能说出"内存账对不上要看 kmem/slab"** 的候选人不多。

B7. 共享内存 /dev/shm 事故复盘（模板）

```
现象：PyTorch DataLoader 多 worker 训练任务随机崩溃：Bus error / unexpected EOF
定位：kubectl exec 进入容器 df -h /dev/shm ⇒ 只有 64M（默认）
修复：emptyDir(medium: Memory, sizeLimit: 8Gi) 挂载到 /dev/shm，并同步抬高 memory request/limit
验证：重跑同样 batch 不再崩溃；同时监控 container_memory_usage_bytes 未超限
```

B8. 怎么看一个容器"真正用了多少资源"？（必考实操）

```
# ① 找到宿主上的容器 PID
crictl inspect <ctr-id> | jq .info.pid
PID=<pid>

# ② CPU：权威数据在 cgroup（不是 /proc/stat）
cat /sys/fs/cgroup$(cat /proc/$PID/cgroup | awk -F: '$2 ~ /cpu/ {print $3}')/cpu.stat 2>/dev/null
# 或者用 systemd 视角：
systemctl status $(ps -o slice= -p $PID)

# ③ 内存：
cat /sys/fs/cgroup/.../memory.stat
cat /sys/fs/cgroup/.../memory.events # 有 oom 事件吗?

# ④ 谁占了 fd / socket / thread
ls /proc/$PID/fd | wc -l
cat /proc/$PID/status | grep -E "Threads|VmRSS"
nsenter -t $PID -n ss -s
```

B9. crictl 与 docker 命令对照（排障必备）

目的	crictl	docker（旧）
列 Pod	`crictl pods`	—
列容器	`crictl ps -a`	`docker ps -a`

目的	crictl	docker（旧）
查看镜像	`crictl images`	`docker images`
容器日志	`crictl logs <id>`	`docker logs`
进入容器	`crictl exec -it <id> sh`	`docker exec`
详情	`crictl inspect <id>`	`docker inspect`
清理无用镜像	`crictl rmi --prune`	`docker image prune`
Pod 沙箱	`crictl inspectp <id>`	—

必答：生产环境**不要装 docker 只为排障**，用 `crictl` + `nsenter` + `cgroup 文件` 这套组合就够了（而且更贴近真实数据面）。

B10. 为什么删除 Pod 后宿主机仍有残留进程?

常见原因：

- 进程处于 **D 状态**（不可中断睡眠，多在等待坏掉的 NFS/网络存储）⇒ 无法被 SIGKILL 立即回收；
- 容器内进程未退出（应用不处理信号）；
- kubelet 与 runtime 状态不一致；
- **僵尸**：父进程没 reap。

```
ps aux | awk '$8 ~ /D|Z/' # D=不可中断睡眠，Z=僵尸
cat /proc/<pid>/stack # 看内核栈（需要权限）
cat /proc/<pid>/wchan
```

必答：**D 状态进程是"节点可能需要重启"的强信号**（尤其是后端存储 hang 住的场景），此时应先 cordon 隔离节点，确认业务已漂走后再处理。

B11. 容器启动到底慢在哪？（和冷启动话题联动）

```
拉取镜像 → 解包到 overlay lower 层（IO 密集）
       → runc create：clone namespaces + 建 cgroup + mount
       → runc start：execve 业务进程
       → 应用初始化（JVM/框架）
```

定位手段：

- `systemd-cgtop`、`iolatency` 看 IO；
- `funclatency` / `execsnoop` 看 exec 行为；
- `perf top -p <pid>` 看应用自身热点；
- 度量 → 对比：`kubelet_runtime_operations_duration_seconds`（见 Kubelet 篇）。

B12. `/sys/fs/cgroup` 只读挂载导致的坑

容器内如果把 cgroup 文件系统**以可写方式挂载**，配合 `CAP_SYS_ADMIN` 就能改写自己的限额（提权手段）。所以：

- 默认只**只读**挂载 cgroupfs；
- 应用若需要读 cgroup 值，读只读挂载即可（足够）。

B13. cgroup delegation 与"容器里再跑容器"

前提是 **cgroup v2 delegation**：把一棵子树的控制器委托给子层级 ⇒ 容器内的 systemd 才能管理自己的 cgroup。 常见场景：CI runner / DinD（Docker in Docker）/ KubeEdge 类场景。**注意它显著增加逃逸面**，一般不推荐在生产无条件开放。

B14. 哪些系统级服务应该被限制以免拖垮节点?

- **容器运行时**本身要进 `kubeReserved`（通过 cgroup 真正限制）；
- **日志采集/Fluent Bit** 容易内存/CPU 失控 ⇒ 必须设 limits 与 backpressure；
- **监控 agent / node exporter** 同样要有 limits，避免被业务拖累反向拖累；
- **DaemonSet 全部要设 requests/limits**（它们分布在每个节点，溢出会以 N 倍放大影响）。

Part C • 大规模节点内核 / 系统调优篇（14 题）

> 定义：**大规模 = ≥ 1000 节点 / 单节点 ≥ 50 Pod / 高并发短连接**。所有 sysctl 建议值都必须在你的机型与业务上压测校准。

C1. 一份可直接落地的节点 sysctl 基线（示意，务必逐项验证）

```
cat > /etc/sysctl.d/99-k8s-node.conf <<'EOF'
# ---------- 网络转发与桥接 ----------
net.ipv4.ip_forward = 1
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1

# ---------- conntrack（★ 短连接高并发必看）----------
net.netfilter.nf_conntrack_max = 1048576 # 建议初值，按实测并发调整
net.netfilter.nf_conntrack_buckets = 262144 # hash 桶数 ≈ max/4（部分环境需模块加载时设置）
net.netfilter.nf_conntrack_tcp_timeout_established = 86400
net.netfilter.nf_conntrack_tcp_timeout_time_wait = 30
net.netfilter.nf_conntrack_tcp_timeout_close_wait = 60

# ---------- 连接与端口 ----------
net.core.somaxconn = 32768 # listen backlog，默认常过小
net.core.netdev_max_backlog = 10000 # 网卡收包队列，高 PPS 建议调大
net.ipv4.ip_local_port_range = 1024 65535
net.ipv4.tcp_tw_reuse = 1
net.ipv4.tcp_max_syn_backlog = 8192
net.ipv4.tcp_syncookies = 1

# ---------- 文件与 inotify ----------
fs.file-max = 1048576
fs.inotify.max_user_instances = 1024 # 大量 Pod 挂载 Secret/ConfigMap 时常耗尽
fs.inotify.max_user_watches = 1048576

# ---------- 虚拟内存 ----------
vm.swappiness = 10 # K8s 通常不建议大量依赖 swap（见 C7）
vm.max_map_count = 262144 # JVM / ES 类必看
vm.overcommit_memory = 0

# ---------- 其它 ----------
kernel.pid_max = 4194304
EOF
sysctl -p /etc/sysctl.d/99-k8s-node.conf
```

> **交付原则**：sysctl 是**节点级全局变更**，必须做到：① 配置版本化（Ansible/镜像）；② 按节点池灰度 1 台 → 10% → 50%；③ 保留回滚文件；④ 每次变更后跑一遍基准压测。

C2. conntrack 容量怎么算？（和网络篇互补，这里给"账怎么算"）

```
nf_conntrack_max ≈ 预计并发连接数 × 1.5~2（安全系数）
buckets = nf_conntrack_max / 4 （每个桶一个链表，桶多则链表短）
每条连接约占 ~300 字节内核内存（量级估算，以实测为准）
```

```
node_nf_conntrack_entries / node_nf_conntrack_entries_limit > 0.75
rate(node_nf_conntrack_insert_failed[5m]) > 0
```

必答：**Cilium 的 socket-LB 路径不依赖 conntrack**，这是短连接极高并发场景从 Linux 层缓解此问题的关键手段（见网络篇）。

C3. somaxconn 与 backlog：为什么默认不够?

- `net.core.somaxconn` 限制的是 **listen 队列长度**；应用如果有 backlog 参数（如 Nginx `listen ... backlog=`）取的是较小值；
- 队列满 ⇒ 新连接被丢弃（客户端看到超时/重传），且**服务端可能完全没有错误日志**；
- 高并发短连接服务要连带看 `tcp_max_syn_backlog`、`tcp_abort_on_overflow`（一般不建议开）。

C4. epoll / fd：三层限制与"too many open files"

见 A20。补充高层手段：

```
# systemd 服务级默认（会影响以 systemd 方式运行的容器运行时等）
systemctl set-property containerd.service TasksMax=infinity
cat /etc/systemd/system.conf.d/99-defaults.conf
# [Manager]
# DefaultLimitNOFILE=1048576
# DefaultTasksMax=infinity
```

C5. inotify 耗尽：为什么 K8s 节点特别容易中招?

原因：kubelet 对每个以 volume 方式挂载的 ConfigMap/Secret 都要 watch 文件变化，每个都要占 inotify instances/watches；Pod 密度高 + 大量 Secret ⇒ 直接耗尽。

表现：kubelet 报 `too many open files` / `inotify_init: failed`，节点上的 Pod 挂载开始失败。

治理：

- 调大 `fs.inotify.max_user_instances` 与 `max_user_watches`；
- **Secret 设** `immutable: true`（正解，能去掉大量 watch）；
- 降低"每个 Pod 挂很多 Secret"的架构模式。

```
# node_exporter 需开启相应 collector，指标名以实际为准
node_filefd_allocated / node_filefd_maximum > 0.8
```

C6. THP（透明大页）：为什么数据库类负载常常要求关掉?

THP 让内核自动把连续小页合并为 2MiB 大页，**在内存分配/回收频繁的场景会引入明显延迟尖峰**（khugepaged 整理、缺页中断变重）。Java heap、Redis、MySQL、MongoDB、各类数据库/中间件官方建议多为**关闭 THP**（或只保留 `madvise`）。

```
cat /sys/kernel/mm/transparent_hugepage/enabled
# 常见建议值（需实测）：[always] madvise never  → 关闭改为 never 或写入启动项 transparent_hugepage=never
echo never > /sys/kernel/mm/transparent_hugepage/enabled
```

C7. swap：Kubernetes 到底该不该用?

立场	说明
传统	kubelet 默认要求关闭 swap（`--fail-swap-on`），因为 swap 会让"内存用量"失去确定性，干扰 OOM 决策
演进	新版本对 swap 提供了受控支持（如仅限 cgroup v2 的受限 swap 行为，版本相关请核对 Release Notes）
实践建议	**在线延迟敏感服务不要依赖 swap**（一次 swap-in 可能带来数量级的延迟）；离线批处理可在受控场景下用 swap 提高装箱率

配套：

- `vm.swappiness` 调低（*建议 1~10*）；
- 若启用，**必须监控 swap 使用率与 swap-in/out 速率**，并明确有 kubelet、cgroup v2 的支持条件。

```
node_memory_SwapFree_bytes / node_memory_SwapTotal_bytes < 0.5
rate(node_vmstat_pswpin[5m]) > 0
```

C8. 文件系统与磁盘：为什么推荐 imagefs 与 nodefs 分离?

问题：镜像层 + 容器可写层 + 容器日志共享一块磁盘时，任何一类暴涨都会导致 kubelet/运行时自身无法工作（**镜像/I/O 抖动 ⇒ PLEG 慢 ⇒ Node NotReady**）。

治理：

项	建议
分区	`/var/lib/containerd`（imagefs）独占磁盘；`/var/lib/kubelet` 与根分区另设或留足空间
文件系统	ext4 / xfs 皆可；大量小文件/超大目录场景 xfs 常更合适（**请实测**）
挂载参数	`noatime,nodiratime`（减少元数据写）
SSD	开启周期性 discard（`fstrim.timer`）或在线 discard；监控 SSD 磨损
监控	分别对 imagefs、nodefs、inode 做容量告警

```
systemctl enable --now fstrim.timer
df -i /var/lib/containerd
```

C9. PID / 线程：宿主与容器的双上限

```
容器级：pids.max（Pod 级由 --pod-max-pids 等设置；也与 pause loop 相关）
宿主级：kernel.pid_max、kernel.threads-max
子系统：systemd TasksMax（默认可能为 15% 之类，容易踩到）
```

典型故障：`resource temporarily unavailable` / `cannot allocate memory`（fork 失败常被误读成内存问题）。

```
# 节点 PID 使用率
node_procs_running / node_procs_threads > 0.8 # 仅为示意：实际请用 pids.current 或 exporter 指标
```

C10. IO 隔离的现实手段（因为 K8s 没有稳定的 IO 限制 API）

优先级：

1. **专用节点池**（taint + toleration）——最有效；
2. **存储侧 QoS**（云盘 IOPS/吞吐配额、Ceph 的 mClock/qos）——最贴近；
3. **cgroup v2 io.weight / io.max**（需在运行时层或平台侧配置，**K8s 不直接提供**；务必兼容性与压测）；
4. 监控 ≥ 限制：至少先把"设备 IO 延迟/队列深度/IOPS"看清楚再谈限制。

```
# 磁盘 IO 利用率（node_exporter）
rate(node_disk_io_time_seconds_total[5m])
# IO 等待是对 CPU 延迟的直接伤害
rate(node_cpu_seconds_total{mode="iowait"}[5m])
```

C11. 高 PPS 下的网络栈调优包

项	建议（需实测）
网卡多队列 RSS	队列数 ≥ CPU 核数，中断分散到多核
irqbalance	开启（或手动绑 IRQ，避免所有中断压在 CPU0）
RPS / RFS	虚拟接口/隧道场景可考虑开启分散软中断

项	建议（需实测）
网卡 ring buffer	高 PPS 适当调大（`ethtool -g/-G`）
`net.core.netdev_budget` / `netdev_budget_usecs`	影响一次软中断处理包数，**调整需谨慎压测**
MTU	物理网络允许时用 jumbo frame 降 PPS；**注意隧道的头开销**（见网络篇）

```
# 软中断分布不均（单核打满的典型信号）
topk(5, rate(node_softirqs_total[5m]))
# 网卡丢包
rate(node_network_receive_drop_total[5m])
rate(node_network_receive_errs_total[5m])
```

C12. NUMA 与 CPU 绑定的节点级配合

- `isolcpus` / `nohz_full` / `rcu_nocbs`（延迟敏感 + DPDK 类场景），但会减少通用调度域，慎用；
- 与 kubelet 的 `cpuManagerPolicy: static` + `topologyManagerPolicy` 一起做，且必须配 `reservedSystemCPUs`；
- 检查：`numactl -H`、`cat /proc/interrupts`、`numastat`（看跨 NUMA 内存访问）。

C13. PSI（Pressure Stall Information）：节点压力的最现代画像

cgroup v2 时代，内核在 `/proc/pressure/{cpu,io,memory}` 提供压力指标：

```
some avg10=... avg60=... avg300=... total=...
full avg10=...  (full 表示所有任务都被阻塞，最危险的信号)
```

价值：比"CPU 使用率"更能说明"应用到底有没有被饿到"——**CPU 使用率 100% 但 PSI cpu-some 很低，说明只是在跑，不饿；反之 PSI 高说明排队严重。**

```
# node_exporter 的 psi collector（需开启，指标名以 exporter 版本为准）
node_pressure_cpu_waiting_seconds_total
node_pressure_memory_waiting_seconds_total
node_pressure_io_waiting_seconds_total
# 建议先看 "some avg10" 的 gauge：node_pressure_cpu_some_avg10 之类
```

C14. 节点级变更的标准灰度与回滚（Linux 层最容易忽视的）

阶段	范围	观察（≥30min，含一次高峰）	回滚触发
Step 1	**1 台**（隔离/预发池）	PLEG relist、RT 抖动、软中断分布、 conntrack	任一指标超基线 1.5×
Step 2	1% 节点	+ Pod 启动 P99、iowait	同上 / 业务 P99 恶化
Step 3	10% → 50%	全 SLO Dashboard	同上
Step 4	100%	稳定 ≥24h	—

回滚要点：

- sysctl 回滚 = 恢复上一版配置并 `sysctl -p`（秒级）；
- **内核/cgroup 驱动/文件系统类变更不能原地回滚** ⇒ 必须 drain + 重建节点（这就是它们必须排在大版本升级窗口的原因）；
- 所有变更写入 "节点基线镜像"，保证新节点天然一致，避免配置漂移（**漂移是 Linux 层事故的最大来源**）。

Part D · 反模式清单（可直接用于生产巡检）

#	反模式	证据（怎么发现）	等级	修复动作	验证方式
D1	延迟敏感在线服务设 CPU limit=requests	throttling 占比 >10%	🔴 高	去掉 limit 或放宽到 request 的 2~4 倍	`nr_throttled` 速率归零 + P99 改善
D2	容器用 shell 形式入口（无 init）	信号不生效、僵尸进程堆积	🔴 高	exec 形式或 dumb-init	`kill -TERM 1` 生效、无 `<defunct>`
D3	生产用 `privileged: true`	Pod spec 审计	🔴 高	收敛 capability + PSS restricted	策略引擎拦截生效
D4	kubelet 与运行时 cgroup driver 不一致	两条 cgroup 路径、limit 不生效	🔴 高	统一为 systemd	`systemd-cgls` 层级一致
D5	只看平均 CPU 不看 throttling	RT 抖动却无头绪	🟠 中	加 throttled 指标与告警	压测时能复现并归因
D6	配置节点后不做基线版本化	节点间参数漂移	🔴 高	sysctl/运行时配置纳入节点镜像	巡检脚本比对一致
D7	inotify/ulimit/fd 无监控	突然大量挂载失败	🟠 中	调参 + Secret immutable + 告警	压力密度下无 limitations 报错
D8	imagefs 与 nodefs 共用根盘	镜像暴涨导致节点不可用	🔴 高	分离磁盘 + GC 阈值	写满演练不影响 kubelet
D9	忽视 inode（`df -h` 只看容量）	空间够却报 No space	🟠 中	`df -i` 纳入监控	inode 告警提前触发
D10	THP 未关（数据库节点）	RT 偶发尖峰	🟠 中	关闭 THP（按厂商建议）	压测 P99 平稳
D11	swap 随意开启且无监控	偶发巨长尾延迟	🟠 中	明确策略 + 监控 swap-in/out	在线服务无 swap 抖动
D12	DaemonSet 不设 resources	节点被 agent 拖垮	🟠 中	全部设 requests/limits	压测下节点资源有冗余
D13	未在 CI 里校验 cgroup limit 与应用参数匹配	JVM 按宿主内存算堆	🟠 中	cgroup-aware + 显式参数	容器压测不 OOM
D14	直接 `chmod 777` 解决挂载权限	掩盖 SELinux/fsGroup 真因	🟠 中	正确 relabel / fsGroup + OnRootMismatch	无 777 且功能正常
D15	内核一次性全量升级	大面积驱动/运行时不兼容	🔴 高	节点池灰度 + drain/回滚窗口	灰度台各项指标达标

Part E • 可观测性：指标与告警

E1. 必看的指标清单（先 `curl http://<node>:9100/metrics` 与 cgroup 文件核对）

层面	指标 / 数据源	用途
CPU 限额	`container_cpu_cfs_throttled_periods_total` / `_periods_total` / `_throttled_seconds_total`	throttling 检测（**核心**）
CPU 使用	`container_cpu_usage_seconds_total`	真实用量
内存	`container_memory_working_set_bytes` / `_rss` / `_cache`	内存账；working_set 是 OOM 判定更接近的量
OOM	`kube_pod_container_status_terminated_reason{reason="OOMKilled"}`	容器 OOM
cgroup 原生	`cpu.stat(nr_throttled/throttled_usec)`、`memory.stat`、`memory.events(oom)`、`pids.current`、`io.stat`	**权威真相**（hostCom基层党组织）
进程	`node_procs_running` / `_blocked`（D 状态）/ `_zombies`	进程健康
fd / inode	`node_filefd_allocated` / `node_filesystem_files_free`	fd 与 inode
内存	`node_memory_*` + `node_vmstat_pswpin/pswpout`	节点内存与 swap
网络	`node_network_*`、softirq、丢包	高 PPS 与软中断
conntrack	`node_nf_conntrack_*`	NAT 表容量
PSI	node_exporter psi collector（开启后）	真实的"被饿到"程度
时钟	`node_timex_offset_seconds`	时钟漂移

E2. Prometheus 告警规则（可直接粘贴，阈值需实测校准）

```
groups:
  - name: linux-container
    rules:
      - alert: ContainerThrottlingHigh
        expr: |
          sum by (namespace, pod, container) (rate(container_cpu_cfs_throttled_periods_total[5m]))
            / clamp_min(sum by (namespace, pod, container) (rate(container_cpu_cfs_periods_total[5m])), 0.001) > 0.1
        for: 15m
        labels: {severity: warning}
        annotations:
          summary: "容器 {{ $labels.namespace }}/{{ $labels.pod }}/{{ $labels.container }} CPU 被限流比例过高"

      - alert: ContainerOOMKilled
        expr: increase(kube_pod_container_status_terminated_reason{reason="OOMKilled"}[15m]) > 0
        for: 2m
        labels: {severity: critical}
        annotations:
          summary: "发生容器 OOMKill: {{ $labels.namespace }}/{{ $labels.pod }}"

      - alert: NodeConntrackNearFull
        expr: node_nf_conntrack_entries / node_nf_conntrack_entries_limit > 0.75
        for: 10m
        labels: {severity: warning}

      - alert: NodeFileDescriptorNearLimit
        expr: node_filefd_allocated / node_filefd_maximum > 0.8
        for: 10m
        labels: {severity: warning}

      - alert: NodeInodeAlmostExhausted
        expr: node_filesystem_files_free / node_filesystem_files < 0.1
        for: 15m
        labels: {severity: warning}
        annotations:
          summary: "{{ $labels.mountpoint }} inode 剩余不足 10%"

      - alert: NodeManyBlockedProcesses
        expr: node_procs_blocked > 10
        for: 10m
        labels: {severity: warning}
        annotations:
          summary: "节点 {{ $labels.instance }} 存在较多 D 状态进程，可能存在存储 hang"

      - alert: NodeZombiesHigh
        expr: node_procs_zombies > 50
        for: 15m
        labels: {severity: info}
        annotations:
          summary: "僵尸进程偏多，检查容器 PID 1 是否 reap"

      - alert: NodeClockDrift
        expr: abs(node_timex_offset_seconds) > 0.05
        for: 10m
        labels: {severity: warning}

      - alert: NodeSwapUsed
        expr: |
          (node_memory_SwapTotal_bytes - node_memory_SwapFree_bytes) / node_memory_SwapTotal_bytes > 0.1
        for: 15m
        labels: {severity: info}
        annotations:
          summary: "节点开始使用 swap，在线服务可能出现长尾延迟"
```

E3. 建议的 SLO（初始建议值，必须实测校准）

SLI	建议目标	度量
容器 CPU 限流占比	≤ 5%（在线）	throttled_periods / periods
容器 OOMKill 次数	在线 ≈ 0	terminated_reason=OOMKilled
conntrack 使用率	≤ 75%	node exporter
inode 剩余	≥ 10%	node exporter
时钟偏移	≤ 50ms	node_timex_offset_seconds
节点 D 状态进程峰值	≤ 10	node_procs_blocked
fd 使用率	≤ 80%	node_filefd_*

Part F · 演练（GameDay）计划表

场景	注入方式	期望表现	验证的 SLO
CPU throttling 事故复现	给某容器设很小的 cpu limit 后跑计算	告警能识别出 throttling 而非"CPU 高"	限流占比 SLO
容器 OOM 复现	容器疯狂写文件制造 page cache	能从 `memory.events` 的 oom 计数与内核日志确认归因，而非误判为 RSS 超限	OOM 归因准确性
/dev/shm 耗尽	跑多 worker 训练任务	有明确报错与处理手册	故障处理 MTTR
fd/inotify 耗尽	制造大量挂载/打开文件	告警提前触发；immutable Secret 能缓解	密度上限认知
D 状态进程	让后端存储 hang	能识别 cordon 决策链	节点隔离流程
内核参数漂移	手动改一台机器的 sysctl	巡检能发现不一致	配置一致性
conntrack 打满	短连接压测	告警早于丢包	连接成功率
时区/时钟漂移	停止 chrony 并手动偏移	时钟告警触发，证书/lease 行为可解释	时钟 SLO
镜像盘写满	疯狂拉取镜像	imagefs 告警 + kubelet 仍存活（磁盘分离有效）	磁盘隔离有效性
cgroup v2 迁移演练	独立节点池先迁	指标、PSI、编译差异、运行时兼容全部验证	迁移可行性

Part G • 10 张面试速记卡（临阵磨枪）

1. **容器 = namespace（看得见）+ cgroup（用多少）+ capabilities/seccomp（能做什么）+ rootfs（文件系统）**。开场说这句，后面全部挂得上去。
2. **八个 namespace**：mount / pid / net / uts / ipc / user / cgroup / time；`/proc/<pid>/ns/*` 存在 ⇒ namespace 不灭。
3. **PID 1 两件事**：转发信号（**未注册的终止信号会被内核忽略**）与 reap 僵尸。解法：exec 入口或 dumb-init。
4. **cgroup memory 含 page cache** ⇒ "进程没超内存却被 OOM"的真相；`memory.events` 的 oom 计数是最硬证据。
5. **CFS throttling**：平均 CPU 不高也会被限流，看 `cpu.stat` 的 `nr_throttled`；在线服务建议少设/放宽 CPU limit。
6. **cgroupDriver 必须和 runtime 一致**（通常 systemd），否则两套层级、限制失效。
7. `privileged` ≈ **宿主 root**；正确的姿势是 drop ALL + 按需加 + `runAsNonRoot` + `readOnlyRootFilesystem` + seccomp RuntimeDefault。
8. **overlayfs 的代价是 inode**：`No space left on device` 先看 `df -i`，再看有没有进程持有已删除文件（`lsof | grep deleted`）。
9. **容器里 `top/free` 看的是宿主**（procfs 不隔离）；权威数据来源是 `/sys/fs/cgroup` 与 cAdvisor。
10. **节点参数必须版本化**：sysctl/运行时配置进节点镜像，按池灰度 1 台 → 1% → 100%，内核/cgroup 驱动类变更不可原地回滚（要 drain 重建）。

第二篇 • APIServer 深度解析

> 请求链路 · 认证鉴权准入 · watch/etcd · 大规模调优

Part 0：答题方法论

面试官考察 APIServer 时，本质上在看三件事：

考察维度	表现出来的是	高分回答的特征
原理深度	能不能画出请求从 `kubectl` 到 `etcd` 的完整链路	说出「认证 → 鉴权 → 准入 → 校验 → 存储」，并能补 APF、序列化、乐观锁
规模意识	知不知道 5000 节点时什么先挂	能说清 **写放大源**（Lease/Event/Endpoint）、**读放大源**（List 全量 / watch 风暴）
排障能力	给一个报错能否定位	能给出：报错 → 指标 → 日志 → 配置 → 验证五步

答题模板（强烈建议背下来）：

1. 一句话定位（它是什么 / 解决什么问题）
2. 关键机制（2–3 个核心技术点）
3. 大规模下的失效模式（这是拉开差距的地方）
4. 可观测指标 + 修复动作 + 回滚条件

Part 1：架构与组件原理（Q1–Q16）

Q1：APIServer 在 Kubernetes 中的定位是什么？为什么它可以是无状态的？

答案要点

- APIServer 是整个集群的**唯一数据入口**：所有组件（kubelet、scheduler、controller-manager、operator、kubectl）都不直接读写 etcd，全部通过 APIServer 的 RESTful API。
- 它是**唯一与 etcd 通信的组件**（除 etcd 自身运维），因此承担：认证、鉴权、准入、校验、序列化/反序列化、版本转换、审计、限流。
- **无状态的原因**：状态全部落在 etcd。APIServer 本地只有 **watch cache**（每个资源类型一份内存缓存），它是 etcd 数据的增量副本，丢失后可重建，不构成持久状态。

为什么重要

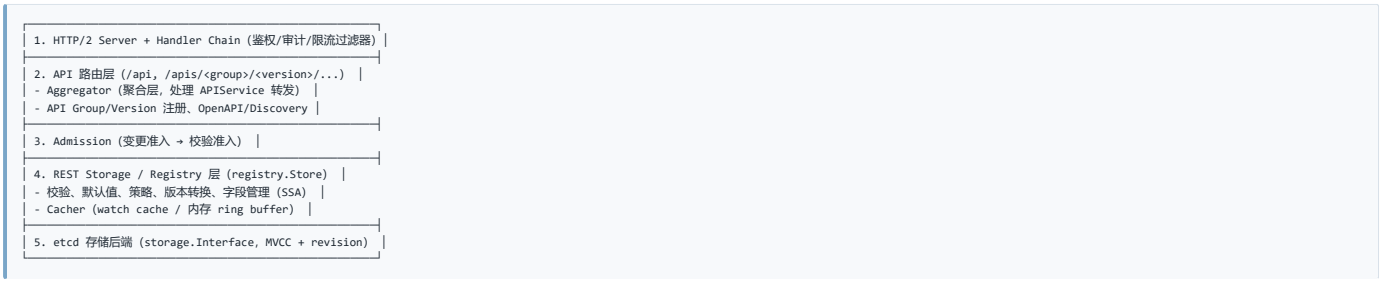
- 决定了 APIServer 可以**水平扩容**（N 副本 + LB），这是大规模集群抗压的基础手段。
- 也决定了任何"绕过 APIServer 直接改 etcd"的操作都是高危反模式（会绕过准入、审计、校验，且可能破坏数据格式）。

加分项

- 提到 `--enable-priority-and-fairness`、`--goaway-chance`（HTTP/2 连接打散，避免 LB 粘滞导致负载不均）。
- 提到 APIServer 之间的**一致性**由 etcd 的线性一致读保证，而非 APIServer 自身。

Q2：画一下 kube-apiserver 的内部架构分层。

答案要点（自顶向下五层）



追问：Aggregator 和 CRD 的区别？ 见 Q15。

Q3：`kubectl apply -f pod.yaml` 发出后，APIServer 内部发生了什么？（高频送命题）

完整链路

1. **客户端侧**：kubectl 读取 kubeconfig → 认证信息（客户端证书 / token）→ 计算请求签名 → 发送 `POST /api/v1/namespaces/default/pods`（Content-Type 通常是 `application/vnd.kubernetes.protobuf` 或 json）。
2. **传输层**：LB → 任一 APIServer 实例（HTTP/2 长连接可能粘滞）。
3. **Handler Chain（过滤器链，从外到内主干顺序）**： Panic Recovery / RequestInfo 解析（verb、resource、namespace、name） **Audit**（审计记录，含 audit-id） **Authentication**（认证，得到 user/groups） **Impersonation**（模拟身份，`--as` 场景） **Priority \& Fairness**（APF 限流/排队，按 user/groups/verb/resource 分流） **Authorization**（RBAC 鉴权决策） **Admission**：先 Mutating（改对象）→ 再 Validating（只读校验）
4. **反序列化 + 版本转换**：把请求体转成内部版本（`internal`），填默认值。
5. **校验**：字段校验、不可变字段校验（immutable field）。

6. **存储**：`registry.Store.Create` → 乐观锁检查（create 时 uid/resourceVersion 为空）→ 写入 etcd（`etcd3.store`，带 TTL/lease 视资源而定）→ 返回带 `resourceVersion` 的对象。
7. **响应**：序列化回请求的版本 → 返回客户端。
8. **异步**：写入成功触发 etcd watch 事件 → Cacher 更新 watch cache → 所有 watch 该资源的 client（kubelet/scheduler/controller 的 informer）收到事件。

> 版本提示：Handler Chain 的**细粒度顺序在不同版本有微调**（例如 APF 与审计的相对位置），答题抓主干「认证 → APF → 鉴权 → 准入 → 校验 → 存储」即可；APF 位于认证之后，因为它需要 user/groups 才能做 FlowSchema 匹配。

加分项

- 提到 `kubectl apply` 是 **Server-Side Apply**（1.22 GA），走 `PATCH + application/apply-patch+yaml`，携带 `fieldManager`。
- 提到 APIServer 会打 **Warning header**（如已废弃 API）。

Q4：APIServer 支持哪些认证方式？生产上怎么选？

方式	原理	适用	生产建议
X509 客户端证书	CA 签发的客户端证书，CN=用户名，O=组	管理员、kubelet、控制面组件	kubeadm 默认；注意证书轮转
ServiceAccount Token	签发的 JWT（1.24+ 需通过 TokenRequest API 动态申请，不再自动生成 Secret）	集群内 Pod	用 `projected volume` + 短时效 token
OIDC / OpenID Connect	外部 IdP 签发 ID token	人类用户、SSO	企业首选，配合 `--oidc-*` 参数
Webhook Token Review	转发 token 给外部服务校验	对接自有账号体系	注意 webhook 是 APIServer 的强依赖，需高可用
Bootstrap Token	短期 token，用于节点引导加入	节点扩容	用完即删
Anonymous	匿名请求，映射到 `system:anonymous`	默认开启	**生产建议：`--anonymous-auth=false`**

关键追问

- **ServiceAccount Token 从 1.24 起不再自动生成 Secret**（LegacyServiceAccountTokenNoAutoGeneration），必须调用 `TokenRequest` API。旧代码依赖 `sa.Secrets` 会失效。
- **证书里的 CN/O** → user/groups，这是 RBAC 绑定的依据（`system:masters` 组等价于 root，慎用）。

Q5：RBAC 的鉴权流程？`Role` 和 `ClusterRole` 的关系？

答案要点

- RBAC 由 `rules`（apiGroups / resources / verbs / resourceNames）构成，通过 `RoleBinding`（命名空间级）或 `ClusterRoleBinding`（集群级）把主体（User/Group/ServiceAccount）绑定到角色。
- **鉴权顺序**：APIServer 按 `--authorization-mode` 配置的顺序依次询问各 authorizer（如 `Node,RBAC`），**任一 Allow 即放行，全部 NoOpinion 则拒绝**；没有 Deny 优先（RBAC 是白名单模型）。
- 决策被缓存（`--authorization-webhook-cache-*`，对 webhook 模式有效），RBAC 本身基于 informer 内存数据，规则变更有秒级传播延迟。

反模式（高频考点）

- 把 `cluster-admin` 绑给 ServiceAccount（等价于给每个 Pod root）。
- 用 `system:masters` 组作为日常操作账号 —— **绕过所有 RBAC 检查**，审计里也难以区分操作者。
- wildcard `resources: ["*"], verbs: ["*"]`。

Node Authorizer 考点

- `system:nodes` 组的 kubelet 只能读写**自己节点**相关的资源（Node、本节点的 Pod、本节点 Pod 的 Secret/ConfigMap、Lease、PV 附着等）。
- **NodeRestriction** 准入插件配合，防止 kubelet 越权改别人的 Pod / 给自身加 label（`node-restriction.kubernetes.io/` 前缀限制）。

Q6：Admission Control 是什么？Mutating 和 Validating 的执行顺序？

答案要点

- Admission 是**对象持久化前的最后一道关卡**，能看到完整的请求对象和（变更阶段的）修改结果。
- **执行顺序**：**MutatingAdmission**（可以修改对象）：内置插件（NamespaceLifecycle、LimitRanger、ServiceAccount、DefaultStorageClass、PodSecurity...）→ **MutatingAdmissionWebhook**（内置 mutate 全部跑完后才调用外部 webhook，保证 webhook 看到最终形态）**ValidatingAdmission**（只读，不改对象）：内置校验插件 → **ValidatingAdmissionWebhook**（基本在最后）
- 原因：webhook 必须看到内置插件处理后的完整对象，否则注入的 sidecar 会被后续插件忽略。

配置要点（webhook 必背参数）

```
failurePolicy: Fail # Fail=webhook 挂了就拒绝请求（强依赖）；Ignore=放行（弱依赖）
sideEffects: None # 声明无外部副作用，配合 dry-run
timeoutSeconds: 3 # 【建议值】≤5s；越长大面积超时越容易拖垮 APIServer
reinvocationPolicy: Never  # Mutating webhook 修改后是否再次调用
matchPolicy: Equivalent # 是否匹配资源的等价版本
objectSelector: {} # 缩小命中范围 — 大规模集群最重要的性能开关
```

大规模风险

- webhook 是**同步阻塞**的：一个慢 webhook 会同步抬高所有请求的 P99，并占用 APIServer 的并发额度（inflight 请求持有内存）。
- 反模式：`failurePolicy: Fail` + 单点部署 + 无 `objectSelector`（命中全部 Pod 创建）。

Q7：resourceVersion 是什么？乐观锁怎么实现？

答案要点

- `resourceVersion` 本质是 **etcd 的 MVCC revision**（全局单调递增的整数），并不透明 —— 但**语义上不保证可比较**，客户端只能做「相等/不等」判断，不能做大小比较。
- **乐观锁流程**： 客户端 GET 拿到 `metadata.resourceVersion` UPDATE 时带上该值 APIServer 转换为 etcd 的 `txn`： `if revision == X then put else fail` 不匹配 → 返回 **409 Conflict**，客户端必须**重新 Get 再重试**（不能直接重试旧对象）
- **意义**：无锁并发，避免分布式锁开销；代价是写冲突时需要重试（热对象上写冲突明显）。

实战建议

- 客户端必须实现 **指数退避 + 冲突重试**（client-go 的 `retry.RetryOnConflict`）。
- 控制器里常见反模式：先 Get 再改再 Update，且缓存陈旧 → 频繁 409。

Q8：List-Watch 机制是怎么工作的？Informer 为什么用它？

答案要点

1. **List**：informer 首次启动时做一次全量 List（可分页），拿到所有对象 + 当前 `resourceVersion`。
2. **Watch**：用该 `resourceVersion` 建立长连接，APIServer 持续推送增量事件（ADDED/MODIFIED/DELETED/BOOKMARK）。
3. **本地 Store + DeltaFIFO**：informer 把对象放本地 indexer 缓存，业务代码从缓存读，**不直接打 APIServer**。
4. **Resync**：周期性（数小时量级，【需实测】按集群规模配置）把本地对象重新入队触发 handler，**不产生 APIServer 请求**（这是常见误解 —— resync 不重新 List）。
5. **Relist**：当 watch 断连且资源版本过期（收到 410 Gone / ErrExpired）时，才重新 List。

为什么不用轮询：推模式把"读放大"从 O(客户端数 × 频率) 降到 O(变更数)；且 APIServer 侧有 watch cache，事件分发不落 etcd。

大规模风险

- informer **重建风暴**：APIServer 重启/滚动升级 → 所有客户端同时 relist → 瞬时 List 请求打爆 APIServer 和 etcd。
- 缓解：客户端 relist 加**抖动（jitter）**；升级 APIServer 时错峰；1.27+ 的 **WatchList** 特性（用 watch 代替 list 做初始化，走 watch cache，避免 etcd 全量读）值得关注。

Q9：watch cache 是什么？会带来什么问题？

答案要点

- 每个（资源类型 × 版本）在 APIServer 内存中维护一个 **Cacher**：内部是 etcd 数据的全量快照 + 按 revision 索引的 **ring buffer**（事件历史）。
- **List 请求**：默认（不指定 resourceVersion 或 `resourceVersion=0`）从 watch cache 读，**不读 etcd**（性能好，但可能**读到几毫秒前的旧数据**）。
- **强一致读**：显式指定 `resourceVersion`（非 0）会穿透到 etcd 做 quorum read，保证不回退，但代价高。
- **Watch 请求**：直接消费 watch cache，不压 etcd。

关键参数

- `--watch-cache=true`（默认开）
- `--default-watch-cache-size`【默认值 100】—— ring buffer 槽位数
- `--watch-cache-sizes=pods#1000,nodes#500,namespaces#10` —— 按资源单独调

失效模式

- 客户端请求的 `resourceVersion` 太旧（已被 ring buffer 淘汰或被 etcd compact）→ 返回 **410 Gone / "too old resource version"** → 客户端必须 relist。
- ring buffer 太小 + 高频变更资源 → 大量 410 → relist 风暴 → etcd 读放大。
- 大对象（如 Pod 列表大）会吃掉大量 APIServer 内存。

调优建议（需实测）

- 观察指标：`apiserver_watch_cache_*`、`apiserver_cache_list_fetched_objects_total` vs `apiserver_cache_list_returned_objects_total`（两者接近说明缓存命中差/过滤发生在缓存之后）。
- 高频变更资源（pods、events、leases、endpointslice）适当放大 `--watch-cache-sizes`，但要核算 APIServer 内存：`槽位数 × 单对象平均大小 × 副本数`。

Q10：Bookmark 事件 / Progress Notify 解决什么问题？

答案要点

- **问题**：客户端 watch 长时间无事件时，无法区分「确实没变更」还是「连接已假死但 TCP 未断」，也不知道自己落后到什么程度。
- **Bookmark**：APIServer 周期性发送一个**只有 resourceVersion、无对象体**的事件（`type=BOOKMARK`）。客户端更新本地"已知最新 RV"。
- **价值**：客户端重连时可以从 bookmark 记录的 RV 续传，而**不必从头 relist**；同时能检测到连接存活。
- **开启**：客户端 watch 时设置 `allowWatchBookmarks=true`。
- **Progress Notify**：APIServer 侧的周期性推进（可通过 `--watch-progress-notify-interval` 调整，**默认值随版本变化，请以目标版本 `--help` 或 kubeadm 配置为准**）。

面试加分：能说清「无变更资源（如 ConfigMap）的 watch 也必须有心跳，否则无法保活和推进 RV」。

Q11：APIServer 与 etcd 的交互细节？etcd 里存的是什么格式？

答案要点

- 存储路径：`/registry/<resource-plural>/<namespace>/<name>`，例如 `/registry/pods/default/nginx`。
- 存储编码：**protobuf**（默认，二进制、体积小）而非 JSON；历史上是 JSON（老集群可用 `etcdctl get` 直接看，1.6+ 起默认 protobuf）。
- 每次写操作产生新的 **etcd revision**，旧版本保留（MVCC），靠 **compaction** 回收。
- APIServer 的写路径：`storage.Interface.GuaranteedUpdate` → etcd `txn`（compare-and-swap）。

关键运维参数

- `--etcd-compaction-interval`【默认值 5m】：APIServer 定期压缩 etcd 历史 revision（保留最近 5 分钟）。
- etcd `--quota-backend-bytes`【默认 2GB，生产常见 4–8GB】：超限后 etcd 触发 **NOSPACE alarm**，集群**只读**。
- etcd `--max-request-bytes`【默认约 1.5MB】：单个对象超限报 "request is too large"（Endpoint 大对象经典问题）。

Q12：APIServer 如何做限流？从 `--max-requests-inflight` 到 APF 的演进？

答案要点

旧方案（已过时）：`--max-requests-inflight`（默认 400）+ `--max-mutating-requests-inflight`（默认 200）全局计数器 —— **问题**：无法区分请求来源，一个写 CRD 的批处理任务就能把 kubelet 心跳挤掉。

新方案：API Priority and Fairness（APF，KEP-1040）

- 自 **1.23 起默认开启**（Beta），后续版本 GA；启用后旧的两个 flag 失效/移除。
- **两个核心对象**：`FlowSchema`：匹配规则（subjects 用户/组/SA + resourceRules/nonResourceRules）+ `matchingPrecedence` + `distinguisherMethod`（按 user 或 namespace 分流）。`PriorityLevelConfiguration`：`assuredConcurrencyShares`（保证并发份额）+ `queuing`（queues / queueLengthLimit / handSize，实现**洗牌分片 shuffle-sharding**，避免一个 noisy neighbor 占满全部队列）。
- **exempt 优先级**：`system:masters` 等系统高优请求**不排队、不限流**，保证"再怎么炸，管理员还能登录排查"。
- **默认内置 FlowSchema**：`exempt`、`probes`、`system-leader-election`、`workload-leader-election`、`system-nodes`、`kube-controller-manager`、`kube-scheduler`、`kube-system-service-accounts`、`service-accounts`、`global-default`。\（较新版本引入 `node-high` 优先级，专门保障 kubelet 心跳。）

诊断指标

```
apiserver_flowcontrol_rejected_requests_total{priority_level,flow_schema,reason}
   # reason: queue-full / concurrency-limit / time-out / cancelled
apiserver_flowcontrol_current_inqueue_requests{priority_level,flow_schema}
apiserver_flowcontrol_request_queue_length_after_enqueue{priority_level,flow_schema}
apiserver_flowcontrol_dispatched_requests_total
apiserver_flowcontrol_request_exec_seconds / request_wait_seconds
```

调优动作（灰度 + 回滚）

- 给关键控制面组件单独建 FlowSchema、提 `assuredConcurrencyShares`。
- 给批处理/CRD 同步类客户端建**低优先级 + 有限队列**（或直接要求他们降速）。
- 回滚：删除自定义 FlowSchema/PriorityLevelConfiguration，系统会回落到 `global-default`（`kubectl delete` 即回滚，无状态，风险低）。

Q13：APIServer 的 API 版本与 Discovery 机制？kubectl 为什么第一次执行慢？

答案要点

- API 组织为 **Group / Version / Resource (GVR)**；同一资源可有多个版本（`apps/v1`、`apps/v1beta1`），APIServer 内部用 **internal version** 做 trung gian，通过生成的 conversion 函数或 **conversion webhook** 转换。
- **Discovery**：`/api`、`/apis`、`/apis/<group>/<version>`、`/openapi/v2`、`/openapi/v3`。kubectl 首次运行会拉取 discovery 并缓存到 `~/.kube/cache/discovery`（TTL 约 10 分钟）。
- **为什么第一次慢**：需要先拉 discovery + OpenAPI 文档（几 MB），后续走本地缓存。
- **大规模风险**：discovery 文档随 CRD 数量膨胀；几百个 CRD 会让 `/openapi/v2` 达到数十 MB，kubectl 冷启动和 APIServer 内存都受影响。 缓解：控制 CRD 数量、用 `kubectl --cache-dir`、让 CI 复用缓存、避免频繁创建无用的 CRD。
- **版本弃用（Deprecation）**：alpha（可能随时变）/ beta（兼容 3 个 minor 或 9 个月）/ GA（长期兼容）。面试常问：`extensions/v1beta1` Ingress、Policy API 的迁移。

Q14：CRD 和 Aggregated API（APIService）的区别？什么时候该用聚合？

维度	CRD	Aggregated API（APIService）
存储	**共享集群 etcd**	**自带存储**（可独立 etcd / 数据库）
实现	只写 YAML schema，无需代码	需要实现并部署一个扩展 APIServer
能力	受限于内置策略（subresource、字段校验有限）	任意 API 语义、自定义 subresource、任意存储后端
性能影响	**直接吃主 etcd 的容量和 QPS**	与主集群**故障隔离**
典型例子	cert-manager、Prometheus Operator CR	**metrics-server**、kube-aggregator 下的 service-catalog

选择原则（大规模集群重点）

- 对象数量大 / 写 QPS 高 / 需要与主集群故障隔离 → **Aggregated API + 独立 etcd**（metrics-server 是典型：它的数据量极大，绝不进主 etcd）。
- 对象少、生命周期简单 → CRD。
- **反模式**：把百万级 CR（如自定义指标、审计记录）塞进 CRD → 主 etcd 被拖垮。

聚合层转发链路

```
kubectl get --raw /apis/metrics.k8s.io/v1beta1/nodes
  → kube-aggregator 匹配 APIService → 转发到 service/namespace
  → 扩展 APIServer 返回 → 聚合层原样返回
```

- APIService 不可用时，请求返回 **503 Service Unavailable**（`kubectl top` 的经典报错）。

Q15：Server-Side Apply（SSA）与 `kubectl apply` 的字段管理？

答案要点

- 传统 `kubectl apply` 依赖 `metadata.annotations["kubectl.kubernetes.io/last-applied-configuration"]`（三路合并），多控制器同时改同一对象会**互相覆盖**。
- **SSA**：客户端发送 `PATCH` + `Content-Type: application/apply-patch+yaml`，携带 `?fieldManager=xxx&force=...`；APIServer 在 `metadata.managedFields` 中记录**每个字段由哪个 manager 拥有**。
- **冲突处理**：若目标字段已被其他 manager 拥有 → 返回 **409 Conflict（Apply failed）**；`force=true` 则强制抢占并转移所有权。
- **大规模注意**：`managedFields` 会**增大每个对象的体积**（etcd 存储 + 网络传输 + watch 事件），大规模集群可考虑关掉（`--feature-gates=ServerSideApply=false` 会失去能力，更常见做法是保留但精简字段管理器数量）。

Q16：Secret 的静态加密（Encryption at Rest）怎么做？

答案要点

- 配置 `EncryptionConfiguration`，通过 `--encryption-provider-config` 挂载。

```yaml
apiVersion: apiserver.config.k8s.io/v1
kind: EncryptionConfiguration
resources:
  - resources: ["secrets", "configmaps"] # 可扩展到 CRD 的 secret 类资源
    providers:
      - aescbc: # 或 aesgcm / kms v2
          keys:
            - name: key1
              secret: <base64 32字节随机密钥>
      - identity: {} # 兜底：读取未加密的旧数据
```

- **provider 顺序**：写入用**列表第一个**；读取时**逐个尝试**直到解出 → 支持密钥轮转（新 key 放最前，旧 key 保留可读）。
- **KMS v2（1.29 GA）**：密钥托管在云 KMS/HashiCorp Vault，APIServer 通过 unix socket 与 KMS plugin 通信。
- **运维要点** 只能加密**新的写入**；存量需执行 `kubectl get secrets -A -o json | kubectl replace -f -` 强制重写。 修改配置后必须**重启所有 APIServer**（滚动），且期间不要写入正在加密的资源类型。 **回滚预案**：保留 `identity` 在列表中，出问题可回退到明文读取；但已加密数据不可"降级"（除非重写）。

Part 2：请求链路与对象语义（Q17–Q28）

Q17：一个 Pod 从创建到被调度、启动，APIServer 参与哪些环节？

```
kubectl create pod
  ① APIServer: 认证/鉴权/准入 → etcd (Pod, spec.nodeName 为空)
  ② Scheduler: watch 到未绑定 Pod → 预选/打分 → 通过 APIServer 写回 binding (Pod.spec.nodeName)
  ③ Kubelet: watch 到调度到本节点的 Pod → 拉镜像、创建容器
  ④ Kubelet: 通过 APIServer 更新 Pod.status (容器状态、IP)
  ⑤ Kubelet: 更新 Node.status + Lease (心跳)
  ⑥ 各 controller: watch 并 reconcile (ReplicaSet/Endpoint/Service…)
```

关键点

- **APIServer 从不主动调用任何人**，一切靠 watch 驱动（声明式 API 的核心：控制循环 = 观察当前状态 → 计算期望差 → 写回）。
- 每个环节的写入都是一次独立的 etcd 写，写放大由此产生。

Q18：为什么 APIServer 的 API 是"声明式"的？声明式和命令式的区别？

- **命令式**：`kubectl run`、`kubectl scale` —— 客户端告诉系统"做什么"，失败需要客户端重试，无法处理并发。
- **声明式**：`kubectl apply -f` —— 客户端声明"期望终态"，由 controller 的**控制循环**持续逼近，天然支持重试、幂等、自我修复。
- **代价**：终态收敛需要时间，观测上表现为"最终一致"，需要容忍短暂不一致。 控制器 bug 会导致"抖动"（flapping）—— 两个控制器对同一字段期望不同，反复互相覆盖。

Q19：`Finalizer`、`OwnerReference`、垃圾回收（GC）的原理？

- **Finalizer**：`metadata.finalizers` 非空时，DELETE 只会设置 `deletionTimestamp`（进入 Terminating），**对象不真正删除**，直到 finalizer 被移除。用于外部资源清理（PV、LoadBalancer、云盘）。 **故障模式**：finalizer 的 controller 挂了 → 对象卡在 Terminating → 需人工 `kubectl patch` 移除 finalizer（有风险，需确认外部资源已释放）。
- **OwnerReference + GC**：`garbage collector` controller 通过 watch 维护**依赖图**，删除 owner 时级联删除 dependent（`--cascade=orphan/background/foreground`）。 **大规模风险**：CRD 数量多、图庞大 → GC controller 内存和 CPU 吃紧；级联删除时产生**瞬时写风暴**。

Q20：APIServer 的审计（Audit）机制？生产怎么配才不拖垮性能？

- **阶段**：`RequestReceived` → `ResponseStarted`（长连接 watch）→ `ResponseComplete` → `Panic`。
- **级别**：`None` / `Metadata`（只记元数据）/ `Request`（含请求体）/ `RequestResponse`（**含响应体，最贵**）。
- **后端**：`log`（文件）、`webhook`（外部收集）、`dynamic`（AuditSink，已废弃）。

生产建议（性能优先）

```
apiVersion: audit.k8s.io/v1
kind: Policy
rules:
  # 1) 不记录只读低价值请求
  - level: None
    users: ["system:kube-proxy"]
    verbs: ["watch"]
    resources: [{group: "", resources: ["endpoints","services","endpointslices"]}]
  # 2) 不记录高频心跳
  - level: None
    users: ["system:serviceaccount:kube-system:generic-garbage-collector"]
    verbs: ["list","watch"]
  # 3) 敏感资源请求记元数据（不记 body，避免 Secret 明文落盘）
  - level: Metadata
    resources: [{group: "", resources: ["secrets","configmaps"]}, {group: "rbac.authorization.k8s.io"}]
  # 4) 写操作全记
  - level: Request
    verbs: ["create","update","patch","delete"]
  # 5) 兜底
  - level: Metadata
```

- **性能开关**：`--audit-log-maxsize/--audit-log-maxbackup/--audit-log-maxage`（日志轮转）、`--audit-webhook-batch-buffer-size` + `--audit-webhook-batch-max-size`（异步批量，避免 webhook 阻塞请求）。
- **反模式**：全量 `RequestResponse` 且写本地盘 → APIServer IO 打满、P99 飙升、磁盘被打爆。

Q21：什么是 API 的幂等性？APIServer 怎么保证？

- **Create**：靠 `metadata.uid` + name 唯一性保证（同名重复创建返回 409 AlreadyExists）。
- **Update/Patch**：靠 `resourceVersion` 的乐观锁（409 冲突）。
- **Apply（SSA）**：靠 `managedFields` 幂等（相同输入重复 apply 不产生变更）。
- **Delete**：幂等（删除不存在的对象返回 404，客户端视为已完成）。
- **面试重点**：控制器代码必须**幂等**，因为 reconcile 会被重复触发（informer resync、事件重放、重启）。

Q22：`--dry-run=server` 的作用和限制？

- 服务端执行完整的 **admission + validation + 默认值填充**，但**不持久化**（`dryRun=true` 传给 storage 层，etcd 写入被跳过）。
- **限制**： 依赖 admission webhook 支持 dry-run（必须声明 `sideEffects: None` 或 `NoneOnDryRun`）。 有副作用的 webhook（如自动创建云资源）必须声明 `sideEffects: Some`，否则 dry-run 时会被拒绝。 **不校验资源配额是否真的够**（配额是并发相关的）。
- **用途**：CI 里做 YAML 合规校验、策略预检（Kyverno/Gatekeeper 的 dry-run 模式）。

Q23：APIServer 优雅关闭（Graceful Shutdown）与滚动升级会发生什么？

- `--shutdown-delay-duration`：收到 SIGTERM 后延迟关闭，让 LB 摘流、客户端感知。
- **关闭顺序**：先停止接收新请求 → 关闭 watch 连接 → 等待 inflight 请求完成（受 `--shutdown-send-timeout` 限制）。
- **客户端影响（大规模考点）**： APIServer 实例下线 → 所有连到它的 watch 断连 → 客户端**同时**重连 + relist → **惊群（thundering herd）**。 缓解：APIServer 多副本、错峰滚动、客户端带 jitter 的退避重连、适当调大 watch cache 使 relist 命中缓存。
- 相关特性：`--goaway-chance`（APIServer 随机发送 HTTP/2 GOAWAY，主动打散长连接，避免 LB 粘滞导致负载不均）—— **大规模集群建议开启并【需实测】调整**。

Q24：kubelet 与 APIServer 的交互有哪些？为什么 Node 心跳要用 Lease？

- kubelet 会 **watch**：分配给本节点的 Pod、相关 Secret/ConfigMap、Service/EndpointSlice。
- kubelet 会 **写**：Pod status、Node status、Node Lease、CSI volume attach 状态、Event。
- **Node 状态更新频率**：`--node-status-update-frequency`【默认值 10s】（有变化时上报），`--node-status-report-frequency`【默认值 5m】（无变化时也强制上报）。
- **为什么引入 Lease**（1.14 GA）： 每次更新完整 Node 对象 = 一次大对象写（Node.status 含 images、conditions、allocatable...），5000 节点 × 每 10s = **约 500 次写/s 的大对象写入**，etcd 直接被压垮。 Lease 是极小的对象（约 250B），心跳只更新 Lease；Node 对象的 status 只在**真正变化时**才写。 Node controller 通过 Lease 判定 NotReady（`--node-monitor-grace-period`【默认 50s】）。
- **大规模收益**：把节点心跳的 etcd 写压力降低一到两个数量级（**具体倍数需在你的集群上实测统计** `apiserver_request_total{verb=PUT,resource=leases}` **与** `resource=nodes` **的比例**）。

Q25：Event 为什么是大规模集群的头号杀手？怎么处理？

- Event 由各组件高频产生，默认 `--event-ttl`【默认值 1h】。
- **问题**：Event 是 CR（events.k8s.io/v1），每个都是 etcd 写；异常时（如镜像拉取失败反复重试、CrashLoopBackOff）会**指数级放大**。
- **处理方案（按优先级） 独立 etcd**： `--etcd-servers-overrides=/events#https://etcd-events-1:2379;...`，把 Event 从主 etcd 剥离（最有效）。 缩短 `--event-ttl`（如 30m，【建议值】需结合排障需求）。 审计策略里对 `system:serviceaccount:kube-system:*` 的 event 写入设为 `None`。 用 `--event-qps` / `--event-burst`（kubelet 侧）限流。 治理根因：修掉反复重启的 Pod，而不是只清 Event。

Q26：Endpoint vs EndpointSlice？为什么要切分？

- 旧 Endpoint 对象：一个 Service 一个 Endpoint 对象，包含**所有**后端 IP。

- **问题**：etcd 单对象上限约 1.5MB（`--max-request-bytes`），大型 Service（上千 Pod）会导致： 写入失败（"request is too large"） 单次变更触发**全对象重写**（一个 Pod 变更 → 整个大对象重写 → 写放大） 所有 kube-proxy watch 到巨大事件
- **EndpointSlice**（1.21 GA）：默认每个 Slice **最多 100 个 endpoint**（`--max-endpoints-per-slice`，【默认值 100】，可调 1–1000），变更粒度小、增量传播。
- **大集群收益**：kube-proxy 的 watch 事件体积和 etcd 写放大显著下降（**需实测对比**）。

Q27：client-go 的限速（QPS/Burst）为什么会影响 APIServer?

- client-go 内置 **令牌桶 + 指数退避**：默认 QPS=5、Burst=10（**各组件版本间有差异，以实际启动参数为准**）。
- **大规模问题**： 默认值太低 → 控制器启动慢、reconcile 积压（5000 节点下 kube-controller-manager 常年打满 QPS）。 客户端被限速 → 大量 429 → client-go 退避重试 → 进一步堆积。 客户端被 APF 拒绝（429 + `Retry-After`）→ 若客户端不遵守 `Retry-After`，重试风暴会雪上加霜。
- **配置示例（kubelet）**

```
apiVersion: kubelet.config.k8s.io/v1beta1
kind: KubeletConfiguration
kubeAPIQPS: 50 # 【建议值】需结合节点数与 Pod 密度实测
kubeAPIBurst: 100
```

- **controller-manager / scheduler**：`--kube-api-qps` / `--kube-api-burst`。

Q28：APIServer 的证书体系？证书过期会发生什么?

- **证书分类**： `ca.crt`（集群 CA） `apiserver.crt`（服务端证书，含 `--apiserver-cert-extra-sans` 中的所有 LB VIP/域名） `apiserver-kubelet-client`（APIServer 访问 kubelet） `front-proxy-ca/front-proxy-client`（聚合层 → 扩展 APIServer） `apiserver-etcd-client` / `etcd-server` / `etcd-peer` ServiceAccount 签名密钥对（`--service-account-key-file` / `--service-account-signing-key-file`）
- **过期后果**：整集群不可用（`x509: certificate has expired`），kubectl 全部失败。
- **运维** kubeadm： `kubeadm certs check-expiration`、`kubeadm certs renew all`（**注意：renew 后需重启 APIServer/CM/Scheduler 的静态 Pod**）。 监控：用 `apiserver_client_certificate_expiration_seconds` 设告警（**剩余 \< 30 天告警**）。 **反模式**：只在证书过期当天才发现；SA 签名密钥轮转未留旧 key 导致存量 token 全部失效。

Part 3：大规模集群高频问题（Q29–Q46）

> 先记住官方"大规模集群"参考上限（**社区文档口径，随版本演进**）： **节点 ≤ 5000，Pod ≤ 150000，容器 ≤ 300000，单节点 Pod ≤ 110。** 超过这些量级需要专门调优，且必须自行压测验证。

Q29：大规模集群下，APIServer 最先出现的瓶颈是什么?

按经验排序（**需在你的集群用监控验证顺序**）：

1. **etcd 写入放大**（Lease、Event、EndpointSlice、Pod status、CR 状态更新）
2. **etcd 读放大**（全量 List、无分页、watch cache miss、informer relist 风暴）
3. **APIServer 内存 / CPU**（watch cache 太大、大 List 请求、审计日志、protobuf 编解码）
4. **限流与排队**（APF 队列满、客户端限速）
5. **webhook 时延**（同步阻塞，直接计入请求 P99）
6. **网络与 LB**（长连接粘滞、单 LB 带宽、跨 AZ 时延）

Q30：如何给 APIServer 做容量规划与压测?

必测项

- 稳态：`apiserver_request_total` 按 verb/resource 拆分的总 QPS。
- 时延：`histogram_quantile(0.99, sum(rate(apiserver_request_duration_seconds_bucket[5m])) by (le, verb, resource))`。
- 并发：`apiserver_current_inflight_requests`、`apiserver_flowcontrol_current_inqueue_requests`。
- 对象数：`etcd_debugging_mvcc_keys_total`、`apiserver_storage_objects`（按资源）。
- etcd 健康：`etcd_disk_wal_fsync_duration_seconds`（**P99 应 \< 10ms，这是硬指标**）、`etcd_server_slow_apply_total`、`etcd_mvcc_db_total_size_in_bytes`。

压测方法

- 用 `kubemark`（hollow-node）模拟大规模节点，比真买机器便宜。
- 用真实业务流量的**影子回放**（录制 `apiserver_request_*` 分布，用脚本重放）。
- **压力测试必须在非生产集群做**；生产只做**灰度观察**。

Q31：etcd 出现 `mvcc: database space exceeded`（NOSPACE）怎么救?

原因：db 达到 `--quota-backend-bytes`，etcd 触发 alarm 并进入**只读**。

修复步骤（生产操作需审批 + 变更窗口）

```
# 1) 查看状态（先确认 db size 与 revision）
ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  endpoint status --write-out=table

# 2) 先看 alarm
etcdctl alarm list

# 3) 压缩到当前 revision（会短暂影响读）
rev=$(etcdctl endpoint status --write-out="json" | jq -r '.[0].Status.header.revision')
etcdctl compact "$rev" --physical

# 4) 每个成员逐个 defrag（defrag 会阻塞该成员，务必逐个做、且先做 follower）
etcdctl defrag --endpoints=... --command-timeout=60s

# 5) 解除 alarm（不解除则仍只读）
etcdctl alarm disarm
```

根因治理

- 确认 APIServer 的 `--etcd-compaction-interval` 正常生效（默认 5m）。
- 排查写放大源头（Event/Lease/CR 高频写）。
- 提高 `--quota-backend-bytes`（**提高配额只是延后爆炸时间，不是解法**）。
- 定期 defrag（**低峰期、逐个成员、有 leader 切换预案**）。

回滚/兜底：操作前必须有 etcd **快照备份**（`etcdctl snapshot save`），灾难恢复演练至少每季度一次。

Q32：如何减少 etcd 的写放大？（综合题，必考）

手段	做法	收益等级
Node 心跳用 Lease	默认开启，确认 kubelet 未禁用	高
Event 独立 etcd	`--etcd-servers-overrides=/events#...`	高
Event TTL 缩短	`--event-ttl=30m`	中
EndpointSlice	默认开启，调 `--max-endpoints-per-slice`	中
控制 CR 状态更新频率	控制器里只在**状态真正变化**时 Update；用 `apiequality.Semantic.DeepEqual` 比较	高
避免 `status` 与 `spec` 打同一对象高频写	用独立 CR 存状态	中
关闭不必要的 controller / addon	每个 controller 都有 watch+写	中
大 CRD 迁移到聚合 API	独立 etcd	高

代码层反模式（面试加分）

```
// 反模式：无条件 Update，即使 status 没变也会产生一次 etcd 写 + 一次事件广播
s.Status.ObservedGeneration = s.Generation
r.Status().Update(ctx, s) // ← 每次 reconcile 都写

// 正确：先比较
if !apiequality.Semantic.DeepEqual(old.Status, new.Status) {
    r.Status().Update(ctx, s)
}
```

Q33：如何减少 etcd 的读放大 / List 压力?

1. **禁止全量 List**：所有客户端必须分页（`limit` + `continue`）或用 informer。
2. `resourceVersion=0`：List 请求显式带 `resourceVersion=0` → 走 watch cache，不压 etcd（默认行为已是缓存读，但显式声明更清晰）。
3. **字段/标签选择器下推**：`--field-selector`、`label selector` 在 APIServer 侧过滤（注意：watch cache 只对 **namespace + name + 部分 field** 建索引，其他选择器需要在缓存后遍历 → 观察 `apiserver_cache_list_fetched_objects_total` 判断浪费）。
4. **Metadata-only 请求**：只要元数据时用 `Accept: application/json;as=PartialObjectMetadataList`（client-go 的 `metadataOnly` option），体积和 CPU 大幅下降。
5. **共享 informer**：同一进程内多个控制器复用同一个 SharedInformer，避免重复 watch。
6. **减少客户端数量**：DaemonSet 类 agent 尽量每个节点只跑一份、用同一个 informer。

Q34：为什么 APIServer 会 OOM? 怎么排查与预防?

常见原因

- **大 List 请求**：一次性 List 全部 Pod/Event（不带 limit）→ 服务端序列化巨型对象，内存峰值 = 对象数 × 单对象大小 × 并发度。
- **watch cache 过大**：`--watch-cache-sizes` 调得过大，尤其是 pods/events。
- **审计日志 `RequestResponse` 全量**：请求体在内存堆积 + 磁盘爆。
- **大量并发 watch**：每个 watch 连接在服务端持有 buffer。
- **巨型 CRD 对象**（单个 CR 几 MB）。

排查

```
# 1) 找出大请求
topk(10, sum by (resource,verb,user) (rate(apiserver_request_duration_seconds_sum[5m])))
# 2) 找出返回对象数最多的 list
topk(10, apiserver_storage_list_fetched_objects_total)
# 3) 进程内存与 GC
container_memory_working_set_bytes{container="kube-apiserver"}
go_goroutines{job="apiserver"} # goroutine 泄漏
```

- `kubectl get --raw /debug/pprof/heap` 抓 heap profile（需开启 profiling，注意安全）。

预防（可直接套用）

- 强制分页：自研平台在 API 网关层强制注入 `limit`。
- 合理设置 `requests/limits`（**内存 limit 需实测后设置，且 APIServer 内存与对象数强相关，不要照抄别人家的值**）。
- 审计策略降级为 `Metadata`。
- 用 `--max-request-bytes` 限制单个请求体大小。

Q35：APF 队列被打满（大量 429 / `queue-full`）怎么定位?

定位三步

```
# 1) 谁在被拒绝
sum by (flow_schema, priority_level, reason) (
  rate(apiserver_flowcontrol_rejected_requests_total[5m])
)
# 2) 队列积压情况
sum by (priority_level, flow_schema) (apiserver_flowcontrol_current_inqueue_requests)
# 3) 等待时长（判断是否只是排队慢而非拒绝）
histogram_quantile(0.99,
  sum by (le, flow_schema) (rate(apiserver_flowcontrol_request_wait_seconds_bucket[5m]))
)
```

根因与对策

现象	根因	对策
`global-default` 大量 `queue-full`	无分类的客户端太多	给主要客户端建专属 FlowSchema
某个业务 SA 占满 `workload-low`	noisy neighbor	降低其 `assuredConcurrencyShares` 或用 `distinguisherMethod: ByUser` 隔离
`concurrency-limit`	总并发超配	提升对应 PL 的 shares（注意总盘子有限，此消彼长）
`time-out`	请求堆积太久	缩短 `queues` / `queueLengthLimit` 让失败更快，客户端退避重试

变更纪律：APF 对象变更是**热生效、无状态**，`kubectl delete` 即回滚；但仍建议先在灰度集群验证。

Q36：5000 节点集群，kubelet 的心跳与 Pod status 会给 APIServer 多大压力?

量级估算方法（不给死数字，给算式，面试时这样答最稳）

```
Lease 写 QPS ≈ 节点数 / node-status-update-frequency(10s) = 5000/10 = 500 QPS
Pod status 写 QPS  ≈ 稳态很小；但滚动发布/驱逐时会瞬时放大（需实测）
Event 写 QPS ≈ 异常期可达数千（需实测），稳态接近 0
EndpointSlice ≈ 与 Service 变更频率相关
```

把每一项都变成 PromQL 实测：

```
sum(rate(apiserver_request_total{verb=~"PUT|PATCH",resource="leases"}[5m]))
sum(rate(apiserver_request_total{verb=~"PUT|PATCH",resource="pods"}[5m]))
sum(rate(apiserver_request_total{verb="create",resource="events"}[5m]))
```

降压手段

- 心跳间隔（**改这个会影响故障检测时延，必须评估 SLO**）：`--node-status-update-frequency` 从 10s 调到 20s 可降一半写，但 Node NotReady 检测时延从 50s 起算会增加 → **必须与业务 SLO 一起决策。**
- 独立 etcd 放 Lease（`--etcd-servers-overrides=/registry/leases#...`）。
- 只在必要时上报 Node status（默认机制已做）。

Q37：APIServer 多副本 + LB 会遇到什么坑?

1. **HTTP/2 连接粘滞**：客户端复用长连接 → 大量连接落在少数 APIServer 实例上 → 负载不均。 对策：`--goaway-chance`（随机断开让客户端重连，重新打散），LB 层用 least-connection / 连接数上限。
2. **watch 长连接**：watch 是长连接，LB 的 idle timeout 会切断 → 客户端需正确处理重连（`410`/EOF → relist）。
3. **LB 健康检查**：用 `/readyz`（不要用 `/healthz`，后者含更多检查可能误摘流）；用 `/livez` 做存活。
4. **跨 AZ 流量**：etcd 有 leader，跨 AZ 的写会增加时延 → APIServer 与 etcd **同 AZ 部署优先。**
5. **滚动升级惊群**：见 Q23。

Q38：`/healthz`、`/readyz`、`/livez` 有什么区别?

- `/healthz`：传统健康检查（1.16 起被拆分），现在默认等同于 ping + log + 若干检查。
- `/readyz`：**是否可接流量**（含 etcd 连通性、poststarthook 完成状态）。LB 必须用这个。 `kubectl get --raw='/readyz?verbose'` 可以看每一项。

- `/livez`：**是否存活**（不含会导致重启的依赖检查，避免级联重启）。

排障技巧：`/readyz?verbose` 能直接告诉你哪个 post-start hook（如 `aggregator-openapi`、`crd-informer-synced`）没就绪 —— 聚合 APIService 挂了导致 APIServer 不 Ready 是经典故障。

Q39：大集群下 CRD 的管理有哪些坑?

1. **CRD 数量膨胀 discovery/OpenAPI**（见 Q13）。
2. **CR 数量巨大** → 主 etcd 容量与写压力 → 迁移到聚合 API（Q14）。
3. **CRD 的 schema 校验开销**：复杂 `openAPIV3Schema` 会显著增加 APIServer CPU（每次请求都要校验）。 对策：避免超深嵌套 schema；必要时用 x-kubernetes-validations（CEL）但注意性能。
4. **CRD 版本转换**：conversion webhook 是同步调用，与 admission webhook 一样有 P99 风险。
5. **finalizer 卡死** → 大量 Terminating CR 堆积。
6. **CRD 删除会连带删除所有 CR**（危险操作，需审批）。

Q40：APIServer 的性能调优参数清单（可以当 checklist 背）

参数	作用	备注
`--enable-priority-and-fairness=true`	APF 限流	1.23+ 默认
`--watch-cache-sizes=`	按资源调 watch cache	【需实测】
`--default-watch-cache-size`	默认槽位（默认 100）	【默认值】
`--etcd-compaction-interval=5m`	历史 revision 压缩	【默认值】
`--event-ttl=1h`	Event 保留时长	【默认值】
`--etcd-servers-overrides=`	按资源前缀拆分 etcd	高收益
`--max-requests-inflight`	旧限流（APF 启用后无效）	注意版本
`--goaway-chance`	打散 HTTP/2 长连接	【需实测】
`--shutdown-delay-duration`	优雅关闭延迟	配合 LB 摘流
`--audit-*`	审计策略与异步批量	见 Q20
`--encryption-provider-config`	静态加密	见 Q16
`--anonymous-auth=false`	关闭匿名	安全基线
`--tls-cipher-suites`	限定 TLS 套件	安全基线
`--request-timeout`	非长连接请求超时	【默认值】1m（1.30+ 有变更）

Q41：为什么"每节点 Pod 数 ≤ 110"这个限制存在?

- **本质不是 110 个 Pod 放不下**，而是： 每节点 Pod 太多 → 单个 Node 对象上的 Pod 列表变大 → kubelet 的 watch 事件变多 IPv4 地址耗尽（每个 Pod 一个 IP） 一个节点故障影响面过大（爆炸半径） 单节点上的容器密度导致内核资源（cgroup、inode、进程数）压力
- 突破手段：更大的 Pod CIDR、IPv6/dual-stack、节点池拆分、提高密度前先压测 kubelet + containerd。

Q42：多集群 / 联邦 vs 单集群扩容，怎么选?

方案	优点	代价	适用
单集群纵向调优	运维简单、资源池统一、调度全局最优	有物理上限、爆炸半径大	绝大多数场景（\<3000 节点优选）
拆分多个集群	故障隔离、可按业务/租户定制	多套控制面、跨集群服务发现/调度复杂	多租户强隔离、跨地域
联邦（KubeFed / 自研）	统一分发	成熟度低、语义复杂、最终一致难保证	谨慎评估

面试观点：**优先把单集群调优到极致，再考虑拆分。**联邦的复杂度远高于收益，除非有硬性隔离/地域要求。

Q43：如何为 APIServer 设定 SLO?

示例 SLO（【建议值】，需与业务共同确定）

SLO	目标	测量方式
可用性	99.95%（月度）	`1 - (5xx + 超时) / 总请求`，数据源 `apiserver_request_total{code=~"5.."}`
读请求时延	P99 \< 1s（LIST/GET 内置资源）	`apiserver_request_duration_seconds`
写请求时延	P99 \< 1.5s	同上
限流拒绝率	\< 0.1%	`apiserver_flowcontrol_rejected_requests_total`
etcd WAL fsync	P99 \< 10ms	`etcd_disk_wal_fsync_duration_seconds`

错误预算：99.95% → 月度错误预算约 **21.9 分钟**。预算耗尽即冻结变更（这是 SRE 的核心纪律）。 **注意**：APIServer 的 SLO ≠ 业务 SLO。要单独定义"控制面可用"与"业务可用"，前者不可用会导致**无法自愈**（不能调度、不能扩容），影响是长尾的。

Q44：APIServer 挂了，正在运行的业务会怎样？

- **已运行的 Pod 不会立即挂**：kubelet 本地有期望状态，容器继续跑；网络（CNI 数据面）不依赖 APIServer。
- **会立刻失效的**： 无法调度新 Pod、无法扩缩容（HPA 失效） Pod 重建失败（重启后 kubelet 需要 watch 新 Pod 才能创建 —— 实际上 kubelet 本地有 checkpoint，能短暂维持，但无 APIServer 时无法感知变更） Service DNS / EndpointSlice 不再更新（新扩的副本不会进负载均衡） `kubectl` 全部失败 节点 NotReady 后无法驱逐（**雪崩风险**）
- **结论**：APIServer 的 MTTR 直接决定恢复能力 → 必须做**慢启动演练**（定期演练"APIServer 全挂 → 恢复"）。

Q45：如何做 APIServer 的可观测性建设？（三位一体）

Metrics（关键指标）

```
apiserver_request_total{code,verb,resource,subresource}
apiserver_request_duration_seconds_bucket{verb,resource,scope}
apiserver_current_inflight_requests
apiserver_flowcontrol_*  （限流全家族）
apiserver_cache_list_fetched_objects_total / _returned_objects_total
apiserver_watch_events_sizes_bucket
etcd_request_duration_seconds{operation,type}
etcd_mvcc_db_total_size_in_bytes / etcd_debugging_mvcc_keys_total
etcd_disk_wal_fsync_duration_seconds_bucket
etcd_server_slow_apply_total / etcd_server_leader_changes_seen_total
apiserver_client_certificate_expiration_seconds
aggregator_unavailable_apiservice_total
```

Logs

- APIServer 日志级别 `--v=2`（生产），排障临时 `--v=4`（**注意：高 verbosity 性能开销明显，只能短时间开**）。
- 审计日志 → 集中采集（审计是"谁改了什么"的唯一真相来源）。

Tracing

- 关键：**APF 排队的等待时间**要单独看（`apiserver_flowcontrol_request_wait_seconds`），因为它常常被误认为是"APIServer 慢"，实际是"被限流了"。

告警降噪原则

- 只对 **SLO 相关信号**告警（`5xx burn rate`、P99 时延、etcd fsync、证书过期、leader 切换频繁）。
- 不要对"单个请求 409"告警（这是正常的并发冲突）。

Q46：举一个你做过的大规模集群 APIServer 优化案例（开放式题的答题框架）

用 **STAR + 数据** 结构回答：

1. **Situation**：规模（节点/Pod/CR 数量）、业务特征、部署方式。
2. **Problem/痛点**：用**指标**描述（例如"APIServer P99 从 200ms 劣化到 3s，etcd WAL fsync P99 达 40ms，DB 接近 8GB 配额"）。
3. **Action**：分层治理（写放大 / 读放大 / 限流 / webhook / 客户端），每一步给出配置片段与灰度比例。
4. **Result**：**量化前后对比**（P99、QPS、DB size、告警数），并说明回滚条件与验证方式。
5. **Learn**：沉淀为规范/自动化检查（如 CI 里禁止无 limit 的 List、CRD 上线评审 checklist）。

> 面试官真正想听的是：**你有没有用数据说话，有没有灰度与回滚意识。**

Part 4：故障排查实战（Q47–Q58）

Q47：`409 Conflict` 是什么？怎么处理？

- **含义**：乐观并发冲突（resourceVersion 不匹配），或 SSA 的字段所有权冲突。
- **处理**： Update 类：重新 GET → 应用变更 → 重试（client-go：`retry.RetryOnConflict`）。 Apply 类：查看 `managedFields` 找出冲突的 manager，决定 `force=true` 还是让对方先让出字段。
- **不是故障**：分布式系统里 409 是正常现象，只要比例不高（**建议监控 409 占比，突增说明有控制器在打架**）。

Q48：`429 Too many requests` 有哪几种来源？

来源	特征	处理
APF 拒绝	响应体含 "Too many requests"，apiserver 侧 `apiserver_flowcontrol_rejected_requests_total` 上升	调 FlowSchema/PL，或让客户端降速
client-go 令牌桶	客户端本地限速，**不会**在 APIServer 侧留下 429	调 `--kube-api-qps/burst`
EventRateLimit 准入插件	针对 Event 资源限流	调 EventRateLimit 配置或治理 Event 源
云厂商 API 网关/限流	托管集群特有	提配额

关键：先看响应是否有 `Retry-After`，客户端**必须遵守**它，否则重试风暴。

Q49：`etcdserver: request timed out` / `context deadline exceeded` 怎么排查？

定位顺序

1. **磁盘**：`etcd_disk_wal_fsync_duration_seconds` P99（**>10ms 就是磁盘不达标**）。 物理机：确认 SSD/NVMe，不是网络盘（云盘需注意 IOPS 与突发配额）。 压测：`fio --rw=write --ioengine=sync --fdatasync=1 --size=22m --bs=2300 --name=mytest`（社区推荐姿势）。
2. **网络**：`etcd_network_peer_round_trip_time_seconds`（**同 AZ 应 \< 数 ms；跨 AZ 部署是常见坑**）。

3. **CPU/内存**：etcd 进程是否被打满（与大 APIServer 抢资源）。
4. **db 大小**：接近 quota → 写放大 → 慢。
5. **leader 抖动**：`etcd_server_leader_changes_seen_total` 频繁上升 → 检查网络/磁盘/心跳参数（`--heartbeat-interval`【默认 100ms】、`--election-timeout`【默认 1000ms】，跨地域部署需要调大）。

Q50：`the server could not find the requested resource` 怎么排查？

按顺序检查：

1. **API 版本/路径写错**（GVR 拼错）。`kubectl api-resources` 确认。
2. **CRD 未创建或未 Established**：`kubectl get crd <name> -o yaml` 看 `status.conditions[Type=Established]`。
3. **聚合 APIService 不可用**：`kubectl get apiservices` 看 `Available=False` → 常见原因：扩展 APIServer Pod 挂了、Service/Endpoint 不通、证书校验失败（需 `--enable-aggregator-routing` 或 CA 配置正确）。
4. **RBAC 拒绝时**是 `403 Forbidden`（不是 404）—— 注意区分：现代 APIServer 对无权限且资源不存在的情况可能返回 404 以避免资源枚举。
5. **多集群/上下文错误**：`kubectl config current-context`。

Q51：`Internal error occurred: failed calling webhook "xxx"` 怎么排查？

排查清单

1. webhook Pod 是否 Running / 副本数是否为 1（**单点** → 就绪即断流）。
2. **证书**：webhook 的 `caBundle` 是否匹配服务端证书、是否过期（TLS 错误会明确提示 x509）。
3. **网络**：APIServer → webhook Service 的连通性（CNI 策略、NetworkPolicy）。
4. **超时**：`timeoutSeconds` 是否太小（默认 10s，建议 3–5s）。
5. **selector**：是否命中了不该命中的请求（导致放大）。
6. **failurePolicy**：`Fail` 时 webhook 不可用 = 业务不可用。关键路径 webhook 应有多副本 + PDB + 独立节点池。

应急（需审批）：把 `failurePolicy` 临时改为 `Ignore`，或删除 webhook 配置。 **风险控制**：改 Ignore 意味着策略失效（安全/合规风险），必须同步通知安全团队并在事后补齐。

Q52：`kubectl exec/logs` 失败与 APIServer 的关系？

- `exec/logs/attach/port-forward` 走的是 **SPDY/WebSocket 升级**（`/api/v1/ns/x/pods/y/exec`），再经 APIServer **反向代理到 kubelet 的 10250 端口。**
- **依赖**：`apiserver-kubelet-client` 证书被 kubelet 信任（`--client-ca-file`） kubelet 的 `--anonymous-auth=false` + webhook 授权（APIServer 需要 `nodes/proxy`、`nodes/log` 权限 —— RBAC `system:kubelet-api-admin` 或内置 `system:kube-apiserver-to-kubelet` ClusterRole） 节点地址类型（`--kubelet-preferred-address-types=InternalIP,Hostname,...`）
- **常见报错**：`error: unable to upgrade connection: Forbidden`、`x509: certificate signed by unknown authority`、`pod does not exist`。
- **大规模注意**：exec/logs 是**长连接**，计入 inflight 但走 long-running 通道，不被 APF 排队 —— 大量 exec（如 CI）会占用连接资源。

Q53：APIServer 日志里 `too old resource version` 是什么意思？

- 客户端请求的 `resourceVersion` 已被 watch cache 淘汰（超出 ring buffer）或已被 etcd compact。
- 返回 **410 Gone**，客户端必须 **relist。**
- **大规模危害**：如果大批量客户端同时收到 410 → 同时 relist → **惊群。**
- **缓解**：调大对应资源的 `--watch-cache-sizes` 客户端开启 bookmark（`allowWatchBookmarks=true`）保持 RV 推进 客户端 relist 加 jitter 排查为什么客户端会持有过旧的 RV（例如 informer 长时间卡住、GC 停顿）

Q54：如何判断是"APIServer 慢"还是"etcd 慢"还是"被限流了"？

决策树

```
请求 P99 高
 ├─ apiserver_flowcontrol_request_wait_seconds 高
 │  → 被 APF 限流/排队（不是慢，是被限）
 ├─ etcd_request_duration_seconds{operation=write} 高
 │  → etcd 慢 → 看 etcd_disk_wal_fsync_duration_seconds
 │  ├─ fsync 慢 → 磁盘问题（换 NVMe / 调 IOPS）
 │  └─ fsync 快但 request 慢 → 网络/leader/大对象
 └─ 两者都正常但 request_duration 高
      → APIServer 自身瓶颈：CPU（编解码/校验）、GC、watch cache miss、webhook
        看 apiserver_flowcontrol_request_exec_seconds 与 webhook 耗时
```

Q55：APIServer 无法启动 / CrashLoopBackOff 常见原因？

1. **etcd 不可达或证书错误**（最常见）：`--etcd-servers`、`--etcd-cafile/certfile/keyfile`。
2. **端口/地址冲突**：`--advertise-address`、`--bind-address`、`--secure-port`。
3. **证书 SAN 不含 LB 地址**：客户端报 x509。
4. `--service-cluster-ip-range` **与已有 Service CIDR 冲突。**
5. **静态 Pod manifest 语法错误**（kubeadm 场景看 `/etc/kubernetes/manifests/kube-apiserver.yaml`）。
6. **内存 limit 太小** → OOMKilled（`dmesg` / `kubectl describe` 看 Last State）。

7. **feature gate / flag 在升级后被移除** → 启动即失败（**升级前必须校验 flag 兼容性**）。

Q56：APIServer 升级需要注意什么？（变更管理题）

标准流程

1. **兼容性校验**：目标版本的废弃 API、移除的 flag、feature gate 状态变化（`kubeadm upgrade plan`、官方 changelog）。
2. **备份**：etcd snapshot + 静态 Pod manifest 备份。
3. **灰度**：先升级 1 个 APIServer 副本并观察（`--shutdown-delay-duration` 配合 LB 摘流）。 **灰度比例建议**：控制面 1/N → 观察 30 分钟（**【建议值】需按业务 SLO 调整**）→ 全量。
4. **观察指标**：5xx 率、P99、APF 拒绝数、etcd 慢请求、`/readyz`。
5. **回滚触发条件（明确写死）**： 5xx 率 > 1% 持续 5 分钟 P99 较基线劣化 > 2 倍持续 10 分钟 APF 拒绝率 > 1% 关键 webhook 大面积失败
6. **回滚方式**：还原静态 Pod manifest 到旧镜像 tag；**注意 API 版本兼容性 —— 一旦新版本写入了新字段/新存储版本，回退可能无法读取**（所以必须先备份 etcd 并确认 storageVersion 迁移策略）。

Q57：如何保护 APIServer 免于"一个坏客户端拖垮全集群"？

分层防御（**面试高分答案**）：

1. **客户端侧**：client-go 令牌桶限速、共享 informer、分页、退避重试遵守 `Retry-After`。
2. **APF 侧**：为不同客户端分配 FlowSchema + 有限队列 + `distinguisherMethod: ByUser` 隔离噪声源；关键系统流量走高优先级。
3. **准入侧**：`EventRateLimit`、`ResourceQuota`、webhook 的 `objectSelector` 缩小范围。
4. **存储侧**：etcd 拆分（`--etcd-servers-overrides`）、独立 etcd 实例。
5. **网关侧**：自研平台在 APIServer 前加一层 API 网关做**按租户限流**（托管集群常见做法）。
6. **治理侧**：CRD 上线评审（对象量级预估）、禁止无 limit 的 List、定期巡检 `apiserver_request_total` TopN 用户。

Q58：说出 5 个 APIServer 的反模式（收尾题，最容易翻车）

1. 业务代码/运维脚本**直接读写 etcd**。
2. 给 ServiceAccount 绑 `cluster-admin`。
3. 用 `system:masters` 做日常操作。
4. 审计策略全量 `RequestResponse`（性能 + Secret 明文落盘双重风险）。
5. webhook `failurePolicy: Fail` + 单副本 + 无限超时 + 无 selector。
6. 客户端全量 List 不分页、不复用 informer。
7. 单 etcd 承载所有资源（含 Event/metrics），且不设监控告警。
8. 把百万级对象做成 CRD 塞进主 etcd。

Part 5：反模式与风险清单

> 每条含：反模式 / 证据 / 风险等级 / 修复动作 / 验证方式。

#	反模式	证据（现象/配置）	等级	修复动作	验证方式
1	全量 List 不分页	`apiserver_storage_list_fetched_objects_total` 单请求数万；APIServer 内存尖峰	高	客户端强制 `limit` + `continue`；改用 informer	观察单次 List 返回对
2	Event 与主资源共 etcd	`etcd_mvcc_db_total_size_in_bytes` 快速逼近 quota	高	`--etcd-servers-overrides=/events#< 独立 etcd>`；`--event-ttl=30m`	DB size 增长率下降；
3	无差别 `failurePolicy: Fail` webhook	webhook Pod 单点；APIServer 报 failed calling webhook	高	多副本 + PDB + `objectSelector` + `timeoutSeconds≤5`；关键路径评估改 `Ignore`	混沌演练：kill 一个 SLO
4	控制器无条件 Update status	etcd 写 QPS 与 reconcile 次数 1:1	中高	`DeepEqual` 比较后再写	`rate(apiserver_r <cr>})` 下降
5	百万级 CR 走 CRD	主 etcd keys 数千万、DB 超限	高	迁移到 Aggregated API + 独立存储	主 etcd DB size 与 k
6	审计全量 RequestResponse	审计日志 TB/天；APIServer CPU/IO 高	中高	降级 `Metadata`；异步批量后端	APIServer P99 回落；
7	无 APF 分类，全走 global-default	`apiserver_flowcontrol_rejected_requests_total{flow_schema="global-default"}` 高	中高	为核心组件建 FlowSchema	拒绝率 \< 0.1%；等
8	HTTP/2 长连接粘滞	各 APIServer 副本 CPU/QPS 严重不均	中	`--goaway-chance`；LB least-conn	副本间 CPU 差值 \<
9	使用 `system:masters` 日常运维	kubeconfig 中 `O=system:masters`	中高	改用最小权限 RBAC 账号	审计日志可追溯到具
10	升级前不校验 flag/API 兼容性	APIServer CrashLoopBackOff	中高	`kubeadm upgrade plan` + 变更评审	灰度副本 `/readyz`

#	反模式	证据（现象/配置）	等级	修复动作	验证方式
11	证书过期无告警	集群整体不可用	高	监控 `apiserver_client_certificate_expiration_seconds`，\<30 天告警	演练：`kubeadm cer`
12	无 etcd 备份与恢复演练	故障后无法恢复	高	定期 snapshot + 每季度演练	演练 RTO/RPO 达标

Part 6：可直接套用的配置片段

6.1 APF：为关键组件提优先级、为批处理降优先级

```
# 1）给核心控制器一个专属高优先级
apiVersion: flowcontrol.apiserver.k8s.io/v1
kind: PriorityLevelConfiguration
metadata:
  name: platform-critical
spec:
  type: Limited
  limited:
    assuredConcurrencyShares: 100 # 【建议值】按实际并发实测调整
    limitResponse:
      type: Queue
      queuing:
        queues: 64
        queueLengthLimit: 200
        handSize: 6 # shuffle sharding，隔离噪声源
---
apiVersion: flowcontrol.apiserver.k8s.io/v1
kind: FlowSchema
metadata:
  name: platform-critical-controllers
spec:
  priorityLevelConfiguration:
    name: platform-critical
  matchingPrecedence: 100 # 越小越优先（1 最高，1000 最低）
  distinguisherMethod:
    type: ByUser
  rules:
    - subjects:
        - kind: ServiceAccount
          serviceAccount: {name: "platform-controller", namespace: "kube-system"}
        - kind: ServiceAccount
          serviceAccount: {name: "csi-provisioner", namespace: "kube-system"}
      resourceRules:
        - apiGroups: ["*"]
          resources: ["*"]
          verbs: ["*"]
---
# 2）给批处理/数据同步类客户端一个"可牺牲"优先级
apiVersion: flowcontrol.apiserver.k8s.io/v1
kind: PriorityLevelConfiguration
metadata:
  name: batch-low
spec:
  type: Limited
  limited:
    assuredConcurrencyShares: 5
    limitResponse:
      type: Queue
      queuing:
        queues: 16
        queueLengthLimit: 20
        handSize: 4
---
apiVersion: flowcontrol.apiserver.k8s.io/v1
kind: FlowSchema
metadata:
  name: batch-sync-clients
spec:
  priorityLevelConfiguration:
    name: batch-low
  matchingPrecedence: 900
  distinguisherMethod:
    type: ByUser
  rules:
    - subjects:
        - kind: ServiceAccount
          serviceAccount: {name: "data-sync", namespace: "ops"}
      resourceRules:
        - apiGroups: ["*"]
          resources: ["*"]
          verbs: ["*"]
```

> **回滚**：`kubectl delete flowschema/<name>` + `kubectl delete prioritylevelconfiguration/<name>`，立即回落默认行为，无状态残留。

6.2 一个"生产级"的 Admission Webhook 配置

```
apiVersion: admissionregistration.k8s.io/v1
kind: MutatingWebhookConfiguration
metadata:
  name: sidecar-injector
webhooks:
  - name: injector.example.com
    admissionReviewVersions: ["v1"]
    sideEffects: None # 声明无副作用，允许 dry-run
    timeoutSeconds: 3 # 【建议值】≤5，越短越安全
    failurePolicy: Fail # 关键注入用 Fail；非关键用 Ignore
    reinvocationPolicy: IfNeeded # 其他 webhook 改了对象后是否再调用
    matchPolicy: Equivalent
    clientConfig:
      service:
        name: injector
        namespace: injector-system
        path: /mutate
        port: 443
      caBundle: <BASE64_CA>
    namespaceSelector:
      matchExpressions:
        - key: kubernetes.io/metadata.name
          operator: NotIn
          values: ["kube-system"] # 排除系统命名空间，减小爆炸半径
    objectSelector: # 关键：只命中带特定标签的 Pod
      matchLabels:
        inject.example.com/enabled: "true"
    rules:
      - apiGroups: [""]
        apiVersions: ["v1"]
        operations: ["CREATE"]
        resources: ["pods"]
        scope: "Namespaced"
```

6.3 kubeadm 场景下的 APIServer 额外参数

```
# /etc/kubernetes/manifests/kube-apiserver.yaml  → spec.containers[0].command
- --enable-priority-and-fairness=true
- --etcd-compaction-interval=5m # 【默认值】
- --event-ttl=1h # 高 Event 量可降到 30m
- --etcd-servers-overrides=/events#https://etcd-events-0:2379;https://etcd-events-1:2379;https://etcd-events-2:2379
- --watch-cache-sizes=pods#1000,nodes#500,namespaces#10,endpointslices#500 # 【需实测】
- --anonymous-auth=false
- --audit-policy-file=/etc/kubernetes/audit/policy.yaml
- --audit-log-path=/var/log/kubernetes/audit.log
- --audit-log-maxage=7
- --audit-log-maxbackup=5
- --audit-log-maxsize=200
- --shutdown-delay-duration=20s # 配合 LB 摘流【建议值】
```

> **变更纪律**：静态 Pod manifest 修改后 kubelet 会自动重启 APIServer；**必须逐个节点修改并观察** `/readyz`，禁止同时改所有 master。

6.4 关键告警规则（Prometheus）

```
groups:
  - name: apiserver-slo
    rules:
      - alert: APIServerHigh5xxRate
        expr: |
          sum(rate(apiserver_request_total{code=~"5.."}[5m]))
          / sum(rate(apiserver_request_total[5m])) > 0.01
        for: 5m
        labels: {severity: critical}
        annotations: {summary: "APIServer 5xx 率 > 1% (SLO 烧穿) "}

      - alert: APIServerLatencyP99
        expr: |
          histogram_quantile(0.99,
            sum by (le) (rate(apiserver_request_duration_seconds_bucket{verb!~"WATCH"}[5m]))
          ) > 1
        for: 10m
        labels: {severity: warning}

      - alert: APIServerFlowControlRejecting
        expr: |
          sum(rate(apiserver_flowcontrol_rejected_requests_total[5m]))
          / sum(rate(apiserver_request_total[5m])) > 0.001
        for: 5m
        labels: {severity: warning}

      - alert: EtcdWALFsyncSlow
        expr: |
          histogram_quantile(0.99,
            sum by (le) (rate(etcd_disk_wal_fsync_duration_seconds_bucket[5m]))
          ) > 0.01
        for: 10m
        labels: {severity: critical}
        annotations: {summary: "etcd WAL fsync P99 > 10ms, 磁盘不达标"}

      - alert: EtcdDBNearQuota
        expr: etcd_mvcc_db_total_size_in_bytes / etcd_server_quota_backend_bytes > 0.8
        for: 15m
        labels: {severity: critical}

      - alert: EtcdLeaderChanges
        expr: increase(etcd_server_leader_changes_seen_total[30m]) > 3
        labels: {severity: critical}

      - alert: APIServerCertExpiringSoon
        expr: |
          min by (job) (apiserver_client_certificate_expiration_seconds) < 30*24*3600
        labels: {severity: warning}
```

Part 7：面试自检清单

能不看资料回答以下问题，说明 APIServer 这块过关了：

- [ ] 能画出 `kubectl apply` → etcd 的完整链路，并说出每个过滤器在做什么
- [ ] 说清 Mutating / Validating 的先后与原因
- [ ] 解释 `resourceVersion` 与乐观锁，以及 409 的正确处理方式
- [ ] 说清 List-Watch + Informer 的三段：List → Watch → Resync/Relist，且知道 resync **不**打 APIServer
- [ ] 解释 watch cache 的作用与 410 Gone 的成因
- [ ] 说出 Bookmark 的作用（保活 + RV 推进 + 避免 relist）
- [ ] 讲清 APF 的 FlowSchema / PriorityLevel / shuffle sharding，能写出 YAML
- [ ] 说出大规模集群的三大压力源：Lease/Event/CR 写放大、List 读放大、webhook 时延
- [ ] 知道官方规模上限与"每节点 Pod ≤ 110"的原因
- [ ] 能给出 etcd NOSPACE 的处置步骤（含 alarm disarm 与 defrag 顺序）
- [ ] 能给出 APIServer P99 高时的决策树（限流 / etcd / 自身）
- [ ] 能给出 SLO + 错误预算的具体数字与测量 PromQL
- [ ] 能把任意优化动作配上"灰度比例 + 回滚触发条件 + 验证指标"

免责与边界：本文为面试复习与架构评审材料，不针对任何具体生产集群直接执行操作。所有生产变更请走：变更评审 → 备份 → 灰度 → 观察 → 回滚预案。文中【默认值】请以目标版本的官方参数文档为准，【需实测】项请以你的监控数据为准。

第三篇 • etcd / controller-manager / kube-proxy

> 控制面存储 · 控制器范式 · 服务转发

0. 使用说明

0.1 控制面四件套的职责速览

组件	一句话职责	是否有状态	是否选主
kube-apiserver	集群唯一的 API 入口：认证/授权/准入/校验/审计 + 读写 etcd	**无状态**（可水平扩容）	否
etcd	唯一的持久化存储（所有对象的唯一真相源）	**有状态**	是（Raft leader）
kube-controller-manager	跑各种控制器，让"实际状态"收敛到"期望状态"	基本无状态（依赖 apiserver）	**是**（Lease）
kube-scheduler	给未绑定 Pod 选节点	无状态	是（Lease）
kube-proxy	节点数据面：把 Service/EndpointSlice 翻译成转发规则	无状态	否
cloud-controller-manager	云厂商相关逻辑（节点生命周期/LB/路由）	无状态	是（通常）

0.2 常用端口清单（面试高频，容易混）

组件	端口	用途
kube-apiserver	**6443**（默认 secure port）	API + `/healthz` `/readyz` `/livez` `/metrics`
kubelet	**10250** API、**10248** healthz、10255 read-only（应关闭）	见 Kubelet 篇
kube-scheduler	10259	healthz + metrics（HTTPS）
kube-controller-manager	10257	healthz + metrics（HTTPS）
kube-proxy	**10249** metrics、**10256** healthz	端口可能因版本/参数而变，请核对
etcd	2379 client、**2380 peer**	客户端 / 集群内部通信
kube-controller-manager	10252 / kube-scheduler 10251	已废弃的 insecure 端口

> **提示**：端口表最适合在面试开始时主动交代，能立刻体现"干过"。

Part B • etcd 篇（12 题）

> etcd 是 K8s 唯一的持久化真相源，**它的健康状况直接等于集群的健康状况。**

关于 kube-apiserver 部分

> ℹ️ **去重说明**：本篇原有的「kube-apiserver 原理篇（A1–A15，共 15 题）」与第二篇《APIServer 深度解析》（Q1–Q58）逐题重合，已整体并入第二篇，此处不再重复。本篇保留 etcd、controller-manager、kube-proxy 三部分。

B1. etcd 的数据模型长什么样?

- **MVCC + bbolt**：每次写生成一个新 revision（单调递增），读可以在某个 revision 上做快照；
- key 的组织形式大致是 `/registry/<resource>/<namespace>/<name>`（v3）；
- `resourceVersion` **本质上就是 etcd 的 revision**；
- **compaction（压缩）** 负责删掉历史 revision；**defrag（碎片整理）** 负责把磁盘空间真正归还。

必答：etcd **不是**"对象数据库"，它既不理解 Kubernetes 语义，也不提供二级索引——所有过滤都在 apiserver 侧完成，这解释了为什么"不加 selector 的 list 全量请求"那么贵。

B2. Raft 与 quorum：为什么必须是奇数节点?

成员数	quorum（可写所需多数）	可容忍故障节点数
3	2	1
5	3	2
4	3	1（**不划算**）

结论：偶数成员除了多花钱没有任何容错收益 ⇒ 用 **3（常规）/ 5（大集群或跨 AZ 要求更高）**。

部署要点：

- **leader 选举与写都要经过 leader**，写延迟 ≈ 到 leader 的 RTT + WAL fsync；
- **跨地域不要做同一 etcd 集群**（RTT 直接决定写性能），要跨地域就做多集群；
- 同城多可用区是常见做法，但要接受"跨 AZ RTT"带来的写延迟代价，并压测；
- `learner` 成员可以让你安全地添加/预热新成员（不参与 quorum）。

B3. 最经典的 etcd 事故：`mvcc: database space exceeded`（NOSPACE）

现象：集群突然不能写（Pod 无法创建、删除对象也失败），但读一般还能进行。

根因链：

```
写入量大（Event/ConfigMap/频繁更新）→ 历史 revision 未及时压缩 → 达到 quota-backend-bytes → 触发 NOSPACE alarm → etcd 进入只读
```

处置步骤（必须记住）：

```
# 1）确认状态
etcdctl endpoint status --write-out=table --cluster
etcdctl alarm list
# 2）压缩到当前 revision
rev=$(etcdctl endpoint status --write-out="json" | python3 -c "import sys,json;print(json.load(sys.stdin)[0]['status']['header']['revision'])")
etcdctl compact "$rev" --physical
# 3）每个成员做碎片整理（会引起瞬时阻塞，逐个执行）
etcdctl defrag --cluster
# 4）清除报警
etcdctl alarm disarm
# 5）确认恢复
etcdctl endpoint status --write-out=table --cluster
```

预防：

- 配自动压缩：`--auto-compaction-mode=revision --auto-compaction-retention=1000`（示意值，**请与你所用发行版核对**）；
- 适当调大 `--quota-backend-bytes`（etcd 默认 2GiB；K8s 集群常见设为更大值，**务必先压测且注意恢复时间**）；
- Event 独立集群 + `--event-ttl`；
- 监控 `etcd_db_total_size_in_bytes / etcd_server_quota_backend_bytes`；
- **定期做 defrag 演练**（这是最容易"不敢在生产执行"的操作）。

```
etcd_db_total_size_in_bytes / etcd_server_quota_backend_bytes > 0.8
etcd_server_quota_backend_bytes
```

B4. etcd 的头号性能指标是什么?

磁盘。 etcd 官方硬件建议的量级参考（**请以 etcd 官方文档为准**）：

- **WAL fsync P99 应 \< 10ms**；
- **backend commit P99 应 \< 25ms**；

- 典型症状：`etcd to commit took too long`、`apply request took too long`、`leader heartbeat took too long`。

```
histogram_quantile(0.99, rate(etcd_disk_wal_fsync_duration_seconds_bucket[5m]))
histogram_quantile(0.99, rate(etcd_disk_backend_commit_duration_seconds_bucket[5m]))
# 网络 RTT（Peer 之间）
histogram_quantile(0.99, sum by (To, le) (rate(etcd_network_peer_round_trip_time_seconds_bucket[5m])))
# leader 变更频率（频繁切主是大忌）
increase(etcd_server_leader_changes_seen_total[30m])
# 慢请求/失败提议
increase(etcd_server_slow_applies_total[10m])
increase(etcd_server_proposals_failed_total[10m])
rate(etcd_server_proposals_pending[1m]) > 5
```

必答的治理动作：NVMe SSD 独占磁盘（不要和其它 IO 密集进程共享）、网络延迟达标、不要跑在"共享/节流型"云盘上、使用独立节点不要与 workload 混部。

B5. K8s 里 etcd 到底存了什么？什么不存?

存	不存
所有 Kubernetes 对象的最新状态（Pod/Deployment/Secret/ConfigMap/Service/EndpointSlice/Node/Lease/PVC/VPC-ish CRD 等）	**容器文件系统内容**（在节点上）
事件的当前对象（在有 TTL 的情况下会过期删除）	容器日志（节点磁盘；etcd 里只有 Pod 对象）
CRD 自定义对象	metrics / 监控时序数据
Lease、TokenRequest 相关状态等	实际网络转发规则（节点上）

必答：**etcd 里没有任何"业务数据"和"运行时数据"** —— 所以"把 etcd 备份好 + 有正确的恢复流程"理论上能重建整个集群状态，但**节点上的容器与本地卷数据不在其中**（有状态服务要另行备份）。

B6. etcd 备份与恢复的标准流程？（SRE 必考）

备份：

```
ETCDCTL_API=3 etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  snapshot save /backup/etcd-$(date +%Y%m%d-%H%M).db

# 校验快照可用性（不要等到真出事才发现备份是坏的）
etcdctl snapshot status /backup/etcd-xxx.db --write-out=table
```

恢复（示意，务必先在演练环境验证）：

```
# 1) 停止所有 apiserver（停止写入是关键）
# 2) 停掉 etcd 成员
# 3) 逐成员 restore（每个成员用自己的 --name/--initial-advertise-peer-urls）
etcdctl snapshot restore /backup/etcd-xxx.db \
  --name etcd-1 \
  --initial-cluster "etcd-1=https://10.0.0.1:2380,etcd-2=https://10.0.0.2:2380,etcd-3=https://10.0.0.3:2380" \
  --initial-cluster-token etcd-cluster-token \
  --initial-advertise-peer-urls https://10.0.0.1:2380 \
  --data-dir /var/lib/etcd-restored
# 4) 用新的 data-dir 启动 etcd
# 5) 启动 apiserver，验证资源一致性
# 6) 强制重启 kubelet/重建 Pod 让实际状态对账（必要时按服务逐个滚动）
```

演练要求：恢复演练必须**定期执行**并计时（MTTR），并在演练中验证"恢复后的数据时间点是否符合 RPO 目标"。

B7. etcd 要不要和 apiserver 分开部署?

方案	优点	缺点
同机部署（kubeadm 常见）	简单、少机器	磁盘/网络资源竞争；大规模（>~1000 节点）不推荐
独立 etcd 集群（推荐中大规模）	资源隔离、可独立扩容/调优/备份	多一组机器与证书管理

建议：集群规模上来或稳定性要求高时，**etcd 独立部署**几乎是必选项，且管理面要能做到"先停 apiserver 再动 etcd"。

B8. defrag 有什么风险？怎么安全地做?

- defrag 会**重建 bbolt 数据库**，期间该成员**阻塞/拒绝请求**（视版本），因此对 leader 执行会触发重新选主 ⇒ 抖动；
- 做法：**逐个成员执行**，先在 follower 上做；某些版本支持 `--read-only-timeout` / 延迟 defrag 的规则（`--defrag-rule-method`，请核对）；
- 磁盘空间要足够容纳临时文件；
- 建议在低峰 + 有回滚窗口 + 已备份的前提下执行，且**先在演练集群熟练**。

B9. 证书与 TLS：etcd 的一整套证书体系

etcd 需要：

- **server cert**（给客户端用）、**peer cert**（成员间）、**CA**；
- apiserver 连接 etcd 需要 client cert（`--etcd-certfile` 等）。

典型事故：证书过期 ⇒ apiserver 无法连 etcd ⇒ **整个集群控制面不可用。**

治理：

- 监控证书有效期并提前告警；
- 证书轮换必须走受控流程并在变更后验证 `etcdctl endpoint health`；
- kubeadm/托管服务会自动处理，**不要手动改一半**

```
# 依赖 kubelet 或自采集方式暴露的证书过期指标（指标名以实际环境为准）
# 建议：对 /etc/kubernetes/pki/** 做定时检查并输出 gauge
```

B10. etcd 的 key 个数/对象数太多会怎样?

- 单 key 过大、单 key 个数过多都会增加 bbolt 树深度与 commit 成本；
- **Pod/Event/EndpointSlice/Lease 是数量增长最快的对象**；
- **不要往 ConfigMap/Secret 里塞大二进制**（会影响 watch cache、list 成本，甚至超过对象大小限制）；
- 清理策略：删除无用 CRD 对象、控制 Event TTL、审视 Lease 数量。

```
# 对象数量视图（需要 kube-state-metrics 或 apiserver 侧计数）
apiserver_storage_objects{resource="pods"} # 指标名与可用性请按版本核对
apiserver_storage_objects{resource="endpointslices"}
```

B11. etcd 的成员替换/扩容怎么做?

```
加入新成员（推荐先以 learner 身份加入，等数据追平）
   → etcdctl member add <name> --peer-urls=... --learner
   → 启动新成员进程
   → 追平后 etcdctl member promote <id>
替换旧成员：加入新 → 删除旧（一次只改一个成员，改完必须确认健康）
```

禁止：**同时改多个成员**、在没有健康 quorum 的情况下操作、在没有快照的情况下做成员变更。

B12. etcd 常见日志报错速查?

日志	含义	处置
`database space exceeded`	配额满	B3 处置流程
`apply request took too long`	commit 慢（通常是磁盘）	查 fsync/commit 延迟
`leader heartbeat took too long`	网络或磁盘慢	查 peer RTT + fsync
`request timed out`	客户端超时、集群忙	查 in-flight 与慢请求
`etcdserver: mvcc: required revision has been compacted`	客户端 watch 太老	客户端 relist；缩短 list 周期
`authentication failed, invalid peer certificate`	证书错/过期	校验证书
`read-only range request ... took too long`	读慢（大 list / 磁盘慢）	优化 list + 磁盘

Part C • kube-controller-manager 篇（12 题）

C1. 控制器的通用编程范式是什么?

```
Informer(ListAndWatch) → 本地 Store(缓存) → EventHandler 把 key 放进 WorkQueue
                                              ↓
                                        worker 取 key
                                              ↓
                                      Reconcile(key):
                                          读期望状态(API) vs 读实际状态(API/外部系统)
                                          → 有差异则写 API 让它收敛
                                              ↓
                                      失败 → 按限流器(指数退避)重新入队
```

必答的四个要点：

1. **Level-triggered，不是 edge-triggered** —— Reconcile 回答的是"现在应该是什么样"，而不是"发生了什么事件"；
2. **幂等**：同一个 key 反复 reconcile 结果一致；
3. **resyncPeriod** 兜底：即使事件丢失也会周期性重放；
4. **不持有外部真相源**：状态要么从 API 读，要么从外部系统读，绝不假设"我上次记的那份是对的"。

C2. controller-manager 里到底有哪些控制器？（要能报出一大串）

分类	控制器	作用
工作负载	Deployment、ReplicaSet、StatefulSet、DaemonSet、Job、**CronJob**	副本维持、滚动更新、定时任务
网络	EndpointSlice controller、Endpoints controller、ServiceAccount token controller	服务发现后端集合、SA token
节点	Node lifecycle controller、**taint manager**、clusterrole aggregation	NotReady 标记/驱逐、聚合内置 ClusterRole
存储	PV controller、Attach/Detach controller、volume expand controller	卷绑定/挂载/扩容
资源治理	**ResourceQuota controller**、Namespace controller	配额用量计算、命名空间生命周期
对象关系	**Garbage collector controller**（级联删除）	ownerReference 级联删除
证书	CSR approver/signer 相关、Certificate controller	证书签发（**注意：node serving CSR 的批准策略依赖发行版**）
扩缩容	HPA controller（常挂在 controller-manager 或独立部署）	基于指标扩副本
其它	Job TTL controller（finished 后清理）、ownership 相关	TTL 清理等

```
# 按需启停（示意）
kube-controller-manager --controllers=*,bootstrapsigner,tokencleaner
```

C3. controller-manager 怎么保证只有一个在工作?

- 基于 `coordination.k8s.io/v1 Lease` 做 leader election（kube-system 命名空间）
- 只有 leader 执行 reconcile；其它实例待命
- 相关默认lease/renew/retry时长请以所用版本为准

```
# 观察是否有多主或频繁切主（指标名以实际暴露情况为准）
increase(leader_election_master_status[10m])
```

C4. Node lifecycle controller 与 taint manager 的完整时间账

```
① kubelet 每 ~10s 更新 Node Lease
② controller-manager 超过 --node-monitor-grace-period（默认 50s）判定节点不健康
③ 给节点打 condition: Ready=Unknown/False
④ taint manager 打 taint: node.kubernetes.io/not-ready 或 node.kubernetes.io/unreachable (NoExecute)
⑤ Pod 默认注入了 tolerationSeconds=300 的容忍
⑥ 5 分钟后 taint manager 才真正驱逐 Pod → 重新 Pod 调度
```

结论：节点故障约 **5~6 分钟**后工作负载才会被重新拉起到其它节点（与 Kubelet 篇 Q16 完全对应，面试时可联动作答）。

C5. 滚动更新（Deployment）到底是怎么算的?

```
maxSurge = 25% / n: 允许超出期望副本数的比例
maxUnavailable= 25% / n: 允许不可用的比例
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 10
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 2 # 建议：按绝对值给，比百分比更可预测
      maxUnavailable: 0 # 零中断场景建议配 0（需要集群有冗余资源）
  minReadySeconds: 15 # 建议：给"新 Pod 已就绪但还没稳定"留观察窗
  progressDeadlineSeconds: 600
  revisionHistoryLimit: 10
```

必答：

- Deployment 只是"创造了新的 ReplicaSet"并按策略缩旧扩新，实际扩缩数量由 **ReplicaSet controller** 完成；
- `maxUnavailable: 0` + PDB 才能保证真正零中断；
- `ProgressDeadlineExceeded` 说明更新在期限内没完成（常见原因：镜像拉不下、配额不够、探针不过）。

C6. Garbage Collector 的三种删除策略?

`propagationPolicy`	行为
`Foreground`	先删除依赖对象，再删 owner（会进入 deleting 状态等待）
`Background`（常见默认）	立即删 owner，GC 后台清理依赖对象
`Orphan`	删 owner 但**保留**子对象（断开 ownerReference）

必答：`Finalizer` 会**阻塞删除**（对象保留 `deletionTimestamp` 直到 finalizer 被拿掉）——这是"Terminating 卡死"的头号原因。

C7. ResourceQuota controller 与 admission ResourceQuota 的分工?

- **Admission（ResourceQuota 准入插件）**：在创建/更新请求时**检查是否会超配额**，超了直接拒绝（这是"为什么 Pod 创建被 403 forbidden"的原因之一）；
- **Controller（ResourceQuota controller）**：统计各命名空间的已用量并写回 `status.used`（用量修正是异步的）。

坑：异步统计意味着短暂窗口内可能超额；所以要结合配额 + 上层审批 + LimitRange 默认值。

C8. Attach/Detach vs Volume manager 怎么分工?

层	谁	干什么
Attach/Detach（controller-manager）	与控制面/云 API 交互	把云盘"挂到虚拟机"
Volume manager（kubelet）	节点侧	挂载文件系统到 Pod 目录 stage/publish
PV controller	控制面	PVC↔PV 绑定、延迟绑定（WaitForFirstConsumer）

排障分界线：`FailedAttachVolume` 属控制面/云 API 问题；`FailedMount` 多半是 kubelet/CSI 节点侧问题。

C9. HPA 的指标管线与常见参数?

```
cAdvisor → kubelet summary/metrics → metrics-server →
      apiserver aggregated API(metrics.k8s.io) → HPA controller → Deployment 副本数
自定义/外部指标:
      Prometheus Adapter → custom.metrics.k8s.io / external.metrics.k8s.io
```

常见参数（请核对版本默认值）：`--horizontal-pod-autoscaler-sync-period`、`--horizontal-pod-autoscaler-downscale-stabilization`（防抖动）、`--horizontal-pod-autoscaler-tolerance`。

实践：给**扩容快、缩容慢**（different stabilization windows，`behavior.scaleDown.stabilizationWindowSeconds`），避免"抖动导致副本数反复横跳"。

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 0
      policies: [{type: Percent, value: 100, periodSeconds: 60}]
    scaleDown:
      stabilizationWindowSeconds: 300 # 建议初值，需要按业务调
```

C10. 大规模下 controller-manager 的调优参数?

```
--kube-api-qps=100 --kube-api-burst=150 # 建议初值，需实测
--concurrent-replicaset-syncs=20
--concurrent-deployment-syncs=10
--concurrent-endpoint-syncs=10
--concurrent-service-endpoint-syncs=10
--concurrent-gc-syncs=20
--concurrent-resource-quota-syncs=10
--node-monitor-grace-period=50s
--node-monitor-period=5s
```

风险：并发调高会放大对 apiserver/etcd 的压力（尤其 endpoint sync），**必须与 apiserver/etcd 容量一起压测**。

C11. Deployment 卡住（rollout 卡住）怎么排查?

症状/证据	原因
`ImagePullBackOff`	镜像/仓库/凭据问题
Pod Pending 且 FailedScheduling	资源/亲和/污点/配额（见 Scheduler 篇）
`forbidden: exceeded quota`	ResourceQuota
更新没法进展	PDB 限制导致旧 Pod 无法删除
新 Pod 一直 NotReady	探针/应用/网络策略
`ProgressDeadlineExceeded`	上述任一原因超时
RS 数量异常多	`revisionHistoryLimit` 过大/频繁发布

```
kubectl rollout status deploy/web
kubectl describe deploy/web | tail -30 # Conditions 有原因
kubectl get rs --sort-by=.metadata.creationTimestamp | tail
kubectl describe rs <new-rs> # ReplicaSet 的 Events 最有信息量
```

C12. 自研控制器要注意什么?

- 用 **controller-runtime（CR 生态）** 而不是手写 informer;
- Reconcile 快速返回 + 用 `RequeueAfter` /事件驱动;
- **写操作要有 check-then-act 的乐观锁重试**（409 处理）;
- 合理限制并发 worker 数与 QPS/burst;
- **不要高频写 Event**（会放大 etcd 压力）;
- 提供清晰 status/conditions，方便排障;
- 健康检查 + metrics + leader election（单副本语义时）必备。

Part D • kube-proxy 专项篇（8 题）

D1. kube-proxy 到底"转发"了吗？（最大误区）

没有。 kube-proxy **只写规则**，真正转发数据包的是内核（netfilter/NAT 或 IPVS），或者完全不经过 netfilter（Cilium socket-LB 在 socket 层完成）。

```
Service/EndpointSlice 变化
    → kube-proxy watch 到
    → 计算规则集合
    → 通过 iptables-restore / ipvsadm-equivalent netlink / nft 下发
    → 内核按规则转发包（kube-proxy 不参与单包处理）
```

结论：kube-proxy 挂掉 ⇒ **已下发的规则仍然生效**（已有连接与 Service 访问不会立刻断），但**后续 Service/Endpoint 变更不再被同步**（这是"为什么节点重启前后不一致、某些后端没更新"的根因）。

D2. kube-proxy 有哪几种模式？各自的关键差异？

模式	数据面	查找复杂度	主要优点	主要缺点
userspace	进程代理	—	历史实现	已弃用
iptables	netfilter 链	O(规则数)	默认、依赖少	规模大时规则膨胀、更新全量下发、无真调度算法
ipvs	IPVS + ipset	O(1)	规模友好、有调度算法	仍用 iptables 处理 SNAT/包过滤；内核模块依赖
nftables	nftables	较好	较新的替代方案	**版本相关（处于演进阶段），请以 Release Notes 确认成熟度**
无 kube-proxy（CNI 替代）	eBPF socket-LB	好	绕开 netfilter/conntrack、保留源 IP	需 Cilium 等支持，依赖较新内核

D3. iptables 模式下的一条链路长什么样？

```
外部访问 ClusterIP:80
  PREROUTING → KUBE-SERVICES 链（匹配 ClusterIP）
       → per-service 链 KUBE-SVC-XXX（statistic random → 挑一个后端）
       → per-endpoint 链 KUBE-SEP-YYY（DNAT 到 PodIP:Port）
  NodePort 场景：先经 KUBE-NODEPORTS
  出集群/源 SNAT：POSTROUTING → KUBE-POSTROUTING → MASQUERADE/RETURN（由 ipset KUBE-LOOP-BACK 等控制）
```

排障命令：

```
iptables-save -t nat | grep -A5 <clusterIP>
iptables-save -t nat | wc -l # 规则规模
ipvsadm -Ln --stats # IPVS 模式
nft list ruleset # nftables 模式
```

D4. kube-proxy 的关键参数有哪些？

```
apiVersion: kubeproxy.config.k8s.io/v1alpha1
kind: KubeProxyConfiguration
mode: ipvs
# 通用
bindAddress: 0.0.0.0
clusterCIDR: 10.244.0.0/16
healthzBindAddress: 0.0.0.0:10256 # 端口请按版本核对
metricsBindAddress: 0.0.0.0:10249
detectLocalMode: ClusterCIDR
# 抖动控制：避免 Endpoint 频繁变化引起规则疯狂重写
iptables:
  syncPeriod: 30s
  minSyncPeriod: 5s
ipvs:
  syncPeriod: 30s
  minSyncPeriod: 5s
  scheduler: rr
  excludeCIDRs: []
  tcpTimeout: 0s
conntrack:
  maxPerCore: 131072 # 建议初值，需实测
  tcpEstablishedTimeout: 24h
  tcpCloseWaitTimeout: 1h
```

要点：`minSyncPeriod` 太小会让 kube-proxy 在 Endpoint 抖动时频繁全量刷新规则（CPU 尖峰）；太大则收敛变慢。

D5. kube-proxy 挂了会怎样？怎么发现？

表现	说明
已有 Service 访问正常	规则还在内核里
新增/删除 Service、Endpoint 变更不生效	规则不同步
Pod 访问新上线的后端失败	旧规则残留

```
# 规则同步是否还活着（关键指标，务必先核对目标版本是否暴露）
histogram_quantile(0.99, sum by (le) (rate(kubeproxy_sync_proxy_rules_duration_seconds_bucket[5m])))
# 最后一次成功同步时间：长时间不更新即意味着 kube-proxy 卡了
time() - kubeproxy_sync_proxy_rules_last_timestamp_seconds > 300
# 进程存活
up{job="kube-proxy"} == 0
```

D6. `healthz` 与 LB 健康检查的关系?

- kube-proxy 提供 `/healthz`（默认端口 10256，请核对），可用于 DaemonSet 的 liveness/readiness；
- **它不代表数据面一定正常**，只说明进程活着；
- 真正要监控的是"规则同步是否及时" + "业务合成探测是否成功"。

D7. 大规模集群 kube-proxy 的优化清单?

1. **切 IPVS**（或 nftables/Cilium）：摆脱 iptables 规则线性增长；
2. 合理设置 `minSyncPeriod`，避免抖动放大；
3. conntrack 扩容 + timeout 调优（见网络篇 C5）；
4. 控制 Service/Endpoint 数量增长（架构侧）；
5. EndpointSlice 默认切分更细可以减少单次更新 payload（`-max-endpoints-per-slice`）；
6. kube-proxy 的 API 客户端限流（`--kube-api-burst/qps`）别太小，否则大规模下追不上。

D8. 什么时候该"去掉" kube-proxy?

满足以下多数条件时可考虑 **Cilium kube-proxy replacement**：

- Service/Endpoint 数量大且持续增长；
- 短连接高并发，conntrack 成为瓶颈；
- 需要保留客户端真实 IP 而不想用 `externalTrafficPolicy: Local`；
- 已经具备 eBPF 的排障能力（团队 skill 到位）。

反之（保守为宜）：内核版本不满足、有强依赖 netfilter 的遗留组件、团队缺乏 eBPF 排障经验。

Part E • 大规模控制面优化篇（12 题）

E1. 控制面的容量到底由什么决定?

```
apiserver QPS 上限 ≈ min(
    CPU: 序列化/准入/审计的 CPU 成本,
    etcd 写延迟: 每次写都要 WAL fsync + Raft quorum,
    watch fan-out: 副本数 × 各组件 watch 数,
    list 成本: 大对象/全量 list 打穿缓存
)
```

结论（必答）：**apiserver 可以横向扩容，etcd 不可以轻易扩容。**控制面性能治理的终点几乎总是落在 etcd 磁盘延迟与"少写少 list"。

E2. 最常见的两类"控制面杀手"：大 list 与 watch 洪水

反模式	后果	治理
控制器全集群 list 且不带 selector	巨大响应体 + 序列化 CPU	强制走 informer + field/label selector + 分页
每个组件都自己 list/watch 同一资源	N 倍流量	使用 **SharedInformer**；或走 informer 缓存代理
watch 不带 bookmarks / 反复断开重放	消息洪峰	开启 `allowWatchBookmarks`、指数退避、合理 relist 间隔
客户端忘记限流	打满 apiserver	client-go `--kube-api-qps/burst` 或 client 侧 rate limiter

```
# 谁在大量请求？（找吵闹邻居的第一条语句）
sum by (user, verb, resource) (rate(apiserver_request_total[5m]))
# 响应体大小（大 list 的直接证据）
histogram_quantile(0.99, sum by (resource, verb, le) (rate(apiserver_response_sizes_bucket[5m])))
```

E3. 对象数量与体积治理

- **避免在 ConfigMap/Secret 里放大对象**（有大小限制；大对象会拖慢 list/watch/etcd）；
- **Event 治理**（TTL、准入限速、独立 etcd）；
- **减少 Lease/CRD 的无效增长**；
- 定期审计：`kubectl get crd`、按对象类型统计 Top N。

```
# 按对象类型快速估算数量（示意思路）
kubectl get --raw /metrics | grep apiserver_storage_objects
```

E4. APF 的生产配置思路

```
① 先区分"关键流量": leader-election、kubelet 心跳/status、scheduler 绑定、控制面探针
② 为它们配独占 PriorityLevel + 队列（避免被吵闹邻居挤掉）
③ 对"全集群 list / 非关键 controller"配低优先级 + 短队列（打不赢就让它们失败重试）
④ 监控 rejected/queue depth，逐步调 shares
```

> **注意**：错误的 FlowSchema 会**误伤关键组件**，任何 APF 变更都应在测试集群验证，并保留可快速删除（回滚）的变更记录。

E5. watch cache 与 per-resource 调优

- 热点资源（pods、endpointslices、nodes、configmaps、secrets）优先加大 cache；
- 观察 `apiserver_cache_*` 指标与 **apiserver 内存 RSS**；
- 内存与一致性之间取值：**建议先给少量关键资源加大，观察 7 天再决定是否继续调。**

E6. etcd 侧的大规模最佳实践（汇总）

项	建议
磁盘	独立 NVMe SSD，不共享
配额	按数据量规划 `--quota-backend-bytes`，并留出恢复时间余量
压缩	开启自动压缩；保留策略按审计/回溯需求设定
defrag	定期低峰执行 + 演练过
Event	独立 etcd 集群（大集群）
机密性	Secret 静态加密；注意 KMS 可用性风险
拓扑	3 或 5 成员，**同地域低延迟网络**，跨可用区需压测
备份	定时快照 + 定期恢复演练（RPO/RTO 写进 SLO）

E7. 控制面组件升级的标准顺序与风险

```
① 先备份 etcd 快照（这一步永远排第一）
② 升 etcd（一次一个成员，确认 health 再下一个）
③ 升/滚动 apiserver（先灰度一个副本，观察后再全量）
④ 升 kube-controller-manager / kube-scheduler（依赖 leader election 自动切换）
⑤ 升 cloud-controller-manager、kube-proxy、CNI、CoreDNS
⑥ 最后升 kubelet（按节点池灰度，遵循 skew ≤ 3 且不得高于 apiserver）
```

风险与回滚：

- **etcd 版本回退几乎不可行**（数据格式/dbVersion 一般只能向前），所以 etcd 升级必须"先快照、慢一点、一次一个"；
- apiserver 回滚 = 换回旧版本镜像/二进制，**要注意 API 是否已经对新字段写入**（可能不可逆）；
- kube-proxy/CNI 回滚请见网络篇 C13 的双池灰度法。

E8. 多控制面副本的流量与优雅下线

- LB 健康检查建议用 `/readyz`；
- 副本滚动时要保证 **watch 连接平滑迁移**（客户端要有重连退避）；
- 注意 `--shutdown-delay-duration`（或对应机制）配合 LB 摘除延迟，避免"已经停止服务但 LB 还在转发"。

```
# apiserver 就绪度
sum(up{job="apiserver"}) / count(up{job="apiserver"}) < 0.66
# 请求 5xx 比例
sum(rate(apiserver_request_total{code=~"5.."}[5m])) / sum(rate(apiserver_request_total[5m])) > 0.01
```

E9. 灾备：etcd 快照 vs 对象级备份

方案	覆盖	恢复粒度	适用
etcd 快照	全部对象状态	整集群（很难只恢复单个 ns）	集群级灾难恢复
Velero/对象导出	指定资源/命名空间	**细粒度**	误删恢复、迁移、审计留存

建议：**两者都要** —— velero 负责日常误操作恢复（高频、低风险），etcd 快照负责集群级重建（低频、高风险、必须演练）。

E10. 控制面 SLO 怎么定？（SIG Scalability 口径思路）

社区对 API 延迟有明确的 SLI 定义（**口径请以 SIG Scalability 文档为准**）：

- mutating / read-only 请求的 P99 延迟（按 request scope 区分：cluster / namespace / resource-level）；
- 通常 apiserver 会暴露专门的 SLI histogram（如 `apiserver_request_sli_duration_seconds_bucket`，**指标是否存在请先在目标节点 `/metrics` 核对**）。

```
# 通用版本（可直接用）
histogram_quantile(0.99, sum by (le) (rate(apiserver_request_duration_seconds_bucket{verb!="WATCH"}[5m])))
# 分 scope 观察
histogram_quantile(0.99, sum by (scope, le) (rate(apiserver_request_duration_seconds_bucket{verb!="WATCH"}[5m])))
# 写操作单独看
histogram_quantile(0.99, sum by (le) (rate(apiserver_request_duration_seconds_bucket{verb=~"POST|PUT|PATCH|DELETE"}[5m])))
```

E11. 控制面故障的三类典型剧本

剧本	现象	首要动作
etcd NOSPACE	全部写失败	B3：compact → defrag → alarm disarm
etcd 慢/频繁切主	全集群 API 延迟高、大量重试	查 fsync/commit/Peer RTT；查磁盘和网络
证书过期	组件互相 TLS 失败、无人/auth 失败	轮换证书（**平时就该有提前告警**）
Webhook Fail 卡全集群	所有变更被拒	立即删除/禁用该 ValidatingWebhookConfiguration
APF 配错误伤	关键组件 timeout	删除/修正 FlowSchema-PriorityLevel
LB 导到坏 apiserver	部分请求失败	健康检查用 `/readyz`，确认 LB 已摘除坏实例

E12. 给你一个 2000 节点的新集群，控制面怎么设计基线？（综合题）

按顺序答：

1. **问 SLO**：可用性目标？RPO/RTO？业务对控制面抖动的容忍度？
2. **拓扑**：apiserver ≥ 3 副本跨可用区 + LB 用 `/readyz`；etcd 3/5 成员、**同城低延迟**、独立节点与独占磁盘（NVMe）。
3. **容量规划**：对象数、Event 量、list/watch 主来源；是否 Event 独立 etcd。
4. **参数基线**：`--max-requests-inflight/-mutating`、`--watch-cache-sizes`、`--event-ttl`、`--auto-compaction-*`、`--quota-backend-bytes`。
5. **APF**：为 kubelet/scheduler/controller/leader-election/探针保留优先级。
6. **安全**：关闭匿名、最小 RBAC、NodeRestriction、Secret 加密、审计策略、webhook 作用域限制。
7. **可观测**：apiserver/etcd/controller-manager/kube-proxy 指标 + 8 条核心告警（见 G 篇）。
8. **备份**：etcd 定时快照 + 定期恢复演练 + Velero 细粒度备份。
9. **演练**：GameDay（见 H 篇）。
10. **变更治理**：E7 的升级顺序 + 灰度 + 回滚预案，全部落到工单与脚本。

Part F · 反模式清单（可直接用于生产巡检）

#	反模式	证据（怎么发现）	等级	修复动作	验证方式
F1	图省事给 `cluster-admin`	RBAC 审计里大量通配绑定	🔴 高	收敛到最小权限 ClusterRole/Role	权限审计脚本通过
F2	Admission Webhook `failurePolicy: Fail` 且作用于所有 ns（含 kube-system）	webhook 挂 ⇒ 全集群变更失败	🔴 高	加 namespaceSelector 排除系统 ns + 多副本 + 短超时	停掉 webhook 副本，系统 ns 仍可变更
F3	控制器全集群大 list（无 selector/无分页）	`apiserver_response_sizes` 巨大、CPU 高	🔴 高	informer + selector + 分页	`apiserver_request_duration_seconds` 下降
F4	未配置自动压缩/定期 defrag	`etcd_db_total_size_in_bytes` 持续爬升	🔴 高	配 auto-compaction + 定期 defrag 演练	空间占用回到稳定水位
F5	etcd 与 workload 混部 / 共享机械/节流盘	fsync P99 高、慢日志	🔴 高	独立节点 + NVMe	fsync P99 达标（实测）
F6	没有 etcd 备份或从未演练恢复	无快照文件/无恢复记录	🔴 高	定时快照 + 定期 restore 演练	演练记录中恢复成功且计时达标
F7	证书无过期告警	证书到期突然全集群不可用	🔴 高	证书有效期监控 + 提前轮换	告警在到期前 30 天触发
F8	APF 未配置（全靠总量限流）	某客户端打满限额饿死 scheduler/kubelet	🟠 中	配 FlowSchema 保障关键组件	压测吵闹邻居时关键组件不受影响
F9	Event 无 TTL / 无限速	Event QPS 巨大	🟠 中	`--event-ttl` + EventRateLimit + 独立 etcd	Event 写入速率回归合理区间
F10	大规模仍用 iptables kube-proxy	规则同步慢、CPU 尖峰	🟠 中	切 IPVS / eBPF	`kubeproxy_sync_proxy_rules_duration_seconds` 下降
F11	Secret 未静态加密	合规不达标	🟠 中	EncryptionConfiguration（注意 KMS 可用性）	etcd 快照中 secret 非明文
F12	`--profiling=true` 或暴露不必要端口	安全扫描告警	🟠 中	关闭 profiling、收紧安全组	端口探测与基线一致
F13	控制器并发参数盲目调大	apiserver/etcd 被打爆	🟠 中	与容量一起压测后再定	压测无 5xx 与 etcd 慢日志
F14	HPA 缩容无稳定窗口	副本数反复横跳	🟡 低	配 `stabilizationWindowSeconds`	观察副本数曲线平稳
F15	直接升级/维护 etcd 未先快照	出问题无法回退	🔴 高	每次变更先备份 + 一次改一个成员	变更工单含备份与回滚步骤

Part G • 可观测性：指标与告警（可直接落地）

G1. 指标清单（请先 `curl -k https://<apiserver>:6443/metrics`、`curl http://<node>:2382/metrics`（etcd）核对后再上线）

组件	指标	用途
apiserver	`apiserver_request_duration_seconds` / `_ request_total`	延迟与 QPS（按 verb/resource/code/scope）
apiserver	`apiserver_current_inflight_requests` / `_mutating`	并发与限流
apiserver	`apiserver_flowcontrol_dispatched_requests_total` / `_rejected_requests_total` / `_current_inqueue_requests`	APF 健康
apiserver	`apiserver_cache_list_duration_seconds` / `_watch_duration_seconds`	watch cache 命中与重放
apiserver	`apiserver_admission_webhook_admission_duration_seconds` / `_fail_open_count`	准入 webhook 是否拖慢/失效
apiserver	`apiserver_audit_event_total` / `_request_total` / `_level_total`	审计健康与成本
apiserver	`apiserver_storage_objects`（可用性依版本）	对象数量趋势
etcd	`etcd_disk_wal_fsync_duration_seconds` / `_backend_commit_duration_seconds`	**最重要的磁盘指标**
etcd	`etcd_server_leader_changes_seen_total` / `etcd_server_has_leader`	选主稳定性
etcd	`etcd_network_peer_round_trip_time_seconds`	Peer 网络 RTT
etcd	`etcd_db_total_size_in_bytes` / `etcd_server_quota_backend_bytes`	容量
etcd	`etcd_server_proposals_failed_total` / `_pending` / `etcd_server_slow_applies_total`	写入健康
controller-manager	`rest_client_*`、`queue depth/workqueue_*`（按实现）	客户端行为与队列积压
kube-proxy	`kubeproxy_sync_proxy_rules_duration_seconds` / `_last_timestamp_seconds`	规则同步健康

G2. Prometheus 告警规则（可直接粘贴，阈值需实测校准）

```yaml
groups:
  - name: k8s-controlplane
    rules:
      - alert: ApiserverDown
        expr: up{job="apiserver"} == 0
        for: 2m
        labels: {severity: critical}

      - alert: ApiserverHighLatency
        expr: |
          histogram_quantile(0.99,
            sum by (resource, verb, le) (rate(apiserver_request_duration_seconds_bucket{verb!="WATCH"}[5m]))) > 4
        for: 10m
        labels: {severity: warning}
        annotations:
          summary: "{{ $labels.verb }} {{ $labels.resource }} P99 达 {{ $value }}s"

      - alert: ApiserverHighErrorRate
        expr: |
          sum(rate(apiserver_request_total{code=~"5.."}[5m]))
            / clamp_min(sum(rate(apiserver_request_total[5m])), 0.001) > 0.01
        for: 10m
        labels: {severity: critical}

      - alert: EtcdNoLeader
        expr: etcd_server_has_leader == 0
        for: 2m
        labels: {severity: critical}

      - alert: EtcdLeaderFrequentChange
        expr: increase(etcd_server_leader_changes_seen_total[30m]) > 3
        for: 5m
        labels: {severity: critical}

      - alert: EtcdDiskWalFsyncSlow
        expr: histogram_quantile(0.99, rate(etcd_disk_wal_fsync_duration_seconds_bucket[5m])) > 0.01
        for: 10m
        labels: {severity: warning}
        annotations:
          summary: "etcd WAL fsync P99 {{ $value }}s (建议 < 0.01s) "

      - alert: EtcdBackendCommitSlow
        expr: histogram_quantile(0.99, rate(etcd_disk_backend_commit_duration_seconds_bucket[5m])) > 0.025
        for: 10m
        labels: {severity: warning}

      - alert: EtcdQuotaNearlyFull
        expr: etcd_db_total_size_in_bytes / etcd_server_quota_backend_bytes > 0.8
        for: 30m
        labels: {severity: critical}
        annotations:
          summary: "etcd 数据库用量已达配额 {{ $value | humanizePercentage }}，尽快 compact/defrag"

      - alert: AdmissionWebhookSlow
        expr: |
          histogram_quantile(0.99,
            sum by (name, le) (rate(apiserver_admission_webhook_admission_duration_seconds_bucket[5m]))) > 0.5
        for: 10m
        labels: {severity: warning}
        annotations:
          summary: "准入 webhook {{ $labels.name }} P99 耗时 {{ $value }}s，可能拖慢所有变更"

      - alert: ApfRequestsRejected
        expr: sum by (priority_level) (rate(apiserver_flowcontrol_rejected_requests_total[5m])) > 0.1
        for: 10m
        labels: {severity: warning}
        annotations:
          summary: "APF 优先级 {{ $labels.priority_level }} 开始拒绝请求"

      - alert: KubeProxySyncStalled
        expr: time() - kubeproxy_sync_proxy_rules_last_timestamp_seconds > 300
        for: 5m
        labels: {severity: critical}
        annotations:
          summary: "实例 {{ $labels.instance }} 的 kube-proxy 已 5 分钟未同步规则"
```

🔗 **关联阅读**：第二篇 · 6.4 关键告警规则（Prometheus）

G3. 建议的控制面 SLO（初始建议值，必须实测校准）

SLI	建议目标（起点）
API 可用性（非 5xx 比例，按 5 分钟窗口）	≥ 99.9%
读请求 P99（cluster-scoped 除外）	≤ 1s
写请求 P99	≤ 2s
集群限定范围读 P99	≤ 5s
etcd WAL fsync P99	≤ 10ms
etcd leader 变更次数（30 天）	≤ 3

SLI	建议目标（起点）
控制面 RTO（灾难恢复）	≤ 1h（按演练实测）
控制面 RPO	≤ 15min（按备份频率）

> 错误预算：30 天滚动窗口，burn rate 14.4×（短期紧急）/ 6×（长期工单）两级升级。

Part H · 演练（GameDay）计划表

场景	注入方式	期望表现	验证的 SLO
apiserver 单副本挂	停一个 Pod/进程	LB 摘除，业务无感；剩余副本够承载	控制面可用性
etcd 单成员挂	停一个成员	quorum 仍在，写不中断；成员能重新加入	容错 1 成员
etcd WAL 变慢	人为加 IO 压力	告警先触发，业务降级可控	磁盘 SLI
etcd NOSPACE	写满/触发 alarm	运维按 B3 流程在 RTO 内恢复	恢复流程有效性
etcd 恢复演练	用快照 restore 到隔离环境	数据一致，时间点符合 RPO	RPO/RTO
证书到期	临时替换短有效期证书	告警提前触发且轮换流程可用	证书治理
Webhook Fail	故意停用 webhook 服务	因为 scope 限制，核心 ns 不受影响	变更可用性
吵闹邻居	起一个疯狂 list 的 Pod	APF 保住 kubelet/scheduler 配额	APF 有效性
kube-proxy 挂	停 DaemonSet Pod	已有通信不断，变更不同步；能被发现	数据面可观测
控制面网络分区	模拟 apiserver 与 etcd 不通	快速定位，有明确告警	MTTR

Part I • 10 张面试速记卡（临阵磨枪）

1. **唯一入口**：apiserver 是**唯一**与 etcd 交互的组件，本身**无状态**，可水平扩容；真正的瓶颈永远在 **etcd 磁盘**。
2. **请求主线**：认证 → 审计 →（模拟）→ 授权 → 准入 → 校验 → 存储；**审计在授权之前**，所以被拒绝的请求也会被记录。
3. **APF**：用 FlowSchema + PriorityLevel 给关键组件留额度，替代"总量限流饿死邻居"的老做法。
4. **resourceVersion 就是 etcd revision**：更新必须带 RV，冲突 ⇒ **409**；watch RV 太老 ⇒ **410 Gone**，必须 relist。
5. **list 一致性**：不带 RV = quorum read（强一致、贵）；`resourceVersion=0` = 读 apiserver watch cache（快、可能略旧）。
6. **etcd 三条命**：WAL fsync P99（\<10ms）、quota 用量（\<80%）、leader 变更（要极少）。NOSPACE = 全集群只读。
7. **etcd 操作铁律**：一次改一个成员；每次变更前先快照；**版本回退基本不可行**。
8. **控制器范式**：informer + workqueue + **幂等 reconcile（level-triggered）** + 退避重排 + resync 兜底。
9. **节点故障时间账**：lease 10s → grace 50s → NotReady taint → Pod 默认容忍 300s ⇒ **约 5~6 分钟**才重调度。
10. **kube-proxy 不转发包**，它只下规则（iptables/IPVS/nftables/eBPF）；挂了**已有规则还在**，只是不再同步变更。

第四篇 • Informer 与 client-go

> Reflector · DeltaFIFO · Indexer · workqueue

Part 0：答题方法论

面试官考察 Informer，本质上看三件事：

考察维度	表现出来的是	高分回答的特征
组件拆解	能不能说出 Reflector / DeltaFIFO / Indexer / Controller / Processor 各自的职责	能画出数据流向，并说清"为什么需要 DeltaFIFO 这一层"
语义精确	resync vs relist 是否混淆	能明确指出 **resync 不产生任何 APIServer 请求**
规模意识	是否知道 15 万 Pod 时 informer 会怎样	能量化内存模型：进程数 × 对象数 × 单对象大小，并给出 Transform / selector / 分片方案

答题模板

1. 一句话定位（解决什么问题）
2. 关键机制（2–3 个核心技术点，最好带组件名）
3. 大规模下的失效模式
4. 可观测指标 + 修复动作 + 回滚条件

Part 1：架构与核心组件（Q1–Q16）

Q1：Informer 解决什么问题？为什么不能直接轮询 APIServer?

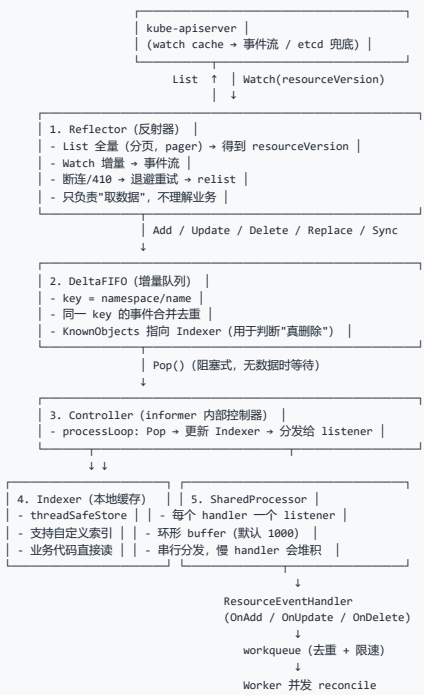
答案要点

- **问题**：控制器需要持续知道"集群里现在有什么"。朴素做法是每个控制循环都 `GET/LIST` 一次 APIServer。
- **代价**：N 个控制器 × 每秒 M 次轮询 = 请求量随客户端数线性放大；且绝大多数轮询是**无效读**（没变更）。
- **Informer 的解法**： **List 一次**拿全量 + `resourceVersion` **Watch 长连接**接收增量事件（推模式） **本地缓存（Indexer）**：业务代码从内存读，**零 API 请求**
- **收益**：把读压力从 `O(客户端数 × 轮询频率)` 降到 `O(实际变更数)`；同时本地查询是**微秒级**，控制器吞吐大幅提升。

加分项

- 这正是 Kubernetes "声明式 API + 控制循环" 能成立的性能基础：**APIServer 从不主动调用任何人，一切由 list-watch 驱动。**

Q2：画出 Informer 的完整架构与数据流向。（必考）



一句话记忆：Reflector 取、DeltaFIFO 攒、Controller 转、Indexer 存、Processor 发。

Q3：Reflector 具体做什么？watch 断了会怎样?

核心职责

- 调用 `ListerWatcher.List()` 拿全量对象 → `DeltaFIFO.Replace()`

- 调用 `ListerWatcher.Watch(resourceVersion)` 拿增量 → `DeltaFIFO.Add/Update/Delete`
- 维护 `lastSyncResourceVersion`，保证断连后能从断点续传

断连处理（关键）

情况	行为
网络抖动、连接被 LB 切断、APIServer 重启	watch 返回 error → **指数退避重试 Watch**（用同一个 RV 续传，不 relist）
返回 `410 Gone` / `ErrExpired`（RV 太旧）	**触发 relist**（全量 List → Replace）
APIServer 返回 429	客户端退避（client-go 令牌桶 + APF 的 `Retry-After`）
请求超时	重试

退避参数（client-go Reflector 的 backoffManager，【默认值】，**请以目标版本源码核对**）

- 初始延迟约 **800ms**，最大约 **30s**，倍增系数 **2.0**，重置周期约 **2min**。

watch 超时设计（面试加分）

- client-go 给每个 watch 请求设置 **随机超时**（基础值约 5 分钟，实际取值在 5–10 分钟区间随机，【默认值】）。
- **目的**：避免所有 Reflector 在同一时刻重连造成惊群（thundering herd）。源码注释明确说明这是"故意不与 APIServer 用同一个超时"。

Q4：DeltaFIFO 是什么？为什么需要它，而不是直接把事件塞给 handler?

答案要点

DeltaFIFO 是一个 **按 key 去重的增量队列**，元素为 `Deltas{ Type, Object }`，`Type` 有五种：

类型	产生时机	语义
`Added`	watch 收到 ADDED	新对象
`Updated`	watch 收到 MODIFIED	对象变更
`Deleted`	watch 收到 DELETED / relist 时缓存中存在但新列表中没有	对象消失
`Replaced`	List 全量结果（relist）	全量同步（较新版本引入，旧版本用 `Sync`）
`Sync`	resync 触发	周期性重新投递，**不代表对象真的变了**

为什么需要它（三个核心价值）

1. **去重**：高频变更的对象（如 Pod status 每秒变）在队列里只保留累积的 delta，避免 handler 被淹没 —— 但注意：**同 key 的多个 delta 会合并为一个 Deltas 列表，handler 会按顺序全部处理，不会真的丢弃中间状态**（这点常被误解）。
2. **保证顺序**：同一 key 的事件严格按序处理（FIFO），避免旧状态覆盖新状态。
3. **兜底删除检测**：`Replace()` 时会对比 `KnownObjects`（Indexer）与新列表，**新列表中缺失的对象会补发 Deleted 事件** —— 这是解决"watch 断连期间删除事件丢失"的核心机制。

Q5：Indexer / ThreadSafeStore 是什么？怎么用自定义索引?

- **Indexer** = 本地缓存 + 索引。底层是 `threadSafeStore`（`map[key]object` + `map[indexName]map[indexValue]set[key]`），读写锁保护。
- 默认 key 函数：`MetaNamespaceKeyFunc` → `namespace/name`（无 namespace 则只有 name）。
- 默认索引：`namespace`（按 namespace 反查）。

自定义索引示例（见 Part 6.4） 常见用途：按 `spec.nodeName` 索引 Pod、按 `spec.serviceAccountName` 索引。

面试追问：Indexer 里的对象是**深拷贝**还是引用?

- 存入时 client-go 会做浅拷贝/深拷贝处理（`DeepCopy`），但**业务代码从 Indexer 取出的对象不应直接修改**（会污染缓存），必须先 `DeepCopy()` 再改。这是高频踩分点。

Q6：Informer 里的 "Controller" 和 Kubernetes 的 "Controller" 是一个东西吗?

不是，这是命名上的经典混淆：

名称	位置	职责
`cache.Controller`（informer 内部）	`client-go/tools/cache/controller.go`	驱动 Reflector + DeltaFIFO，把事件落到 Indexer 并分发给 handler。**不含业务逻辑**
业务 Controller / Reconciler	用户代码	注册 handler → 入队 → 执行调谐逻辑

`NewInformer` / `NewIndexerInformer` 内部都会创建这个 `Controller`。

Q7：`SharedInformer` 为什么叫 "Shared"？共享了什么？（高频）

共享的是：同一个资源类型的 **一个 Reflector + 一个 DeltaFIFO + 一个 Indexer**（即一条 watch 连接和一份缓存），多个 handler 通过 `SharedProcessor` 共享同一份事件流。

价值

- 一个进程内 10 个控制器都关心 Pod → **只有 1 次 List、1 条 Watch、1 份缓存**，而不是 10 份。
- 大规模集群下这是**最重要的内存/连接优化手段。**

用法

```
factory := informers.NewSharedInformerFactory(clientset, 0) // 0 = 不自动 resync
podInformer := factory.Core().V1().Pods().Informer()
podInformer.AddEventHandler(handlerA)
podInformer.AddEventHandler(handlerB) // 同一个 informer，共享缓存
factory.Start(ctx.Done())
factory.WaitForCacheSync(ctx.Done())
```

注意：`SharedInformerFactory` 有 `Shutdown()`；调用后需重新创建 factory 才能再次 start（informer 不支持 restart）。

Q8：`AddEventHandlerWithResyncPeriod` 的 resync period 是什么语义？

- 可为**单个 handler** 设置不同的 resync 周期；`SharedInformerFactory` 的整体 resyncPeriod 与所有注册 handler 的周期取**最小值**驱动。
- resync 到达时，把 Indexer 中**所有对象**以 `Sync` 类型重新入队 → 触发对应 handler 的 `OnUpdate`。
- **关键**：resync **不会**触发 APIServer 请求（见 Q18）。

Q9：`WaitForCacheSync` / `HasSynced` 的语义？为什么必须在 Start 之后调用？

- `HasSynced()` 返回该 informer 的 DeltaFIFO 是否已完成**首次全量同步**（即第一次 List 的结果已全部 Pop 并写入 Indexer）。
- `WaitForCacheSync(stopCh)` 阻塞等待所有已注册 informer 同步完成；返回 `map[type]bool`。
- **必须先 `Start()` 再 `WaitForCacheSync()`**，否则永远等不到（informer 没启动）。
- **语义边界**：只代表"**首次**同步完成"。之后 relist / 断连重连**不会**让 HasSynced 变回 false，业务代码也不应依赖它判断"数据是否最新"。
- **反模式**：`WaitForCacheSync` 返回 false 却继续执行 → 控制器基于空缓存工作 → 会误判"对象不存在"而**疯狂创建/删除**。必须检查返回值并处理。

Q10：ResourceEventHandler 的三个回调语义？UpdateFunc 的 old/new 从哪来？

```
type ResourceEventHandler interface {
    OnAdd(obj interface{}, isInInitialList bool) // 较新版本增加 isInInitialList
    OnUpdate(oldObj, newObj interface{})
    OnDelete(obj interface{})
}
```

- **OnAdd**：首次 List 期间的每个对象都会回调一次；之后 watch 到新对象也回调。 `isInInitialList=true` 用于区分"启动时的存量对象"与"运行期新增"（**重要**：避免启动时对 15 万个存量对象做全量业务动作）。
- **OnUpdate**：`oldObj` 来自 Indexer 中的旧值，`newObj` 是新值。resync 时也会回调（old == new）。
- **OnDelete**：对象从缓存移除时回调。 **陷阱**：收到的是 `DeletedFinalStateUnknown`（包含 `Key` + `Obj`）的情况 —— 当 DeltaFIFO 无法拿到最终状态（如 watch 断连期间的删除）时，只能拿到 key。业务代码**必须处理这个类型**，否则会漏删外部资源（如云盘、DNS 记录）。

Q11：workqueue 的作用？去重、限速、延迟分别怎么实现？

核心能力

1. **去重**：同一 key 在队列中只保留一份（正在处理时再次 Add 只会标记 dirty，处理完 `Done()` 后重新入队）。天然避免"同一对象被并发 reconcile"。
2. **限速（RateLimiter）**：控制 key 重新入队的节奏。 默认组合（`DefaultTypedControllerRateLimiter`，【默认值】，以源码为准）： `ItemExponentialFailureRateLimiter(5ms, 1000s)`：失败次数越多，延迟指数增长 `ItemBucketRateLimiter(10 qps, burst 100)`：整体令牌桶兜底
3. **延迟入队**：`AddAfter(key, duration)`（延迟队列）。
4. **忘记失败计数**：`Forget(key)`（成功后清除退避计数）。
5. **优雅关闭**：`ShutDownWithDrain()`（先排空再关闭，用于原地升级/重启不丢任务）。

关键纪律

- `Get()` 后**必须** `Done(key)`，否则该 key 永远被认为"正在处理"，不再入队 → **静默卡死**（这是最经典的 Operator bug）。
- 处理失败要 `AddRateLimited(key)` 而非 `Add(key)`。
- 处理成功要 `Forget(key)`。

Q12：为什么 handler 里只应该做 `enqueue`，不能做业务逻辑？（高频）

原因（三个层次）

1. **阻塞分发**：`SharedProcessor` 对每个 listener 用**单个 goroutine 串行分发**。handler 慢 → 该 listener 的 buffer 堆积 → 事件延迟。
2. **内存增长**：`processorListener` 有一个**环形 buffer（【默认值】1000）**，满了之后新通知会进入**无界的** `pendingNotifications` **切片** → handler 持续慢会导致内存线性上涨直至 OOM。
3. **阻塞整个 informer**：分发是同步调用链的一部分，慢 handler 会拖慢 DeltaFIFO 的消费速度，进而影响**所有** listener（因为 processor 是同步遍历的）。

正确姿势

```
func (c *Ctrl) OnAdd(obj interface{}, _ bool) {
    key, _ := cache.MetaNamespaceKeyFunc(obj)
    c.queue.Add(key) // 只入队，微秒级返回
}
// 真正的业务逻辑放在 worker goroutine 里，从缓存取对象处理
```

Q13：Informer 的 goroutine 模型是怎样的？一个进程有多少 goroutine？

粗略模型（每个 informer）：

- 1 个 `Reflector.Run` goroutine（List/Watch 循环）
- 1 个 `processLoop` goroutine（Pop 并分发）
- **每个注册的 handler 1 个** `processorListener.run` + 1 个 `pop` goroutine
- 用户自起的 N 个 worker goroutine

面试考点：注册 handler 越多，goroutine 越多；且**每个 handler 都要完整遍历一遍所有事件**，所以 handler 数量直接放大 CPU 开销。大规模下应尽量复用同一个 handler/controller。

Q14：`dynamic` / `unstructured` / `metadata-only` informer 的区别与取舍？

类型	返回对象	内存	类型安全	适用
Typed informer	`*v1.Pod` 等	中（但可 Transform）	✅	常规控制器（首选）
Dynamic informer	`*unstructured.Unstructured`	**大**（map[string]interface{}，比结构体膨胀数倍）	❌	需要处理任意 CRD（如多集群同步、通用审计）
Metadata-only	`*metav1.PartialObjectMetadata`	**极小**（仅 metadata）	部分	只关心存在性/标签/命名空间（如配额统计、NS 生命周期）

大规模建议

- 只关心 metadata 的场景用 `DynamicSharedInformerFactory` + `metadata` GVR（`metav1.APIResource{Name:"pods", ...}` 走 `metadata.k8s.io` 的 PartialObjectMetadata）→ 请求头带 `Accept: ...;as=PartialObjectMetadataList`，**APIServer 侧和客户端侧都省。**
- 避免用 unstructured 缓存大资源（内存和 GC 压力显著）。

Q15：controller-runtime 的 `Manager` / `Cache` 与 client-go informer 的关系？

- controller-runtime 的 `cache.Cache` 是对 `SharedInformerFactory` 的**封装**，按 GVK 懒创建 informer（`GetInformer(ctx, obj)`）。
- `DelegatingClient`：**读（Get/List）→ 走 Cache（本地内存）**，零 API 请求 **写（Create/Update/Delete/Patch）→ 走 APIServer**
- `manager.Start()` 内部会启动所有 informer 并 `WaitForCacheSync`。
- **Leader 选举**：controller-runtime 默认**只有成为 leader 后才启动 cache**（`LeaderElection` + `LeaderElectionReleaseOnCancel`）。非 leader 副本不启动 informer → **大规模下避免了 N 副本 × 全量缓存的浪费**（对比：controller-runtime 之前的版本所有副本都 watch）。

关键配置（1.15+）

```
cache.Options{
    ByObject: map[client.Object]cache.ByObject{
        &corev1.Pod{}: {
            Label: labels.SelectorFromSet(labels.Set{"app": "myapp"}),  // 只缓存部分
            Transform: func(obj interface{}) (interface{}, error) { ... }, // 裁剪字段
        },
    },
    DefaultTransform: ...,
}
```

Q16：Informer 的事件顺序与一致性保证到什么程度？

- **单 key 顺序保证**：DeltaFIFO 保证同一 key 的事件按到达顺序处理。
- **跨 key 无顺序保证**：Pod A 和 Pod B 的处理顺序不确定，控制器**不能依赖跨对象顺序。**
- **不保证"每个变更都被 handler 看到"**：高频变更对象的中间状态可能被合并（但同 key 的 delta 列表会依次处理，仍是最终一致）。
- **最终一致**：断连期间的变更由 relist + `Replace` 兜底对齐。
- **缓存不是强一致**：读本地缓存可能读到几百毫秒前的旧值；对一致性要求高的操作（如写前校验）应直接读 APIServer（`client.APIReader` 或 `cache.Uncached`）。

Part 2：事件与生命周期语义（Q17–Q30）

Q17：List 阶段 informer 是怎么做的？分页吗？

- Reflector 用 `pager` 做**分页 List**（【默认值】每页 500 个对象；可通过 `Reflector.WatchListPageSize` 覆盖，**默认值请核对源码**）。
- 请求带 `resourceVersion=0` → 让 APIServer **走 watch cache**，不压 etcd（这是关键设计：informer 的初始化应该命中缓存而非打穿到 etcd）。
- 全量结果 → `DeltaFIFO.Replace(list, rv)` → 之后用该 `rv` 建 watch。

大规模影响

- 15 万 Pod / 500 = **300 次 List 往返**；单次 List 大对象会吃内存。
- 调大页大小可减少往返次数，但会增大单次内存峰值 —— **【需实测】权衡。**

Q18：resync 和 relist 的区别？（经典高频，最容易答错）

维度	resync（重同步）	relist（重列）
触发	定时器（`resyncPeriod`）	watch 断连且 RV 过期（410 / ErrExpired）、首次启动
是否请求 APIServer	❌ **完全不请求**	✅ 全量 List 请求

维度	resync（重同步）	relist（重列）
数据来源	本地 Indexer 中已有对象	APIServer
事件类型	`Sync`	`Replaced` / `Sync`（旧版本）
目的	周期性把对象重新投递给 handler，兜底"事件丢失/处理失败"	重新建立基线，对齐断连期间的变更
代价	本地 CPU + 队列压力	**APIServer / etcd 压力**（惊群风险）

> **面试必背**：**resync 不产生任何 APIServer 请求**。很多人误以为 resync 会重新 List，这是错的。

Q19：resync 设成多久合适？设成 0 行不行？

- **设 0（关闭）**：完全事件驱动。优点是无额外开销；缺点是**漏事件无法自愈**（例如 handler 处理失败但没重新入队、外部系统状态漂移）。
- **常见做法**：【建议值】**数小时量级**（controller-runtime 的 `SyncPeriod` 默认约 **10 小时**，【默认值】）。
- **反模式**：设成秒级（如 10s）→ 15 万对象每 10 秒全部重新入队 → **reconcile 风暴**，CPU 和队列被打爆，虽然不打 APIServer，但控制器自己会被压垮。
- **更优实践**：resync 设为 0 或很大，改为在控制器内部做**定期全量对账**（可控节奏 + 可观测）。

Q20：删除事件丢失会怎样？Informer 如何兜底？

场景：watch 连接中断 30 秒，期间某对象被删除 → 重连后不会收到这条 DELETED。

兜底机制

1. 若能用原 RV 续传：APIServer 会正常补齐（etcd 有历史 revision）。
2. 若 RV 已过期 → relist → `DeltaFIFO.Replace()` → 对比 `KnownObjects`（本地缓存）与新列表：**缓存里存在但新列表中没有的对象，补发 Deleted 事件。**
3. 若补发时已拿不到最终对象 → 投递 `DeletedFinalStateUnknown`（只有 key）。

业务代码必须做的

```
func (c *Ctrl) OnDelete(obj interface{}) {
    if d, ok := obj.(cache.DeletedFinalStateUnknown); ok {
        obj = d.Obj // 可能为 nil，只能用 d.Key
        key = d.Key
    }
    ...
}
```

并且：所有对外部系统（云盘、DNS、数据库）的删除操作必须**幂等**，且要能通过"定期全量对账"兜底（对比 K8s 侧与外部侧的存量）。

Q21：为什么 watch 会"假死"？Bookmark 在这里的作用？

- **假死**：TCP 连接未断（LB/防火墙未超时），但**实际上已无数据流动**（例如中间设备静默丢包）。客户端以为连接正常，实际已经收不到事件 → **informer 静默不更新**，控制器看不到任何变化。
- **Bookmark 事件**：APIServer 周期性发送一个**只含 resourceVersion、无对象体**的事件。客户端（需 `allowWatchBookmarks=true`）用它： 更新本地已知最新 RV（重连时可续传，避免 relist） **检测连接存活** —— 超过预期时间没收到任何事件（含 bookmark）→ 判定连接异常，主动断开重建
- **实践**：client-go 较新版本在 watch 初始化时会请求 bookmark；LB / 云厂商的 **idle timeout** 必须大于 bookmark 间隔，否则会被静默切断。

Q22：informer 的 stop channel 与生命周期？能不能重启？

- `factory.Start(stopCh)`： `stopCh` 关闭 → 所有 informer 的 Reflector、processor、processLoop 都退出。
- `SharedInformer` **不支持重启**：一旦停止，必须**重新创建 factory 和 informer**（源码层面没有 restart 语义）。
- **优雅关闭顺序**（生产级）： 关闭业务 worker（等待处理中的 reconcile 完成或超时） `queue.ShutDownWithDrain()`（排空队列） 关闭 informer 的 stopCh 释放外部连接
- **反模式**：只 `close(stopCh)` 而不排空 workqueue → 在途任务丢失（对有外部副作用的控制器是数据丢失）。

Q23：`OnUpdate` 被调用就一定说明对象变了吗？

不一定。两种"假更新"：

1. **resync**：会无条件对所有对象触发 `OnUpdate(old, new)`，且 `old` 与 `new` 内容相同。
2. **无意义变更**：如 `managedFields`、`resourceVersion`、`lastAppliedConfig`、`status` 的无关抖动。

正确做法：在 reconcile 入口做"有意义变更"判断

```
// 1) resync 场景：内容相同直接跳过
if apiequality.Semantic.DeepEqual(oldObj.Spec, newObj.Spec) &&
   oldObj.GetDeletionTimestamp() == nil {
    return
}
// 2) reconcile 入口：比较 Spec/关键注解，而非整对象
// 3) 更彻底：给对象打 hash 注解（spec-hash），变了才处理
```

大规模收益：这一层过滤能砍掉绝大部分无效 reconcile，**【需实测】但通常是数量级差异。**

Q24：如何只 watch 部分对象（label/field selector）？这对大规模有多重要?

```
factory := informers.NewSharedInformerFactoryWithOptions(
    clientset, 10*time.Hour,
    informers.WithNamespace("prod"), // 只看一个命名空间
    informers.WithTweakListOptions(func(o *metav1.ListOptions) {
        o.LabelSelector = "team=platform"
        o.FieldSelector = "spec.nodeName=node-1" // 仅部分字段可用
    }),
)
```

价值（大规模集群最重要的优化之一）

- 内存从"全集群对象"降到"我关心的那部分"
- 事件量同比例下降
- **注意**：selector 下推到 APIServer 侧过滤，但 watch cache 只对部分字段建索引，复杂 selector 会在缓存后遍历（观察 `apiserver_cache_list_fetched_objects_total` 判断是否浪费）。

Q25：缓存里的对象能不能直接修改?

不能。Indexer 中的对象是共享的，直接修改会：

1. 污染本地缓存（其他 handler 读到被改过的对象）
2. 导致后续 `OnUpdate` 的 `oldObj` 与真实历史不一致
3. **如果误传给 Update 请求，可能把本地修改写进集群**（严重事故）

正确做法

```
pod := informer.GetIndexer().GetByKey(key).(*corev1.Pod).DeepCopy()  // 必须 DeepCopy
pod.Labels["x"] = "y"
client.Update(ctx, pod)
```

Q26：Informer 的 List 请求会不会打穿到 etcd?

- 默认带 `resourceVersion=0` → APIServer 从 **watch cache** 返回，**不打 etcd。**
- 若显式指定非 0 的 `resourceVersion` → 走 **quorum read**（强一致，压 etcd）。Reflector 首次 List 用的是 "0"，所以是缓存读。
- **大规模意义**：informer 重启/relist 时，只要 APIServer 的 watch cache 还有数据，就不会打穿 etcd。**这也意味着调大 `--watch-cache-sizes` 能显著降低 relist 的 etcd 压力。**
- WatchList 特性（较新版本）：直接用 `watch + sendInitialEvents` 完成初始化，进一步减少 etcd 全量读（**特性开关与版本支持情况请以目标版本为准**）。

Q27：多个副本（多 Pod）的控制器，informer 怎么工作?

两种模式

模式	行为	代价	适用
全部副本都 watch	每个副本一份全量缓存	N × 内存、N × watch 连接	需要每个副本都能快速接管、读多写少
仅 leader watch（controller-runtime 默认）	只有 leader 启动 cache	切主时有冷启动 List 开销	大规模首选

切主代价：新 leader 需要**全量 List + 建立缓存**，大资源下可能要几十秒到数分钟 → 期间控制器不可用。

- 缓解：缩小缓存范围（selector / Transform / metadata-only）、预留足够内存、监控冷启动耗时。

Q28：Informer 与 APF（限流）的关系？被限流会怎样?

- Informer 的 List 请求属于某个 FlowSchema（通常按 ServiceAccount/用户匹配，如 `service-accounts`、`kube-controller-manager`）。
- 被限流 → List 变慢或返回 **429** → 控制器**启动变慢**（`WaitForCacheSync` 超时）、relist 变慢、事件延迟。
- **危险正反馈**：APIServer 压力大 → 限流 → 客户端 relist 变慢 → 客户端数据更旧 → 客户端重试更多 → APIServer 压力更大。
- **缓解**：为核心控制器配置专属 FlowSchema/高优先级（见 APIServer 篇 Part 6.1），并为客户端设置合理的 `QPS/Burst`。

Q29：如何给 Informer 相关逻辑做单元测试?

层次	工具	说明
reconcile 逻辑	`sigs.k8s.io/controller-runtime/pkg/client/fake`	纯内存客户端，测业务逻辑
需要真实 APIServer 语义	**envtest**（`setup-envtest`）	启动真实 kube-apiserver + etcd，测 informer/webhook/CRD 行为
事件驱动逻辑	直接构造 `DeltaFIFO` / 手动调用 handler	无需集群
集成/规模验证	kind + kubemark / 压测集群	验证规模行为

注意：fake client 不支持 watch 语义，测不出"事件丢失/重放"相关问题，这类必须上 envtest 或真实集群。

Q30：Informer 相关的可观测指标有哪些?

客户端侧（需在控制器中自行暴露）

```
# workqueue (controller-runtime 默认暴露)
workqueue_depth{name} # 队列深度 — 最直接的积压指标
workqueue_adds_total / workqueue_retries_total
workqueue_queue_duration_seconds # 在队列中待了多久
workqueue_work_duration_seconds # 处理耗时
workqueue_unfinished_work_seconds

# controller-runtime
controller_runtime_active_workers{controller}
controller_runtime_reconcile_total{controller,result} # result=error/requeue/success
controller_runtime_reconcile_time_seconds
controller_runtime_max_concurrent_reconciles
```

客户端进程侧

```
process_resident_memory_bytes # 缓存对象数量直接反映在内存上
go_goroutines # handler/goroutine 泄漏
```

APIServer 侧（判断 informer 造成的压力）

```
apiserver_request_total{verb="list",resource=...,client=...}
apiserver_request_total{verb="watch"}
apiserver_current_inflight_requests # watch 长连接占用
apiserver_flowcontrol_rejected_requests_total # 是否被限流
apiserver_watch_events_sizes_bucket # 事件体积
```

建议告警（【建议值】，需按 SLO 调整）

- `workqueue_depth` 持续 > 1000 或单调增长 → 处理不过来
- `controller_runtime_reconcile_total{result="error"}` 错误率 > 1%
- 客户端 RSS 超过预期的缓存规模（说明有泄漏或缓存范围失控）

Part 3：大规模集群高频问题（Q31–Q46）

Q31：大规模集群下，Informer 的三大问题是什么?

问题	表现	根因
① 内存	控制器 Pod RSS 数 GB、OOMKilled	本地缓存 = 全量对象副本，且 进程数 × 对象数 × 单对象大小
② 惊群	APIServer 重启后 etcd 读 QPS 暴涨、List P99 飙升	所有 reflector 同时断连 → 同时 relist
③ 事件风暴	CPU 打满、workqueue 积压、reconcile 延迟分钟级	高频变更资源（Pod status/Lease/Event）+ resync 过密 + 无意义 Update

---

Q32：量化 Informer 的内存开销（面试给算式，别给死数字）

```
单进程缓存内存 ≈ Σ(每种资源的对象数 × 该资源单对象平均大小 × 膨胀系数)
               + DeltaFIFO 队列 + workqueue + Go 运行时开销

膨胀系数（经验）：
  typed struct ≈ 1.0（结构体本身）
  unstructured ≈ 3~5x（map[string]interface{} 开销）【需实测】
  metadata-only ≈ 0.05~0.1x【需实测】
```

举例（示意，不是结论）：全集群 15 万 Pod，单 Pod 对象按结构化表示约数 KB 量级 → 单个控制器进程仅 Pod 缓存就可能是**数 GB**。再乘以副本数（若非 leader-only）→ 直接 OOM。

测量方法（必须实测，不要猜）

```
kubectl top pod <controller> -n <ns>
# 或运行时取 heap
go tool pprof http://<pod>:<port>/debug/pprof/heap
# 看 inuse_space 的 top，通常 cache.threadSafeStore / unstructured 占大头
```

Q33：如何降低 Informer 的内存？（综合题，必考）

按收益排序：

手段	做法	收益
1. 缩小缓存范围	`WithNamespace` + `LabelSelector` + `FieldSelector`	**最高**（对象数直接下降）
2. Transform 裁剪字段	controller-runtime `cache.Options.ByObject[].Transform`，只保留 metadata + 需要的 spec 字段	**很高**（单对象大小可降一个量级）
3. metadata-only	只要 metadata 时用 PartialObjectMetadata	高
4. 避免 unstructured	用 typed 对象	高
5. 仅 leader 启动 cache	controller-runtime 默认行为	中高（减少副本倍数）
6. 按 namespace/租户分片	多控制器分片各自 watch 一部分	中高
7. 不缓存大资源	`client.Options{Cache: &client.CacheOptions{Unstructured: false}}` / `cache.Uncached(...)`	中
8. 减少 handler 数量	复用 SharedInformer	中

Transform 示例见 Part 6.2。 注意：Transform 会**丢失字段**，被裁剪的对象**不能**用于 Update/Patch（会把裁剪后的对象写回 APIServer，导致字段丢失！）。写入时必须重新 Get（走 APIReader）或确保该类型不使用裁剪缓存。

Q34：什么是"惊群（Thundering Herd）"？怎么缓解？（必考）

场景：APIServer 滚动升级 / 网络抖动 / etcd leader 切换 → 所有 Reflector 的 watch 同时断开 → 同时重试 → 大量同时 relist（全量 List）→ APIServer 和 etcd 压力暴涨 → 进一步失败 → **雪崩**。

缓解手段（分层）

层	手段
客户端	client-go 自带**指数退避 + 随机 watch 超时（5–10min）**；确保使用共享 informer（减少连接数）
客户端	relist 时确保带 `resourceVersion=0`（走 watch cache 而非 etcd）
客户端	减少控制器副本数 / 仅 leader 启动 cache
APIServer	调大 `--watch-cache-sizes`，保证 relist 命中缓存
APIServer	`--goaway-chance` 打散长连接，避免负载不均
APIServer	滚动升级 + `--shutdown-delay-duration`，避免多副本同时下线
架构	用 WatchList 特性（较新版本）让初始化走 watch 而非 List
架构	降低"客户端数量"：DaemonSet agent 合并、禁止在业务 Pod 里起 informer

验证方式：演练（非生产）：同时 kill 所有 APIServer → 观察 `apiserver_request_total{verb="list"}` 峰值与恢复时间；目标：**不出现 etcd 读 QPS 超过基线 N 倍**（N 需按集群容量定义）。

Q35：为什么"禁止在业务 Pod 里起 informer"？

- 每个起 informer 的 Pod = **1 条 watch 长连接 + 1 份全量缓存。**
- 1000 个业务 Pod 都 watch 全集群 Pod → **1000 条连接 + 1000 份缓存** → APIServer 连接数和内存直接被打爆，客户端侧内存也爆炸。
- **正确架构**：**控制器模式** —— 中心化部署 1–3 个副本的控制器做 watch + reconcile，业务 Pod 只通过自己的 API/CR 交互。
- **例外**：确实需要本地感知的 agent（如 CNI、CSI node plugin）→ 必须**严格限制 selector**（只看本节点相关对象）+ metadata-only。

面试加分：这是"控制器模式 vs 分布式 agent 模式"的架构权衡题，能说出爆炸半径和连接数模型就很加分。

Q36：高频变更资源（Pod status / Lease / Event）怎么处理？

问题：这些资源变更极频繁，informer 的事件量和 CPU 消耗会被它们主导，而业务往往只关心 spec/存在性。

对策

1. **不要 watch Event**（改用 EventRecorder 写入，或按需查）。
2. **Lease 只在 node/leader-election 场景 watch**，普通控制器不碰。
3. **Pod 用 Transform 裁掉** `status`（只保留 metadata + spec.nodeName 等）。
4. **用** `spec-hash` **注解 / DeepEqual 过滤**：只处理 spec 真正变化的事件。
5. **Event handler 侧做过滤**：`OnUpdate` 里比较 `old.ResourceVersion` 不够（每次都变），要比较业务字段。

Q37：控制器分片（sharding）怎么做？

场景：单一控制器无法处理全量对象（CPU/内存瓶颈）。

方案

方案	做法	优点	代价
按 namespace 分片	每个副本负责一组 namespace（一致性 hash 或静态划分）	简单、缓存范围小	需处理 namespace 增删与再平衡
按对象 key hash 分片	所有副本 watch 全量（或用 selector 无法分片），但 reconcile 时按 key hash 判断归属	天然负载均衡	**缓存仍是全量**，只省 CPU 不省内存
按 label/租户分片	每个副本 watch 不同 label	缓存和 CPU 都降	需要 label 体系支撑
一致性 hash + 协调器	用 CR 记录分片归属	动态再平衡	复杂

关键：**只有缩小了 watch 范围的方案才省内存**；仅按 key hash 分 reconcile 不省内存（缓存还是全量）。这是常见的认知误区。

Q38：Informer 冷启动慢（WaitForCacheSync 超时）怎么排查和优化？

排查

- 观察 List 请求耗时：`histogram_quantile(0.99, rate(apiserver_request_duration_seconds_bucket{verb="list",resource="pods"}[5m]))`
- 对象数量：`kubectl get pods -A --no-headers | wc -l`
- 是否被限流：`apiserver_flowcontrol_rejected_requests_total`

优化

1. 缩小缓存范围（selector / namespace）← **最有效**
2. Transform 减小单对象大小
3. 调大 List 页大小（减少往返，**【需实测】**内存换时间）
4. 确保客户端使用 **protobuf** 而非 JSON（client-go 对内置类型默认 protobuf；自定义客户端需显式配置 `ContentConfig.ContentType`）
5. 提高客户端 `QPS/Burst`（注意别超过 APF 配额）

6. **放宽** `WaitForCacheSync` **的超时，并处理超时返回值**（不能假成功）

Q39：Informer 与控制器"无意义 reconcile"的成本有多高?

成本模型

```
每次 reconcile ≈ 1 次缓存读（微秒）+ 1 次状态判断
              + 概率性的 1~N 次 APIServer 写（毫秒级 + etcd 压力）
```

- 若 15 万对象每 10 秒 resync 一次 → **1.5 万次/秒**的 reconcile → CPU 打满，且任何一次"误判需要写"都会放大成 etcd 写风暴。
- **对策**（见 Q23）：resync 设 0 或数小时；reconcile 入口做语义比较；写前用 `Semantic.DeepEqual` 判断状态是否真的变了。

Q40：什么是"幽灵对象"？如何避免基于过期缓存做危险操作?

- **幽灵对象**：对象在集群中已删除，但本地缓存还没收到删除事件（watch 断连、事件积压）→ 控制器认为它还存在。
- **危险场景**：控制器基于幽灵对象去**删除外部资源**（云盘、DNS）或**创建替代对象**（ReplicaSet 看到 Pod 消失就新建 → 实际 Pod 还在 → 超量创建）。
- **对策** 写操作前对关键对象做**直读 APIServer 校验**（`APIReader`，绕过缓存） 所有删除外部资源的操作必须 **幂等 + 确认存在性** 利用 `DeletionTimestamp` + finalizer 保证顺序 定期**全量对账**（对账周期按业务容忍度设定）

Q41：多集群场景下 Informer 怎么用?

- 每个集群一个 `Manager` / `SharedInformerFactory` + 一个 rest.Config。
- **注意点** 缓存爆炸：N 个集群 × 全量对象 → 必须用 selector/Transform 或 metadata-only 事件风暴：多集群事件汇聚到一个控制面，workqueue 需限速 故障隔离：单集群 APIServer 不可用不应拖垮全局控制器（每个集群独立的 stopCh + 独立的健康检查） 常用库：controller-runtime 的 `mcsv1alpha1` / Cluster API / Karmada / OCM

Q42：Informer 会不会导致 APIServer 的 watch 连接数打爆?

会。模型：

```
watch 连接数 ≈ Σ(每个客户端进程 × 该进程 watch 的资源类型数)
```

- 100 个控制器 Pod × 各 watch 5 种资源 = 500 条连接（可接受）
- 1000 个业务 Pod × 各 watch 3 种 = 3000 条（危险，尤其配合长连接内存开销）
- **APIServer 侧每个 watch = 1 goroutine + buffer**，连接数过多会导致内存上涨和调度开销。

对策：共享 informer、减少客户端数量、中心化控制器、监控 `apiserver_current_inflight_requests`（长连接计入）。

Q43：大规模下 CRD 控制器的特殊注意点?

1. **CR 数量可能远大于内置资源**（百万级）→ 缓存内存是第一瓶颈 → 必须 Transform / 分片 / metadata-only。
2. **CRD 的 status 更新会触发 informer 自己产生的事件** → 若控制器"改 status → 触发 OnUpdate → 又改 status"会形成**无限自激循环**（hot loop）。 必须：只在状态真的变化时写；或用 `Generation` / `observedGeneration` 判断。
3. **conversion webhook 参与**：CR 多版本时每次读写都可能触发 webhook 调用（同步、有 P99 成本）。
4. **finalizer 卡死** → 大量 Terminating CR 堆积在缓存里。

Q44：Informer 与"控制器重启后 CPU 飙高"是什么关系?

现象：控制器重启后 CPU 和 APIServer 压力明显高于稳态，持续数分钟。

原因

1. **冷启动全量 List**（大资源慢）
2. **全量事件回放**：首次 List 的 N 个对象全部以 `Added` 事件触发 handler → N 次 reconcile 一次性涌入 workqueue
3. resync（若配置）叠加

优化

- 用 `isInInitialList` 区分存量对象：启动时只做**轻量登记**，不做重业务动作
- 对存量对象做**分批/限速入队**（`AddAfter` 打散）
- 提高 worker 并发但**限制对 APIServer 的写速率**（避免启动时写风暴）
- 冷启动期设置"静默期"（等待缓存稳定 + 事件消化完成再开始有副作用的操作）

Q45：如何为 Informer 驱动的控制器设定 SLO?

SLO	建议目标	测量
调谐时延（对象变更 → 处理完成）	P99 \< 30s（【建议值】，按业务定）	需自行埋点：事件发生时间 → reconcile 完成时间
调谐成功率	> 99.9%	`controller_runtime_reconcile_total{result}`
队列深度	稳态 \< 100，且不单调增长	`workqueue_depth`
冷启动时间	缓存同步 + 存量处理 \< N 分钟	启动日志埋点

SLO	建议目标	测量
控制器可用性	99.9%（含切主时间）	leader 选举 + readiness

错误预算：调谐失败率超预算 → 冻结控制器发版，先修稳定性。

Q46：讲一个你优化过 Informer 的案例（开放式题框架）

STAR + 数据：

1. **S**：规模（对象数、控制器副本、资源类型）
2. **P**：用指标描述（"控制器 RSS 从 1.2GB 涨到 6GB 并 OOMKilled；workqueue_depth 峰值 8 万；冷启动 12 分钟"）
3. **A**：分层（缩小范围 → Transform → 关闭 resync → 分批入队 → 限速），每步给代码与灰度比例
4. **R**：前后对比（RSS、启动时间、队列深度、APIServer List QPS），并说明回滚条件
5. **L**：沉淀规范（CRD 上线评审必填"预估对象数 + 缓存策略"）

Part 4：故障排查实战（Q47–Q56）

Q47：`WaitForCacheSync` 一直返回 false / 超时？

排查顺序

1. 是否 `Start()` 之后才调 `WaitForCacheSync()`？（最常见：顺序写反）
2. **RBAC**：客户端有没有该资源的 `list` + `watch` 权限？缺权限会持续失败（日志里有 `forbidden`）。
3. 是否被 APF 限流（429）？看 `apiserver_flowcontrol_rejected_requests_total`。
4. List 是否超时（对象太多 / APIServer 慢）？
5. CRD 是否处于 `Established`？CRD 未就绪 → List 报 404 → 永远同步不了。
6. `stopCh` 是否已被提前关闭（context 取消）。

验证：`kubectl auth can-i list pods --as=system:serviceaccount:ns:sa`；看控制器日志的 reflector 报错。

Q48：控制器内存持续上涨直至 OOM？

定位

```
# 1）看是不是缓存本身大（正常）还是泄漏
go tool pprof -inuse_space http://pod:port/debug/pprof/heap
# 2）常见泄漏点
```

原因	特征	修复
缓存范围过大（最常见，非泄漏）	heap top 是 `threadSafeStore` / `unstructured`	selector / Transform / metadata-only
processorListener buffer 堆积	heap 里有大量 notification	handler 只 enqueue，不做慢操作
workqueue 无限增长	`workqueue_depth` 单调增长	检查是否漏 `Done()` / 处理速度不足
goroutine 泄漏	`go_goroutines` 单调增长	检查未关闭的 stopCh、未退出的循环
unstructured 缓存	内存是 typed 的数倍	改用 typed 或 Transform
finalizer 卡死导致对象不释放	缓存里 Terminating 对象堆积	修复 finalizer 处理逻辑

Q49：控制器"卡住"，对象变了但没反应？

决策树

```
控制器无反应
├─ workqueue_depth > 0 但 worker 不动？
│  → 漏了 Done(key)？worker panic？并发数=0？
├─ workqueue_depth == 0 且缓存没更新？
│  → watch 假死（见 Q21）：检查是否收到 bookmark
│  → LB idle timeout 静默切断
│  → Reflector 在退避重试中（看日志 backoff）
├─ 缓存更新了但 handler 没触发？
│  → handler 注册前 informer 已 Start？
│  → OnUpdate 里被"无意义变更"过滤掉了？
└─ 事件有但 reconcile 失败？
   → 看 reconcile_total{result="error"} 与日志
```

watch 假死验证：对比 APIServer 侧 `apiserver_request_duration_seconds{verb="watch"}` 的连接存活情况，或直接看客户端是否长时间无任何事件（含 bookmark）。

Q50：日志大量 `too old resource version` / `410 Gone`？

- RV 过期 → 触发 relist。
- **偶发**：正常（尤其长时间无变更的资源，bookmark 未启用时）。
- **频繁**：说明客户端经常长时间不消费事件，或 APIServer watch cache 太小 / compaction 太激进。
- **处理**： 调大对应资源的 `--watch-cache-sizes` 确保客户端开启 bookmark（`allowWatchBookmarks`） 排查客户端为什么卡住（handler 慢、GC 停顿、CPU 打满） **大规模危害**：大量客户端同时 410 → 同时 relist → 惊群

Q51：日志出现 `DeletedFinalStateUnknown` 怎么处理?

- **含义**：删除事件到达时已拿不到对象的最终状态，只有 key。
- **成因**：watch 断连期间的删除，relist 时通过 Replace 补发，但拿不到对象体。
- **必须处理**：

```
if d, ok := obj.(cache.DeletedFinalStateUnknown); ok {
    // 只有 key，没有 Obj
    namespace, name, _ := cache.SplitMetaNamespaceKey(d.Key)
    // 用 key 去清理外部资源（必须幂等）
}
```

- **反模式**：类型断言只处理 `*v1.Pod` 而忽略这种情况 → **外部资源泄漏**（云盘不释放是最典型的线上事故）。

Q52：控制器启动时把存量对象"重做了一遍"，造成事故?

场景：控制器升级/重启 → 15 万存量对象全部走 `OnAdd` → 触发全量业务动作（如全量重启 Pod、全量调用外部 API）。

对策

1. 用 `OnAdd(obj, isInInitialList)` 区分：存量对象只做**登记/对账**，不做破坏性动作。
2. 首次同步完成后再开启"有副作用"的处理（用 `HasSynced()` 门控）。
3. 对存量处理做**限速 + 分批**（`AddAfter` 打散）。
4. 幂等：即使重放也不会产生副作用。

Q53：APIServer 侧看到大量来自某控制器的 List 请求?

排查

```
sum by (client,resource) (rate(apiserver_request_total{verb="list"}[5m]))
```

常见根因

根因	特征	修复
频繁 relist（410 循环）	list 与 410 交替出现	见 Q50
每个 handler 一份 informer（没共享）	同资源多份	用 SharedInformerFactory
业务代码绕过缓存直接 List APIServer	代码里有裸 `client.List`	改为从缓存读（或用 `APIReader` 但控制频率）
控制器反复崩溃重启	restartCount 高	修复崩溃根因
多副本都启动 cache	副本数 × 全量	仅 leader 启动

Q54：`reconcile` 出现大量 409 Conflict?

- 正常现象（乐观锁），但**比例过高**说明： 多个控制器/副本在改同一对象（分片逻辑有问题，多个副本都认为自己负责） 缓存太旧（Update 用的 resourceVersion 过期）→ 应先重新 Get 热对象被高频并发写
- **处理**：`retry.RetryOnConflict` + 重新 Get（从 **APIReader 直读**，不要用可能陈旧的缓存对象）。
- **监控**：统计 409 占写请求比例，突增报警。

Q55：控制器切主（leader 切换）后长时间不可用?

- **原因**：新 leader 需要冷启动缓存（全量 List）+ 消化存量事件。
- **优化**：缩小缓存范围、Transform、提高 List 并发/页大小、预留内存避免 GC 抖动。
- **监控**：埋点"从成为 leader 到首次成功 reconcile"的耗时，设告警。
- **兜底**：切主期间保持 readiness=false，避免被判定为健康但不干活。

Q56：如何优雅地重启/升级一个 Informer 控制器?

标准流程

1. 新版本 Pod 启动 → 先 `WaitForCacheSync`（readiness 未就绪，不接活）
2. 缓存同步完成 → readiness 就绪 → 参与 leader 竞选
3. 旧 Pod 收到 SIGTERM： 停止接收新任务 `queue.ShutDownWithDrain()` 排空在途任务（带超时） leader 释放（`LeaderElectionReleaseOnCancel=true`，让新副本快速接管） 关闭 informer stopCh
4. **回滚触发条件**：新版本 `reconcile_total{result="error"}` 错误率 > 基线 2 倍，或冷启动超时 → 回滚镜像。
5. **注意**：若控制器有外部副作用，升级前确认**幂等性**，避免新旧版本双写（确保 leader 释放及时）。

Part 5：反模式与风险清单

#	反模式	证据（现象/配置）	等级	修复动作	验证方式
1	在业务 Pod 里起全量 informer	每 Pod 一份缓存 + 一条 watch；APIServer watch 连接数 = Pod 数	高	改为中心化控制器；agent 必须用 selector + metadata-only	`apiserver_current_inflight_requests` 与客户端 RSS 下降
2	未使用 SharedInformerFactory	同进程多个控制器各起一份 informer	高	统一 factory + 共享 informer	进程 RSS 与 goroutine 数下降
3	handler 里做慢业务逻辑	`processorListener` buffer 堆积；内存线性上涨	高	handler 只 enqueue；业务放 worker	buffer 深度归零、内存平稳
4	漏调 `queue.Done(key)`	该 key 永久卡住，不再入队	高	用 `defer queue.Done(key)`	`workqueue_depth` 不出现"有 depth 但无进展"
5	resync 设成秒级	reconcile QPS 与对象数成正比，CPU 打满	中高	设 0 或数小时；改为定期全量对账	CPU 与 reconcile_total 下降
6	忽略 `DeletedFinalStateUnknown`	外部资源（云盘/DNS）泄漏	高	显式处理该类型，删除操作幂等	演练：断网删除对象后恢复，外部资源被正确回收
7	直接修改缓存对象	缓存被污染，甚至误写回集群	中高	取出即 `DeepCopy()`	Code review + 单测
8	用 unstructured 缓存大资源	内存是 typed 的数倍	中高	改 typed / Transform	heap profile 对比
9	缓存全集群对象但只关心一小部分	heap top 是无关对象	高	namespace + label/field selector	缓存对象数（自埋点）下降
10	启动时对存量对象做破坏性动作	升级后全量重启 Pod / 全量调用外部 API	高	`isInInitialList` 区分 + 限速分批	演练：重启控制器，无破坏性副作用
11	控制器自激循环（改 status → 触发 → 再改）	APIServer 写 QPS 持续高位、无限事件	高	只在状态真变时写；用 observedGeneration 门控	写 QPS 回落到预期
12	Transform 裁剪后仍用该缓存对象做 Update	字段被写丢	中高	写入前直读或该类型不裁剪	对比写入前后对象字段完整性
13	多副本全部启动全量 cache	N × 内存	中	仅 leader 启动（controller-runtime 默认）	副本内存总量下降
14	WaitForCacheSync 不检查返回值	空缓存状态下开始 reconcile → 误删/误建	中高	检查返回值，false 则退出或重试	故障注入：RBAC 撤权后控制器应拒绝启动
15	无 workqueue/控制器指标	出问题只能猜	中	暴露 workqueue + reconcile 指标并告警	有 dashboard 与告警

Part 6：可直接套用的代码与配置

6.1 生产级 SharedInformerFactory 骨架

```
factory := informers.NewSharedInformerFactoryWithOptions(
    clientset,
    10*time.Hour, // resyncPeriod【建议值】；设 0 关闭
    informers.WithNamespace(""), // 空=全命名空间；按需收敛
    informers.WithTweakListOptions(func(o *metav1.ListOptions) {
        o.LabelSelector = "app.kubernetes.io/managed-by=my-operator"
    }),
)

podInformer := factory.Core().V1().Pods()

podInformer.Informer().AddEventHandler(cache.ResourceEventHandlerFuncs{
    AddFunc: func(obj interface{}) {
        if key, err := cache.MetaNamespaceKeyFunc(obj); err == nil {
            queue.Add(key) // 只入队
        }
    },
    UpdateFunc: func(old, new interface{}) {
        oldPod := old.(*corev1.Pod)
        newPod := new.(*corev1.Pod)
        // 关键：过滤无意义变更（resync 与 status 抖动）
        if oldPod.GetResourceVersion() != newPod.GetResourceVersion() &&
           apiequality.Semantic.DeepEqual(oldPod.Spec, newPod.Spec) &&
           oldPod.GetDeletionTimestamp().IsZero() == newPod.GetDeletionTimestamp().IsZero() {
            return
        }
        if key, err := cache.MetaNamespaceKeyFunc(new); err == nil {
            queue.Add(key)
        }
    },
    DeleteFunc: func(obj interface{}) {
        // 必须处理 DeletedFinalStateUnknown
        var key string
        if d, ok := obj.(cache.DeletedFinalStateUnknown); ok {
            key = d.Key
        } else if k, err := cache.MetaNamespaceKeyFunc(obj); err == nil {
            key = k
        } else {
            return
        }
        queue.Add(key)
    },
})

factory.Start(ctx.Done())
// 必须检查返回值!
if !cache.WaitForCacheSync(ctx.Done(), podInformer.Informer().HasSynced) {
    return fmt.Errorf("failed to sync pod informer cache")
}
```

6.2 controller-runtime：缩小缓存 + Transform 裁剪

```
mgr, err := ctrl.NewManager(cfg, ctrl.Options{
    Cache: cache.Options{
        // 默认同步周期【建议值】数小时；0 表示不自动 resync
        SyncPeriod: ptr.To(10 * time.Hour),
        ByObject: map[client.Object]cache.ByObject{
            // Pod 只保留需要的字段，大幅降内存
            &corev1.Pod{}: {
                Transform: func(obj interface{}) (interface{}, error) {
                    pod, ok := obj.(*corev1.Pod)
                    if !ok { return obj, nil }
                    // 注意：Transform 后的对象不能用于 Update!
                    trimmed := &corev1.Pod{
                        ObjectMeta: metav1.ObjectMeta{
                            Name: pod.Name,
                            Namespace: pod.Namespace,
                            Labels: pod.Labels,
                            Annotations: map[string]string{}, // 只留需要的 key
                            UID: pod.UID,
                            ResourceVersion: pod.ResourceVersion,
                        },
                        Spec: corev1.PodSpec{
                            NodeName: pod.Spec.NodeName,
                        },
                    }
                    return trimmed, nil
                },
            },
            // 只缓存带特定标签的 ConfigMap
            &corev1.ConfigMap{}: {
                Label: labels.SelectorFromSet(labels.Set{"my-operator/cache": "true"}),
            },
        },
        // 完全不缓存的大资源（读走直连）
        // Uncached 通过 client.Options 配置
    },
})
```

6.3 workqueue 限速器与正确用法

```
queue := workqueue.NewTypedRateLimitingQueue(
    workqueue.NewTypedWithMaxWaitRateLimiter(
        workqueue.NewTypedItemExponentialFailureRateLimiter(5*time.Millisecond, 1000*time.Second),
        workqueue.NewTypedItemBucketRateLimiter(10, 100),
        2*time.Second, // 最大等待上限【建议值】，避免过长退避
    ),
)

func (c *Ctrl) runWorker(ctx context.Context) {
    for c.processNext(ctx) {}
}

func (c *Ctrl) processNext(ctx context.Context) bool {
    key, shutdown := c.queue.Get()
    if shutdown { return false }
    defer c.queue.Done(key) // 必须！否则该 key 永久卡死

    if err := c.reconcile(ctx, key); err != nil {
        c.queue.AddRateLimited(key) // 失败：退避重试
        return true
    }
    c.queue.Forget(key) // 成功：清除退避计数
    return true
}

// 优雅关闭
func (c *Ctrl) Shutdown() {
    c.queue.ShutDownWithDrain() // 排空在途任务，避免丢失
}
```

6.4 自定义 Indexer（按 nodeName 索引 Pod）

```
informer := factory.Core().V1().Pods().Informer()
err := informer.AddIndexers(cache.Indexers{
    "byNode": func(obj interface{}) ([]string, error) {
        pod, ok := obj.(*corev1.Pod)
        if !ok || pod.Spec.NodeName == "" {
            return nil, nil
        }
        return []string{pod.Spec.NodeName}, nil
    },
})
if err != nil { return err }

// 使用
objs, err := informer.GetIndexer().ByIndex("byNode", "node-1")
```

> 注意：Indexer 在 informer **启动前**添加；启动后 AddIndexers 会报错。

6.5 分片控制器（按 namespace hash）

```
// 简单一致性分片：只处理属于自己的 key
func (c *Ctrl) owns(key string) bool {
    h := fnv.New32a()
    h.Write([]byte(key))
    return h.Sum32()%c.totalShards == c.shardIndex
}

// 更省内存的做法：每个分片用 WithNamespace 只 watch 自己负责的 namespace
factory := informers.NewSharedInformerFactoryWithOptions(
    clientset, resync,
    informers.WithNamespace(myNamespaces...),
)
```

6.6 客户端 QPS/Burst（避免被限速拖慢冷启动）

```
cfg := rest.CopyConfig(origCfg)
cfg.QPS = 50 // 【建议值】需结合对象规模与 APF 配额实测
cfg.Burst = 100
```

> **注意**：提高客户端 QPS 会加大 APIServer 压力，大规模场景必须**同时**确认 APF 的 FlowSchema 配额，避免出现"客户端提速 → 被 APF 拒绝 → 更慢"的负反馈。

6.7 客户端侧指标埋点（建议暴露）

```
var (
    reconcileTotal = prometheus.NewCounterVec(prometheus.CounterOpts{
        Name: "myctrl_reconcile_total",
    }, []string{"result"})
    reconcileLatency = prometheus.NewHistogramVec(prometheus.HistogramOpts{
        Name: "myctrl_reconcile_duration_seconds",
        Buckets: prometheus.DefBuckets,
    }, []string{})
    cachedObjects = prometheus.NewGaugeVec(prometheus.GaugeOpts{
        Name: "myctrl_cached_objects",
    }, []string{"resource"})
)
```

> `cached_objects` 是最容易被忽略但极有价值的指标：**它直接暴露缓存范围是否失控**，比内存更早发现问题。

Part 7：面试自检清单

- [ ] 能画出 Reflector → DeltaFIFO → Controller → Indexer → Processor 的数据流
- [ ] 说清 DeltaFIFO 存在的三个理由（去重/顺序/删除兜底）
- [ ] **明确指出 resync 不产生任何 APIServer 请求**，并说清它与 relist 的区别
- [ ] 知道 watch 超时是随机的（5–10min）及其惊群缓解目的
- [ ] 能写出"handler 只 enqueue"的正确骨架，并知道漏 `Done()` 的后果
- [ ] 知道 `DeletedFinalStateUnknown` 必须处理，否则外部资源泄漏
- [ ] 能列出降低缓存内存的 5 种手段，并知道哪些省内存、哪些只省 CPU
- [ ] 说清"按 key hash 分片不省内存"（缓存仍全量）
- [ ] 知道 controller-runtime 默认只有 leader 启动 cache
- [ ] 能给出内存开销的量化算式（对象数 × 单对象大小 × 膨胀系数）
- [ ] 能说出惊群的成因与 4 层以上缓解手段
- [ ] 知道 Transform 后的对象不能用于 Update
- [ ] 能给出冷启动优化与"存量对象破坏性动作"的防护
- [ ] 能把任意优化配上下灰度比例、回滚触发条件与验证指标

免责与边界：本文为面试复习与架构评审材料，不针对任何具体生产集群直接执行操作。所有生产变更请走：变更评审 → 灰度 → 观察 → 回滚预案。文中【默认值】请以目标版本 client-go / controller-runtime 源码为准，【需实测】项请以你的监控与 profile 数据为准。

第五篇 • kube-scheduler 调度

> 调度框架 · 队列 · 抢占 · 打分 · 性能

0. 使用说明与答题方法论

0.1 面试官真正在考什么

层级	考点	表现
L1 会用	知道 requests/limits、亲和、污点怎么写	能背 YAML
L2 懂原理	说得清 Scheduling Framework 12 个扩展点、队列三级结构、抢占流程	能画图、能讲为什么这样设计
L3 有规模感	知道 1000/5000 节点下谁先崩、吞吐瓶颈在哪、怎么量化	能报指标名、说得清 trade-off
L4 有事故经验	能讲清"现象 → 证据 → 定位路径 → 止血 → 根治 → 防复发"	有 PromQL、有灰度、有回滚

> **答题模板（强烈建议每道题都套）**：结论先行 → 讲清机制/数据流 → 给一个可落地证据（配置片段 / 指标名 / 事件字段）→ 补一个坑或边界条件。

0.2 本文档的数值约定

按照 SRE 规范，本文档严格区分三类数字：

- **官方/源码值**：标注出处（如 `KubeSchedulerConfiguration` 字段），并提示用 `kube-scheduler --write-config-to=conf.yaml` 现场核对；
- **建议值（推荐初值）**：标注为 *建议*，必须结合业务实测调整；
- **实测值**：本文档 **不提供任何未实测的性能数字**，凡涉及吞吐/耗时均给出**测量方法**，避免编造。

Part A • 原理篇（22 题）

Q1. kube-scheduler 到底负责什么？职责边界在哪？

一句话：kube-scheduler 只做一件事——**为 `.spec.nodeName` 为空的 Pod 选一个 Node，并把绑定结果写回 API Server**。它不运行容器，不保证 Pod 真的能跑起来。

职责边界（高频送分/送命点）：

事项	是否 scheduler 负责	实际负责人
选节点 + 写绑定	✅ 是	kube-scheduler
`ResourceQuota` 校验	❌ 否	admission controller（ResourceQuota 插件）
`LimitRange` 注入默认值	❌ 否	admission controller
校验 requests ≤ limits	❌ 否	apiserver validation
Node 故障驱逐 Pod	❌ 否	NodeLifecycleController（taint-manager）+ kubelet
资源不足时驱逐 Pod	❌ 否	kubelet eviction manager
抢占低优先级 Pod	✅ 是	kube-scheduler（PostFilter / DefaultPreemption）
容器层面能否起来	❌ 否	kubelet admission + runtime

加分点：正因为 scheduler 不校验 quota，**Pod 被 scheduler 成功绑定后，仍可能在 binding 阶段被 admission/Kubelet 拒绝**（例如 quota 超限、快照卷不满足），这就是 `FailedScheduling` 之外还能看到奇怪失败的原因。

Q2. 讲一次完整调度循环（scheduleOne）的数据流

```
┌──────────────── 一个 profile = 一个 goroutine 串行跑 scheduleOne ────────────────┐
NextPod()  ← 阻塞地从 SchedulingQueue pop 一个 Pod
   │
   ├─ 若 Pod 已在 assumedPod 集合中（重复入队）→ 跳过
   │
   ├─【1】获取 Node 快照 Snapshot（从 scheduler cache 拷贝 NodeInfo 列表，不含已删除 Node）
   │
   ├─【2】SchedulingFramework.RunPreFilterPlugins  ← 单个 PreFilter 返回 NonSuccess 即失败
   │   └─ 典型：PodFitsResources(with PreFilter extenders)、NodePorts、VolumeRestrictions
   │
   ├─【3】findNodesThatFitPod
   │   └─ 并发（默认 parallelism=16）对所有可行节点跑 Filter 插件
   │   └─ percentageOfNodesToScore 生效：达到节点发现比例即提前停止
   │
   ├─【4】若可行节点列表为空 → RunPostFilterPlugins（DefaultPreemption 抢占）
   │   └─ 抢占成功 → nominatedNodeName，回队列；失败 → requeue（backoff）
   │
   ├─【5】若仅 1 个候选节点 → 直接选中（不再打分，快路径）
   │
   ├─【6】RunPreScorePlugins → RunScorePlugins（并发 16）→ RunNormalizeScorePlugins
   │   └─ 加权求和：nodeScore = Σ(pluginScore(plugin, node) * pluginWeight)
   │
   ├─【7】SelectHost：在所有"最高分"节点中**随机**取一个（避免总打偏同一个节点）
   │
   ├─【8】Reserve 阶段调用 **cache.assumePod()**（乐观假设绑定：先把 Pod 记入 NodeInfo 占用资源，仅写调度器本地 cache，不落 API Server）
   │
   ├─【9】RunPermitPlugins（可以让 Pod 进入"等待"状态，如 Gang Scheduling 配额等待、某些流量网关）
   │
   └─【10】异步 goroutine：Bind 阶段（PreBind → Bind → PostBind）
               └─ 调用 API Server 的 binding subresource（或新版直接更新 Pod），
                  失败则 Unreserve（回滚 assume）+ Pod 重新入队
```

关键设计点（要说出来才加分）：

1. **每个 profile 是单 goroutine 串行的。**不是"所有 Pod 并发调度"，而是"一次调度周期内部的**节点遍历**是并发的"。这是单机吞吐上限讨论的根。
2. **Assume/乐观绑定**：Reserve 之后到真正 Bind 成功之间有一段时间窗，此时调度器 cache 已"假装"占用了资源，让**下一个 Pod 不必等 API Server 往返**即可继续调度 —— 这是吞吐优化的核心手法，代价是短暂的 cache 与真实状态不一致。
3. **CycleState**：每个调度周期一个 pod-scoped 状态对象，插件之间传递中间计算结果；它随周期销毁，不跨 Pod。

Q3. Scheduling Framework 有哪些扩展点？作用与典型插件是什么？

这是必考题，表格要能默下来：

#	扩展点	阶段	作用 / 特点	典型内置插件
1	`PreEnqueue`	队列	Pod 进 ActiveQ 前的前置门禁	部分版本结合别的项目用；自定义 gate 常用
2	`QueueSort`	队列	决定 pop 顺序	`PrioritySort`（按 priority 排）
3	`PreFilter`	调度周期	全局预检 \& 预计算；**任何一个返回失败就整体失败**	`NodeResourcesFit` 、 `NodePorts` 、 `VolumeRestrictions` 、 `PodTopologySpread` 、 `InterPodAffinity`
4	`Filter`	调度周期	逐节点过滤（可并发）	`NodeUnschedulable` 、 `TaintToleration` 、 `NodeAffinity` 、 `NodeResourcesFit` 、 `VolumeBinding` 、 `VolumeZone`
5	`PostFilter`	调度周期	**Filter 全失败后才执行**，做抢占	`DefaultPreemption`
6	`PreScore`	调度周期	打分前预计算	`PodTopologySpread` 、 `InterPodAffinity`
7	`Score`	调度周期	逐节点打分 0~100	`NodeResourcesFit` 、 `NodeResourcesBalancedAllocation` 、 `ImageLocality` 、 `PodTopologySpread`

#	扩展点	阶段	作用 / 特点	典型内置插件
8	`NormalizeScore`	调度周期	分数归一化到 0~100	多数 Score 插件自带
9	`Reserve`	绑定周期	预留资源 / assume	`VolumeBinding`
10	`Permit`	绑定周期	批准或延迟绑定（可 `Wait`）	自研 Gang/Coscheduling 插件
11	`PreBind`	绑定周期	绑定前准备（如 VolumeBinding 在这里真正下发 PVC/PV 的绑定请求）	`VolumeBinding`
12	`Bind`	绑定周期	真正写 `nodeName`	`DefaultBinder`
13	`PostBind`	绑定周期	绑定后通知（异步、信息性）	自研

必答的边界：

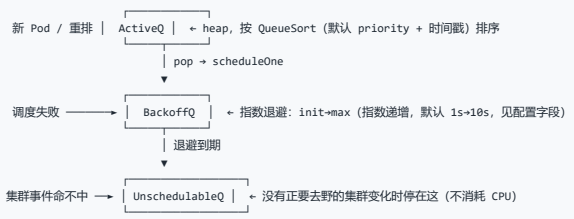
- `PreFilter` 是**全局**的（一次整个周期只跑一次），`Filter` 是**逐节点**的；
- `Reserve` 失败会触发 `Unreserve` 回滚；
- `Permit` 与其他的区别：它可以返回 `Wait` / `Timeout`，把 Pod 挂起而不算失败。

可直接套用的配置片段（`KubeSchedulerConfiguration`）：

```
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
profiles:
  - schedulerName: default-scheduler
    plugins:
      filter:
        enabled:
          - name: NodeResourcesFit
          - name: NodePorts
          - name: NodeAffinity
          - name: TaintToleration
        disabled:
          - name: "*" # 先全关再逐个启用，白名单模式更安全
      score:
        enabled:
          - name: NodeResourcesFit
            weight: 2
          - name: PodTopologySpread
            weight: 2
    pluginConfig:
      - name: NodeResourcesFit
        args:
          scoringStrategy:
            type: MostAllocated # 装箱更紧，降低碎片；在线业务建议 LeastAllocated
            resources:
              - name: cpu
                weight: 1
              - name: memory
                weight: 1
```

> **验证方式**：改完配置后，先 `kube-scheduler --config=conf.yaml --dry-run`（若版本支持）与 `--write-config-to-dump.yaml` 对比默认导出，确认插件集合符合预期再上线。

Q4. 调度队列为什么是三级的？Pod 什么时候会被搬移？



搬移条件（核心考点）：

1. **触发事件类型**：Node/PV/PVC/StorageClass/CSINode/Service 等 add/update 时，会调用 `MoveAllToActiveOrBackoffQueue`；
2. `schedulingCycle` **机制**：记录在队列内的 cycle 计数。某 Pod 是在第 N 周期被判定 unschedulable，只有"周期 N 之后"发生的集群事件才算"新鲜事件"，才会把它搬回 ActiveQ —— 避免老事件反复触发无用调度；
3. **兜底定时器**：即使没有任何事件，也会周期性 flush 回来重排（源码常量 `podMaxInUnschedulablePodsDuration`，默认 `5m`，在你的版本里 `grep podMaxInUnschedulablePodsDuration pkg/scheduler/internal/queue` 核对）；
4. **QueueingHint（新版本）**：进一步精细化——只有"该 Pod 失败的某个插件认为这个事件可能让它重新可行"时才搬移，大幅减少大规模集群里无意义的 requeue。

可直接套用：

```
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
podInitialBackoffSeconds: 1 # 建议初值（源码默认 1）
podMaxBackoffSeconds: 10 # 建议初值（源码默认 10）
podMaxInUnschedulablePodsDuration: 5m  # 源码默认 5m
parallelism: 16 # 节点遍历并发度，源码默认 16
```

> **大规模场景的坑**：大批 Job Pod pending + 集群频繁节点变更 ⇒ 事件一来就把成千上万 unschedulable pod 全部搬回 ActiveQ，造成**重调度风暴**。解法：开 QueueingHint（新版本）、pod 拆分 Queue（多 profile）、资源提前预留、以及见 L4。

Q5. percentageOfNodesToScore 是什么？为什么要有它？

问题：5000 节点的集群，如果每个 Pod 都完整遍历 5000 个节点跑一遍 Filter，一次调度周期的成本与节点数线性相关——节点越多吞吐越低。

机制：只遍历一定比例（或一定数量）的节点，**找到足够多的可行节点后就停止**，从这些里打分选最优。

判定逻辑：

- 节点总数 ≤ 100 → **100% 遍历**（小规模无意义减小；源码里的 min 阈值 100）；
- 节点总数 > 100 → 使用 `percentageOfNodesToScore`；
- **未显式配置（=0）时的自适应公式**：`50 - numAllNodes / 125`，并 clamp 到 `[5, 100]`。即 5000 节点时约 10%（*该公式为社区实现，以源码为准*）。

风险（必答）：这是**牺牲最优性换吞吐**的近似算法——可能导致 Pod 落到"次优"节点（碎片、镜像缺失、跨可用区）。

- **离线批处理**：可以设 5~15%，换吞吐；
- **在线低延迟/亲和敏感业务**：设 100%（或保证国产/敏感业务量大在为数不多的节点池里，节点基数小本身就不触发近似）。

```
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
percentageOfNodesToScore: 30 # 建议：普通集群 20~30；CPU/GPU 贵资源集群考虑 100
```

验证方式：改前后对比 `scheduler_framework_plugin_duration_seconds{...Filter...}` 的 P99 与 `scheduler_pod_scheduling_duration_seconds` P99。

Q6. 抢占（Preemption）的完整流程是什么？

触发点：`Filter` 后**一个可行节点都没有** → 进入 `PostFilter`。

流程：

```
1. 优先级判定：Pod 必须有更高 PriorityClass；victim 只能来自同/更低优先级
2. 对所有节点做"假设 + 试删"：
      └ 逐步把最低优先级的 Pod 标记为 victim（"把资源挤出来"），直到本 Pod 能放下
      └ 受 PodDisruptionBudget 保护：若删除会违反 victim 的 PDB，该候选节点会被**重罚分**（天然排在后面甚至被排除）
3. 选一个"代价最小"的节点：优先级越低越好、被抢 Pod 越少越好、违反 PDB 越少越好
4. 通过 Eviction/Delete 删除 victim Pod（victim 会进入优雅退出）
5. 给抢占者设置 `status.nominatedNodeName = <node>`，Pod 回队列等待
6. 下一轮调度周期：
      └ PreFilter/Filter 前会先"把 victims 从 NodeInfo 里暂借移出"再试算，
        即 nominated pod 能"看到"即将释放的资源
7. victim 真的消失后（deletion timestamp → 对象移除），抢占者完成绑定
```

四个必答细节：

细节	说明
`nominatedNodeName`	只对 Terminating 的 victim 有意义；是"预约席"，不是绑定结果
抢占失败/虚假抢占	victim 迟迟没消失（优雅退出期长、finalizer 卡住），抢占者会一直等待或按退避重新计算；社区称这种现象为 bogus preemption（虚假抢占），可通过缩短 victim 的优雅退出期、治理 finalizer 来规避
`preemptionPolicy: Never`	高优先级 Pod 声明"我不抢别人"，只作为 victim 候选外的旁观者
`PodDisruptionBudget`	保护的是 **victim**（`minAvailable`），含义是"不会为了抢占而毁掉已有的可用性承诺"

可直接套用：

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: prod-critical
value: 1000000
globalDefault: false
preemptionPolicy: PreemptLowerPriority
description: "核心在线业务，允许抢占低优离线任务"
---
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: offline-batch
value: 1000
globalDefault: true
preemptionPolicy: Never # 关键：批处理永不抢占，避免抢占风暴
description: "离线批处理，可被抢占，但不抢别人"
```

> **注意**：`globalDefault: true` 全局只能有一个。给"默认优先级"设 NEVER 抢占，是防大规模抢占风暴最有效的一招。

Q7. 抢占 vs 驱逐 vs PDB vs Descheduler：别再混为一谈

机制	谁触发	目标	是否考虑 PDB	典型场景
Scheduler Preemption	kube-scheduler	给 Pending 的高优 Pod 腾地方	是	高优在线挤走离线
Kubelet Eviction	kubelet	节点本身快不行了（内存/磁盘/inode/PID 不足）	否（节点级自保）	node-pressure eviction
Node Lifecycle Eviction	taint-manager	节点 NotReady/Unreachable 后被标记	部分版本/路径有限支持	节点宕机重调度
API-Initiated Eviction	外部调用 Eviction API	运维手动搬迁	**是**	节点排水（drain）/灰度腾空
Descheduler	独立组件	平衡已经运行起来的集群（去碎片、去重、去违反亲和）	是（按策略）	长期均衡

必答一句：**抢占是"为了调度新 Pod"，驱逐是"为了保节点/回收坏节点"，Descheduler 是"为了均衡"**，三者目标不同、不可替代。

Q8. 打分是怎么算的？默认插件和权重是多少？

公式：`nodeScore = Σ (normalize(pluginScore_i(node)) × weight_i)`，每个插件分数先归一化到 `[0, 100]`，再按插件权重加权求和。

默认 profile 大致如下（请用 `kube-scheduler --write-config-to=dump.yaml` 现场核对，官方默认插件集与权重会随版本调整）：

插件	默认权重	作用
`NodeResourcesFit`	1	默认 `LeastAllocated`（越空闲越优）
`NodeResourcesBalancedAllocation`	1	CPU/内存/GPU 使用比例是否均衡（避免单一资源打满）
`ImageLocality`	1	本地已有镜像加分（拉镜像快）
`InterPodAffinity`	1	满足 Pod 亲和/反亲和程度
`NodeAffinity`	1	节点亲和匹配度
`PodTopologySpread`	2	拓扑分布均匀性（默认给的权重较大）
`TaintToleration`	3	节点上污点越少越好；能容忍的污点越多则分数略降
`VolumeBinding`	1	卷等待节点亲和与否、存储拓扑

常考追问：怎么让调度"更紧"？→ `NodeResourcesFit.scoringStrategy.type: MostAllocated` + `NodeResourcesBalancedAllocation` 加权（给大数据离线/弹性批量用）。但要注意：**在线延迟敏感业务用 MostAllocated 会造成热点叠加**，通常在线用 `LeastAllocated`、离线 Mixed 才上 MostAllocated。

Q9. Snapshot（NodeInfo 快照）与 scheduler cache 的一致性怎么保证？

结构：

- `SchedulerCache`（thread-safe，由 informer 更新）维护 `Nodes map[string]*NodeInfo`，每个 `NodeInfo` 维护该节点上 Pods 的资源聚合；
- 每次调度周期拿到的是**快照**（快照期间不再变），保证同周期内 Filter/Score 看到一致的视图。

不一致来源与自愈：

场景	处理
Assume 之后还没 Bind 成功	pod 在 `assumedPods` 里带 TTL；超时/绑定失败 → `ForgetPod` + 重新入队
实际 Pod 已删除但 cache 未更新	informer 的 Delete event 修正
Bind 失败	`Unreserve` 回滚 assume
apiserver watch 断连 relist	重新 list 重建 cache

必答：正因为是"乐观 + 最终一致"，才会出现 **"调度器觉得有资源，Kubelet 起来时却发现不够"**（尽管很少），以及必须依赖 **informer 的修正**而不能靠分布式锁。

Q10. 亲和性三兄弟 + 污点 + 拓扑分布怎么选？

手段	粒度	是硬约束还是软约束	规模风险	典型用法
`nodeSelector`	节点标签	硬	无	最简单过滤
`nodeAffinity`	节点标签	硬（`requiredDuringScheduling`）/ 软（`preferred`）	低	机型/zone/GPU 型号
`podAffinity`	Pod 间	硬/软	**中高**（要扫同 topology 的所有 Pod）	缓存靠近、数据本地性
`podAntiAffinity`	Pod 间	硬/软	**高**	单实例打散、避免同机故障域
`topologySpreadConstraints`	拓扑域	硬（`DoNotSchedule`）/ 软（`ScheduleAnyway`）	**中高**（大规模算 scrutiny 收益差）	跨 zone/机架均匀分布
`taints & tolerations`	节点	排斥/容忍 + NoSchedule/NoExecute	低	专用节点池、隔离 GPU/ARM

强烈建议的实践：优先用 `topologySpreadConstraints` 而不是 `podAntiAffinity` 做"打散"。原因：

- `podAntiAffinity` 只区分"有没有满足条件的 Pod"，是**布尔**判定，无法表达 skew；
- TopologySpread 可以表达"每个 zone 之间副本数差不超过 1"，语义精确且能软着落（`ScheduleAnyway` 保证不会因为打散而调度不出去）。

可直接套用（推荐组合拳）：

```
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: topology.kubernetes.io/zone
                operator: In
                values: ["cn-north-1a", "cn-north-1b", "cn-north-1c"]
  topologySpreadConstraints:
    - maxSkew: 1
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: ScheduleAnyway # 宁可挤一点也不能调度不出去
      minDomains: 3 # 保证至少覆盖 3 个 zone
      labelSelector:
        matchLabels:
          app: web
    - maxSkew: 1
      topologyKey: kubernetes.io/hostname
      whenUnsatisfiable: DoNotSchedule # 单机最多 1 副本
      labelSelector:
        matchLabels:
          app: web
  tolerations:
    - key: dedicated
      operator: Equal
      value: online
      effect: NoSchedule
```

Q11. VolumeBinding 插件与 `WaitForFirstConsumer` 为什么要存在?

背景：PVC 详情在下述 StorageClass 下会选择即刻分配 PV：

- `volumeBindingMode: Immediate` → PVC 一创建就 bind + 分配 PV，**PV 的拓扑（zone）可能与 Pod 最终落点不一致** → Pod Pending；
- `volumeBindingMode: WaitForFirstConsumer` → 延迟到 Pod 首次调度时才 bind，**让 PV 的 chosen zone 与调度结果一致。**

调度器侧做了两件事：

1. **Filter**：检查该节点能否满足 PV 的 `nodeAffinity`（`accessibleTopology`、CSI 写入的拓扑标签），过滤掉不可能节点；
2. **Reserve/PreBind**：真正触发 bind（`BindCSIStorage` 逻辑），并把结果写回 API Server。

必答的坑：

- WaitForFirstConsumer 让**调度需要考虑存储拓扑** ⇒ 节点更多/器件braska 拓扑复杂时，这一层会成为瓶颈之一（见大规模 L10）；
- 云盘一般只允许挂载到同 zone ⇒ 这是最常见的 "0/N nodes are available: N node(s) had volume node affinity conflict"。

Q12. 为什么 requests 比 limits 更影响调度?

- 调度只看 `.spec.containers[].resources.requests`（以及 initContainers 的 max/update-strap gaps 取最大）；**limits 完全不参与调度**；
- CPU request → CFS shares（争抢时的权重）；CPU limit → CFS quota（硬上限，会造成 throttling）；
- Memory request → 调度占位 + QoS 分级；memory limit → OOMKill 阈值（超过即被杀）。

QoS 与驱逐优先级：Guaranteed（requests==limits，且 CPU/内存都设）> Burstable > BestEffort。**升级包: BestEffort Pod 最先被节点压力驱逐。**

必答陷阱：

- `requests` 不设 → Burstable/BestEffort，调度器认为**不用资源**，会造成节点过量售卖 + 后续 OOM 集中崩；
- CPU limit 设太低 → throttling 导致 P99 毛刺；
- 建议：CPU 的 request 按真实常驻设、limit 可不设（避免 throttling），内存的 request==limit（保稳定）。这是"经验建议"，需压测校准。

Q13. PriorityClass / QoS / Quota 分别解决什么问题?

机制	解决	作用点	是否影响调度
PriorityClass	谁先调度 / 谁能抢谁	队列排序 + 抢占	**是**（强影响）
QoS	节点压力时谁先死	kubelet	否（但影响稳定性）
ResourceQuota	命名空间能用多少总量	admission	否（admission 拒绝，pod 不会进队列）
LimitRange	命名空间默认值/上下界	admission	否

Q14. 多个调度器共存是怎么工作的? 多 profile 与多调度器进程的区别?

两种模式：

```
# 模式 A：一个 kube-scheduler 进程跑多个 profile（推荐）
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
profiles:
  - schedulerName: default-scheduler # 在线业务：均衡 + 严格
    plugins:
      score:
        enabled:
          - name: NodeResourcesFit
            weight: 1
  - schedulerName: batch-scheduler # 离线批处理：装箱 + 快
    plugins:
      score:
        enabled:
          - name: NodeResourcesFit
            weight: 5
    pluginConfig:
      - name: NodeResourcesFit
        args:
          scoringStrategy:
            type: MostAllocated
```

```
# Pod 侧选择（注意：schedulerName 不可变）
spec:
  schedulerName: batch-scheduler
```

对比	多 profile（同进程）	多调度器进程 / 独立部署
资源	共享同一进程，省内存	各自独立，隔离更好
队列	**各自独立 ActiveQ**	各自独立
故障域	一个进程挂全挂	隔离好（一个挂了不影响另一个）
适用	绝大多数场景	极高隔离/独立演进诉求

必答：`spec.schedulerName` **是不可变字段**，Pod 一旦创建就绑定了该名字；若没有对应调度器运行 → Pod 永远 Pending 且没人管（连 event 都没有，最难受的一类事故）。

Q15. Scheduler Extender 为什么被废弃物这么多?

维度	Extender（HTTP 外挂）	Framework 插件（编译/多 profile）
性能	**每次调度都要 HTTP 交互，串行阻塞**	进程内，无网络开销
可靠性	extender 挂了 ⇒ 调度卡住/拒绝	同进程，故障域小
能力	只能做 Filter/Prioritize + 少量钩子	12+ 扩展点全支持
缓存	自己维护，易与 scheduler cache 不一致	共享 CycleState/框架状态

面试口径：Extender 已经不推荐作为主路径（社区倾向用框架插件或多 profile + 自定义插件），但在"已有系统难改动/跨语言"时仍可用，需要注意加**超时 + 降级 + 幂等**。

Q16. kube-scheduler 如何做到高可用? 双主会不会重复调度?

- 基于 `coordination.k8s.io/v1 Lease` 做 leader election（与 controller-manager 同一套机制），只有 leader 进程执行 `scheduleOne`；
- 默认 Lease/Renew/Retry（componentbaseconfig 常见默认 15s/10s/2s，**以运行版本为准**对应 `leaderElection` 字段）；
- 非 leader 进程不干事，只是等待抢锁。

```
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
leaderElection:
  leaderElect: true
  leaseDuration: 15s
  renewDeadline: 10s
  retryPeriod: 2s
  resourceLock: leases
  resourceName: kube-scheduler
  resourceNamespace: kube-system
```

为什么必须单点 leader：调度本质是对共享资源（全网 Node 资源视图）做决策，多主同时决策会产生**并发超卖**（两个 Pod 同时算账互相之间看不到对方 assume）。

必答的异常：

- Lease 抖动频繁 ⇒ 调度暂停窗口变长 ⇒ `scheduler_pending_pods` 突增 + metric 断更；
- 版本滚动升级时尤其要注意 election 参数一致性。

Q17. Descheduler 的价值与常用策略? 和调度器冲突吗?

调度器是"**一次性决策**"，不会回头。随着时间推移：节点标签变了、新节点加入、资源碎片多了 ⇒ 集群越来越不平衡，需要 Descheduler/重调度。

策略	作用
`RemovePodsViolatingNodeAffinity`	节点标签变了，Pod 不再满足 nodeAffinity
`RemovePodsViolatingInterPodAntiAffinity`	违反 Pod 反亲和
`RemovePodsViolatingTopologySpreadConstraint`	违反拓扑分布（很重要，新增节点后常用）

策略	作用
`RemoveDuplicates`	同 controller 下过多副本在同节点
`LowNodeUtilization`	节点利用率过低 ⇒ 迁出，便于缩容省钱
`HighNodeUtilization`	提高装箱密度（与 Evict 结合）
`RemovePodsHavingTooManyRestarts`	老 CrashLoop Pod
`RemoveFailedPods` (aka RemovePodsViolatingPss/ExpiringPods)	清理失败 Pod 等

必答风险（这是高分点）：

- Descheduler 通过 **Evict API** 驱逐，**严格遵守 PDB**，否则会引发可用性事故；
- 驱逐后由上层 controller 重建 ⇒ 会重新进入调度 ⇒ 要防止"**驱逐-重建-再驱逐**"的无限循环（对**无上层控制器管理的裸 Pod** 要特别小心，驱逐等于直接消失）；
- 必须配置 `--evict-local-storage-pods=false`、优先级门槛（`priorityThreshold`）避免驱逐系统组件。

Q18. Gang Scheduling / Coscheduling 怎么做？为什么必须用 Permit 扩展点？

背景：AI 训练/大数据作业要求"要么全部 Pod 起来，要么一个都不要"；否则会**死锁占资源**（部分 Pod 起来了等不到同伴，占着 GPU 空转）。

实现：

- `Permit` 插件：当一组 Pod（通过 PodGroup CR 或 label 关联）达到满足条件的最小数量时才批准绑定；否则返回 `Wait` 让 Pod 挂起，并设超时回滚；
- `PostFilter`：超时/资源无法凑齐时，整组放弃（或抢占）；
- `QueueSort`：**必须**按 PodGroup 创建时间排序，避免"后来者先占资源"导致先来者饿死。

必答的坑：

- `Permit` 处于**绑定周期**，等待的 Pod 已经通过 Filter/Score 并 `Assume` 了资源 ⇒ **等待期间资源是被占住的。**这就是"占位等待"，需要配套超时和配额，否则变资源泄漏；
- 大规模下 coscheduling 的 wait 会显著抬升 `scheduler_pending_pods` 与绑定延迟。

Q19. Kubernetes 原生支持多调度扩展点，那么"自定义调度器"有几种做法？

方案	复杂度	性能	维护成本	适用
调 Redis 的参数/权重 + 多 profile	低	好	低	**首选**
编译自定义 Framework 插件（in-tree 或 out-of-tree）	中/高	最好	中	通用需求
Scheduler Extender（HTTP）	低	差	中	遗留/快速验证
MutatingWebhook 改 nodeName（不推荐变相排 Secheduler）	中	好	高（灾难）	不建议
完全自研独立调度器	高	取决于实现	高	特殊领域（如 HPC）

Q20. 调度器如何降低对 API Server 的压力？

- **Pod Informer 带 field selector**：只 watch 未调度 Pod（`spec.nodeName` 为空），不加候选 Pod 的全量流；
- **Assume 乐观绑定**：减少等待 API Server 往返；
- **Bind 异步**：由独立 goroutine 执行绑定，不阻塞下一个 Pod 的调度周期；
- **client 侧限流**：`yaml clientConnection: qps: 100 # 建议初值（源码常见默认 50/100 量级，请 --write-config-to 核对） burst: 150`
- **事件侧**：大量 Pending 时会产生海量 `FailedScheduling` Event ⇒ 需限流/聚合，必要时调 `-v` 级别。

Q21. Device Plugin / Extended Resource / NUMA 三者关系？

层	谁负责	关键字段
上报资源量	kubelet device plugin（`ListAndWatch`）	node.status.capacity/allocatable `nvidia.com/gpu: 8`
调度	scheduler `NodeResourcesFit`（只看数量）	`resources.limits["nvidia.com/gpu"]: 1`
分配具体设备	device plugin `Allocate()` 在 kubelet 侧	返回 envs/挂载/annotations
亲和局部性	kubelet **TopologyManager** + `CPUManager`（single-numa-node 策略）	`--topology-manager-policy`
GPU 型号/拓扑标签暴露	GPU Operator / NFD / Device plugin 自身或 DRA	标签如 `nvidia.com/gpu.product`

必答认知：**原生调度器做 GPU 只认"数量"，不认"拓扑/型号细节"。**想做 GPU 型号/连通感拓扑/显存共享，需要：

- 用 label/taint 做粗粒度隔离（NFD）；
- 或上 **DRA（Dynamic Resource Allocation）**；
- 或用离屏/第三方调度器（Koordinator、Volcano、HAMi 等）。

Q22. DRA（Dynamic Resource Allocation）解决了 Device Plugin 什么痛点？

痛点（Device Plugin）	DRA 的解决方式
资源模型只有"整数计数"，无法表达"显存 40G / 特定型号 / 特定拓扑"	ResourceClaim + DeviceClass + ResourceSlice，**结构化参数**

痛点（Device Plugin）	DRA 的解决方式
只有 kubelet 参与，调度时无法提前参与设备选择	调度器介入 ResourceClaim 分配（structured parameters 下），避免"调度完才发现不匹配"
无法共享同一设备给多 Pod，也无法网络挂载设备	支持 shareable / 网络附加设备声明
资源发现与属性难以表达	`ResourceSlice` 由 DRA driver 发布，带属性选择器

> **注意**：DRA 的 alpha→beta 演进较快，**是否已 GA、structured parameters 版本请以 Release Notes 为准**；面试时重点说清"它要解决的是表达力与调度介入，不是简单替代 device plugin"。

Part B • 大规模集群篇（16 题）

> 定义：**大规模 = ≥ 1000 节点 / ≥ 15000 Pod**。以下每道题都按"现象 → 根因 → 量化方法 → 解法 → 验证"给。

L0. 官方宣称的规模上限是什么？（必须先有基准）

Kubernetes 官方提出的扩展目标（**1.x 后期版本，以官方文档为准**）：

维度	上限
单节点 Pod 数	110
Node 数	5000
集群总 Pod 数	150000
总容器数	300000

SLO 口径（SIG Scalability 定义，大规模集群的性能验收）：

- **Pod startup latency**（含镜像拉取与否两种口径）的分位数；
- **API Server 调用延迟**；
- 调度侧：Scheduler 使用 `scheduler_pod_scheduling_sli_duration_seconds` 作为 SLI（**该统计过滤掉多轮重试/抢占路径的 attempt，口径以版本为准**）。

> **重要提醒**：本文档**不提供"某集群实测 X pods/s"这种数字**——它强依赖节点规模、插件集、亲和约束复杂度、存储拓扑。**你的答案应该是：给出测量方法 + 给出你自己的实测数字。**

如何压测（必须会答）：社区做法是用 **Kubemark（hollow node）** 或 **KWOK** 模拟 N 千节点 + clusterloader2 / scheduler-perf 套件，测 `PodScheduled` 吞吐。

```
# 1. 起一个 fake/kwok 集群并注入 5000 个 hollow node
# 2. 跑 scheduler-perf 套件（perf-tests/scheduler），产出 metrics 耗时分布
# 3. 关注：scheduler_pod_scheduling_duration_seconds 的 P50/P90/P99 +
# scheduler_framework_plugin_duration_seconds 按 plugin 分解
```

L1. 大规模集群调度吞吐的瓶颈到底在哪？（必考综述题）

从数据流上拆，瓶颈永远落在这 5 个位置：

#	瓶颈点	量化指标	典型解法
1	**单 profile 串行调度**	`scheduler_pod_scheduling_duration_seconds` 随 QPS 线性抬升	多 profile 拆流量 / 多进程拆分 / 业务独立集群
2	**节点遍历成本 O(N)**	`scheduler_framework_plugin_duration_seconds{extension_point="Filter"}`	`percentageOfNodesToScore`；节点池化（taint + nodeAffinity 缩小候选集）
3	**亲和 /Spread 计算 O(Pods)**	`scheduler_framework_plugin_duration_seconds{plugin="InterPodAffinity"}`	限制 `topologyKey` 粒度、限制 labelSelector 基数、避免大量 Pod 在同一 domain
4	**存储拓扑参与调度**	`scheduler_framework_plugin_duration_seconds{plugin="VolumeBinding"}`	减少涉及 WaitForFirstConsumer 的 Pod 比例
5	**API Server 交互**	`apiserver_request_duration_seconds`、`scheduler` 侧 client 限流错误	client qps/burst 调优、降低 Event 量、apiserver 扩容

标准答法：

> "大规模下调度器不是单一瓶颈，而是分层的。我会先看插件级耗时分解（`scheduler_framework_plugin_duration_seconds` 按 plugin/extension_point），找出 Top 2 的插件，再决定是缩候选节点集（降低 N）还是简化约束（降低每节点成本）。这两条路径是乘法关系。"

L2. 节点池化 / 隔离为什么是解决大规模调度的第一银弹？

原理：`percentageOfNodesToScore` 是按"全部节点数"算比例的。**如果每个 Pod 通过 `nodeAffinity` / `taints` 先把候选集缩到 100~200 个节点，Filter 的成本就是常数级**，远强于调比例。

```
全集群 5000 节点
  ├─ online-pool (2000 nodes)  taint: dedicated=online:NoSchedule
  ├─ gpu-pool (300 nodes) taint: nvidia.com/gpu=present:NoSchedule
  ├─ arm-pool (500 nodes) label: kubernetes.io/arch=arm64
  └─ batch-pool (2200 nodes)  taint: workload=batch:NoSchedule
```

配套做法：

- 给业务 Pod 显式 `tolerations` + `nodeAffinity`；
- 给敏感/专用机型加 taint，防止普通 Pod 误调度（`NoSchedule` 是"防呆"）；
- 用 `resourceQuota` / `priorityClass` 做租户配额。

验证指标：改前后对比同一业务队列的 Filter P99 与整体 `scheduler_pod_scheduling_duration_seconds` P99。

L3. 集群事件批量触发导致的"重调度风暴"怎么办？

现象：集群批量加节点 / CSInode 更新 / 大量 PVC 变更 ⇒ UnschedulableQ 里上万 Pod 一次性搬回 ActiveQ ⇒ `scheduler_pending_pods{queue="active"}` 暴涨，CPU 打满，正常 Pod 排队变慢。

根因：`MoveAllToActiveOrBackoffQueue` 是粗粒度搬移 + 大量 Pod 依赖同一批事件。

解法组合：

1. **开启 QueueingHint**（较新版本）：让"事件 → 重新入队"精细化，只有真正可能被救活的 Pod 才搬；
2. **调大退避参数**：确认抖动严重时，可适当加大退避上限（建议把 `podMaxBackoffSeconds` 从 10 调到 20~30 观察），给集群自愈留窗口；
3. **按业务拆分 profile**：让离线风暴不要污染在线队列；
4. **不要一次性创建海量 Pod**：上层控制器加分批发放（batch sizing / Kueue quota 门控）；
5. **控制 Pending 总量**：给离线 Namespace 加 ResourceQuota，天然封顶。

告警（可落地）：

```
# 活跃队列堆积（阈值需按集群容量实测校准）
sum(scheduler_pending_pods{queue="active"}) > 500

# backoff 堆积，说明大量 Pod 在反复失败重试
sum(scheduler_pending_pods{queue="backoff"}) > 200

# 调度失败率（5m 窗口）
sum(rate(scheduler_schedule_attempts_total{result="unschedulable"}[5m]))
  / clamp_min(sum(rate(scheduler_schedule_attempts_total[5m])), 0.001) > 0.1
```

L4. 抢占风暴怎么防？（大规模经典反模式）

现象：夜里批处理高峰，大量高优 Pod 抢低优 ⇒ `scheduler_preemption_victims` 飙升、在线 Available 抖动、API Server 压力涨。

根因：无节制使用高优先级 + `preemptionPolicy: PreemptLowerPriority` 滥用。

解法（按优先级）：

1. **默认 PriorityClass 设为 `preemptionPolicy: Never`**（强烈推荐），只有极少数真正核心业务才开抢占；
2. **给 victim 侧配 PDB**，让抢占无法突破可用性底线；
3. **离线队列用 ResourceQuota 封顶**，从源头减少"需要抢"的场景；
4. **错峰 + 弹性扩队长 Len Cleveland**：离线高峰时先扩 nodepool（Cluster Autoscaler / Karpenter），而不是抢在线；
5. **在线/离线分节点池**（taint 隔离），把抢占变成"物理隔离"而非"频率抢占"。

```
# 抢占活跃度告警（>0 就应该被人群 Review）
sum(rate(scheduler_preemption_attempts_total[5m])) > 0.1
sum(rate(scheduler_preemption_victims[5m])) > 0
```

L5. Pending Pod 排障的标准路径（面试必背）

```
1. kubectl get pod -A --field-selector=status.phase=Pending -o wide | head
2. kubectl describe pod X # 看 Events: FailedScheduling 的 reason 是关键
     常见原因串：
       "0/N nodes are available: N Insufficient cpu." → 资源不够
       "N node(s) had untolerated taint {key: value}" → 污点
       "N node(s) didn't match Pod's node affinity/selector" → 亲和/标签
       "N node(s) had volume node affinity conflict" → 存储拓扑
       "N node(s) didn't match pod anti-affinity rules" → 反亲和
       "N Insufficient nvidia.com/gpu" → 扩展资源
3. 若 Pod 完全没有任何 Event → 极可能是：
       a) spec.schedulerName 写错 / 没有对应调度器在跑（最高频！）
       b) 调度器进程挂了或 lease 争抢失败
       c) 变更的选择器/field selector 出错导致 Pod 没进 informer
4. 查调度器进程：kubectl -n kube-system get lease kube-scheduler
5. 查 metrics：scheduler_pending_pods / scheduler_schedule_attempts_total
6. 必要时开 -v=5 看单个 Pod 的决策日志（临时操作，注意恢复日志级别）
```

> **金牌追问**：为什么有的 Pod "Pending 但没有事件"？答：`schedulerName` 错误/调度器未运行是最常见原因，因为**没有任何组件认领它**，自然就没有 event。

L6. 大规模下 FailedScheduling Event 洪水怎么治？

现象：上万 Pending Pod 持续失败 ⇒ 调度器疯狂写 Event ⇒ API Server `apiserver_current_inflight_requests` 打满，`apiserver_request_duration_seconds{verb="create",resource="events"}` 恶化 ⇒ 连带影响其它组件。

治理清单：

手段	做法
Event 限流	调整调度器 client qps/burst；给 Event 单独的 client 参数（版本支持时）
日志降级	恢复 `-v=2`（不要把 `-v=5` 常开）
根因减量	治理 pending 本身（批处理配额、弹性扩容、调度整形）
Event 保留策略	apiserver `--event-ttl`（注意版本差异）、避免 Event 表膨胀影响 etcd
新版本特性	部分版本引入 FailedScheduling 事件**聚合**能力（门控名/可用性以所用版本 Release Notes 为准），升级后可大幅降低 Event 数

告警：

```
sum(rate(apiserver_request_total{verb="create",resource="events",code=~"2.."}[5m])) > 500
```

L7. 资源碎片导致"总量够但调度不出去"怎么办?

现象：`Free CPU: 200 cores`，但一个新的 16C Pod 就是调度不出去（每个节点余 4~8C）。

根因：历史上用 LeastAllocated 打分 + 无装箱 ⇒ 资源被打散。

解法：

1. **打分策略改为** `MostAllocated`（针对离线/批处理 profile），让新 Pod 优先填坑；
2. 开启/加权 `NodeResourcesBalancedAllocation`，避免 CPU 用完内存剩一堆（反之亦然）；
3. **Requests 治理**：治理"申请虚高"（requests 远大于实际用量），这是碎片的根本来源 —— 用量画像（VPA recommendation / Prometheus 实测）驱动 requests 收敛；
4. **引入 Descheduler**（`HighNodeUtilization` / `LowNodeUtilization`）做二次整理（严格守 PDB）；
5. **规格标准化**：限制 node 机型数量、限制 Pod 规格档位（如只允许 2C/4G、4C/8G、8C/16G、16C/32G），碎片概率大幅下降。

验证：用分配率指标看改善：

```
# 集群 CPU 分配率（request / allocatable）
sum(kube_pod_container_resource_requests{resource="cpu"})
  / sum(kube_node_status_allocatable{resource="cpu"}) > 0.75

# 但"碎片"更要看：无法满足某规格的节点占比（需用自定义计算）
```

L8. 一个 5000 节点集群 vs 5 个 1000 节点集群，怎么选?

维度	单大集群	多小集群
资源利用率	高（池化效应，削峰补谷）	低（需预留 buffer）
故障爆炸半径	**大**（一次故障全站）	小
运维复杂度	低（一套控制面）	**高**（多套升级/配置漂移）
etcd/API 压力	大（需要专职调优）	分摊
成本	低（overhead 少）	高

标准答法：

> "不要为了分区而分区。分区的真正理由是**故障域隔离与变更隔离**，而不是性能。当出现以下信号才考虑拆：控制面单点故障会造成跨业务不可用、升级必须全局暂停、不同业务 SLO 差异极大（延迟敏感 vs 批处理）、或租户隔离合规要求。性能问题优先用节点池化 + profile 拆分 + 参数调优解决。"

L9. 大规模 GPU / AI 场景的调度难点?

难点	现象	解法
GPU 是"整块"资源	一张卡只能给一个 Pod（原生），利用率低（\<30%）	分片方案（MIG / 时间片共享厂商方案 / DRA）；实则要在 device plugin/第三方层实现
Gang Scheduling	部分起来 ⇒ 死锁占卡	coscheduling（Permit 插件）+ PodGroup + Kueue/Voice/Volcano
拓扑亲和	NVLink/NVSwitch 亲和，跨卡通信慢	label/NFD + nodeAffinity；或用专业的网络拓扑感知调度
抢占	高优训练任务等不到卡	优先级 + 合理 Quota；谨慎用抢占（训练中断成本极高）
Extender 性能	自研调度逻辑 O(N) 串行 HTTP	改为 in-tree/out-of-tree framework 插件

L10. 弹性扩缩节点时调度侧会踩什么坑?

场景：CA/Karpenter 扩容 1000 节点 ⇒ 大量 Pod 立即被调度到新节点，但节点 CNI/CSI 还没就绪 ⇒ Pod `ContainerCreating` 长时间 / `FailedMount` / `ImagePullBackOff` / CNI not ready。

解法：

1. 依赖 `node.kubernetes.io/not-ready` / `network-unavailable` / **cloud provider uninitialized taint** 天然防呆（scheduler 不会往 NoSchedule 污点节点调）；
2. **启动污点**（如 `node.cilium.io/agent-not-ready`、`node.kubernetes.io/network-unavailable`）由 CNI 就绪后移除；
3. 关键业务加 `minReadySeconds` / readinessGate 防"节点起来就接流"；
4. **弹性节奏控制**：客户端侧的批次发放避免瞬时打爆调度器和 API Server；
5. 给新节点预留 / 扩容预热（提前创建 Warm Pool）。

收缩反向问题：缩容时大量 Pod 同时被驱逐 ⇒ 瞬时 ΔQPS 爆发 ⇒ 同时进入调度队列，需要注意 PDB + 分批。

L11. 大规模下 Kubernetes Control Plane 与 Scheduler 的相互伤害?

- **Scheduler watch 未调度 Pod**：大批 Pod 瞬时创建 ⇒ informer 事件流爆发 ⇒ 调度器内存/CPU 涨；
- **Scheduler 写 Event / Bind**：反向给 API Server 增压；
- **etcd 延迟上涨** ⇒ informer relist / resourceVersion 过期 "too old resource version" ⇒ scheduler cache 需要重建 ⇒ 调度暂停。

解法：

- 拆分 informer（多 profile 会各自 habere 一份 pod informer，注意内存随 profile 数涨）；
- 控制 apiserver `--max-requests-inflight` / Priority \& Fairness (APF) 给 scheduler 分配专用 FlowSchema，**保障调度器不被其它组件饿死**；

- etcd 独立盘 + 监控 fsync 延迟（`etcd_disk_wal_fsync_duration_seconds` P99）。

```
# APF 给调度器保障配额（示意，字段以版本为准）
apiVersion: flowcontrol.apiserver.k8s.io/v1
kind: PriorityLevelConfiguration
metadata:
  name: system-scheduler
spec:
  type: Limited
  limited:
    assuredConcurrencyShares: 100
    limitResponse:
      type: Queue
      queuing:
        queueLengthLimit: 500
```

> 🔗 **关联阅读**：第二篇 · 6.1 APF：为关键组件提优先级、为批处理降优先级

L12. 多租户 / 多个产品共用集群怎么防止互相影响?

组合拳：

1. **节点池 + taint**（硬隔离）；
2. **profile 拆分**（策略隔离：在线/离线不同打分）；
3. **ResourceQuota**（总量封顶）+ **LimitRange**（默认 requests，防 Pod 开 0 request）；
4. **PriorityClass 分层** 且默认层 `preemptionPolicy: Never`；
5. **Priority \& Fairness** 保证调度器的 apiserver 配额。

```
# 防呆：给每个 NS 注入默认 requests（否则业务创建 0 request 的 Pod 会引发过度调度 + OOM）
apiVersion: v1
kind: LimitRange
metadata:
  name: defaults
  namespace: tenant-a
spec:
  limits:
    - type: Container
      defaultRequest:
        cpu: 500m
        memory: 512Mi
      default:
        cpu: "2"
        memory: 2Gi
      max:
        cpu: "32"
        memory: 64Gi
```

L13. 怎么判断"scheduler 本身慢"还是"apiserver/etcd 慢"?

三步定位法：

```
# ① 调度器自身：插件级耗时分解（若是这里高 → 是 scheduler 侧算法/规模问题）
histogram_quantile(0.99,
  sum by (plugin, extension_point, le) (
    rate(scheduler_framework_plugin_duration_seconds_bucket[5m])
  )
)

# ② 到 apiserver 的往返（若这里高 → 是 apiserver/etcd 或客户端限流）
histogram_quantile(0.99,
  sum by (verb, resource, le) (
    rate(apiserver_request_duration_seconds_bucket[5m])
  )
)

# ③ etcd 自身（若这里高 → 根子在 etcd）
histogram_quantile(0.99,
  rate(etcd_request_duration_seconds_bucket[5m])
)
histogram_quantile(0.99, rate(etcd_disk_wal_fsync_duration_seconds_bucket[5m]))
```

判定表：

现象	结论
① 高、②③ 正常	scheduler 算法/规模瓶颈 → 调 percentageOfNodesToScore、简化约束、节点池化
①②③ 都高	etcd 慢，级联到 apiserver 再到 scheduler → 先治 etcd（磁盘/压缩/对象数）
① 正常、② 高、③ 正常	apiserver 侧：对象数过多 / Event 洪水 / 限流分流 → 看 APF、看 verb=create events

---

L14. 亲和/反亲和在大规模下为什么是性能杀手？怎么优化?

成本模型：

- `podAffinity/podAntiAffinity` 需要对每个候选节点，**遍历该 topology domain 下所有匹配 labelSelector 的 Pod**；
- 若 topologyKey=hostname 且 Pod 很多 ⇒ 每次要聚合全集群 Pod ⇒ 与 Pod 总数近似线性。

社区优化（答出来很加分）：

- PreFilter 阶段预先计算 matching pod 列表 / topology pairs，避免每个节点重复扫；
- 限制同一拓扑域内的候选数量；
- `topologyKey` 不要乱选 hostname（除非必要），用更粗的 topologyKey 能减少聚合开销。

治理建议：

1. **优先用** `topologySpreadConstraints` **取代** `podAntiAffinity` 表达打散；
2. labelSelector **基数要小**（不要用它匹配全量 Pod）；
3. 周期性检查 spike：`scheduler_framework_plugin_duration_seconds{plugin=~"InterPodAffinity|PodTopologySpread"}` P99。

L15. 调度策略变更如何灰度与回滚？（生产变更治理）

任何调度器配置变更都必须走这条路：

阶段	灰度比例	观察指标（≥ 30min）	回滚触发条件
Step 1	**1%** Pod（通过独立 `schedulerName` 灰度，而不是直接改默认调度器）	P99 调度时延、失败率、Pending 数	失败率 > 基线 +2pp，或 P99 > 基线 ×1.5
Step 2	10%	+ 业务 P99 延迟、错误率	同上，或任一业务 SLO 告警触发
Step 3	50%	全量 SLO Dashboard	同上
Step 4	100%	稳定 ≥ 24h	—

回滚要点：

- **优先"改回 schedulerName"**（把 Pod 切回 default-scheduler），而不是回滚二进制；这是秒级、幂等的；
- 配置用 ConfigMap/文件 + 版本化管理，回滚就是切回上一个版本文件；
- *注意*：回滚旧 scheduling 配置**不会重新调度**已经在跑的 Pod ⇒ 已受影响的 Pod 需要靠 Descheduler 或人工滚动重启才能回到期望分布（这点必须在方案里写明）。

示例：通过新 profile 灰度

```
# 1) 先在同一个 kube-scheduler 里加一个灰度 profile
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
profiles:
  - schedulerName: default-scheduler # 老策略，保持不变
  - schedulerName: default-scheduler-canary # 新策略
    plugins:
      score:
        enabled:
          - name: NodeResourcesFit
            weight: 2
    pluginConfig:
      - name: NodeResourcesFit
        args:
          scoringStrategy:
            type: MostAllocated
            resources:
              - {name: cpu, weight: 1}
              - {name: memory, weight: 1}
```

```
# 2) 把 1% 的 Deployment 切进去（用 patch，可观察、可秒回）
kubectl -n app patch deploy web -p '{"spec":{"template":{"spec":{"schedulerName":"default-scheduler-canary"}}}}'
# 3) 观察 30min ... 出问题：
kubectl -n app patch deploy web -p '{"spec":{"template":{"spec":{"schedulerName":"default-scheduler"}}}}'
```

L16. 给你一个新集群，怎么设计其"调度治理"方案？（综合题/架构题）

按这个顺序答，面试官会认为你干过：

1. **先问 SLO**：业务的可用性目标是多少？P99 延迟目标？成本预算约束？（**不先问 SLO 就谈调度的都是耍流氓**）
2. **节点池化设计**：按机型/业务线/租户划 pool（taint + label），天然缩小调度候选集。
3. **优先级体系**：3~5 个 PriorityClass（system > prod > default > batch-low），默认层 `preemptionPolicy: Never`。
4. **requests/limits 治理**：LimitRange 给默认值；定期用实测用量做 requests 校准（VPA recommendation / Prometheus）。
5. **多 profile**：在线用 LeastAllocated + 均衡；离线用 MostAllocated + 装箱。
6. **可观测性**：scheduler metrics + 关键告警（Pending、失败率、P99、抢占、插件耗时 Top）。
7. **混沌演练**：定期 GameDay —— 杀调度器、模拟节点宕机、模拟 Pod 洪峰，验证 SLO 与 MTTR。
8. **变更治理**：灰度/回滚流程 + Descheduler 兜底。

Part C · 反模式清单（SRE 视角，可直接用于生产巡检）

每条含：反模式 / 证据 / 风险等级 / 修复动作 / 验证方式

#	反模式	证据（怎么发现）	等级	修复动作	验证方式
C1	容器不设 requests / requests 远低于实际用量	`kube_pod_container_resource_requests` vs 实际用量；节点 OOM 频繁	🔴 高	LimitRange 注入默认；VPA recommend 校准	节点 OOMKill `kube_pod_container_status_term` 归零
C2	只用 `podAntiAffinity` 做打散，且 labelSelector 基数巨大	`scheduler_framework_plugin_duration_seconds{plugin="InterPodAffinity"}` P99 高	🔴 高	改用 `topologySpreadConstraints`；收敛 selector	该插件 P99 下降 ≥ 30%（实测）
C3	默认 PriorityClass 允许抢占	存在多个 value > 0 且 `preemptionPolicy` 非 Never 的默认类	🔴 高	默认层设 `preemptionPolicy: Never`	`scheduler_preemption_victims` 速
C4	scheduleName 拼错 / 无对应调度器	Pod Pending 且**无任何 Event**	🟠 中	校验 webhook 检查 schedulerName 是否在白名单；巡检脚本	巡检脚本输出异常 Pod 数为 0
C5	生产长期开 `-v=5` 调试日志	scheduler 日志量巨大、CPU 高	🟠 中	恢复 `-v=2`；按需临时打开	CPU 使用率回落；日志量下降
C6	一个超大 RuntimeClass/Pool 无任何隔离	`kube_node_info` 5000 节点单一 labels	🟠 中	池化 + taint	Filter P99 下降
C7	用 Scheduler Extender 做核心逻辑	extender 依赖 + kube-scheduler 超时日志	🟠 中	迁移为 framework 插件	无 HTTP 调用；错误率归零
C8	Descheduler 未配 PDB / priorityThreshold	驱逐关键 Pod、业务抖动	🔴 高	配 evict 策略 threshold + PDB	演练：驱逐时段业务可用性不跌破 SLO
C9	集群无调度 SLO 与告警	无 PromQL / 无 Dashboard	🟠 中	建立 Part D 的告警集	演练：注入 Pending 洪峰，告警 5min 内
C10	一次性创建上万 Pod（无批次控制）	apiserver inflight 打满 + scheduler CPU 满	🟠 中	上层控制器分批 + Quota 封顶	创建速率平稳，无 QoS 挤兑
C11	MostAllocated 用于在线延迟敏感业务	同节点多实例叠加，P99 恶化	🟠 中	在线 profile 用 LeastAllocated	业务 P99 回到基线
C12	不监控 scheduler lease	调度暂停无告警	🔴 高	加 lease 存活/频率告警	模拟抢主，告警触发

Part D • 可观测性：SLO、指标与告警（可直接落地）

D1. 建议的调度 SLO（初始值，必须按业务实测校准）

SLI	建议目标（起点）	计算方式
调度成功率（5min 窗口）	≥ 99.9%	`1 - rate(result="unschedulable") / rate(total)`
P99 调度时延（首调度路径）	≤ 2s	`histogram_quantile(0.99, scheduler_pod_scheduling_sli_duration_seconds_bucket)`
Pending > 5min 的 Pod 占比	≤ 0.1%	需自定义（Pod annotation 时间戳或审计）
抢占事件频率	在线 profile ≈ 0	`rate(scheduler_preemption_attempts_total[5m])`

> 错误预算：以 30 天滚动窗口，消耗速率 > 14.4×/6×（Google SRE burn rate）分别触发紧急/工单告警。

D2. 核心指标清单（先 `curl :10259/metrics` 核对指标名）

指标	含义	用于
`scheduler_pending_pods{queue=...}`	各级队列长度	堆积、风暴
`scheduler_pod_scheduling_duration_seconds{attempts}`	端到端调度耗时	P99 时延、attempts 分层分析
`scheduler_pod_scheduling_sli_duration_seconds`	SLI 口径耗时（排除复杂重试路径，口径以版本为准）	SLO 度量
`scheduler_schedule_attempts_total{profile,result}`	结果分布	成功率
`scheduler_framework_plugin_duration_seconds{plugin,extension_point,status}`	**插件级耗时**（最有价值的分解指标）	性能瓶颈定位
`scheduler_preemption_attempts_total` / `scheduler_preemption_victims`	抢占活动	抢占风暴
`scheduler_queue_incoming_pods_total{event}`	入队来源	队列抖动
`scheduler_scheduler_cache_size{type}`	cache 对象数	内存/cahce 泄漏
`scheduler_goroutines`	goroutine 数	泄漏/卡住

D3. Prometheus 告警规则（可直接粘贴，阈值请实测校准）

```
groups:
  - name: kube-scheduler
    rules:
      - alert: SchedulerNoMetrics
        expr: absent(up{job="kube-scheduler"} == 1)
        for: 3m
        labels: {severity: critical}
        annotations:
          summary: "kube-scheduler 指标消失，可能进程挂掉或 lease 争抢失败"

      - alert: SchedulerPendingPodsHigh
        expr: sum(scheduler_pending_pods{queue="active"}) > 500
        for: 10m
        labels: {severity: warning}
        annotations:
          summary: "调度活跃队列堆积 {{ $value }}，检查资源/亲和/配额"

      - alert: SchedulerUnschedulableRateHigh
        expr: |
          sum(rate(scheduler_schedule_attempts_total{result="unschedulable"}[5m]))
            / clamp_min(sum(rate(scheduler_schedule_attempts_total[5m])), 0.001) > 0.1
        for: 10m
        labels: {severity: critical}
        annotations:
          summary: "5 分钟内调度失败率 {{ $value | humanizePercentage }}"

      - alert: SchedulerSchedulingLatencyP99
        expr: |
          histogram_quantile(0.99,
            sum(rate(scheduler_pod_scheduling_duration_seconds_bucket[5m])) by (le)) > 5
        for: 10m
        labels: {severity: warning}
        annotations:
          summary: "P99 调度时延 {{ $value }}s"

      - alert: SchedulerPreemptionActive
        expr: sum(rate(scheduler_preemption_attempts_total[5m])) > 0.1
        for: 5m
        labels: {severity: warning}
        annotations:
          summary: "调度器正在频繁抢占，检查是否出现抢占风暴"

      - alert: SchedulerCacheTooLarge
        expr: scheduler_scheduler_cache_size{type="pods"} > 200000
        for: 15m
        labels: {severity: info}
        annotations:
          summary: "scheduler cache 中 Pod 数异常，检查是否有 assumed pod 泄漏"
```

> **10259 端口是 HTTPS**，抓取需要 `scheme: https` + `insecureSkipVerify` + bearer token（或 kubeconfig RBAC），CA/SAN 处理务必放到经审批的方案里，别口头 monkey-patch。

Part E • 演练（GameDay）计划表

场景	注入方式	期望表现	验证的 SLO
调度器主失联	删除 scheduler Pod / 强制抢 lease	30s 内新主接管，Pending 短暂上涨后回落	调度可用性
Pod 洪峰	一次性创建 3000 Pod	无 unschedulable 溢出、apiserver 无 5xx	调度成功率、时延
节点宕机	Stop 一个 100 Pod 的节点	5min 内重调度完成，业务 P99 不破线	可用性 + MTTR
资源碎片	人为造成每节点仅剩 4C 空闲	16C 的新 Pod 要么调度失败率超过阈值、要么被 MostAllocated 策略正确装箱；二者必有其一且结论可解释	装箱策略有效性
存储拓扑冲突	跨 zone PVC + Pod	调度到正确 zone（或给出明确事件）	存储亲和正确性
抢占演练	注入高优 Pod	Victim 被优雅终止、PDB 不被突破	抢占安全性

Part F • 10 张面试速记卡（临阵磨枪）

1. **一次调度 = NextPod → 快照 → PreFilter → Filter(并发) → [PostFilter 抢占] → Score → 随机选最高分 → assume(Reserve) → Permit → 异步 Bind**。每 profile 一个 goroutine 串行。
2. **扩展点 13 个**：PreEnqueue / QueueSort / PreFilter / Filter / PostFilter / PreScore / Score / NormalizeScore / Reserve / Permit / PreBind / Bind / PostBind（+ Unreserve 回滚）。
3. **队列三级**：ActiveQ / BackoffQ / UnschedulableQ；UnschedulableQ 靠集群事件 + `podMaxInUnschedulablePodsDuration` 兜底重排。
4. **抢占**：只对"Filter 全失败"生效；`nominatedNodeName` 是预约不是绑定；PDB 会压制；默认优先级应 `preemptionPolicy: Never`。
5. **调度只看 requests，不看 limits**。limits 是运行时的事。
6. **ResourceQuota 是 admission 的事，不是 scheduler 的事。**
7. **打散用 TopologySpread，不用 podAntiAffinity**；`maxSkew` + `whenUnschedulable` + `minDomains` 三件套。
8. **大规模第一银弹 = 节点池化**（缩小 N），第二才是 `percentageOfNodesToScore`（牺牲最优性）。
9. **排障第一问**：有没有 Event？没有 ⇒ 先看 `spec.schedulerName` 和调度器进程活着没。
10. **所有性能数字必须自己测**：用 Kubemark/KWOK + scheduler-perf，按 `scheduler_framework_plugin_duration_seconds` 分解找瓶颈。

第六篇 • kubelet 与节点

> SyncLoop · PLEG · CRI/CNI/CSI · 探针 · 驱逐

0. 使用说明

0.1 kubelet 面试题的考察层级

层级	考点	表现
L1 会用	会 `kubectl logs/exec`，知道三种探针	只会 describe
L2 懂原理	说得清 SyncLoop / PLEG / CRI / CNI / CSI / EvictionManager / cgroup 分层	能画节点内架构图
L3 有规模感	知道高密度节点下 PLEG、镜像拉取、心跳、驱逐谁先崩	能报 metric 名
L4 有事故经验	能讲清"Pod 卡 ContainerCreating / 优雅下线 502 / 节点失联 5 分钟才驱逐"的根因与治理	有 PromQL、有灰度、有回滚

> **答题模板**：结论 → 组件/数据流 → 给证据（配置项 / metric 名 / 事件字段）→ 补坑或算时间账。

Part A • 原理篇（24 题）

Q1. kubelet 负责什么？和 scheduler / controller-manager 的边界在哪？

一句话：kubelet 是**节点代理**——它把 API Server 里"应该在本节点运行的 Pod"翻译成真实的容器、网络、存储、资源限制，并持续汇报节点与 Pod 状态。

事项	kubelet	说明
决定是否在本节点跑	❌（scheduler 决定 nodeName）	kubelet 只认账不管分配
创建/启动/停止容器	✅	通过 CRI
挂载卷 / 卸载卷	✅（VolumeManager + CSI Node 插件调用）	attach/detach 由 controller-manager 的 AttachDetach controller 或 CSI 自己完成
配置容器网络	✅（调用 CNI 插件）	IPAM 由 CNI 负责
探针探测	✅	liveness 失败则重启容器
节点/ Pod 状态上报	✅	Node status + Lease
驱逐 Pod	✅ **节点压力驱逐**	与 scheduler 抢占完全不同
准入 Pod（节点级）	✅ `CanAdmit` 检查	资源、cgroup、拓扑亲和等，失败 → `UnexpectedAdmissionError`
镜像/容器/日志 GC	✅	ImageGC / ContainerGC / ContainerLogManager

必答要点：kubelet 是"**单点自治**"的——即使 API Server 不可达，它仍会继续维持现有 Pod 的运行（这是设计目标），只是无法获取新指令。

Q2. kubelet 内部有哪些核心组件？画一下



高频追问：PodManager 里存的是什么？ 答：期望状态（来自 apiserver/static file/http），与实际 runtime 状态的"对账源"。PLEG 提供真实事件，两者在 SyncLoop 里汇合，驱动 `syncPod` 做**幂等收敛**。

幂等性（加分点）：`syncPod` 可以被反复调用且结果一致，这是 kubelet 允许"周期性重放 + 事件驱动"混合模式的基础。

Q3. SyncLoop 的四个事件源是什么？为什么设计成这样？

`syncLoopIteration` 同时从 4 个 channel 收事件：

Channel	来源	作用
`configCh`	PodManager（apiserver watch / static file / http）	期望状态变化（新 Pod、删除 Pod）
`plegCh`	PLEG（周期性 relist runtime）	真实状态变化（容器死了/启动了/变了）
`syncCh`	定时器 `--sync-frequency`（默认 1m）	**兜底全量重同步**，防事件丢失导致状态漂移
`lifecycleCh`	上次 syncPod 失败/需要重试的 Pod 回灌	失败重放
`housekeepingCh`	周期性清理任务	GC / 卷清理等

必答的设计思想：

1. **事件驱动 + 周期兜底（"level-triggered 而非 edge-triggered"）**——丢事件不会导致永久错 상태，因为最多 1 分钟就被 sync 拉回来；
2. `--sync-frequency` 越小收敛越快，但 CPU/API 开销越大（ Node-agent 的内部 arning）。

Q4. PLEG 是什么？为什么它一出问题，节点上所有 Pod 都跟着 Pending？（必考）

PLEG = Pod Lifecycle Event Generator。

旧版（GenericPLEG）：**定期（默认 1s）relist** —— 调用 CRI 列出节点上所有 sandbox/container，与上一次快照 diff，生成 `ContainerStarted / ContainerDied / ContainerRemoved / ContainerChanged` 事件，塞进容量为 1000 的 channel，供 SyncLoop 消费。

为什么它是单点故障放大器：

现象	根因
`PLEG is not healthy: pleg was last seen active 3m ago; threshold is 3m`	relist 周期超过阈值（源码默认 3m），kubelet 直接把 Node 置 **NotReady**
所有 Pod 瞬间变 `ContainerCreating` / `Unknown`	拿不到 runtime 状态 ⇒ 无法收敛
`Event channel is full, discard this relist()`	一次 relist 事件超过 channel 容量（1000），**事件被丢弃** ⇒ 状态漂移，只能等 1 分钟兜底 sync

relist 为什么会变慢：

- 容器密度高 ⇒ 单次 relist 要遍历成千上万个容器，且 **CNI/CSI/CRI 调用是串行的**；
- runtime（containerd/docker）本身卡顿（overlayfs IO、磁盘满、shim 泄漏）；
- 节点 CPU 被打满（含被低优任务抢占）。

解法：

1. **Evented PLEG**（新版本的 feature gate）：改为 runtime 主动推送事件 + 低频全量 relist 兜底，显著降低 relist 开销（**是否 GA、版本号以 Release Notes 为准**）；
2. 降 Pod 密度 / 调 `--pod-pids-limit`；
3. 治理 runtime 健康（containerd shim/fs/inode）；
4. 监控 `kubelet_pleg_relist_duration_seconds` 提前预警。

```
# PLEG relist 耗时 P99（阈值需实测校准，集群内 >5s 就要查）
histogram_quantile(0.99, sum by (le) (rate(kubelet_pleg_relist_duration_seconds_bucket[5m])))
# relist 间隔
histogram_quantile(0.99, sum by (le) (rate(kubelet_pleg_relist_interval_seconds_bucket[5m])))
```

Q5. CRI 是什么？PodSandbox 又是什么？

CRI = Container Runtime Interface：kubelet 通过 gRPC（`RuntimeService` + `ImageService`）调用运行时，由运行时负责真正拉镜像、创建/删除容器。**kubelet 自己不直接操作 runc/容器进程。**

常见运行时：**containerd**（主流，内置 CRI 插件）、**CRI-O**、（Docker 经由 dockershim 的路径已在 1.24 起移除，必须转 containerd/CRI-O）。

PodSandbox（Pause 容器）的作用（必答）：

- 为同一 Pod 里的所有容器**共享 network namespace**（Pod IP 在 sandbox 上，容器加入同一个 netns）；
- 承担 PID namespace 共享、共享 volume 生命周期；
- **Sandbox 没起来 = Pod 绝不可能起来** ⇒ `ContainerCreating` 卡住最常见的就是 sandbox 创建失败（CNI 失败、runtime 失败、资源不足）。

CRI 相关 metric（排障利器）：

```
# 按操作类型看 runtime 调用耗时，能精确分离"拉镜像慢 / 创建容器慢 / 停容器慢"
histogram_quantile(0.99,
  sum by (operation_type, le) (rate(kubelet_runtime_operations_duration_seconds_bucket[5m])))

# runtime 错误率
sum by (operation_type) (rate(kubelet_runtime_operations_errors_total[5m]))

# 调用量（密度视角）
sum by (operation_type) (rate(kubelet_runtime_operations_total[5m]))
```

> `operation_type` 通常包含 `create_container`、`start_container`、`stop_container`、`pull_image`、`image_status`、`container_status`、`podsandbox_status` 等（**指标取值请以目标节点 `curl -k https://127.0.0.1:10250/metrics` 核对**）。

Q6. kubelet 在网络上到底做了什么？CNI 是怎么被调用的？

kubelet **不实现任何网络**，它只是在 sandbox 创建/销毁时按 CNI 规范调用二进制插件：

```
SyncPod → CRI RunPodSandbox → CNI ADD(/opt/cni/bin/<plugin>, conf from /etc/cni/net.d)
                            → plugin 创建 veth / 调 IPAM 分配 IP / 写路由 / 返回 IP
删除 Pod → CRI StopPodSandbox → CNI DEL
```

必答要点：

- 网络插件（Calico/Cilium/Flannel 等）负责 **IPAM、跨节点路由、网络策略**，kubelet **只负责触发 CNI 调用并把结果写回 Pod status**；
- **CNI 失败** ⇒ `FailedCreatePodSandBox`，典型错误：`failed to allocate for range`（IP 耗尽）、`plugin type="calico" failed (add): ...`、CNI DaemonSet 未就绪；
- `network-unavailable` taint：CNI 未就绪时由网络组件或云管控在节点上打污点，防止 Pod 被调度到无网络节点。

Q7. CSI 与 kubelet 的存储挂载流程？

kubelet 侧是 **VolumeManager**，两个核心 reconciler：

1. **DesiredStateOfWorldPopulator**：算出本节点该挂哪些卷；
2. **Reconciler**：执行 mount/unmount。

对 CSI 卷的典型调用链：

```
kubelet VolumeManager
   ├─ NodeStageVolume（格式化 + 挂载到 staging 路径）— 可选，取决于驱动
   ├─ NodePublishVolume（bind mount 到 Pod 目录 /var/lib/kubelet/pods/<uid>/volumes/...）
   └─ NodeExpandVolume（在线扩容时）
卸载时反向：NodeUnpublishVolume → NodeUnstageVolume
```

attach/detach 不在 kubelet：块存储的 Attach/Detach 由 **AttachDetach controller（controller-manager）** 或 CSI attacher 处理；有些 CSI 驱动走 `skip attach` 模式（由 kubelet 直接调用 CSI NodePublish）。

高频坑：

- in-tree 存储插件已全面**迁移到 CSI**（社区推动 CSI migration；具体哪些卷类型已下线请以版本 Release Notes 为准，旧参数/annotation 在新版本会失效）；
- CSINode 对象记录该节点上驱动信息与拓扑；**驱动 Pod 崩 ⇒ 卷挂不上 ⇒ Pod 卡 ContainerCreating**；
- subPath 挂载的 **ConfigMap/Secret 不支持热更新。**

Q8. 一次 Pod 启动，从 bind 到 Ready，全链路经过哪些阶段？慢在哪？（必考综合题）

```
t0  scheduler 绑定 nodeName
t1  kubelet watch 到本节点 Pod → PodManager → SyncLoop → podWorker.syncPod
t2  WaitForAttachAndMount：等待卷 attach + mount
t3  CRI RunPodSandbox（创建 pause 容器）
t4  CNI ADD（分配 IP、建 veth、设路由）
t5  拉取镜像（可能串行排队！）
t6  CRI CreateContainer + StartContainer（含 postStart hook）
t7  startupProbe / readinessProbe 通过
t8  Pod Ready → Endpoint 更新 → kube-proxy/Ingress 生效（这一步还有额外传播延迟！）
```

慢点定位顺序（可直接背）：

现象	大概率阶段	证据
`ContainerCreating` 很久	镜像拉取 / 卷挂载 / CNI	Events + `kubelet_runtime_operations_duration_seconds{operation_type="pull_image"}`
Pod 起来了但很久才 Ready	探针太保守 / 应用冷启动	`kubelet_pod_start_duration_seconds`、探针 threshold
Ready 了但流量还没切过来	Endpoint 异步传播	Endpoint/EndpointSlice 更新延迟、ingress reload
Pending 卡在 kubelet 之前	调度或 admission	上一篇 L5 的排障路径

可观测的三个黄金 metric：

- `kubelet_pod_start_duration_seconds`（kubelet 视角的 Pod 启动耗时）
- `kubelet_pod_worker_duration_seconds`（syncPod 本身耗时）
- `scheduler_pod_scheduling_duration_seconds`（前一段）

注意：这三者相加再加上"镜像未缓存时的下载时间"才接近体感冷启动时长。

Q9. 容器重启策略与 CrashLoopBackOff 的退避规律?

`restartPolicy`	行为
`Always`（默认）	容器退出就重启
`OnFailure`	仅当退出码非 0 时重启
`Never`	从不重启

CrashLoopBackOff 退避：**指数退避**，`10s → 20s → 40s → 80s → 160s → 300s`，上限 **300s（5 分钟）**（源码默认；版本相关请核对）。**容器稳定运行一段时间后计数值会重置。**

必答的坑：

- CrashLoopBackOff 是**结果不是原因**，必须看 `kubectl logs --previous` 和退出码；
- 退出码 **137 = SIGKILL**（多半 OOMKill 或主动 kill 或蒸发）；**143 = SIGTERM**（优雅终止中）；**139 = 段错误**；
- 应用启动慢但 liveness 阈值太小 ⇒ **应用还没起来就被反复杀**，正确解法是配 `startupProbe`。

```
livenessProbe:
  httpGet: {path: /healthz, port: 8080}
  periodSeconds: 10
  timeoutSeconds: 1
  failureThreshold: 3
startupProbe: # 关键：给慢启动应用豁免窗口
  httpGet: {path: /healthz, port: 8080}
  periodSeconds: 5
  failureThreshold: 60 # 最多等 5 * 60 = 300s
```

Q10. 三种探针的区别与常见误用?

探针	失败后果	典型误用
`livenessProbe`	**重启容器**（按 restartPolicy）	依赖了外部中间件 ⇒ DB 抖动导致全站重启雪崩
`readinessProbe`	**摘除 Endpoint**（不重启）	忘了配 ⇒ 滚动更新期间流量打到未就绪实例
`startupProbe`	失败不重启但阻塞 liveness 启动	慢启动应用没配 ⇒ 被 liveness 反复杀

必答两条铁律：

1. **liveness 只应检查"进程自身是否死锁/失活"**，绝不能检查外部依赖（DB、下游服务）——否则外部抖动会造成**全集群级重启风暴**；
2. **readiness + 滚动更新 +** `minReadySeconds` 才能避免"流量打到还没预热的实例"。

探针默认值（不配时的默认，版本相关）：`periodSeconds=10`、`timeoutSeconds=1`、`failureThreshold=3`、`successThreshold=1`。

Q11. Pod 优雅终止到底是怎么回事？为什么会出现 502？（超高频事故题）

```
1. API Server 收到删除请求 → Pod 进入 Terminating，设 deletionTimestamp
2. kubelet 发现 → 触发 SyncTerminatingPod
3. ① 执行 preStop hook（阻塞式，必须执行完）
   ② 随后发 SIGTERM 给容器主进程（PID 1）
4. 等待优雅退出，最多 terminationGracePeriodSeconds（默认 30s）
5. 超时 ⇒ SIGKILL
6. 卸载卷、CNI DEL、清理 cgroup
```

为什么会出现 502 / 连接中断（必答）：

> `deletionTimestamp` **落地的同时，Endpoint controller 会把该 Pod 移出 Endpoint**——但这个过程是**异步**的，kubelet 侧几乎同时就开始发 SIGTERM。于是：**进程已经停了，但 Ingress/kube-proxy/客户端侧的负载均衡列表还没更新** ⇒ 这段时间内的新请求打到"正在死亡"的 Pod ⇒ 502/连接重置。

标准解法（可直接套用）：

```
spec:
  terminationGracePeriodSeconds: 60 # 给足的总预算
  containers:
    - name: app
      lifecycle:
        preStop:
          exec:
            command: ["sh", "-c", "sleep 15"] # 15s < 60s，用于等待 endpoint 传播
      readinessProbe: # 提前把自己变成 notReady，让 LB 先摘
        httpGet: {path: /ready, port: 8080}
        periodSeconds: 2
```

配套：

- 应用自身处理 **SIGTERM**：停止接收新连接、等待存量请求处理完再退出；
- `Deployment` 配 `minReadySeconds` + `maxUnavailable: 0`（或 1）降低滚动更新抖动；
- 灰度/缩容场景配 **PodDisruptionBudget**；
- 长连接场景要留意 `keepalive` 时间。

> **注意**：`preStop` 时长**占用** `terminationGracePeriodSeconds` 预算，不是额外加的；强行缩短 grace 也可能造成"业务没退干净就被 SIGKILL"。

Q12. Eviction Manager 是怎么工作的？（驱逐）

目标：节点自身资源快撑不住时，**主动杀 Pod 保住节点**，与 scheduler 抢占无关。

概念	说明
Hard eviction	触及即杀，**没有宽限期**（用户可配，但立刻给容器发 SIGTERM，随后 SIGKILL）
Soft eviction	触及阈值后等待 `evictionSoftGracePeriod`，期间仍超限才杀
`evictionPressureTransitionPeriod`	压力消失后维持 Node condition 的时长（默认 5m），**防止 condition 抖动导致调度器反复 taint 抖动**
`evictionMinimumReclaim`	每次驱逐至少要回收多少，避免"刚够就停"反复触发
`--eviction-max-pod-grace-period`	软驱逐时给 Pod 的最大优雅期（超时即 SIGKILL）

默认阈值（社区默认，请以所用版本为准）：

信号	默认 Hard 阈值
`memory.available`	`< 100Mi`
`nodefs.available`	`< 10%`
`nodefs.inodesFree`	`< 5%`
`imagefs.available`	`< 15%`（存在独立 imagefs 时生效）
`pid.available`	由 `--pod-max-pids` 决定

驱逐顺序（必答）：

1. 先看**是否超过 requests**（实际用量超出 requests 的 Pod 优先被驱逐）；
2. 再看 **QoS 等级**：`BestEffort` → `Burstable` → `Guaranteed`（Guaranteed 只在自身超 limit 或系统级紧缺时被逐）；
3. 同优先级内再按资源占用量排序。

触发后同时会：给节点打 `MemoryPressure` / `DiskPressure` / `PIDPressure` condition ⇒ controller-manager 打对应 `NoSchedule` **taint**（注意不是 NoExecute）⇒ 新 Pod 不再调度上来。

可直接套用：

```
apiVersion: kubelet.config.k8s.io/v1beta1
kind: KubeletConfiguration
evictionHard:
  memory.available: "500Mi" # 建议：大内存节点按比例给，别用默认 100Mi
  nodefs.available: "10%"
  nodefs.inodesFree: "5%"
  imagefs.available: "15%"
evictionSoft:
  memory.available: "1Gi" # 更早介入，给 Pod 优雅退出的时间
evictionSoftGracePeriod:
  memory.available: "1m30s"
evictionMaxPodGracePeriod: 30 # 软驱逐时最多等 30s 就 SIGKILL
evictionMinimumReclaim:
  memory.available: "0Mi"
  nodefs.available: "500Mi"
  imagefs.available: "2Gi"
evictionPressureTransitionPeriod: "5m"
```

> **关键实践**：把 `memory.available` 的 hard 阈值从默认 100Mi 调大（*建议大内存节点至少 500Mi~1Gi，实测校准*），否则节点会在 OOM kill 的边缘才动手，来不及救。

Q13. Allocatable 是怎么算出来的？为什么要配 kube/system reserved?

```
节点 Capacity（整机）
  - kubeReserved （kubelet / 容器运行时 等节点守护进程）
  - systemReserved （sshd / systemd-journald / 云厂商 agent 等系统进程）
  - evictionThreshold （硬性预留不被算进可用）
  = Node Allocatable（调度器能用的量）
```

必答关键点（极易踩坑）：

> **只写 `-KubeReserved/systemReserved` 只是"记账"，不生效！** 必须同时指定 `--kube-reserved-cgroup` / `--system-reserved-cgroup`，kubelet 才会真的在对应 cgroup 上设限制；否则 Pod 依然可能把系统组件挤到 OOM。

```
apiVersion: kubelet.config.k8s.io/v1beta1
kind: KubeletConfiguration
kubeReserved:
  cpu: "500m"
  memory: "1Gi"
  ephemeral-storage: "1Gi"
systemReserved:
  cpu: "1000m"
  memory: "2Gi"
  ephemeral-storage: "2Gi"
# 真正生效的关键（cgroup v2 路径示例；v1 路径不同，需按版本确认）
kubeReservedCgroup: /runtime.slice
systemReservedCgroup: /system.slice
```

为什么不足预留会出事：节点上跑高密度 Pod 时，Pod 会把 CPU/内存/IO 吃光，导致 **kubelet 自身被 OOM / 抢不到 CPU ⇒ PLEG relist 变慢 ⇒ Node NotReady ⇒ 更大规模雪崩**。这是典型的"自杀式循环"。

Q14. QoS 等级怎么判定？OOM 时谁先死?

QoS	判定条件	内存紧张时的表现
Guaranteed	每个容器 CPU/内存都 `requests == limits`（都设且相等）	最后才被杀（只在自身超 limit 或系统级紧缺时）
Burstable	至少一个容器设了 requests 但不满足 Guaranteed	中间梯队
BestEffort	所有容器都没设 requests/limits	**第一个被杀**

OOM 判定顺序（内核 oom_score_adj 由 kubelet 设置）：BestEffort(`1000`) > Burstable(`min(1000, 999-1000*request/allocatable)` 量级) > Guaranteed(`-997`)。

必答要点：

- **"没设 limits" ≠ 无限制**：`limits.memory` 不设意味着容器可以用到节点内存上限，**极易被 OOMKill 且不受 cgroup 保护**；
- **CPU 不设 limits 不会 OOM，但可能被 throttling**（若设了 limits）；CPU 是不可压缩资源竞争中的"可压缩资源"，只影响延迟不影响存活；
- 想要"**稳定的节点级资源底线**" = `requests 合理 + QoS=Guaranteed + kube/system reserved 生效`。

Q15. cgroup v1 vs v2、CPU Manager、Memory Manager、Topology Manager?

管理层	作用	常见策略
cgroup driver	kubelet 用什么接口管理 cgroup	`systemd`（推荐，和主机 systemd 一致）vs `cgroupfs`
CPU Manager	独占核/绑核	`none`（默认）/ `static`（需 CPU request 为整数且 Guaranteed）
Memory Manager	内存 NUMA 亲和 / hugepages	`None` / `Static`
Topology Manager	跨 CPU/设备 NUMA 对齐	`none` / `best-effort` / `restricted` / **`single-numa-node`**

```
apiVersion: kubelet.config.k8s.io/v1beta1
kind: KubeletConfiguration
cgroupDriver: systemd
cpuManagerPolicy: static
# cpuManagerPolicyOptions:
# full-pcpus-only: "true" # 各版本实现差异较大，以 Release Notes 为准
reservedSystemCPUs: "0-1" # 给系统/守护进程预留，绝不让 Pod 独占所有核
memoryManagerPolicy: Static
topologyManagerPolicy: single-numa-node
topologyManagerScope: pod # container/pod，版本相关
```

必答的坑：

- `cpuManagerPolicy: static` 时必须配 `reservedSystemCPUs`，否则 kubelet/系统进程会被挤图层用一个核上抖动；
- **static 策略要求 CPU requests 为整数且 QoS=Guaranteed**，否则拿不到独占核，甚至报 `TopologyAffinityError`/拓扑亲和失败；
- GPU 场景常配 `single-numa-node`，但会导致**调度条件变苛刻、部分节点不可用**（这是可用性成本，要量化）；
- **cgroup v2 是新版本推荐路径**，v1 正在逐步退场；迁移前要确认 runtime/CNI/监控组件都支持。

Q16. 节点心跳与 Node NotReady 的完整时间账（超级必考）

```
kubelet 每 node-status-update-frequency（默认 10s） → 更新 Node status（条件变化时才更新）
kubelet 每 10s（lease duration 默认 40s） → 续约 Node Lease（轻量对象，专门给心跳用）
kube-controller-manager --node-monitor-grace-period（默认 50s）→ 超过则置 Node Ready=Unknown/False
       ↓ taint-manager 打 taint：node.kubernetes.io/not-ready 或 unreachable（NoExecute）
       ↓ Pod 默认 tolerationSeconds = 300s（toleration 是 kubelet/apiserver 给 Pod 注入的默认容忍）
       ↓ 5 分钟后才开始驱逐 Pod → 重新调度到其它节点
```

结论（必须背下来）：**节点宕机后，Pod 大约需要 50s + 300s = 约 5~6 分钟才会被真正重调度。** 业务上"节点挂了马上自愈"是不可能的，除非主动缩短 `tolerationSeconds`。

缩短恢复时间的办法（要在 YAML 里显式写）：

```
spec:
  tolerations:
    - key: node.kubernetes.io/not-ready
      operator: Exists
      effect: NoExecute
      tolerationSeconds: 30 # 建议：无状态快恢复业务设 30~60
    - key: node.kubernetes.io/unreachable
      operator: Exists
      effect: NoExecute
      tolerationSeconds: 30
```

必答的 trade-off：

- 缩短 `tolerationSeconds` 会让网络抖动/短时失联期间**误驱逐**有状态服务（数据库/分布式共识组件），可能引发数据风险；
- 有状态服务通常应**容忍更久**（甚至用 `tolerationSeconds` 更大值），并且必须有 PDB + 数据副本保护；
- 缩 `--node-monitor-grace-period` 同样有副作用：**控制面抖动会放大。**

> 🔗 **关联阅读**：第三篇 · C4. Node lifecycle controller 与 taint manager 的完整时间账

Q17. Static Pod vs DaemonSet? Mirror Pod 是什么?

维度	Static Pod	DaemonSet
定义位置	节点磁盘上的 manifest 文件（`--pod-manifest-path`，常见 `/etc/kubernetes/manifests`）	API Server 对象
管理者	**kubelet 自己**（file source）	DaemonSet controller
能否被 API 修改删除	不能（删了 kubelet 会重建）	可以
集群可见性	通过 **Mirror Pod**（只读镜像对象）在 apiserver 里可见	原生可见
典型用途	**引导控制面**：apiserver/etcd/kube-scheduler/controller-manager	日志采集、CNI、监控 agent

必答：

- Static Pod **不会被调度器调度**，不受 schedulerName 影响，也不参与 ResourceQuota；
- Mirror Pod 是只读的，`kubectl delete` 会立刻被 kubelet 重建（这就是"删不掉某些 kube-system Pod"的原因）；
- 名字后缀是节点名，例如 `kube-apiserver-node01`。

Q18. 镜像/容器/日志 GC 是怎么做的? 磁盘怎么防爆?

GC	触发	默认值（请以版本为准）
Image GC	磁盘使用率超过 `imageGCHighThresholdPercent`	High=85，Low=80（清理到 80 为止）；`imageMinimumGCAge` 默认 2m
Container GC	定期（默认 1m）	每个 Pod 保留的已死容器数（`--maximum-dead-containers-per-container`，旧曾是 2）/ 全局总数（旧 `--maximum-dead-containers=100`，**新版本这些开关正在被逐步弃用/隐藏**）
Container log rotation	按文件大小/个数	`--container-log-max-size=10Mi`、`--container-log-max-files=5`（默认）

```
apiVersion: kubelet.config.k8s.io/v1beta1
kind: KubeletConfiguration
imageGCHighThresholdPercent: 85
imageGCLowThresholdPercent: 80
imageMinimumGCAge: "2m"
containerLogMaxSize: "50Mi" # 建议：按业务日志量实测调整，别用默认 10Mi 硬扛重日志业务
containerLogMaxFiles: 5
```

必答实践：

- 生产上**建议把 `/var/lib/containerd`（imagefs）与 `/var/lib/kubelet`、`/`（nodefs）分离磁盘**，避免镜像把根盘撑爆导致 kubelet 本身挂掉；
- 日志要**同时施肥两层防护**：容器日志轮转（防单容器）+ 节点日志采集组件限流（防 agent 写爆）+ 节点 disk 告警；
- `imageMinimumGCAge` 太小会导致刚拉的镜像被清，**反复拉取**。

Q19. Device Plugin 是怎么注册到 kubelet 的?

```
Device plugin Pod
   | 1. 在宿主机 /var/lib/kubelet/device-plugins/<name>.sock 起 gRPC server
   ▼
kubelet PluginManager (watch socket 目录)
   | 2. 调用 Register(gRPC) → kubelet 建立长连接
   | 3. 周期性 ListAndWatch → kubelet 更新 Node allocatable: nvidia.com/gpu: 8
   ▼
kubelet /metrics? popularity: PLEGRESS/pod-resources
   | 4. Pod 创建时 kubelet 调 Allocate() → 拿回注入 env/mounts/annotations
   ▼
容器真正拿到设备
```

必答：

- kubelet 提供 **PodResources API**（`/var/lib/kubelet/pod-resources/kubelet.sock`），让第三方组件（如 GPU 监控/共享方案）能查询"某个 Pod 分到了哪些设备"；
- kubelet 侧只做**数量 + 健康状态**管理，**不理解设备拓扑/型号**（与上一篇 Q21 呼应）；
- Device plugin 挂掉 ⇒ 资源上报停止 ⇒ 已有 Pod 通常继续跑，但新 Pod 可能因资源不存在而失败（取决于实现）。

Q20. kubelet 的权限模型：Node Authorizer / NodeRestriction / TLS Bootstrap（安全必考）

三件事必须一起配：

1. **kubelet 认证**：TLS 客户端证书（`authentication.x509.clientCAFile`）+ Bootstrap kubeconfig；
2. **kubelet 授权**：`authorization.mode=Webhook`（走 SubjectAccessReview）；
3. **API Server 侧**：启用 `--authorization-mode=...,Node` + `NodeRestriction` **admission 插件**。

Node authorizer 能做什么（必须背）：

允许	禁止
读取**自身节点**绑定的 Pod（field selector `spec.nodeName=<self>`）	读其它节点的 Pod
读取自己 Pod **引用到的** Secret/ConfigMap/PVC	读任意 Secret
创建/更新自己的 Node status、Lease	改其它 Node
创建 events、创建 CSR	创建普通 Pod（防止 kubelet 被利用做权限提升）

> `NodeRestriction` **的存在意义**：即使攻击者拿到某节点的 kubelet 证书，也只能操作该节点的资源，**不能借助 apiserver 横向读取全集群 Secret**——这是"Pod 逃逸后的第二道防线"。

TLS Bootstrap 与证书轮换：

```
apiVersion: kubelet.config.k8s.io/v1beta1
kind: KubeletConfiguration
rotateCertificates: true # 客户端证书轮换（默认 true 更有利）
serverTLSBootstrap: true # serving 证书走 CSR 流程，需由 approve 通道批准（见下文说明）
authentication:
  anonymous:
    enabled: false # 必须关!
  webhook:
    enabled: true
  x509:
    clientCAFile: /etc/kubernetes/pki/ca.crt
authorization:
  mode: Webhook
readOnlyPort: 0 # 关闭 10255 只读端口，历史漏洞面
```

必答：

- **关闭 `readOnlyPort`（10255）、关闭匿名认证** 是两条硬基线；
- 证书轮转由 kubelet 的 certificate manager 负责；若 `serverTLSBootstrap: true`，serving CSR 需要 apiserver 侧有 **CSR approver**（具体 approve 方式因发行版/版本而异：有的靠内置 approver、有的靠 kubeadm/CCM 的控制器，也可能要手动 `kubectl certificate approve`），**这一点务必在自己的集群里验证过**；
- "证书过期导致 kubelet 无法注册"是经典事故，务必监控证书有效期（见 D3）。

Q21. ConfigMap/Secret 挂载为什么有时不更新?

使用方式	是否热更新	机制
环境变量注入	❌ 不更新	Pod 创建时快照，必须重建 Pod
Volume 挂载	✅ 会更新（有延迟）	kubelet 周期性同步（受 `--sync-frequency` 影响），通过符号链接**原子切换**；延迟一般是数十秒到 1 分钟级别，**需实测**
`subPath` 挂载	❌ 不更新	subPath 挂载的是具体文件路径，没有符号链接切换机制

必答：

- 想要"立即生效"，必须让应用**监听文件变化重载**或用 Reloader/argocd 之类方案触发滚动重启；
- Secret/ConfigMap 更新后容器进程不会自动重启，这一点常常误解。

Q22. 节点关机时能优雅吗? Graceful Node Shutdown 与 out-of-service taint?

机制	作用
Graceful Node Shutdown（新版本 GA，请以 Release Notes 为准）	kubelet 监听 systemd 的 inhibitor lock，收到关机信号时**主动进入节点优雅下线流程**，按优先级分阶段终止 Pod（普通 Pod → 关键 Pod，`shutdownGracePeriod` / `shutdownGracePeriodCriticalPods`）
Non-graceful shutdown handling	节点**没来得及关机就掉电**的场景：给节点打 `node.kubernetes.io/out-of-service` taint ⇒ 强制把该节点上有状态卷的 Pod 删除，让它们能在别处重建（**必须人工确认节点真的不可恢复**，否则会造成双写/脑裂风险）

```
apiVersion: kubelet.config.k8s.io/v1beta1
kind: KubeletConfiguration
shutdownGracePeriod: 60s
shutdownGracePeriodCriticalPods: 10s
```

必答 caveat：`out-of-service` taint 是**破坏性操作**（强制删除带卷 Pod），生产上必须有"节点已确认下线/不可恢复"的工单与确认流程（某些云厂商已在其托管服务中自动处理）。

Q23. kubelet 自身怎么加固? （生产基线）

项	建议值	理由
`readOnlyPort`	`0`	关闭 10255 未认证只读端口
`authentication.anonymous.enabled`	`false`	禁止匿名访问 10250
`authorization.mode`	`Webhook`	走 SubjectAccessReview
Node 授权 + NodeRestriction	启用	限制横向权限
`protectKernelDefaults`	`true`	防止 kubelet 把内核 sysctl 改错/依赖未设置的值
TLS cipher suites	关闭弱套件	合规
10250 端口	仅限管控网段可达	安全组/主机防火墙
`--streaming-connection-idle-timeout`	设合理值（如 4h）	防止 exec/port-forward 长连接无限挂，同时避免过早断开
版本	**不高于 apiserver**	版本倾斜策略要求

Q24. kubelet 提供哪些排障入口? （面试最后往往会问"你怎么看"）

入口	内容	用途
`https://<node>:10250/metrics`	kubelet + runtime + PLEG 指标	性能定位
`/metrics/cadvisor`	容器 CPU/内存/网络/FS 指标	资源画像
`/metrics/resource`	资源相关（各 cAdvisor Rread）	资源画像
`/metrics/probes`	探针相关 latency	探针开销分析
`/stats/summary`	Summary API	metrics-server 数据源
`/pods`	kubelet 视角的本节点 Pod 期望清单	**对比实际状态，判断 apiserver↔kubelet 是否同步**
`/var/lib/kubelet/pod-resources/kubelet.sock`	Pod↔设备映射	GPU/DPDK 类排查
`journalctl -u kubelet`	日志	权威解释

常用命令组合（可直接背）：

```
# 看 Pod 在 kubelet 侧的描述 vendor vs api
curl -sk --cert /var/lib/kubelet/pki/kubelet-client-current.pem \
     --key /var/lib/kubelet/pki/kubelet-client-current-key.pem \
     https://127.0.0.1:10250/pods | jq '.items[].metadata.name'

# 看 runtime 调用最慢的操作
curl -sk ... https://127.0.0.1:10250/metrics | grep kubelet_runtime_operations_duration

# 容器运行时侧（containerd）
ctr -n k8s.io containers ls | wc -l
crictl ps -a | wc -l
crictl inspectp <sandbox-id>
```

Part B • 大规模集群篇（16 题）

> 定义：**大规模 = ≥ 1000 节点 / 单节点 ≥ 50 Pod（高密度）**。

L0. 规模约束与版本倾斜政策（先立基准）

维度	官方目标（以官方文档为准）
每节点 Pod 数	≤ **110**（`maxPods` 默认即 110）
节点数	≤ 5000
集群 Pod 总数	≤ 150000

版本倾斜政策（必考）：

- kubelet **不得比 apiserver 新**；
- kubelet **最多可以比 apiserver 低 3 个 minor 版本**；
- 升级顺序：**先 apiserver（及其它控制面组件）→ 再 kubelet。**

> 面试可以这么收尾："所以大规模集群的 kubelet 往往是**多版本共存**的，压测和灰度必须以最老的 kubelet 版本为准。"

L1. 大机型（少节点高密度）还是小机型（多节点低密度）？

维度	大机型（如 128C/512G，跑 100 Pod）	小机型（如 8C/32G，跑 10 Pod）
资源利用率	高（碎片少、削峰填谷）	低
故障爆炸半径	**大**（一台宕机影响 100 Pod）	小
节点数	少 ⇒ 控制面压力小	多 ⇒ 心跳/watch 压力大
PLEG/GC 压力	**大**（单节点容器数多，relist 代价高）	小
镜像缓存命中	高（同节点同镜像仅拉一次）	低（每台都要拉）
特殊资源（GPU/NUMA）	更好排布	受限

标准答法：

> "这是**故障半径 vs 资源效率 vs 控制面压力**的三方权衡。实践做法是分层：**在线低延迟业务用中小机型控制爆炸半径，离线批处理用大机型提高装箱率**；单节点 Pod 数建议留 20% 余量，不要贴着 110 跑（要给 DaemonSet、系统组件与突发伸缩留位置）。"

L2. 高密度节点为什么容易 PLEG 不健康？怎么治理？

成本模型：GenericPLEG 每 **1s** relist 全部容器，单次成本约 O(容器数)，期间还要串行查询 runtime 状态。单节点 100+ Pod × 每 Pod 2~3 容器 ⇒ 每次 relist 要遍历数百个实体；一旦 runtime 变慢（创建/删除排队、磁盘 IO 高），relist 就可能超过 3m 阈值 ⇒ **Node NotReady，该节点所有 Pod 全受影响**。

治理清单：

手段	做法	代价
Evented PLEG（按需开启，版本相关）	运行时主动推事件 + 低频全量兜底	需 runtime 支持、版本较新
降低单节点 Pod 数	`maxPods` 下调到实际需要的 1.2 倍	利用率下降
节点池拆分	把"容器生命周期变更极频繁"的业务隔离到独立池	池化复杂度
治理 runtime	containerd shim 泄漏、overlayfs、inode、磁盘 IO	需专项排查
CPU 保障	`kubeReserved` + `reservedSystemCPUs` + cgroup 真正生效，确保 kubelet 不被抢 CPU	通常需重启节点生效

告警（要早于故障）：

```
# relist 耗时异常（阈值为建议初值，需按集群实测校准）
histogram_quantile(0.99, sum by (node, le) (rate(kubelet_pleg_relist_duration_seconds_bucket[5m]))) > 5

# 结合 Node Ready 状态判断
kube_node_status_condition{condition="Ready",status="false"} == 1
```

L3. 镜像拉取为什么会成为大规模瓶颈？怎么优化？

问题：大规模滚动发布时，成百上千节点同时拉同一个大镜像（AI/大数据镜像动辄 GB 级）⇒ **仓库出口带宽打满 + 节点被限流** ⇒ 大面积 `ImagePullBackOff`、Pod 冷启动暴涨。

证据：

- Events：`Failed to pull image ...: toomanyrequests`、`Error response from daemon: Get "https://...": net/http: TLS handshake timeout`；
- `kubelet_runtime_operations_duration_seconds{operation_type="pull_image"}` P99 暴涨。

优化手段：

手段	说明
并发拉取	`serializeImagePulls: false`（**默认是串行的！**）
并发上限	`maxParallelImagePulls`（引入版本较晚，请以 `KubeletConfiguration` 文档为准）
仓库 QPS 限流	`registryPullQPS` / `registryBurst`（*建议初值 5/10*）
P2P / 延迟加载	Dragonfly、Nydus/erofs 快照器等降低冷启动
镜像瘦身	多阶段构建、统一基础镜像、去无用层
就近仓库	避免跨区域拉取；用私有仓库 + 缓存/CDN
DaemonSet 预热	批量发布前先预热镜像到目标节点

```
apiVersion: kubelet.config.k8s.io/v1beta1
kind: KubeletConfiguration
serializeImagePulls: false # 关键：允许并发拉取不同的镜像
registryPullQPS: 5 # 建议初值，按仓库容量实测调整
registryBurst: 10
```

L4. Pod 冷启动慢的系统性治理清单?

按"这段开销能不能省掉"逐层砍：

环节	优化	备注
镜像下载	见 L3（并发/P2P/预热/瘦身）	通常是最大头
镜像解压	Nydus/erofs 延迟加载 + 快照器	大镜像收益明显
卷挂载	减少不必要的卷；关注 CSI 驱动健康	attach/detach 串行要警惕
CNI	DaemonSet 就绪时间、IPAM 性能	IP 耗尽会直接卡住 sandbox
initContainer	**串行执行**且各要拉镜像，能合并就合并	—
sidecar	每个 sidecar 都要拉镜像+探针，累积延迟	注意 sidecar 的重启语义
探针	`startupProbe` 给足窗口	过严会导致"刚起就被重启"
应用自身	预热、连接池初始化	业务侧优化
调度	见调度篇	—

度量方法（必给）：

```
# kubelet 视角 Pod 启动耗时（判断是不是 podWorker 本身慢）
histogram_quantile(0.99, sum by (le) (rate(kubelet_pod_worker_duration_seconds_bucket[5m])))
histogram_quantile(0.99, sum by (le) (rate(kubelet_pod_start_duration_seconds_bucket[5m])))
# runtime 分解（判断是 pull 慢还是 create 慢）
histogram_quantile(0.99, sum by (operation_type, le) (
  rate(kubelet_runtime_operations_duration_seconds_bucket[5m])))
```

L5. 驱逐风暴怎么防?

现象：一批节点同时进入 `MemoryPressure` / `DiskPressure` ⇒ 大量 Pod 被逐 ⇒ 重建后又落到压力节点/其它压力节点 ⇒ 抖动扩散。

根因：

- requests 严重虚低（实际用量远超 requests）⇒ 节点严重超卖；
- 突发批处理、大内存任务叠加；
- 日志/临时文件暴涨把磁盘打满；
- kube/system reserved 没做 ⇒ kubelet 自身资源被抢，形成恶性循环。

治理组合拳：

1. **requests 校准**：用实测用量（Prometheus / VPA recommendation）收敛 requests，让超卖率可控；
2. **预留与阈值**：`kubeReserved` / `systemReserved` 且 **cgroup 真正生效**；`evictionHard.memory.available` 提到 500Mi~1Gi（*建议，需实测*）；
3. **软驱逐 + 优雅期**：给 Pod 体面退出的机会，而不是直接 SIGKILL；
4. **在线/离线分节点池**（taint 隔离），避免离线流量冲击在线；
5. **PDB + 拓扑打散**，保证始终有可用实例；
6. **告警前置**：压力 condition 一出现就介入，而不是等真的驱逐。

```
# 节点压力 condition（依赖 kube-state-metrics）
kube_node_status_condition{condition=~"MemoryPressure|DiskPressure|PIDPressure",status="true"} == 1

# 被驱逐的 Pod 数量增长 => 风暴信号
increase(kube_pod_container_status_terminated_reason{reason="Evicted"}[15m])
```

L6. 磁盘/inode 治理：为什么节点磁盘最容易出事?

四类消耗：

1. **镜像**（imagefs）：滚动发布留下的旧镜像层；

2. **容器可写层 + 容器内临时文件**：业务写 `/tmp`、线程 dump、heap dump；
3. **容器日志**（`/var/log/pods/...`）：最容易忽视的一类，日志量暴涨直接撑爆根盘；
4. **kubelet/运行时元数据**：`/var/lib/kubelet/pods/...`、`/var/lib/containerd`（含 volume 挂载残留）。

治理清单：

手段	配置/做法
磁盘分离	imagefs（容器运行时）与 nodefs（根分区）使用不同磁盘/分区
容器日志轮转	`containerLogMaxSize` / `containerLogMaxFiles`（建议按业务实测调整）
镜像 GC 阈值	`imageGCHighThresholdPercent: 85` / Low 80；必要时下调 High 让 GC 更早介入
业务侧约束	通过 `ephemeral-storage` requests/limits 限制容器可写层 + emptyDir
残留清理	排查孤儿目录：`/var/lib/kubelet/pods/` 里已删除 Pod 的残留卷
监控	分别对 imagefs/nodefs 做容量告警，并监控 inode

```
# 给业务 Pod 加临时存储配额（防止单 Pod 写爆节点）
apiVersion: v1
kind: Pod
metadata:
  name: app
spec:
  containers:
    - name: app
      image: app:1.0
      resources:
        requests:
          ephemeral-storage: "1Gi"
        limits:
          ephemeral-storage: "2Gi" # 超限即被驱逐
```

```
# 容器日志占用的文件系统字节数（指标存在则说明版本支持）
sum by (node) (kubelet_container_log_filesystem_used_bytes)
```

L7. CNI/IPAM 在大规模下的坑?

坑	现象	解法
IP 耗尽	`FailedCreatePodSandBox: failed to allocate for range`	扩容 Pod CIDR / 调整 per-node IP 配额 / 回收泄漏 IP
IP 泄漏	Pod 已删除但 IPAM 未释放（`crictl` 与 IPAM 账不一致）	升级 CNI、检查 kubelet/runtime 异常退出导致的 CNI DEL 丢失
CNI 串行调用	节点批量起 Pod 时 sandbox 创建排队	选择支持并发/更高性能的 CNI，控制单节点 Pod 密度
CNI DaemonSet 未就绪	新扩容节点上的 Pod 全部卡 ContainerCreating	用 cloud provider/CNI 的 `network-unavailable` taint 做防呆
网络策略/转发规则膨胀	eBPF map 或 iptables 规则量过大导致 CPU 升高	规则治理、选型评估（见 L14）

L8. 节点心跳对控制面的压力模型?

账要这么算：

机制	频率	对象	成本
Node Lease（推荐）	每 10s 一次	`Lease` 轻量对象	低（设计目的就是把心跳从 Node 对象里剥离出来）
Node status 更新	状态变化时才更新，另有强制上报周期（默认 5m）	Node 对象（大）	高
Pod status 更新	状态变化时	Pod 对象	中（高密度/探针频繁时会放大）

5000 节点 ⇒ 仅心跳就约 500 QPS 的 Lease 更新，所以必须用 Lease 而不是 Node 对象。

关键参数：

```
apiVersion: kubelet.config.k8s.io/v1beta1
kind: KubeletConfiguration
nodeStatusUpdateFrequency: 10s # 默认值；调小会增加写放大，仅在强检测需求时调整
nodeStatusReportFrequency: 5m # 默认强制上报周期
```

必答 trade-off：把心跳频率调低（省控制面）必然会**拉长故障检测时间**，因为 `node-monitor-grace-period` 必须 ≥ 数倍心跳周期。所以这条只能配合平台侧的 SLO 来说。

L9. 证书过期 / 轮换风暴是经典事故

典型表现：某天一批节点同时 `Unable to register node ... x509: certificate has expired or is not yet valid`，节点被集群踢出，Pod 开始漂移。

原因：

- `rotateCertificates` 没开或 apiserver 侧 CSR approver 缺失 ⇒ 客户端证书过期；
- `serverTLSBootstrap` 开启但 **serving CSR 无人 approve** ⇒ kubelet serving 证书过期（注意：这会让 `kubectl logs/exec` 失败）；
- 节点时钟漂移 ⇒ 证书"尚未生效"或"已过期"误判。

治理：

1. 确认 `rotateCertificates: true` 且 apiserver 侧有可用的 CSR 审批机制（**必须在自己环境验证，不同发行版/版本实现方式不同**）；

2. 打开证书过期告警（见 D3）；

3. 所有节点强制 NTP/chrony 时钟同步并纳入监控；

4. 定期演练："把证书过期TTL临时调短 / 模拟 CSR 不批准"的 GameDay。

L10. 高密度 + NUMA/CPU Manager 的陷阱

陷阱	现象	建议
`cpuManagerPolicy: static` 未配 `reservedSystemCPUs`	kubelet/系统守护进程与 Pod 争抢，节点抖动	必须预留 1~2 核
CPU request 非整数 / 非 Guaranteed	拿不到独占核，行为与预期不符（`TopologyAffinityError`）	绑核场景必须 Guaranteed + 整数 CPU
`topologyManagerPolicy: single-numa-node`	**调度变苛刻**，多 NUMA 机器上部分 Pod 无法放下	权衡后使用；在线延迟敏感可用，普通业务建议 `best-effort` / `restricted`
hugepages 与 memory manager 组合复杂	内存分配失败	先在非生产压测

L11. kubelet 配置变更 / 升级怎么灰度与回滚?

原则：kubelet 是节点级组件，变更必须"以节点池为单位"灰度，且必须排水 Drain。

阶段	范围	观察（≥ 30min，覆盖一次业务高峰）	回滚触发条件
Step 1	**1 个节点**（先在预发池）	Node Ready、PLEG relist、Pod 启动 P99、业务 P99	任一指标超基线 1.5×
Step 2	**1% 节点**（同池内）	+ 驱逐率、运行时错误率	出现 Evicted 增长或 runtime error 上升
Step 3	10% → 50%	全量 SLO Dashboard	同上，或收到业务投诉
Step 4	100%	稳定 ≥ 24h	—

标准操作流程：

```
# 1) 排水并隔离
kubectl cordon <node>
kubectl drain <node> --ignore-daemonsets --delete-emptydir-data --grace-period=60 --timeout=10m

# 2) 变更（示例：改 KubeletConfiguration 后重启）
vim /var/lib/kubelet/config.yaml
systemctl restart kubelet
kubectl get node <node> -w # 等 Ready

# 3) 观察 30min ... 异常则回滚（恢复上个版本配置文件后再次排水重启）
cp /etc/kubernetes/kubelet-config.bak /var/lib/kubelet/config.yaml
systemctl restart kubelet

# 4) 恢复调度
kubectl uncordon <node>
```

必答的风险点：

- **需要先 `cordon` 再 `drain`**，并保证 PDB 允许排水（否则 drain 会挂住或违反可用性）；
- kubelet 版本**不能高于 apiserver**，升级顺序错了无法回退；
- `cpuManagerPolicy` 变更需要清理 `cpu_manager_state` 状态文件（带破坏性，**必须在排水后操作并写入工单**）；
- 回滚 kubelet 版本时若中间已发生过"静态 CPU 分配"等状态变更，务必确认状态一致性。

L12. 常见故障现象速查排障表（面试杀手锏）

现象	常见根因	第一步证据
Pending	调度层问题（见调度篇）	Events 是否有 `FailedScheduling`
ContainerCreating 卡住	镜像拉取慢/失败、CNI 失败、卷挂载失败、secret 缺失	Events（`FailedCreatePodSandBox` / `FailedMount` / `FailedAttachVolume`）、`crictl ps -a`
ImagePullBackOff	仓库认证/限流、镜像不存在、tag 错误	`Failed to pull image` reason、`crictl pull <img>` 在节点上验证
CrashLoopBackOff	应用异常退出、探针过严、配置缺失退出码非 0	`kubectl logs <pod> --previous`、退出码 137/139/143
OOMKilled	memory limit 过小或节点压力	`kube_pod_container_status_terminated_reason{reason="OOMKilled"}`、内核日志
Evicted	节点压力驱逐	`kubectl describe pod` reason=Evicted、节点 condition
Terminating 卡死	Finalizer 卡住、preStop 阻塞、卷卸载失败、runtime 僵死	`kubectl get pod -o yaml` 看 finalizers、`crictl` 状态
Unknown	节点失联后 kubelet 无法上报，Pod 被保留至驱逐完成	Node Ready 状态、lease 心跳
CreateContainerConfigError	ConfigMap/Secret key 不存在	Events 明确指出缺失的 key
整个节点 Pod 全unknown	**PLEG 不健康 / kubelet 挂了 / 网络断**	`journalctl -u kubelet` + PLEG relist 指标

L13. 版本治理：为什么必须盯 version skew?

- kubelet 可低于 apiserver 最多 3 个 minor，不可高于；
- **升级顺序**：控制面（含 apiserver）→ 其它 controller/kube-proxy → kubelet；

- 大规模集群常见问题是"**部分节点忘了升**"，导致行为不一致（如 GC 参数、cgroup v2 支持、feature gate 差异），要提供**版本台账 + 自动巡检**。

```
# 各节点的 kubelet 版本分布（kube-state-metrics）
count by (kubelet_version) (kube_node_info)
```

L14. 大规模下 kubelet 侧可观测性的成本问题

项	风险	建议
`/metrics/cadvisor` 抓取	每指标维度大，抓取成本高	按需精简 metric relabeling，或降低抓取频率
metrics-server → summary API	每个节点调用 kubelet summary API，大规模下压力大	注意采样频率与副本数；新版倾向于 **kepler/cAdvisor 直采 + Prometheus** 组合
`--enable-features` 打开过多	指标维度爆炸	按需开启
探针频率过高	每个探针都要发请求，Pod 多时开销明显	`periodSeconds` 按 SLO 给，不是越小越好

> **经验建议（需实测校准）**：单节点 Pod 数 > 50 时，应核算 `kubelet` 自身 CPU/内存占用，把"节点 agent 开销"计入集群容量规划（一般给节点预留数百 MiB 内存和 1~2 核 CPU 给系统 + 守护进程，具体以实测为准）。

L15. kubelet 安全加固清单（可直接用于巡检）

- [ ] `readOnlyPort: 0`（10255 关闭）
- [ ] `authentication.anonymous.enabled: false`
- [ ] `authorization.mode: Webhook`
- [ ] apiserver 侧启用 Node authorizer + `NodeRestriction` admission
- [ ] 10250 端口仅管控网段可达（安全组/iptables）
- [ ] `protectKernelDefaults: true`
- [ ] 禁用弱 TLS 套件
- [ ] 证书自动轮换开启，并有 CSR approve 通道
- [ ] 关闭 `privileged` Pod 的默认允许（上层策略：PSA/PSS、OPA/Kyverno）
- [ ] 节点 hostPath 挂载受控（Pod Security Admission 或策略引擎）
- [ ] 审计：谁在 exec/attach/port-forward（`audit-policy` + kubelet webhook）

L16. 给你一个新节点池，怎么设计 kubelet 基线？（综合题）

按这个顺序答：

1. **先问 SLO 与工作负载画像**：在线/离线？有无绑核/大页/GPU？单节点跑多少 Pod？
2. **机型与密度**：确定机器规格（CPU/内存/磁盘/NUMA）与 `maxPods`（留 20% 余量）；
3. **资源预留**：`kubeReserved` / `systemReserved` + cgroup 真正生效 + `reservedSystemCPUs`；
4. **驱逐策略**：hard/soft 阈值 + 优雅期 + minimumReclaim；
5. **磁盘**：镜像/容器数据与根分区分离；日志轮转 + GC 阈值；
6. **拉取策略**：`serializeImagePulls: false` + 仓库 QPS + 是否需要 P2P；
7. **CPU/Memory/Topology manager**：按 NUMA 需求定策略；
8. **安全基线**：L15 清单全勾；
9. **可观测性**：kubelet + runtime + PLEG + 节点 condition 告警；
10. **变更治理**：镜像化 kubelet 配置 + 节点池灰度 + drain/cordon 流程 + 回滚工单；
11. **演练**：杀 kubelet、注内存压力、模拟列表 Large image pull，验证 SLO 与 MTTR。

Part C · 反模式清单（可直接用于生产巡检）

每条含：反模式 / 证据 / 风险等级 / 修复动作 / 验证方式

#	反模式	证据（怎么发现）	等级	修复动作	验证方式
C1	容器不设 requests/limits	`kube_pod_container_resource_requests` 为空；节点频繁 OOM	🔴 高	LimitRange 默认值 + 用量画像校准	OOMKill 数下降并稳定
C2	只设 `kubeReserved` / `systemReserved` **没配对应 cgroup**	节点上 kubelet/ 系统进程被 OOM，`config.yaml` 无 reservedCgroup	🔴 高	补齐 `kubeReservedCgroup` / `systemReservedCgroup` 并重启	压力测试下 kubelet 进程不再被杀
C3	`livenessProbe` 检查外部依赖（DB/下游）	依赖抖动时全集群 Pod 重启	🔴 高	liveness 只查自身活性，依赖检查放 readiness	依赖故障演练不再触发重启
C4	不配 `preStop` / `terminationGracePeriodSeconds` 过短	发布窗口出现 502/连接重置	🔴 高	preStop sleep + 放宽 grace + 应用处理 SIGTERM	滚动更新期间错误率不上升
C5	用默认 `evictionHard.memory.available=100Mi`	触发过晚，来不及救	🔴 高	调大并按机型实测	内存压力下有足够时间优雅退出
C6	`readOnlyPort` 未关 / 匿名认证开启	安全扫描、10255 可访问	🔴 高	`readOnlyPort: 0` + 关闭匿名	端口探测关闭
C7	`serializeImagePulls` 保持默认 true	发布时间长、P99 启动高	🟠 中	设 false + 调整仓库限流	发布总时长下降（实测）
C8	日志无上限（用默认 10Mi/5 却发现写爆）	节点 disk 打满、Evicted 增多	🟠 中	按业务调整轮转参数 + ephemeral-storage limit	DiskPressure condition 消失
C9	单节点密度满载 110 Pod	PLEG relist 抖动、NotReady 事件	🟠 中	留 20% 余量；按业务画像下调 maxPods	`kubelet_pleg_relist_duration_se` 回归基线
C10	静态绑核未配 `reservedSystemCPUs`	节点抖动、kubelet 抢不到 CPU	🟠 中	预留 1~2 核	重启后资源画像稳定
C11	证书轮换无 approver 通道	CSR 长期 Pending、证书过期	🔴 高	完善 approve 机制 + 过期告警	演练临时证书过期可自动续签
C12	无节点 PLEG/runtime 监控	故障才发现 relist 早就变慢	🟠 中	建立 D3 告警集	注入故障 5min 内告警
C13	kubelet 版本高于 apiserver / 长期不升	`kube_node_info` 版本台账异常	🟠 中	遵循 skew 策略先升控制面	巡检台账无越界节点
C14	接到态服务未设更长 `tolerationSeconds`	网络抖动被误驱逐	🟠 中	有状态服务放宽容忍或固定节点	网络抖动演练不触发重调度

Part D • 可观测性：SLO、指标与告警（可直接落地）

D1. 建议的 kubelet 侧 SLO（初始建议值，必须实测校准）

SLI	建议目标（起点）	计算方式
节点 Ready 率（按池）	≥ 99.9%	`kube_node_status_condition{condition="Ready",status="true"}`
Pod 启动 P99（kubelet 视角）	≤ 5s（不含镜像首次拉取）	`kubelet_pod_start_duration_seconds`
节点压力驱逐频率	在线池 ≈ 0	`increase(kube_pod_container_status_terminated_reason{reason="Evicted"}[1h])`
PLEG relist P99	≤ 5s	`kubelet_pleg_relist_duration_seconds`
证书剩余有效期	> 30 天告警	`kubelet_certificate_manager_*_expiration_seconds`
容器重启率	单 Deployment \< 0.1%/h	`kube_pod_container_status_restarts_total`

D2. 核心指标清单（先在节点上 `curl -k https://127.0.0.1:10250/metrics` 核对）

指标	含义	用于
`kubelet_pod_start_duration_seconds`	Pod 从进入 kubelet 到所有容器 started 的耗时	冷启动 SLO
`kubelet_pod_worker_duration_seconds`	syncPod 执行耗时（按 operation 分）	定位 kubelet 内部慢点
`kubelet_pleg_relist_duration_seconds` / `..._interval_seconds`	PLEG relist 耗时/间隔	PLEG 健康度
`kubelet_runtime_operations_total` / `_duration_seconds` / `_errors_total`	CRI 调用量/耗时/错误（按 `operation_type`）	**runtime 排障首选**
`kubelet_running_pods` / `kubelet_running_containers`	本机 Pod/容器数	密度与泄漏
`kubelet_started_pods_total` / `kubelet_started_containers_total`	累计启动数	变更/抖动频率
`kubelet_container_log_filesystem_used_bytes`	容器日志占用	磁盘治理
`kubelet_cgroup_manager_duration_seconds`	cgroup 操作耗时	资源压力下的性能
`kubelet_certificate_manager_client_expiration_seconds` / `_server_expiration_seconds`	证书剩余有效期	过期事故预防
`kubelet_node_name`（info 类）	节点存在性/元信息	up 检测
`rest_client_request_duration_seconds`	kubelet 作为客户端访问 apiserver 的耗时	区分是 apiserver 慢还是本节点慢
kube-state-metrics：`kube_node_status_condition`	Node Ready/Pressure condition	节点可用性
kube-state-metrics：`kube_pod_container_status_terminated_reason`	终止原因（Evicted / OOMKilled / Error）	驱逐与 OOM 治理

> **注意**：指标名在不同版本会有增删/改名，**上线前必须先在目标版本节点核对。**

D3. Prometheus 告警规则（可直接粘贴，阈值请实测校准）

```
groups:
  - name: kubelet
    rules:
      - alert: NodeNotReady
        expr: kube_node_status_condition{condition="Ready",status="false"} == 1
        for: 3m
        labels: {severity: critical}
        annotations:
          summary: "节点 {{ $labels.node }} NotReady"

      - alert: NodeUnderPressure
        expr: |
          kube_node_status_condition{condition=~"MemoryPressure|DiskPressure|PIDPressure",status="true"} == 1
        for: 5m
        labels: {severity: warning}
        annotations:
          summary: "节点 {{ $labels.node }} 处于 {{ $labels.condition }}，可能即将驱逐 Pod"

      - alert: PodEvictedSpike
        expr: increase(kube_pod_container_status_terminated_reason{reason="Evicted"}[15m]) > 10
        for: 2m
        labels: {severity: critical}
        annotations:
          summary: "15 分钟内 {{ $value }} 个容器被驱逐，疑似驱逐风暴"

      - alert: PlegRelistSlow
        expr: |
          histogram_quantile(0.99,
            sum by (instance, le) (rate(kubelet_pleg_relist_duration_seconds_bucket[5m]))) > 5
        for: 10m
        labels: {severity: warning}
        annotations:
          summary: "实例 {{ $labels.instance }} PLEG relist P99 达 {{ $value }}s"

      - alert: RuntimeOperationsError
        expr: |
          sum by (instance, operation_type) (rate(kubelet_runtime_operations_errors_total[5m])) > 0.1
        for: 10m
        labels: {severity: warning}
        annotations:
          summary: "{{ $labels.instance }} 的 {{ $labels.operation_type }} 持续报错"

      - alert: ContainerRestartSpike
        expr: increase(kube_pod_container_status_restarts_total[30m]) > 10
        for: 5m
        labels: {severity: warning}
        annotations:
          summary: "容器 {{ $labels.namespace }}/{{ $labels.pod }} 30 分钟内重启 {{ $value }} 次"

      - alert: KubeletCertExpiringSoon
        expr: kubelet_certificate_manager_client_expiration_seconds < 30 * 86400
        for: 1h
        labels: {severity: critical}
        annotations:
          summary: "kubelet 客户端证书剩余 {{ $value | humanizeDuration }}，请检查自动轮换"

      - alert: NodeDiskPressurePredictive
        expr: |
          sum by (node) (kubelet_container_log_filesystem_used_bytes) > 20 * 1024 * 1024 * 1024
        for: 30m
        labels: {severity: info}
        annotations:
          summary: "节点 {{ $labels.node }} 容器日志占用超 20GiB，检查日志轮转"
```

抓取注意：kubelet 的 `10250` 是 HTTPS 且需要认证，Prometheus 侧要配 `scheme: https` + bearer token/kubeconfig CA + RBAC（`system:kubelet-api-admin` 或自建最小权限 ClusterRole）。**权限要最小化，别图省事给 cluster-admin。**

🔗 **关联阅读**：第三篇 · G2. Prometheus 告警规则（可直接粘贴，阈值需实测校准）

Part E • 演练（GameDay）计划表

场景	注入方式	期望表现	验证的 SLO
kubelet 进程挂掉	`systemctl stop kubelet`	节点在 grace period 后 NotReady，Pod 在默认 5min 容忍后被重调度	MTTR、容错时间账
PLEG 不健康	制造 runtime 卡顿/大量容器创建	PLEG 告警先于 NotReady 触发	告警前置能力
内存压力	跑内存压测容器	先软驱逐再硬驱逐，PDB 不被突破	驱逐 SLO
磁盘打满	写大文件至根分区	DiskPressure + Pod 被逐，日志轮转生效	磁盘治理
镜像仓库限流	限制 registry 出口	并发拉取开关生效，P2P/预热兜底	冷启动 SLO
优雅下线	`kubectl delete pod` 观察	无 502、preStop 生效	发布可用性
证书过期	临时缩短 TTL / 不 approve CSR	告警触发且轮换通道可用	证书治理
节点非优雅掉电	强制关机	`out-of-service` 流程按工单执行，带卷 Pod 能重建	有状态容灾

Part F • 10 张面试速记卡（临阵磨枪）

1. **kubelet 是节点代理**：负责建/启/停容器、挂卷、配网络、跑探针、节点压力驱逐、上报状态；**不负责调度，不管集群级编排。**
2. **SyncLoop 四路事件**：configCh（期望） / plegCh（真实） / syncCh（1m 兜底） / lifecycleCh（失败重放）+ housekeeping（GC）。
3. **PLEG 是节点故障放大器**：relist 每 1s 一次、channel 容量 1000、连续 3m 没成功 ⇒ Node NotReady；解法 Evented PLEG / 降密度 / 保 kubelet CPU。
4. **五分钟账**：10s 心跳 → 50s `node-monitor-grace-period` → NotReady/Unreachable taint → Pod 默认 `tolerationSeconds=300` → **约 5~6 分钟才真正重调度。**
5. **驱逐 ≠ 抢占**：驱逐是 kubelet 保节点（看 requests 超限 + QoS：BestEffort > Burstable > Guaranteed）；抢占是调度器为了新 Pod。
6. **预留只记账不生效**：`kubeReserved` / `systemReserved` 必须配对应 `xxxReservedCgroup` 才真正限住。
7. **优雅终止三件事**：`preStop`（等 endpoint 传播）+ `terminationGracePeriodSeconds` + 应用处理 SIGTERM；三者缺一就会 502。
8. **探针铁律**：liveness 只查自身活性（查依赖会引发重启雪崩）；慢启动应用必配 `startupProbe`。
9. **镜像拉取默认串行**：`serializeImagePulls` 默认 true，大规模必须 `false` + 仓库限流 + P2P/预热。
10. **版本 skew**：kubelet 不可高于 apiserver，最多低 3 个 minor；升级顺序先控制面后 kubelet，灰度按节点池 + drain/cordon + PDB。

第七篇 · 网络与 CNI

网络模型 · kube-proxy · DNS · NetworkPolicy · 数据面

0. 使用说明

0.1 网络方向的面试考察层次

层级	考点	表现
L1 会用	会写 Service/Ingress/NetworkPolicy YAML	只会 apply
L2 懂原理	说得清 Pod 出网封包路径、CNI 干了什么、ClusterIP 为什么不存在实体网卡、DNS 怎么走	能画封包路径图
L3 懂选型	能比较 Overlay / 路由 / ENI 直路由三类模式的代价，能报出对应 CNI 的组件与配置项	能说 trade-off
L4 有事故经验	能讲 DNS 5 秒抖动、conntrack 满、iptables 规则膨胀、IP 耗尽、MTU 黑洞的根因与治理	有 PromQL、有灰度、有回滚

答题模板：封包路径（从 Pod 网卡一路讲到目标网卡）→ 谁在这条路上做了什么（CNI/kube-proxy/CoreDNS）→ 给证据（配置项 / metric / 内核参数）→ 补坑。

Part A • 原理篇（26 题）

A1. Kubernetes 的网络模型是什么？为什么这样设计？

四大假设（必须背）：

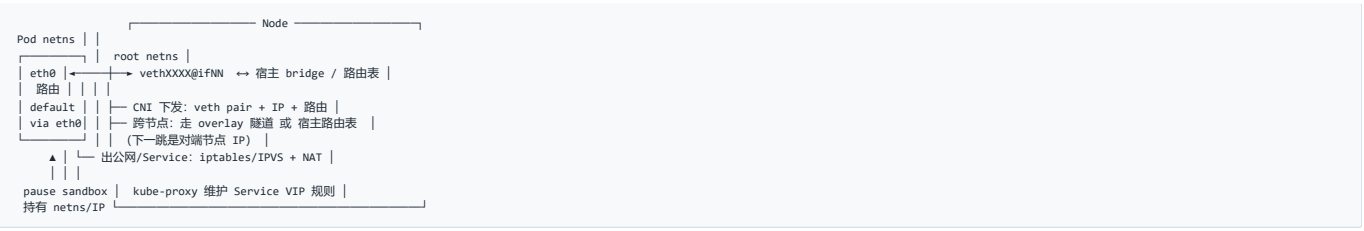
1. **所有 Pod 之间可以不经过 NAT 直接通信**（同一 IP 平面）；
2. **Node 与所有 Pod 之间可以直接通信**（方便健康检查、日志采集、管控）；
3. **Pod 看到的自己的 IP 与别人看到的它的 IP 是同一个 IP**（身份自洽）；
4. 网络的实现由 **CNI 插件**负责，K8s 本身只定义要求、不实现。

为什么这样设计：把网络（网络地址分配与互通）与编排（容器生命周期）解耦，任何满足 CNI 规范的实现都能接入；同时"扁平 IP 网络"让服务发现、网络策略、可观测性都能统一在一套 IP 语义上。

必答补充：K8s **不要求**：

- 不做负载均衡（那是 Service/kube-proxy/LB 的事）；
- 不做 DNS（CoreDNS 是可选组件）；
- 不做网络策略（NetworkPolicy 是 API，需要 CNI 实现，**不实现就完全不生效且默认放通**）。

A2. Pod 到底是怎么"有网"的？把封包路径讲一遍



创建流程（kubelet → CNI）：

```
kubelet → CRI RunPodSandbox → CNI ADD
    Plugin 完成：
      1. 创建 netns（由 sandbox pause 容器持有）
      2. 建 veth pair：一端进 Pod netns 改名 eth0，另一端留在 root netns
      3. 把宿主侧 veth 挂到 bridge / 或直接由 CNI 写路由
      4. 调用 IPAM 申请 IP
      5. 配置 Pod 内 IP + 默认路由 + ARP 代理/neigh
      6. 返回结果（含 IP）→ kubelet 写入 Pod status
删除时反向 CNI DEL
```

必答要点：

- **Pod IP 实际在 pause / sandbox 容器上**，业务容器共享这个 netns（所以同一 Pod 内各容器 `localhost` 互通）；
- **CNI ADD 失败 ⇒** `FailedCreatePodSandBox`，Pod 永远进不了 Running；
- CNI 配置文件在 `/etc/cni/net.d/*.conf*`（按字典序取第一个有效配置），二进制在 `/opt/cni/bin/`。

A3. CNI 规范本身规定了什么？为什么 kubelet 不自己实现 IPAM？

CNI 规定的三件事：

1. 配置文件格式（JSON，含 `cniVersion`、`type`、IPAM 段）；
2. 二进制插件通过 **stdin + 环境变量 + 配置文件路径**被调用；
3. 必须实现 `ADD` / `DEL` / `CHECK` / `VERSION`（kubelet 主要只用 ADD/DEL）。

插件分类：

类型	职责	例子
main / interface 插件	真正建网卡和数据面	bridge、ptp、macvlan、ipvlan、host-local route、calico、cilium-cni
IPAM 插件	只管地址分配	host-local、static、whereabouts、Calico/Cilium 自带的 IPAM
meta 插件	组合与增强	portmap（hostPort）、bandwidth（限速）、tuning、firewall

链式调用：配置里是 list 时，插件按顺序执行，**前一个的结果传给后一个。**

为什么不由 kubelet 实现：这本身就是为了解耦——不同环境（云网络、IDC、DPDK、SR-IOV）需要完全不同的数据面实现，kubelet 只负责"按配置调用二进制、拿到 IP、写进 Pod status"，别的一概不管：

```
{
  "cniVersion": "1.0.0",
  "name": "k8s-pod-network",
  "type": "calico",
  "ipam": { "type": "calico-ipam" }
}
```

必答：

- CNI v1.0+（v0.4 已弃用），新插件应支持 CHECK/GC（`garbage-collect`）；
- **CNI-in-CNI（chained 插件）**用于 Multus 多网卡场景（主线网卡 + 业务网卡）；
- kubelet 侧只看 CNI 返回结果的 IP，其它一概不管。

A4. ClusterIP 到底是什么？为什么说它"没有实体网卡"？

ClusterIP 是一个纯数据面规则意义上的 VIP：

- 它没有对应的网络设备，`ip addr` 里**在本机看不到 ClusterIP**（有些 CNI/kube-proxy 实现会把它放到 dummy 接口 `kube-ipvs0` 上，便于外部发布 CLUSTER IP 到 BGP；这是实现细节）；
- 它的"存在"靠的是 **kube-proxy 下发的 iptables/IPVS/eBPF 规则**：把目的地址换成后端 Pod IP（DNAT）。

```
Pod: curl http://svc:80
  ① Pod DNS 解析 svc → ClusterIP
  ② 封包 dst=ClusterIP:80 → 走 Pod 默认路由出 netns
  ③ 宿主 netns 的 iptables/IPVS/eBPF 规则命中 → DNAT dst=PodIP:8080
  ④ 若跨节点 → 由 CNI 数据面送到对端节点
  ⑤ 到达后端 Pod
```

三种 Service VIP 类型：

类型	特点	注意
ClusterIP	集群内 VIP	最常用
NodePort	每个 Node 上开端口转发	端口范围默认 30000-32767
LoadBalancer	云厂商 LB → NodePort/直连 Pod	云 LB 与云 CNI 的交互因厂商而异
Headless（`clusterIP: None`）	**不做 VIP**，DNS 直接返回 Pod IP 列表	有状态服务的关键

必答：

- Headless Service 才有 `podname.svc.ns.svc.cluster.local` 的稳定 DNS 记录与 SRV 记录，是 StatefulSet 稳定网络身份的基础；
- `externalName` 是纯 CNAME，没有 selector 也没有转发规则。

A5. kube-proxy 三种模式怎么演进的？

模式	机制	复杂度	现状
userspace（最初）	kube-proxy 进程自己做代理转发	差	已弃用
iptables	内核 netfilter 规则做 DNAT	规则数 O(规则数)，匹配是**顺序遍历**	长期默认
IPVS	内核 LVS，哈希表 O(1)	规则再多也是常数查找	大规模可选
nftables（新 gewgaws）	nftables 后端	较新	**版本相关（alpha/beta 阶段），需按 Release Notes 确认**
Cilium socket-LB（替代 kube-proxy）	eBPF 在 socket 层完成转换	最好	方案侧选择

iptables 模式的核心缺点（必答）：

1. **规则数量与 Service/Endpoint 规模线性相关**，且更新要 `iptables-restore` 全量下发；
2. 匹配是**链表顺序匹配**，规则越多单包延迟越高；
3. 负载均衡用 statistic random 模式，**无真正的调度算法**（IPVS 才有 rr/lc/sh/dh/sed/nq）；
4. 大量 Endpoints 更新会引发周期性 CPU 尖峰。

切换 IPVS 的配置（可直接套用）：

```
apiVersion: kubeproxy.config.k8s.io/v1alpha1
kind: KubeProxyConfiguration
mode: "ipvs"
ipvs:
  scheduler: "rr" # rr | lc | dh | sh | sed | nq
  strictARP: true # 配合 MetalLB 等 ARP 型 LB 时需要
  syncPeriod: 30s
  minSyncPeriod: 5s
conntrack:
  maxPerCore: 131072 # 建议初值，需按节点连接数实测校准
  tcpEstablishedTimeout: 24h
  tcpCloseWaitTimeout: 1h
```

> **注意**：切到 IPVS **不等于不用 iptables**——IPVS 模式下 kube-proxy 仍会用 iptables/ipset 处理 MASQUERADE(SNAT)、包过滤与健康检查相关流程。

A6. EndpointSlice 为什么取代了 Endpoints？

背景：一个 Service 下若有 5000 个 Endpoint，Endpoints 是一个巨大的单对象，任何 endpoint 变更都要**全对象更新**，引发：etcd 写放大 + 所有 watch 者收到大 payload。

EndpointSlice 的改进：

- 将 Endpoints **切片**，默认每个 slice 最多 100 个 endpoint（由 kube-controller-manager 的 `--max-endpoints-per-slice` 控制，默认值请核对）；
- 变更时只更新受影响的那一小块 ⇒ 显著减少 etcd 与网络开销；
- 支持 `TopologyAwareHints` / `trafficDistribution: PreferClose`（亲和到同 zone），降低跨可用区流量与延迟（该功能随版本演进，请以 Release Notes 为准）。

```
# EndpointSlice 数量与变更频率（判断有没有因 endpoint 变动导致的写放大）
kube_endpointslice_info
```

A7. 集群 DNS 是怎么工作的？`ndots:5` 是什么坑？（超高频）

```
Pod /etc/resolv.conf:
  nameserver 10.96.0.10 # ClusterIP of kube-dns Service
  search default.svc.cluster.local svc.cluster.local cluster.local
  options ndots:5 # ← 关键点
```

规则：名字里 **点的个数 \< ndots（默认 5）** 时，会先依次按 search 域拼上再做 A/AAAA 查询，**没采中后再查原名。**

```
Pod 内部访问 www.baidu.com（1 个点 < 5）：
  1) www.baidu.com.default.svc.cluster.local → NXDOMAIN
  2) www.baidu.com.svc.cluster.local → NXDOMAIN
  3) www.baidu.com.cluster.local → NXDOMAIN
  4) www.baidu.com → 命中
⇒ QPS 被放大 4~5 倍（IPv4+IPv6 双查询时更夸张，可达 8~10 倍，需实测）
```

治理手段：

手段	做法
写法优化	外部域名写 FQDN，末尾加 `.`（`www.baidu.com.`）⇒ 直接查原名
降 ndots	Pod 侧 `dnsConfig.options: [{name: ndots, value: "2"}]`（**要验证不影响内部短域名**）
NodeLocal DNSCache	节点本地缓存（DaemonSet），查询走本地 TCP/UDP loopback，绕开 conntrack 竞态、降低 CoreDNS QPS
CoreDNS 扩容	`cluster-proportional-autoscaler` 按节点/CPU 数自动扩容

```
# Pod 侧降 ndots（注意：改了可能导致依赖 search 域的短域名失效，务必压测）
spec:
  dnsConfig:
    options:
      - name: ndots
        value: "2"
```

NodeLocal DNSCache 部署（关键片段）：

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-local-dns
  namespace: kube-system
spec:
  template:
    spec:
      hostNetwork: true
      dnsPolicy: ClusterFirstWithHostNet # 关键：hostNetwork 下仍需集群 DNS
      priorityClassName: system-node-critical
      tolerations:
        - key: CriticalAddonsOnly
          operator: Exists
      containers:
        - name: node-cache
          image: registry.k8s.io/dns/k8s-dns-node-cache:1.22.28 # 版本请按自身镜像仓库核对
          args:
            - "-localip"
            - "169.254.20.10" # 每个节点上一个 link-local 地址
            - "-conf"
            - "/etc/coredns/Corefile"
            - "-upstreamsvc"
            - "kube-dns-upstream" # 上游指向原 CoreDNS Service
          ports:
            - containerPort: 53
              name: dns
              protocol: UDP
```

```
# CoreDNS 的 CM：给 nodelocaldns 提供 ClusterIP 做 upstream
kubectl -n kube-system get svc kube-dns-upstream -o yaml # 无 selector 的 Service，手动维护 endpoint
```

> **部署要点**：切换 nodelocaldns 需要把 Pod 的 `nameserver` 指到 `169.254.20.10` ——通常通过 kubelet 的 `--cluster-dns` 配置 + 节点替换灰度实现，**属于有损变更，必须灰度。**

A8. 经典的"DNS 间歇性 5 秒超时"是怎么回事？（事故级必考题）

现象：Pod 内请求偶发卡 5 秒后报错，概率极低但持续存在，和业务/网络负载无关。

根因：**内核 conntrack + UDP NAT 的竞态**（社区长期已知问题）。同一时刻并发的 DNS 查询复用了同一个源端口，DNAT 时 conntrack 表项冲突 ⇒ 其中一个包被丢弃。UDP 无重传机制得当 ⇒ glibc resolver 等 5 秒超时（resolver 默认 `timeout:5 attempts:2` 相关）。

证据：

- CoreDNS 侧看到请求没到 / 重复查询；
- 节点 `nf_conntrack_insert_failed` / `conntrack -S` 的 `insert_failed` 计数增长；
- Pod 侧 `resolv.conf` 里有两个以上 search 域 + ndots=5。

解法（按优先级）：

1. **NodeLocal DNSCache**（最推荐）：本机链路-local 通信，绕开 conntrack NAT 竞态，且降低 CoreDNS QPS；
2. 内核侧：升级到包含相关修复的内核 / 开启 conntrack 相关优化（**具体 patch/参数因内核版本而异，务必在测试环境验证**）；
3. 应用侧使用 TCP / 或 `options single-request-reopen`（串行化，减少冲突概率）；
4. 降低查询次数（FQDN + 降 ndots）。

```
# CoreDNS 侧看超时与错误分布（指标含义请与目标版本核对）
histogram_quantile(0.99, sum by (le) (rate(coredns_dns_request_duration_seconds_bucket[5m])))
sum(rate(coredns_dns_requests_total{rcode="SERVFAIL"}[5m]))
# 缓存命中率（NodeLocal 上线后应显著上升）
sum(rate(coredns_cache_hits_total[5m])) / clamp_min(sum(rate(coredns_cache_hits_total[5m]) + rate(coredns_cache_misses_total[5m])), 0.001)
```

A9. NetworkPolicy 是怎么生效的？为什么配了没效果？

关键点：NetworkPolicy 只是**声明式 API**，K8s 自身（kubelet/kube-proxy）**不实现它**，必须由 **CNI 数据面**落地：

CNI	实现载体
Calico	iptables/ipsets 或 eBPF
Cilium	eBPF（支持 L7、DNS-aware）
Antrea	OVS
Flannel	**不支持**（需搭配其它方案，如 Canal/额外方案）

三个必答的语义：

1. **默认全通**：没有任何 NetworkPolicy 选中某 Pod 时，它可被任意来源访问；
2. **一旦被任意 ingress 策略选中，未被显式允许的方向即拒绝**（egress 同理，互不影响）；
3. namespace/podSelector/ipBlock 三要素组合，标签驱动的。

最佳实践（可直接套用）：先 default-deny，再逐步放行。

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: prod
spec:
  podSelector: {} # 选中本 ns 所有 Pod
  policyTypes: ["Ingress", "Egress"]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-app-to-db
  namespace: prod
spec:
  podSelector:
    matchLabels: {app: db}
  policyTypes: ["Ingress"]
  ingress:
    - from:
        - podSelector:
            matchLabels: {app: web}
      ports:
        - protocol: TCP
          port: 5432
```

排坑清单：

- 忘了包含 DNS 出口 ⇒ 加了 default-deny 后**所有域名解析失败**（务必放行 kube-system/CoreDNS 的 53/UDP、TCP）；
- 健康检查来源（云 LB / ingress / kubelet probe）被拦 ⇒ Probe 失败；
- NetworkPolicy **不影响 Service VIP 层面的负载均衡**，只在 Pod↔Pod 入口/出口处生效；
- 大规模下 iptables 实现的策略会有**规则数线性膨胀**问题（见 C8）。

A10. Overlay vs 路由 vs 云网卡直连：三种数据面模式的本质区别？

模式	代表	封包开销	对底层网络的要求	适用
Overlay（隧道）	Flannel VXLAN、Calico VXLAN/IPIP、Cilium VXLAN/Geneve	有封装头（VXLAN 约 50 字节 IPv4），MTU 变小	只要求节点 IP 互通（可跨子网/跨 VPC）	通用、云上跨可用区、网络不可控场景
路由（纯三层）	Calico BGP / host-gw / Cilium native routing	无封装，直接路由转发	**要求 Pod CIDR 在底层网络可路由**（同二层/路由可达）	IDC、可控网络、追求性能
CrossSubnet 混合	Calico `IPIPMode: CrossSubnet`	同子网直路由、跨子网才封装	—	**IDC/云上最常见折中方案**
云网卡（ENI 直连）	AWS VPC CNI、aliyun Terway、Azure CNI（部分模式）	Pod 直接拿 VPC IP，无隧道	依赖云厂商 API 分配网卡/IP	云托管集群、需要 Pod 具备 VPC IP 的场景

必答：选择的关键不是"哪个更快"，而是 **底层网络是否允许你把 Pod CIDR 通告出去**，以及 **是否需要 Pod IP 在集群外可见（安全组/审计/直连）**。

A11. VXLAN / IPIP / Geneve 的封装代价怎么算？MTU 为什么老出问题？

封装	IPv4 头部开销（估算）	说明
VXLAN	~50 字节	外层 IP(20) + UDP(8) + VXLAN(8) + 以太网(14) 量级
IPIP	~20 字节	只加一层 IP 头
Geneve	可变（通常 ≥50）	支持可变长选项

MTU 计算公式（建议值）：`Pod MTU ≈ 宿主网卡 MTU - 封装开销`，1500 网卡上：

- VXLAN ⇒ 常被配置为 **1440/1450**（不同 CNI 默认值不同，**请以所用插件文档为准**）；
- IPIP ⇒ **1480**。

MTU 不一致的典型故障：

- 大包被丢弃但小包正常（ping 通但 curl 大文件卡死）；
- TLS 握手卡住、gRPC 偶发 reset；
- ICMP 被禁 ⇒ PMTUD 失效 ⇒ **黑洞连接**（只能靠 TCP MSS clamp 缓解）。

治理：

- 统一 MTU 配置（CNI 侧显式指定，不依赖自动探测结果漂移）；
- 必要时开启 **MSS clamping**（Calico/Cilium 都有相关配置）；
- 网络侧放通必要 ICMP type 3 code 4。

A12. 跨节点 Pod 通信：Calico BGP 模式下封包怎么走？

```
Pod A (10.244.1.10) → Node1 路由表：10.244.2.0/24 via Node2-IP dev eth0
    → Node2 收到 → 路由表：10.244.2.10 dev calixxxx scope link → Pod B
这条路被 **BIRD/BGP** 在节点之间互相通告：每个 Node 把自己持有的 Pod CIDR 通告给其它 Node
```

组件分工（必答）：

组件	作用
Felix	数据面执行者：下发路由、iptables/ipsets/eBPF 规则、写 ACL
BIRD（或 GoBGP / FRR）	BGP 客户端，与其他节点/ToR 交换路由
confd	监听 datastore 变化，生成 BIRD 配置
Typha	**大规模关键**：替所有 Felix 统一 watch datastore，减少 datastore 连接数与压力
calico-kube-controllers	同步 node/pod 状态、GC IPAM 泄漏等

规模要点：BGP **full-mesh 的连接数是 O(n²)**，节点数上百后必须上 **Route Reflector（RR）**（*建议 >100 节点启用 RR，具体阈值以官方文档与实测为准*），或由 ToR 交换机做 RR（推荐生产做法，把 BGP 交给网络设备）。

A13. Cilium 的 eBPF 数据面有什么不同？

核心思路：把转发、负载均衡、策略执行放进内核 eBPF 程序（挂在网络设备 TC/XDP、socket 层、cgroup），**尽量绕开 netfilter（iptables）这条链表。**

```
传统：nic → netfilter PREROUTING → iptables 链顺序匹配 → conntrack → 转发
Cilium：nic(TC ingress eBPF) → eBPF map 查表 → 直接转发 / 尾调用，策略在 map 里
```

关键能力：

能力	说明
kube-proxy 替代（socket LB）	在 `connect()` 阶段就把 ClusterIP 转成 PodIP，**不经过 NAT/iptables**；连接建立更快，且天然保真客户端 IP
策略规模可扩展	基于 identity（label 派生）+ eBPF map，**避免每 endpoint 一套规则链**
Hubble 可观测	flow/policy/drop 层面的可观测，含 **drop reason**
XDP / Bandwidth Manager	XDP 加速；EDT + FQ 做带宽整形（替代 tc tbf）
ClusterMesh	多集群网络与服务互联
加密	WireGuard / IPsec 透明加密（有性能成本，需实测）

必答的限制/坑：

- 对 **内核版本有要求**（不同功能要求不同版本，请以 Cilium 官方矩阵核对）；
- socket LB 与某些场景需要特别注意：**hostPort**、某些策略/NAT 交互、与 hostNetwork Pod 的行为差异等，升级前请逐个验证；
- eBPF maps 占用内存，大规模要关注 map 大小限制；
- 排障工具从 `iptables-save` 变成 `bpftool` / `cilium monitor` / Hubble —— **团队技能栈要换。**

A14. 云厂商 CNI（ENI 类）和自建 CNI 的本质差异？

维度	云 CNI（如 VPC CNI / Terway）	自建 CNI（Calico/Cilium 通用模式）
Pod IP 来源	**VPC 地址空间**（集群外也可见、可路由）	集群私有 CIDR
数据面	通常无需隧道（直路由 ENI）	视模式而定

维度	云 CNI（如 VPC CNI / Terway）	自建 CNI（Calico/Cilium 通用模式）
节点 Pod 数上限	**受实例网卡规格限制**（ENI 数 × 每网卡 IP 数）	主要受 `--max-pods` 和 CIDR 规划限制
安全能力	可用 VPC 安全组精细管控 Pod	依赖 NetworkPolicy
运维依赖	强依赖云 API（限流/配额风险）	依赖集群自身组件
迁移性	差（绑定厂商）	好

必答的成本/风险点：

- 云 API 限流会导致 **Pod 创建变慢甚至失败**（IP 分配失败 → `FailedCreatePodSandBox`）；
- **IP 规划必须和 VPC 子网协调**，Pod 数暴涨容易撞到子网耗尽；
- 故障排查要跨越"云网络 + K8s"两层边界。

A15. AWS VPC CNI 的几种模式？（进阶高频）

模式	机制	优势	注意
Secondary IP（默认）	每个 ENI 分配多个辅助私网 IP，Pod 直接用	原生 VPC IP	单节点 Pod 数 = ENI 数 × 每 ENI IP 数（受实例规格限制）
Prefix Delegation	每个 ENI 分配 /28 前缀而非单个 IP	**大幅提升单节点 Pod 密度**，减少 ENI 与 API 调用	需要子网有连续的空闲地址块；版本/参数相关（`ENABLE_PREFIX_DELEGATION`）
Custom Networking	Pod 使用独立子网/安全组	IP 隔离、缓解主子网耗尽	子网规划复杂
IPv6 模式	Pod 拿 IPv6 地址	彻底缓解 IPv4 枯竭	要求配套组件支持双栈

关键调优参数（环境变量，取值以官方版本为准）：

```
# aws-node DaemonSet 常用参数（示意，务必按官方文档核对默认值）
env:
  - name: WARM_ENI_TARGET # 期望保持的空闲 ENI 数
    value: "1"
  - name: WARM_IP_TARGET # 期望保持的空闲 IP 数（推荐用它替代 WARM_ENI_TARGET）
    value: "5"
  - name: MINIMUM_IP_TARGET # 至少保留的 IP 数
    value: "10"
  - name: ENABLE_PREFIX_DELEGATION
    value: "true"
  - name: AWS_VPC_K8S_CNI_CUSTOM_NETWORK_CFG
    value: "false"
```

> **实践建议（需实测校准）**：`WARM_IP_TARGET` 太小 ⇒ 突发扩容时等 IP 导致 Pod 起得慢；太大 ⇒ 长期占用 VPC 地址造成浪费。应结合"Pod 创建速率"与"子网剩余 IP"一起调，并配专属告警。

A16. 常见 CNI 方案横向对比（选型总表，必须能背）

方案	数据面	主要模式	NetworkPolicy	L7 策略	加密	观测/IPAM	适用
Flannel	简单	VXLAN / host-gw	❌ 不支持	❌	❌	弱 / host-local	小集群、学习、无策略诉求
Calico	iptables/ipsets 或 eBPF	BGP / VXLAN / IPIP / CrossSubnet / NoEncap	✅	部分（需企业版/组合方案）	✅ WireGuard	Felix 指标+Typha 规模扩展	**IDC/通用主力方案**
Cilium	eBPF	native routing / VXLAN / Geneve / ENI	✅✅	✅（Envoy/L7，DNS-aware）	WireGuard / IPsec	**Hubble**（最强）	大规模、高性能、可观测需求
Antrea	OVS/OVS-based	Geneve / noEncap / hybrid	✅	部分	IPsec	Octant/Flow 相关能力	VMware 生态、复杂 L2 场景
OVN-Kubernetes	OVN/OVS	Geneve / localnet	✅	—	—	较弱	Openshift / OpenStack 生态
Multus + SR-IOV/DPDK	硬件（VF）	多网卡直通	依赖主 CNI	—	—	—	NFV / 电信 / 超高性能诉求
云厂商 CNI	云网络直通	ENI	视实现	—	—	—	托管集群、需 VPC IP

选型决策树（可直接背）：

```
1. 是否必须使用 VPC IP（合规/安全组/外部直连）？
     是 → 云厂商 CNI（并评估 Pod 密度是否满足）
     否 ↓
2. 底层网络能通告 Pod CIDR（IDC/同二层/可控路由）？
     能 → Calico BGP / Cilium native routing（性能最优）
     不能 ↓
3. 同可用区/同二层居多、少量跨子网？
     是 → CrossSubnet（IPIP/VXLAN 混合）
     否 → 全 Overlay（VXLAN/Geneve）
4. 是否需要 L7/DNS策略、强可观测、是否规模 ≥ 1000 节点或 Service 很多？
     是 → Cilium（eBPF）
5. 纯 NFV/电信 或需要第二块网卡 → Multus + SR-IOV/DPDK/macvlan（接受运维复杂度）
```

A17. Calico 的 IPPool 各项参数怎么配?

```
apiVersion: projectcalico.org/v3
kind: IPPool
metadata:
  name: default-ipv4-ippool
spec:
  cidr: 10.244.0.0/16
  blockSize: 26 # 每个 block 大小，默认 26（64 个地址/块）
  ipipMode: CrossSubnet # Always | CrossSubnet | Never
  vxlanMode: Never # Always | CrossSubnet | Never
  natOutgoing: true # Pod 出集群时对 Pod IP 做 SNAT
  nodeSelector: all()
  allowedUse: ["Workload", "Tunnel"]
```

必答：

参数	说明	坑
`blockSize`	IPAM 分配粒度，**创建后不可改**（必须新建 IPPool 迁移，属重操作）	**上线前必须规划**
`ipipMode: CrossSubnet`	同二层直路由、跨子网才封装	云上最常见配置
`NATOutgoing`	出公网 SNAT	关闭后 Pod IP 需在外部可路由
`allowedUse`	区分 workload 地址与隧道地址（较新能力，版本相关）	—

IPAM 泄漏排查：节点删除后仍占用 block ⇒ 用 `calicoctl ipam check` 识别泄漏并 `calicoctl ipam release --force`（**破坏性操作，须走工单**）。

A18. conntrack 是什么？为什么会成为瓶颈?

conntrack（连接跟踪）：netfilter 维护每条连接的元组表，支撑 NAT 与状态防火墙。

瓶颈表现：

- 日志 `nf_conntrack: table full, dropping packet`；
- 新连接建立失败/延迟上升，尤其短连接高并发场景。

关键参数：

```
# 查看容量与当前使用
sysctl net.netfilter.nf_conntrack_max
cat /proc/sys/net/netfilter/nf_conntrack_count

# 调整（建议值需实测校准；写入 /etc/sysctl.d/ 持久化）
net.netfilter.nf_conntrack_max = 1048576
net.netfilter.nf_conntrack_buckets = 262144 # hashsize，通常为 max/4
net.netfilter.nf_conntrack_tcp_timeout_established = 86400
net.netfilter.nf_conntrack_tcp_timeout_time_wait = 30
```

关系公式：`nf_conntrack_max = nf_conntrack_buckets * 4`（hashsize 是底层哈希桶数）。

治理：

- 按"节点预计并发连接数 × 安全系数"规划 `nf_conntrack_max`（**必须实测**）；
- 缩短 TIME_WAIT 等 timeout；
- **IPVS 模式仍依赖 conntrack**（NAT 模式），而 **Cilium socket-LB 路径不依赖 conntrack**，这是后者在高并发短连接场景的显著优势之一。

```
# 依赖 node_exporter，指标名按 exporter 版本核对
node_nf_conntrack_entries / node_nf_conntrack_entries_limit > 0.8
increase(node_nf_conntrack_insert_failed[5m]) > 0
```

A19. `externalTrafficPolicy` 的两个取值有什么区别？（高频）

取值	行为	优点	缺点
Cluster（默认）	NodePort/LB 流量可在**集群内再次转发**到任意节点的 Pod	负载均衡均匀	**丢失客户端真实 IP**（做了额外一跳 SNAT），多一跳延迟
Local	只转发给**本节点上存在的** Pod	**保留源 IP**、少一跳	若本节点无该服务 Pod ⇒ 该节点健康检查失败；可能**负载不均**

```
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  type: LoadBalancer
  externalTrafficPolicy: Local # 保留源 IP
  # healthCheckNodePort 会由系统自动分配，或显式指定
```

必答的组合要点：

- `Local` 模式下要确保 LB 的**健康检查能感知"本节点有无可用 Pod"**（K8s 会通过 healthz/healthCheckNodePort 返回），否则流量打到无后端节点会失败；
- `Local` + 单副本服务 ⇒ LB 只会把流量导到唯一节点，**请结合多副本 + 拓扑打散**使用；
- 想要源 IP 又想要均衡，可用代理协议（PROXY protocol）或上层网关注入 header。

A20. Pod 访问自身的 Service（hairpin）为什么有时不通?

hairpin/NAT loopback：Pod 通过 Service VIP 访问"自己作为后端"的服务时，封包从 Pod 出去、又被转发回同一个 Pod，需要内核/网桥支持 hairpin（`hairpin_mode`）。

- 排查：kubelet/hostPort/`docker-compose` 类的 bridge hairpin；也可在 Service 侧要求 CNI/kube-proxy 正确处理（不同 CNI 行为不同）；
- 经典解法：应用层避免"自己访问自己的 Service"，改用 localhost 或 Pod IP 直连；
- 也可检查 CNI 是否开启 `hairpinMode` / `promiscuous` 相关配置。

A21. hostNetwork / hostPort 与 DNS 策略?

```
spec:
  hostNetwork: true
  dnsPolicy: ClusterFirstWithHostNet # 关键：hostNetwork 下默认 dnsPolicy 会退化为 Default
```

`dnsPolicy`	含义
`ClusterFirst`（默认）	先问集群 DNS（CoreDNS），不中的再转发上游
`ClusterFirstWithHostNet`	**hostNetwork Pod 想用集群 DNS 必须显式配**
`Default`	用节点 `/etc/resolv.conf`
`None`	完全自定义，配合 `dnsConfig`

hostPort 的坑：

- 端口占用是**节点级**的 ⇒ 同节点多副本必然冲突（调度器看不到这种冲突，会照常调度，然后 Pod 起来失败）；
- 优先用 NodePort / HostPort 替代方案，或用 DaemonSet 固定单实例。

A22. IPv4/IPv6 双栈要注意什么?

- 双栈要求集群/Pod/Service CIDR 均成对配置（`--service-cluster-ip-range`、`Pod CIDR` 双段）；
- Service 的 `ipFamilyPolicy`（`SingleStack` / `PreferDualStack` / `RequiredDualStack`）与 `ipFamilies` 控制行为；
- **注意**：修改已有 Service 的 ipFamilies 有严格约束（通常不可从单栈直接改双栈，需重建）；
- 应用、监控、外部 LB 都要支持 IPv6，端到端才成立。

A23. Service、Ingress、Gateway API 到底什么关系?

层次	对象	职责	谁实现
L3/L4（东西向）	Service（ClusterIP/NodePort/LB）	Pod 集合的稳定入口与负载均衡	kube-proxy / CNI
L7 南北向	Ingress	HTTP/HTTPS 路由、TLS 终止	Ingress Controller（NGINX/APISIX/Traefik…）
新一代南北向/东西向 API	**Gateway API**（GatewayClass/Gateway/HTTPRoute…）	面向角色的 API（基础设施/集群运维/应用开发者），支持更丰富的路由与流量能力	各类 Gateway 实现

为什么有 Gateway API：Ingress 表达能力弱（靠 annotation 扩展导致实现之间不可移植），Gateway API 通过**角色划分 + 细粒度路由 + 明确的一致性测试**来解决。它已是社区推荐的演进方向（**具体 API 版本与 GA 范围请以 Release Notes 为准**）。

A24. Service Mesh 和 CNI 是竞争关系吗?

不是，职责分层不同：

层	提供什么	代表
CNI（L3/L4）	IP 分配、Pod 互通、网络策略执行	Calico / Cilium
Mesh（L7）	服务治理：重试/超时/熔断/mTLS/灰度/可观测	Istio / Linkerd
CNI 的上层扩展	Cilium 提供部分 L7 策略、可替代部分 mesh 功能（如 mTLS/L7 策略）	Cilium Service Mesh
Sidecar vs Ambient	Sidecar：每 Pod 注入代理；Ambient：节点级 ztunnel + 按需 waypoint	Istio ambient

必答：Mesh 是"应用层治理"，CNI 是"网络层连通与策略"。两者可以同时存在（Cilium 提供 CNI + mesh 部分能力），但要注意**双重策略叠加导致的排障复杂度**。

A25. 网络加密（WireGuard / IPsec / mTLS）怎么选?

方案	层	成本	适用
WireGuard（CNI 提供）	L3 Pod 流量	相对较低、现代算法，**有性能成本需实测**	节点间 Pod 流量加密
IPsec 透明加密	L3	相对更高	合规强要求
mTLS（Mesh）	L7	CPU 成本主要在 sidecar	服务身份 + 应用层加密
TLS Ingress/gateway	L7 边界	边缘处	南北向加密

必答：加密会引入 CPU 与 MTU/分片开销（隧道类尤其），且会让**抓包排障变难**，要先按业务合规要求确定范围（全量 vs 特定 Namespace），并做压测。

A26. 带宽限速怎么做?

两条路线:

```
# ① CNI bandwidth 插件: 靠 Pod annotation + tc tbf
apiVersion: v1
kind: Pod
metadata:
  annotations:
    kubernetes.io/ingress-bandwidth: 10M
    kubernetes.io/egress-bandwidth: 20M
spec:
  containers:
    - name: app
      image: app:1.0
```

```
# ② Cilium Bandwidth Manager: EDT + FQ (更现代的实现, 需对应版本支持)
# helm values 示意
bandwidthManager:
  enabled: true
  bbr: true # 开启后影响拥塞控制策略, 务必压测
```

必答: tc tbf 的实现在高 PPS 场景自身有开销; 限速策略通常只用于防"吵闹邻居", 不要当核心流量治理手段。

Part B • 方案 / 模式专项篇（18 题）

> 这一篇按"具体方案和它的各种模式"出牌，是大厂面试网络方向最爱追问的部分。

B1. Flannel 的三种后端有什么区别？

后端	机制	要求	性能	备注
VXLAN（默认）	封装 VXLAN，MAC-in-UDP	只要节点 IP 互通	有封装开销	通用
host-gw	在节点上写静态路由，宿主充当网关	**要求所有节点在同一个二层网络**	接近裸网络（无封装）	二层场景最优，注意 ARP 规模
UDP（老）	用户态转发	—	差	基本不再使用

必答结论：Flannel **不支持 NetworkPolicy**，所以生产上要么换成 Calico/Cilium，要么叠加策略方案。Flannel 只适合"先跑起来"的场景。

B2. Flannel host-gw 为什么不能用跨子网？会出什么问题？

host-gw 依赖"**下一跳就是对端节点 IP，且二层可达**"（靠 ARP 解析 MAC）。跨三层子网后网关不做这个路由 ⇒ ARP 无法解析 ⇒ 封包出不去 ⇒ Pod 之间不通（**且常常是部分通、部分不通**，很容易误判）。

B3. Calico：`ipipMode` 与 `vxlanMode` 怎么选？

场景	建议模式
IDC 全网可被通告 Pod CIDR	`Never`（纯 BGP 路由），性能最好
IDC 同二层 + 少量跨三层	`IPIPMode: CrossSubnet`
云上 VPC（通常不允许通告 Pod 路由/不支持 BGP）	`VXLANMode: Always` 或 `CrossSubnet`
需要 Pod IP 在外部可见	`Never` + 通告路由给 ToR + `natOutgoing: false`

> **注意**：通常二者互斥（不应同时 Always），具体互斥规则以 Calico 版本文档为准。

B4. BGP 规模问题：什么时候必须上 Route Reflector？

问题：full mesh 的 BGP 会话数是 `n(n-1)/2`，500 节点 ⇒ 约 12.5 万条会话，节点 CPU 与收敛都吃不消。

解法：

1. **Route Reflector（RR）**：节点只与少数 RR 建会话 ⇒ 会话数降到 O(n)；
2. **ToR 做 RR**（生产推荐）：把 BGP 交给网络设备，节点只与本机 ToR 建会话；
3. 启用 **BFD / graceful restart** 加快收敛、避免短暂中断（参数按网络团队规范）。

> **建议（需实测校准）**：节点数超过约 100 就该引入 RR；具体阈值按下例回填。

```
# Calico 侧大致流程（示意，请以所用版本文档为准）
# 1）停用默认 node-to-node mesh
calicoctl patch bgpconfiguration default -p '{"spec":{"nodeToNodeMeshEnabled":false}}'
# 2）部署 CR 或用 ToR 承担 RR 角色，配置 node 的 peerSelector
```

B5. Calico Typha 是干什么用的？不装会怎样？

作用：所有节点的 Felix 都要 watch 集群状态（datastore）。节点数很多时，**每个 Felix 一条 watch ⇒ 对 kube-apiserver/etcd 产生 O(n) 倍读压力**。

Typha 就是这层聚合/代理：Felix 连 Typha，Typha 统一 watch 再广播 ⇒ 大幅降低 datastore 连接数与 watch 量。

告警与实践：

- Typha 挂了/数量不足 ⇒ Felix 连不上、策略更新延迟（表现为策略"不生效/延迟生效"）；
- Typha 应跑多副本（一般 3 个以上），并跨故障域分布，**是必须监控的组件**。

B6. Calico eBPF 模式和 iptables 模式怎么选？

维度	iptables/ipsets	eBPF
规则随规模增长	线性、单包延迟随规则数上升	map 查表，规模扩展性更好
源 IP 保留（kube-proxy 场景）	会做 SNAT	可原生保留
内核要求	低	**有版本要求**（按 Cilium/Calico 官方矩阵核对）
排障工具链	iptables/ipset	bpftool / calico-bpf / 日志
成熟度/心智负担	低	中高

建议：要"跟着社区未来走 + 性能敏感 + 规模大"就上 eBPF；要"稳定第一、团队 skill 有限"先用 iptables 模式，并在**独立的灰度集群/节点池**验证 eBPF。

B7. Cilium：native routing vs tunnel vs ENI 模式？

模式	机制	适用
native routing	无封装，依赖 Pod CIDR 可路由（或 ToR 路由）	IDC/可控网络，性能最优

模式	机制	适用
tunnel（vxlan/geneve）	封装	底层不可控/跨子网
ENI / cloud provider IPAM	Pod 用云网卡 IP（cluster-pool vs ENI IPAM 差别）	云托管、需 VPC IP
ClusterMesh	多集群 Pod IP 互通 + 全局服务	多集群/容灾

```
# Helm values 示意 — 具体取值严格按所用版本 Chart 文档核对
tunnel: vxlan
kubeProxyReplacement: strict # 替代 kube-proxy (or probe/partial/false)
ipv4:
  enabled: true
ipam:
  mode: cluster-pool # or eni / kubernetes
autoDirectNodeRoutes: true
bpf:
  masquerade: true
  hostLegacyRouting: false
bandwidthManager:
  enabled: true
hubble:
  enabled: true
  metrics:
    enabled:
      - dns:query
      - flow:source=namespace
```

B8. Cilium 替换 kube-proxy 到底值不值？（进阶高频）

收益：

- ClusterIP 转换发生在 socket 层，**不经过 iptables/nat/nf_conntrack** ⇒ 规则数不再随规模线性增长，短连接高并发场景收益明显；
- **天然保留客户端 IP**（不需要 `externalTrafficPolicy: Local` 的副作用）；
- 支持 DSR（Direct Server Return）、Maglev 一致性哈希等能力（特性可用性以版本为准）。

风险/限制：

- 依赖较新内核（请查矩阵）；
- 与 hostPort、部分 NAT/策略组合存在已知限制，**必须在灰度环境逐项验证**；
- 排障方式改变（要会 `cilium monitor` / Hubble / bpftool）；
- 一些依赖 kube-proxy iptables 规则的遗留组件可能失效。

给出决策口径：

> "值不值取决于 Service 数量与短连接 QPS。Service 上百/上千、短连接密集、已经出现 iptables 规则更新导致的 CPU 尖峰时，替换收益很明显；否则就把它当成下一个演进阶段，先在独立节点池验证。"

B9. Cilium Hubble 能解决哪些传统手段解决不了的问题？

能力	传统方式	Hubble
Pod 间流量拓扑	靠 tcpdump + 人肉	直接出 flow 拓扑
丢包原因（drop reason）	几乎无法定位	输出 policy/l4/ct 等原因
L7（HTTP/DNS/Kafka）可见性	需 sidecar/抓包	部分场景原生可见
策略命中/拒绝	只能靠试	可见 denied/forwarded 判断

排障示例：

```
hubble observe --pod app --verdict DROPPED -f
cilium monitor --type drop
# 或直接看 drop reason 分布指标（指标名按版本核对）
```

B10. ClusterMesh（多集群）要注意什么？

- 要求各集群 **Pod CIDR 不冲突**（这是最容易翻车的点，必须提前规划）；
- 控制面通过共享 KV/ClusterMesh API server 互通，**这是强依赖组件**，要做 HA；
- 全局服务（global service）跨集群发现带来的**跨集群故障域**要评估；
- 跨集群流量会走隧道 ⇒ 关注 MTU 与延迟；
- SRE 视角：**先明确 SLO（是否需要跨集群容灾）再上，别为炫技上。**

B11. Antrea（OVS 系）适合什么场景？

- 基于 OVS/OVS 流水线做数据面，适合**复杂 L2/L3 场景**（如 VM 与 Pod 混部、OpenStack/VMware 生态）；
- 支持 NetworkPolicy 与 Antrea 自定义策略、IPsec 加密、flow 导出等；
- **成本**：OVS 的用户态组件 + datapath 复杂度，资源占用与排障门槛相对高。

B12. Multus + SR-IOV/DPDK 什么时候才该用?

适用场景：NFV/电信、超高 PPS/低延迟要求、需要 Pod 多张网卡（业务网卡与管理网卡分离）。

```
Multus (meta 插件，负责调用多个 CNI)
  ├── 主 CNI (如 Calico) → 提供集群网络 eth0
  └── 附加网卡 (SR-IOV VF / macvlan / ipvlan) → net1
        ↑ 资源由 device plugin 上报: intel.com/intel_sriov_netdevice
```

```
# NetworkAttachmentDefinition (示意)
apiVersion: k8s.cni.cncf.io/v1
kind: NetworkAttachmentDefinition
metadata:
  name: sriov-net
spec:
  config: |
    { "cniVersion":"1.0.0","type":"sriov","ipam":{"type":"host-local","subnet":"192.168.100.0/24"} }
---
apiVersion: v1
kind: Pod
metadata:
  annotations:
    k8s.v1.cni.cncf.io/networks: sriov-net
spec:
  containers:
    - name: app
      resources:
        limits:
          intel.com/intel_sriov_netdevice: "1"
```

代价（必答）：失去"Pod 可迁移/可用度高"的部分优势——SR-IOV VF 与**具体节点网卡绑定**，迁移后 IP/MAC 可能变化，且依赖节点池硬件一致性。

B13. macvlan / ipvlan 的特点与坑?

模式	特点	坑
macvlan	每个子接口有独立 MAC，性能好	**Pod 无法直接访问宿主 IP**（大多数实现下不能用它访问宿主）；交换机需允许大量 MAC
ipvlan L2	共享宿主 MAC	同二层
ipvlan L3	三层转发	广播/ARP 受限，需路由配合

> 二者都不适合作为默认集群网络，常见用途是"给特定 Pod 一个二层/三层直通网卡"。

B14. 怎么验证网络策略真的生效了？（面试常见实操题）

```
# 1) 确认 CNI 支持并开启了策略 enforcement (部分 CNI 有 "policy mode")
# 2) 起一个测试 Pod，从被拒绝侧 curl
kubectl run curler --rm -it --image=curlimages/curl -- sh
curl -m 3 http://target.svc:80 # 期望 timeout / connection refused
# 3) CNI 侧证据
cilium policy get # Cilium
iptables-save | grep cali # Calico (iptables 模式)
hubble observe --verdict DROPPED # Cilium 观测
```

必须验证的两个反向场景：① 允许列表是否漏了 DNS（kube-system 53/UDP+TCP）与健康探测来源；② 加策略前后业务是否有误伤（配 metrics 告警同开）。

B15. Ingress Controller 选型要考虑什么?

维度	关注点
性能与并发模型	NGINX 类 worker 模型、reload 开销（大规模路由的热更新抖动）
动态配置能力	APISIX/Traefik 等支持热更新/插件生态
一致性/Gateway API 支持	未来演进方向
观测	是否方便导出 per-route 延迟/错误率
灰度能力	是否支持基于 header/权重的金丝雀
高可用	多副本 + 拓扑打散 + PDB

生产要点：Ingress 是南北向单点故障源，必须：多副本跨可用区 + PDB + 优雅终止 + 单独的 LB + 容量压测 + 变更灰度。

B16. Kubernetes Service CIDR 和 Pod CIDR 怎么规划?

```
原则：
  Pod CIDR:  按"单节点最大 Pod 数 × 最大节点数"估算，并预留倍数（建议 ≥2 倍）
  Service CIDR: 按最大 Service 数估算，同样预留；注意它**不可在线扩容**（多数实现下扩容属于重建级操作）
  Cloud CNI:  Pod CIDR 需落在 VPC 子网内 ⇒ 必须与云网络团队协同预留
```

禁止的事：

- Service CIDR 与 Pod CIDR 重叠；
- 集群 CIDR 与办公网/IDC 现有网段冲突（会导致路由黑洞）；
- 规划过小又无法在线调整。

B17. 怎么排查"Pod 起来了但服务访问不通"？（综合排障套路）

```
① DNS：nslookup svc / 看 /etc/resolv.conf 是否指向预期 DNS
② Service：kubectl get endpointslices -l kubernetes.io/service-name=svc （有无后端）
③ kube-proxy/CNI 本地规则：iptables-save | grep <clusterIP> 或 ipvsadm -Ln / cilium service list
④ Pod 直连：curl <PodIP>:<port> （排除 Service 层面问题）
⑤ 跨节点：从另一节点 curl PodIP （排除 CNI 跨节点问题）
⑥ 策略：即使没配 NetworkPolicy，也可能有集群默认策略/主机防火墙
⑦ CNI 组件健康：cilium status / calico-node 日志 / felix Ready
⑧ 抓包：tcpdump 在 veth + 宿主网卡同时抓，对比丢在哪一跳
```

B18. 哪些网络问题是"看似随机"的偶发故障？（经验题）

现象	可能根因
DNS 偶发 5 秒超时	conntrack 竞态（A8）
TLS 握手偶发失败、大文件传输卡死	MTU 不一致 / PMTUD 黑洞（A11）
短连接高峰期新连接失败	conntrack 满 / TIME_WAIT / 源端口耗尽
访问自己 Service 偶发失败	hairpin 未开启（A20）
跨可用区偶发延迟高	LB 跨区转发到无后端节点（`externalTrafficPolicy`）+ 重试
滚动更新期间报错	Endpoint 传播延迟 + 优雅终止没配（kubelet 篇 Q11）
特定节点上的 Pod 全不通	CNI DaemonSet 在该节点异常 / IPAM 泄漏 / 路由缺失

Part C • 大规模集群网络优化篇（14 题）

定义：**大规模 = ≥ 1000 节点 / Service 数上千 / 单集群 Pod 数万以上 / 峰值 QPS 高。**

C1. 先量化：网络侧的瓶颈一般落在哪 5 个位置?

#	瓶颈	量化指标	解法方向
1	**iptables/IPVS 规则规模**	`kubeproxy_sync_proxy_rules_duration_seconds`（同步耗时）、节点 `iptables-save`	wc -l`
2	**DNS 查询放大与延迟**	`coredns_dns_requests_total`、`coredns_dns_request_duration_seconds`	NodeLocal DNSCache、降 ndots、CoreDNS 自动扩容
3	**conntrack 容量**	`node_nf_conntrack_entries / limit`	调表大小、缩短 timeout、socket-LB 路径
4	**IPAM / IP 地址**	节点 Pod CIDR 使用率、CNI IPAM 分配错误	CIDR 规划、前缀委派、回收泄漏
5	**跨可用区/跨区流量**	流量计费与延迟	拓扑感知路由（TopologyAwareHints / `trafficDistribution: PreferClose`）

标准答法：

> "我会先用三组指标判断瓶颈位置：① kube-proxy 规则同步耗时与时延分布；② CoreDNS QPS/延迟/缓存命中率；③ 节点 conntrack 使用率。这三个基本能决定我是换数据面、加 DNS 缓存，还是调内核参数。"

C2. iptables 规则膨胀怎么治理?

症状：

- `iptables-save | wc -l` 达到数万甚至十万级；
- Service/Endpoint 每次变更都触发全量 `iptables-restore`，CPU 出现尖峰；
- 网络编程延迟（Service 变更到真正生效）显著变长。

证据与量化：

```
# kube-proxy 规则同步耗时（最关键的领先指标）
histogram_quantile(0.99, sum by (le) (rate(kubeproxy_sync_proxy_rules_duration_seconds_bucket[5m])))

# 网络编程延迟：从 EndpointSlice 变更到规则生效的时延（指标存在与否依版本而定）
histogram_quantile(0.99, sum by (le) (rate(kubeproxy_network_programming_duration_seconds_bucket[5m])))
```

```
# 节点侧规则量（用于基线对比）
iptables-save | wc -l
ipvsadm -Ln | wc -l
```

解法（按力度递增）：

1. **切 IPVS**：规则查找 O(1)，且 ipset 聚合后端集合；
2. **上 Cilium kube-proxy replacement**：彻底绕开 netfilter；
3. 关注**新的 nftables 后端**（版本相关，属演进选项，需先在小集群验证）；
4. 架构侧：减少不必要的 Service（如批处理内部直连 PodIP/Headless/SRV），控制 Service 数量增长。

C3. DNS 大规模优化的完整方案?

```
① NodeLocal DNSCache DaemonSet (hostNetwork + link-local IP)
      ⇒ 大幅降低 CoreDNS QPS，并规避 conntrack 竞态导致的 5 秒超时
② CoreDNS 自动扩容：cluster-proportional-autoscaler
③ CoreDNS 自身调优：缓存、forward 策略、健康检查、autopath（谨慎使用）
④ Pod 侧：FQDN 写法 + 合理 ndots
⑤ 监控：QPS、延迟 P99、SERVFAIL 率、缓存命中率
```

```
# CoreDNS 自动扩容（示意 ConfigMap/linear 模式）
apiVersion: apps/v1
kind: Deployment
metadata:
  name: coredns-autoscaler
  namespace: kube-system
spec:
  template:
    spec:
      containers:
        - image: registry.k8s.io/cpa/cluster-proportional-autoscaler:1.8.9 # 版本自行核对
          name: autoscaler
          command:
            - /cluster-proportional-autoscaler
            - --namespace=kube-system
            - --configmap=coredns-autoscaler
            - --target=deployment/coredns
            - --logtostderr=true
            - --nodelabels=node-role.kubernetes.io/control-plane
            - --poll-period-seconds=15
```

```
# CoreDNS QPS 与延迟（判断是否需要扩容 / 是否已到瓶颈）
sum(rate(coredns_dns_requests_total[5m]))
histogram_quantile(0.99, sum by (le) (rate(coredns_dns_request_duration_seconds_bucket[5m])))
```

> **注意**：CoreDNS 副本数增加会提高 kube-dns Service 后端的均衡性，但过度扩容会放大 EndpointSlice 变更频率，**建议按节点数比例线性扩容（coresPerReplica/nodesPerReplica 需实测校准）**。

C4. NodeLocal DNSCache 上线怎么灰度？（有损变更，必须给方案）

阶段	范围	观察指标	回滚触发条件
Step 1	**1 个节点**（隔离/OFF 搅动池）	Pod DNS 成功率、CoreDNS QPS、业务 P99	DNS SERVFAIL 率上升或成功率跌破基线
Step 2	1% 节点	+ Pod 重启后解析正确性	同上 / 出现解析错误或解析到错误地址
Step 3	10% → 50%	全 SLO Dashboard	同上
Step 4	100%	稳定 24h	—

回滚要点：NodeLocal 的启用通常改的是 kubelet 的 DNS 配置（`-cluster-dns`）⇒ **回滚需要重启 kubelet 并让现有 Pod 重建**才能完全恢复；务必：

1. 保证 `kube-dns-upstream` Service 与 endpoint 配置正确（否则 Cache 找不到上游 ⇒ 全局解析失败）；
2. 灰度期间保留原 CoreDNS Service 不被替换；
3. 明确回滚步骤（改回 kubelet 配置 → 重启 → 滚动替换 Pod）。

> **这是典型的"看似局部实则全节点"的变更**，面试里能讲清它的回滚代价很加分。

C5. conntrack 满的预防与扩容？

```
# sysctl 持久化（建议值需按业务并发实测校准）
cat > /etc/sysctl.d/90-conntrack.conf <<'EOF'
net.netfilter.nf_conntrack_max = 1048576
net.netfilter.nf_conntrack_buckets = 262144
net.netfilter.nf_conntrack_tcp_timeout_time_wait = 30
net.netfilter.nf_conntrack_tcp_timeout_close_wait = 60
EOF
sysctl -p /etc/sysctl.d/90-conntrack.conf
```

注意：`nf_conntrack_buckets` 是**哈希桶数**，变更会重建表（大数据量下有瞬时开销），且部分环境只能在模块加载时设置。**必须在变更窗口执行并观察。**

监控（必配）：

```
node_nf_conntrack_entries / node_nf_conntrack_entries_limit > 0.75
increase(node_nf_conntrack_insert_failed[5m]) > 0
rate(node_nf_conntrack_drops[5m]) > 0
```

C6. IP 地址耗尽怎么提前发现和避免？

容量模型（务必自己算一遍）：

```
集群 Pod 总 IP 需求 = 预期最大节点数 × 单节点最大 Pod 数(maxPods)
（还要考虑滚动更新期间新旧版本并存 ≈ ×2）
```

规划建议（建议初值，需按业务校准）：

场景	建议
通用集群	Pod CIDR 预留 ≥ 2× 实际需求；单节点 maxPods 留 20% 余量
云 CNI（每节点 Pod 数受网卡限制）	确认实例规格 → 计算 `ENI 数 × 每 ENI IP 数`，并开启前缀委派
Calico IPAM	上线前确定 `blockSize`（**创建后不可改**）

监控：

```
# 节点已用 IP vs 上限（CNI 侧指标名依插件而定，以下为思路）
# Calico: felix_ipam_* 系列；Cilium: cilium_ipam_*；需要先 _ 核对
# 兜底：直接用 kubelet 已运行 Pod 数 / maxPods
sum by (node) (kubelet_running_pods) / 110 > 0.8
```

回收泄漏：节点删除后 IPAM block 未释放、早已终止的 Pod 残留 endpoint ⇒ 用 CNI 自带工具核查（`calicoctl ipam check`、`cilium status`）。

C7. Service 数量爆炸怎么办？

- **架构治理**：不必要的 Service 收敛；内部调用用 Headless Service / SRV / 服务网格的直连机制；
- **数据面**：IPVS/eBPF 降低单次转发规则的匹配成本；
- **避免 ClusterIP 耗尽**：Service CIDR 要早规划（多数实现不可平滑扩容）；
- 监控 Service/EndpointSlice 总数增长趋势：

```
kube_service_info
count(kube_endpointslice_info)
```

C8. NetworkPolicy 大规模落地的成本?

实现	规模成本	建议
iptables/ipsets 实现	每个 endpoint 一套链，**规则数随 Pod×策略数增长**	控制策略数量与 selector 复杂度；定期审查僵尸策略
eBPF map 实现	基于 identity + map 查表，增长更平缓	优先（同时满足内核要求）

实践建议：

- 用**分层策略**：default-deny + 少量粗粒度放行，而不是每个服务一张精细表；
- 把策略对象数量、CNI 侧的策略同步延迟纳入监控（如 Cilium 的 `cilium_policy_*`、Calico 的 `felix_*`，**指标名请与目标版本核对**）；
- 每次策略变更都要在**灰度 Namespace**先验证再推全量。

C9. 跨可用区流量怎么降?

```
# ① Service 侧开启拓扑感知（新老两套写法，随版本演进；请以所用版本为准）
apiVersion: v1
kind: Service
metadata:
  name: web
  annotations:
    service.kubernetes.io/topology-aware-hints: auto # 老写法
spec:
  # 新写法（版本相关）：trafficDistribution: PreferClose
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 8080
```

配套动作：

- Deployment 用 **TopologySpreadConstraints** 让副本均匀分布到各可用区（否则"优先本地"会导致某区永远没有后端）；
- 注意：拓扑感知会牺牲**全局均衡**，热点/流量倾斜场景要压测；
- Ingress/网关侧也要做同区优先，否则只治标。

C10. MTU / 分片问题的大规模治理?

- 全集群统一 MTU，并纳入"节点池基线"（配置漂移是最常见的事故源）；
- CNI 侧显式写死 MTU，避免依赖自动探测；
- 网络链路变更（如更换隧道、VXLAN offload）后**必须重新核算 MTU**；
- 开启 MSS clamp 作为兜底；
- 把"大包丢包率"作为常态化巡检项。

```
# 快速验证：从 Pod 内发大包（需要宿主机/对端配合 tcpdump 看是否分片被丢）
kubectl exec -it po -- ping -M do -s 8972 <peer-ip> # 逐步降 size 找到临界值
```

C11. 东西向流量可观测怎么落地?

手段	覆盖	成本
Hubble / CNI flow	L3/L4 + 部分 L7，含 drop reason	较低（CNI 自带）
Service Mesh 遥测	L7 全量指标/trace	sidecar 资源 + 复杂度
主机侧 eBPF 采集	跨容器宿主机视角	需额外组件
传统 netflow	网络设备侧	与 CNI 语义割裂

建议：先由 CNI 提供 L3/L4 与丢包原因（性价比最高），L7 再按业务重要性选择性引入 mesh。

C12. 大规模集群要不要拆"多集群"?

网络侧的真实动机只有三个：**① 故障域隔离；② IP 地址空间不足；③ 地域/延迟**。

- ClusterMesh/Submariner 类方案能打通 Pod IP，但会**把故障域重新连起来**——这与"隔离"目标矛盾；
- 更常见做法：**多集群 + 独立网络 + 上层 DNS/网关做流量调度**（容灾切换），而不是全互通。

C13. 一次 CNI 变更的标准灰度与回滚（全网级变更）

CNI 是集群里最不能随便动的东西之一。规范流程：

阶段	做法	回滚触发
准备	新 CNI 只在**新增节点池**启用（老节点池不动），并确保双方可互通（临时隧道/路由）	—
Step 1	把**无状态、可重建**的测试服务调度到新池（用 nodeSelector/taint 控制）	连通性失败、DNS 失败、策略不生效
Step 2	10% 业务 Pod 迁过去	P99 恶化 / 错误率上升
Step 3	50% → 100%，并逐步引导新节点加入新池	SLO 指标异常
收尾	老节点下线（**注意**：已有 Pod 通常必须重建才能获得新 CNI 的 IP）	—

回滚要点：

- 回滚 = 把 Pod 调度回旧节点池（改 nodeSelector/Pod 调度），**旧节点池必须完整保留到验收通过之后**；
- CNI 迁移通常**无法原地回滚**（已分配的 Pod IP 不会自动变化），所以"双池并存"是唯一安全的做法；
- 必须在窗口内完成，并准备"业务全量重建"的应急脚本。

C14. 新集群网络基线设计（综合题，按顺序答）

1. **问 SLO**：是否需要 Pod IP 集群外可见？是否需要 L7 策略？是否需要跨集群？合规是否要求加密？
2. **CIDR 规划**：Pod/Service CIDR 与现网无冲突、留 2× 余量；云上要与 VPC 团队对齐。
3. **方案选型**：按 A16 决策树。
4. **模式配置**：是否需要 Overlay/CrossSubnet/原生路由；MTU 统一。
5. **规模组件**：Calico Typha/RR 或 Cilium（含 kube-proxy replacement 决策）。
6. **DNS**：NodeLocal DNSCache + CoreDNS 自动扩容 + ndots 治理。
7. **内核参数基线**：conntrack、somaxconn、somax backlog、`ip_forward` 等。
8. **策略**：default-deny 分层策略 + DNS 出口豁免。
9. **观测**：CNI 指标、CoreDNS、kube-proxy、conntrack、Hubble/flow。
10. **变更治理 + GameDay**：按 C13 灰度，定期做"CNI 组件挂掉/节点失联/IP 耗尽"演练。

Part D · 反模式清单（可直接用于生产巡检）

#	反模式	证据（怎么发现）	等级	修复动作	验证方式
D1	用 Flannel 却以为 NetworkPolicy 生效	配了策略仍能访问	🔴 高	换支持策略的 CNI	B14 的验证脚本通过
D2	Service/Ingress 全集群无拓扑感知，跨可用区流量巨大	跨区流量计费、延迟高于同区	🟠 中	开 `PrefastClose`/hints + 拓扑打散	跨区流量占比下降（★实测）
D3	未部署 NodeLocal DNSCache 却有海量 DNS 查询	`coredns_dns_requests_total` 极高、偶发 5s	🔴 高	灰度上线 NodeLocal	SERVFAIL/5s 超时归零
D4	外部域名未写 FQDN + ndots=5	DNS QPS 放大 4~10 倍	🟠 中	写 FQDN / 调 ndots	CoreDNS QPS 下降到放大部分消失
D5	`nf_conntrack_max` 保持默认	`insert_failed` 增长、`table full` 日志	🔴 高	按并发连接实测调参	高压场景无丢计数
D6	Service 数上千仍用 iptables kube-proxy	规则同步 P99 高、CPU 尖峰	🟠 中	切 IPVS 或 eBPF 替代	`kubeproxy_sync_proxy_rules_duration_seconds` 回归基线
D7	MTU 未统一/未写死	大包丢、TLS 偶发失败	🔴 高	统一 MTU + MSS clamp	大包 ping/压测通过
D8	Calico IPPool `blockSize` 未规划就上线	后期发现不可改	🟠 中	新建 IPPool 迁移（重操作）	`calicoctl ipam` 检查通过
D9	BGP full mesh 跑到几百节点	BGP 会话数 $O(n^2)$、收敛慢	🔴 高	上 Route Reflector / ToR RR	会话数降到 O(n)
D10	Typha 副本数不足/未监控	策略更新延迟、Felix 报错	🟠 中	多副本 + 跨故障域 + 告警	模拟杀 Typha 不影响策略下发
D11	default-deny 忘记放行 DNS	加策略后全部解析失败	🔴 高	显式放行 kube-system 53/UDP+TCP	加策略后 DNS 探测成功
D12	`externalTrafficPolicy: Local` 但副本未按区打散	部分区无后端，LB 健康检查失败	🔴 高	副本数 ≥ 可用区数 + TopologySpread	各区均有 Ready 后端
D13	生产直接原地换 CNI	全集群网络中断	🔴 高	双节点池灰度迁移（C13）	回滚到旧池成功
D14	没有 Networks 丢包原因可观测	故障只能猜	🟠 中	Hubble/flow + drop reason 告警	注入策略拒绝能立刻看到 DROPPED 原因
D15	Service CIDR 规划过小	Service 数量接近上限	🟠 中	早期扩容/重建规划（不可平滑改）	容量台账有充足余量

Part E • 可观测性：SLO、指标与告警

E1. 建议的网络侧 SLO（初始建议值，必须实测校准）

SLI	建议目标（起点）	度量方式
Pod 网络连通成功率（同集群 Pod→Service）	≥ 99.99%	合成探测（blackbox-exporter 常驻 Pod）
DNS 解析成功率（内部域名）	≥ 99.99%	blackbox DNS probe + CoreDNS SERVFAIL 率
DNS P99 延迟	≤ 50ms	`coredns_dns_request_duration_seconds`
Service 变更后网络编程延迟 P99	≤ 5s	`kubeproxy_network_programming_duration_seconds`（版本相关）
kube-proxy 规则同步 P99	≤ 2s	`kubeproxy_sync_proxy_rules_duration_seconds`
conntrack 使用率	≤ 75%	node_exporter
丢包（drop）率	≈ 0（非策略拒绝类）	CNI flow

> 错误预算：以 30 天滚动窗口，burn rate > 14.4×（短期）/ 6×（长期）分别升级为紧急告警与工单。

E2. 指标清单（上线前必须在目标节点/组件核对该版本的指标名）

组件	指标	用途
kube-proxy	`kubeproxy_sync_proxy_rules_duration_seconds`	规则同步耗时（核心领先指标）
kube-proxy	`kubeproxy_network_programming_duration_seconds`	变更到生效的网络编程延迟
CoreDNS	`coredns_dns_requests_total` / `_request_duration_seconds` / `_responses_total`	DNS QPS/延迟/响应码
CoreDNS	`coredns_cache_hits_total` / `coredns_cache_misses_total`	缓存命中率
node	`node_nf_conntrack_entries` / `_limit` / `node_nf_conntrack_drops`	conntrack 容量
cilium-agent	`cilium_*`（如 `cilium_policy_*`、`cilium_datapath_*`、`cilium_ipam_*`、`cilium_controllers_failing`）	Cilium 健康/策略/IPAM
hubble	`hubble_...` / export black (dependson)	flow 与 drop reason
calico-node/felix	`felix_*`（如 `felix_int_dataplane_failures`、`felix_active_local_endpoints`、`felix_ipsets_calico`、`felix_resyncs_total`）	Felix 数据面健康
cAdvisor / node	网络接口 PPS、丢包、错包	网卡层问题

E3. Prometheus 告警规则（可直接粘贴，阈值需实测校准）

```
groups:
  - name: k8s-network
    rules:
      - alert: KubeProxySyncSlow
        expr: |
          histogram_quantile(0.99,
            sum by (le) (rate(kubeproxy_sync_proxy_rules_duration_seconds_bucket[5m]))) > 2
        for: 10m
        labels: {severity: warning}
        annotations:
          summary: "kube-proxy 规则同步 P99 达 {{ $value }}s，考虑切换 IPVS/eBPF 或收敛 Service"

      - alert: CoreDNSHighLatency
        expr: |
          histogram_quantile(0.99,
            sum by (le) (rate(coredns_dns_request_duration_seconds_bucket[5m]))) > 0.05
        for: 10m
        labels: {severity: warning}
        annotations:
          summary: "CoreDNS P99 解析延迟 {{ $value }}s"

      - alert: CoreDNSServfail
        expr: sum(rate(coredns_dns_responses_total{rcode="SERVFAIL"}[5m])) > 1
        for: 5m
        labels: {severity: critical}
        annotations:
          summary: "CoreDNS 出现 SERVFAIL，可能出现 5 秒超时问题"

      - alert: ConntrackNearFull
        expr: node_nf_conntrack_entries / node_nf_conntrack_entries_limit > 0.75
        for: 10m
        labels: {severity: warning}
        annotations:
          summary: "节点 {{ $labels.instance }} conntrack 使用率 {{ $value | humanizePercentage }}"

      - alert: ConntrackInsertFailed
        expr: rate(node_nf_conntrack_insert_failed[5m]) > 0
        for: 5m
        labels: {severity: critical}
        annotations:
          summary: "conntrack 插入失败，正在丢连接"

      - alert: CNINotReady
        expr: kube_node_status_condition{condition="NetworkUnavailable",status="true"} == 1
        for: 5m
        labels: {severity: critical}
        annotations:
          summary: "节点 {{ $labels.node }} 网络不可用，检查 CNI DaemonSet"

      - alert: NodePodIPPressure
        expr: sum by (node) (kubelet_running_pods) / 110 > 0.85
        for: 30m
        labels: {severity: info}
        annotations:
          summary: "节点 {{ $labels.node }} Pod 数接近 maxPods 上限"

      # 合成检测：集群内部 DNS/HTTP 成功率，建议用 blackbox-exporter 常驻 Pod
      - alert: PodHTTPProbeFailing
        expr: probe_success{job="kube-network-probe"} == 0
        for: 3m
        labels: {severity: critical}
        annotations:
          summary: "合成探测失败：{{ $labels.target }}"
```

🔗 **关联阅读**：第六篇 · D3. Prometheus 告警规则（可直接粘贴，阈值请实测校准） | 第三篇 · G2. Prometheus 告警规则（可直接粘贴，阈值需实测校准） | 第五篇 · D3. Prometheus 告警规则（可直接粘贴，阈值请实测校准）

Part F · 演练（GameDay）计划表

场景	注入方式	期望表现	验证的 SLO
CoreDNS 全部挂掉	scale NodeLocal/ CoreDNS to 0	NodeLocal 缓存兜住一段时间；告警及时	DNS 可用性
conntrack 打满	短连接压测	告警先于丢包触发，业务降级可控	连接成功率
kube-proxy 挂掉	停 DaemonSet Pod	已建规则不失效，变更不再生效；能快速恢复	变更可用性
CNI agent 挂掉（单节点）	停 CNI Pod	已有 Pod 通信通常不中断、新 Pod 起不来；节点被标记 NetworkUnavailable	MTTR
策略误配	下发错误 NetworkPolicy	立刻能看到 DROPPED + 业务告警，且能秒级删除策略恢复	策略变更安全
IP 耗尽演练	在测试集群塞满 CIDR	新 Pod 起不来且有明确告警	IPAM 容量管理
MTU 不一致	故意调整某节点 MTU	大包失败被巡检发现	数据面一致性
跨可用区故障	隔离一个可用区	拓扑感知把流量导走	容灾 RTO

Part G • 10 张面试速记卡

1. **K8s 网络四句话**：Pod 互通无 NAT；Node 与 Pod 互通；Pod IP 自洽；实现交给 CNI。
2. **封包路径**：Pod eth0 → veth → 宿主（bridge/路由）→（隧道/路由/ENI）→ 对端宿主 → veth → Pod；Service IP 只是 DNAT 规则，没有网卡。
3. **CNI 三类插件**：main（建网卡）/ IPAM（分 IP）/ meta（portmap、bandwidth 等增强），ADD/DEL/CHECK 三接口 + stdin/env 调用。
4. **DNS 两大坑**：ndots=5 让外部域名查询放大 4~10 倍；conntrack 竞态导致 5 秒超时 ⇒ NodeLocal DNSCache 是标准解。
5. **模式选择看的是"底层能否通告 Pod CIDR"**：能 → BGP/原生路由；不能 → Overlay；折中 → CrossSubnet；要 VPC IP → 云 CNI。
6. **MTU 是隐形炸弹**：VXLAN/IPIP 会让可用 MTU 变小；1514 变 1440/1450；配错就是"小包通大包死"。
7. **conntrack 是隐形的表**：规则我调了一百万次却忘了给 conntrack 扩容，短连接高峰就丢连接。
8. **NetworkPolicy 是 CNI 实现的**：Flannel 不支持；default-deny 一定记得放行 DNS。
9. **规模三件套**：Service 多 → IPVS/eBPF；DNS 压力大 → NodeLocal + 自动扩容；BGP 节点多 → Route Reflector（Calico 还有 Typha 扛 watch 压力）。
10. **CNI 迁移只能"双池并存灰度"**，不能原地换；回滚 = 把 Pod 调度回旧节点池，老池要保留到验收通过。

第八篇 · 存储与 CSI

> PV/PVC · StorageClass · CSI sidecar · 拓扑 · 快照

0. 使用说明

0.1 存储方向的考察层次

层级	考点	表现
L1 会用	会写 PV/PVC/StorageClass YAML	只会 apply
L2 懂原理	说得清 PV 绑定状态机、动态供给链路、CSI 三个 gRPC 服务、attach/stage/publish 三段	能画全链路图
L3 懂选型	能比较块/文件/对象、各 CSI 驱动能力矩阵、AZ 拓扑、访问模式	能说 trade-off
L4 有事故经验	能讲"Pod 挂着卷 20 分钟起不来""有状态服务跨可用区调度失败""卷删不掉"的根因与治理	有排查命令、有灰度、有回滚

> **答题模板**：先说这条链路上的**四个参与者**（PV Controller / CSI Controller / kubelet / CSI Node），再说各自做的事，最后给证据（配置、对象字段、metric、日志/事件）。

0.2 一句话区分四个层次

层次	对象	谁定义	生命周期
Pod 内的 Volume 声明	`spec.volumes`	应用	与 Pod 一致
用户申请	**PVC**	应用/平台	独立于 Pod（关键！）
集群资源	**PV**	管理员/动态供给	独立于 Namespace
供给模板	**StorageClass**	平台	长期
真实实现	**CSI driver**	存储厂商/社区	独立于以上全部

Part A • 原理篇（24 题）

A1. K8s 存储的四种抽象各自解决什么问题?

抽象	解决的问题	关键点
Volume（Pod 内声明）	让 Pod 能用上各种"卷"（emptyDir/configMap/secret/csi/pvc...）	与 Pod 生命周期一致
PV	把"具体的存储资源"抽象成集群对象（容量/访问模式/回收策略/后端信息）	**集群级资源**，不属于某个 namespace
PVC	让应用"只要提需求"（多少 GiB、什么访问模式、哪个 SC），不用关心后端	与 PV **一对一绑定**
StorageClass	定义"如何供给"（provisioner、参数、回收策略、绑定模式）	支持**动态供给**，是平台能力的边界

必答：PV 与 PVC 的绑定是 **1:1 且独占** 的；PVC 的生命周期**不跟随 Pod**（这是有状态服务能重建的基础，也是"磁盘泄漏"的来源）。

A2. PV 的状态机是怎么走的？各状态代表什么?



PV 状态	含义
`Available`	空闲，未被任何 PVC 绑定
`Bound`	已绑定到某个 PVC
`Released`	PVC 已删除，但后端存储资产还在回收中
`Failed`	自动回收失败（多为后端 API 报错/凭据失效）

PVC 侧状态：`Pending`（未绑定，最常见）/ `Bound` / `Lost`（PV 丢失或异常）。

必答的回收策略：

- `Retain`：删除 PVC 后 PV 保留，**数据还在**（合规/防误删常选），但没人自动清理 ⇒ 容易堆积；
- `Delete`：删除 PVC ⇒ 删除 PV ⇒ 调 CSI `DeleteVolume` **真删数据**（默认行为要看 StorageClass 的 `reclaimPolicy`）；
- `Recycle`：**已废弃**，不要再说它是推荐做法。

A3. StorageClass 里每个字段都会问什么?

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: ebs.csi.aws.com # ① CSI 驱动名（必须与 CSIDriver 对象一致）
parameters: # ② 传给 CreateVolume 的参数（驱动自定义）
  type: gp3
  iops: "5000"
  throughput: "250"
  encrypted: "true"
  csi.storage.k8s.io/fstype: ext4
reclaimPolicy: Delete # ③ Delete | Retain
volumeBindingMode: WaitForFirstConsumer # ④ Immediate | WaitForFirstConsumer（★ 高频考点）
allowVolumeExpansion: true # ⑤ 是否允许扩容
mountOptions: # ⑥ 挂载参数
  - noatime
  - nodiratime
allowedTopologies: # ⑦ 拓扑约束（配合 CSI 的 accessibleTopology）
  - matchLabelExpressions:
      - key: topology.ebs.csi.aws.com/zone
        values: ["cn-north-1a", "cn-north-1b"]
```

必答：

- `WaitForFirstConsumer` **几乎是所有支持的 StorageClass 的推荐默认**，否则会出现"PV 先落在一个 AZ，Pod 却被调度到另一个 AZ" ⇒ 永久 Pending；
- StorageClass 是**不可变为主**的资源：修改它**不会影响已创建的 PV**（很多人误以为能"改参数生效"）；
- 想给不同租户不同配额 ⇒ 用不同 StorageClass + ResourceQuota 的 `xxx.storageclass.storage.k8s.io/persistentvolumeclaims`。

A4. AccessMode 四种模式到底限制什么?

模式	含义	典型
`ReadWriteOnce` (RWO)	可被**一个节点**读写挂载	云盘、块存储
`ReadOnlyMany` (ROX)	可被**多节点只读**挂载	只读数据集
`ReadWriteMany` (RWX)	可被**多节点读写**挂载	NFS / CephFS / EFS / NAS

模式	含义	典型
`ReadWriteOncePod`（RWOP）	**整个集群只能一个 Pod** 挂载（比 RWO 更严格）	需要严格单写互斥的有状态服务（版本相关，请核对）

必答的两个真相：

1. AccessMode 是**声明**，K8s **不强制**执行（除了 RWOP 会在 kubelet 侧做 Pod 级限制）：驱动不支持 RWX 你写了 RWX，结果就是**多节点挂载失败**或**静默数据损坏**；
2. **RWO ≠ 只能一个 Pod**：同一节点上的多个 Pod 可以同时挂载同一个 RWO PVC（因为都在一个节点）；要防止"第二个 Pod 也挂上来"，必须用 **RWOP**。

A5. volumeMode：Filesystem 与 Block 怎么选?

```
apiVersion: v1
kind: PersistentVolumeClaim
spec:
  volumeMode: Block # Filesystem（默认）| Block
  resources:
    requests:
      storage: 100Gi
---
# Pod 侧使用方式完全不同
spec:
  containers:
    - name: app
      volumeDevices: # 注意：不是 volumeMounts
        - name: data
          devicePath: /dev/xvda
  volumes:
    - name: raw
      persistentVolumeClaim:
        claimName: block-pvc
```

模式	特点	场景
`Filesystem`（默认）	kubelet 做 mkfs + mount	绝大多数应用
`Block`	把裸设备透传给容器	数据库自己管文件系统、需要 Direct IO、部分 RDMA/高性能场景

必答：Block 模式下 kubelet **不创建文件系统也不 mount**，由应用自己处理；扩容/快照行为也与文件系统模式不同。

A6. 一次"动态供给"的完整链路是什么？（核心大题）

```
① 用户创建 PVC（指定 StorageClass）
② PV controller 发现有 StorageClass → 标记 PVC 为 Pending，等待 provisioner
③ external-provisioner sidecar watch 到 PVC
      → 调 CSI ControllerService.CreateVolume(capacity, parameters, accessibilityRequirements, secrets)
④ 存储后端真的创建磁盘；pv provisioner 创建 PV 对象并绑定 PVC
⑤ Pod 被调度到某个节点（若 WaitForFirstConsumer，则 CreateVolume 会带上该节点的拓扑信息）
⑥ 若驱动需要 attach：external-attacher 调 ControllerPublishVolume → 云上把盘挂给这台虚拟机
⑦ kubelet VolumeManager 发现卷已就绪
      → NodeStageVolume（格式化 + mount 到 staging 路径，仅支持 STAGE_UNSTAGE 能力的驱动）
      → NodePublishVolume（bind mount 到 /var/lib/kubelet/pods/<uid>/volumes/...）
⑧ 容器启动，卷可用
⑨ 删除 Pod → NodeUnpublishVolume → NodeUnstageVolume → ControllerUnpublishVolume
⑩ 删除 PVC → 依 reclaimPolicy → DeleteVolume → PV 消失
```

必答：这条路上有 **4 类故障点**：`provisioner 没起来 / 后端配额不足`（停在第 3-4 步）、`AZ 拓扑不匹配`（停在第 5 步）、`attach 失败`（第 6 步，`FailedAttachVolume`）、`mount 失败`（第 7 步，`FailedMount`）。**Events 里的字段名就是最好的定位线索。**

A7. CSI 有哪些核心 sidecar？各自干什么？（必考）

sidecar	干什么	跑在哪
external-provisioner	watch PVC → CreateVolume / DeleteVolume	Controller（Deployment）
external-attacher	watch VolumeAttachment → ControllerPublish/UnpublishVolume	Controller
external-resizer	watch PVC 扩容请求 → ControllerExpandVolume	Controller
external-snapshotter	watch VolumeSnapshot → CreateSnapshot / DeleteSnapshot	Controller
external-health-monitor（controller/agent）	卷健康检测 → 上报 Abnormal 事件	Controller / Node
node-driver-registrar	向 kubelet 注册 driver socket → 生成 CSINode 对象	Node（DaemonSet）
livenessprobe	探活 CSI socket，暴露健康状态	Controller + Node

为什么需要这么多 sidecar：CSI 只定义了 gRPC 接口，**观看 K8s 对象并调用 CSI 这部分"胶水"交给标准 sidecar**，各驱动只需实现存储语义。

A8. ControllerPublish / NodeStage / NodePublish 三者必须分得清（必考）

阶段	调用者	含义	失败时的事件
`ControllerPublishVolume`	external-attacher（控制面）	**把磁盘挂给这台虚拟机**（云 API 层）	`FailedAttachVolume`
`NodeStageVolume`	kubelet（节点）	**格式化 + 挂到节点 staging 路径**（一次操作，可被多 Pod 复用）	`FailedMount`
`NodePublishVolume`	kubelet（节点）	**bind mount 到具体 Pod 目录**（每 Pod 一次）	`FailedMount`

好处：多个 Pod 共享同一个节点上的卷时，**stage 只做一次**，publish 多次 ⇒ 显著减少昂贵操作。

必答：并不是所有驱动都需要 stage/unstage（`STAGE_UNSTAGE_VOLUME` capability）；块存储通常需要，文件存储/NFS 可能跳过。

A9. 三个 gRPC 服务（Identity / Controller / Node）各自的能力?

Service	主要 RPC	部署位置
Identity	GetPluginInfo、GetPluginCapabilities、Probe	两侧都有
Controller	CreateVolume / DeleteVolume / ControllerPublishVolume / ControllerUnpublishVolume / ValidateVolumeCapabilities / ListVolumes / GetCapacity / ControllerExpandVolume / CreateSnapshot / ListSnapshots	通常只有 Controller 实例
Node	NodeStageVolume / NodeUnstageVolume / NodePublishVolume / NodeUnpublishVolume / NodeGetInfo / NodeGetCapabilities / NodeGetVolumeStats / NodeExpandVolume	每个节点

必考追问：Node GetInfo 返回什么？ → **nodeID**（存储视角的节点标识）与 **accessibleTopology**（拓扑段，如 `topology.ebs.csi.aws.com/zone=cn-north-1a`），它们会写进 **CSINode 对象**，是调度器做拓扑决策的依据。

A10. WaitForFirstConsumer 与 CSI 拓扑的完整流程（和调度器联动）

```
① PVC 创建，StorageClass.volumeBindingMode = WaitForFirstConsumer
   → PV controller 不立即供给，只标记 PVC annotation "selected node pending"
② Pod 创建 → 调度器介入：
     - VolumeBinding 插件查：该 PVC 未绑定 ⇒ 进入"延迟绑定"分支
     - Filter 阶段：遍历节点，看每个节点的**拓扑 labels** 是否落在 CSI 驱动的
       accessibleTopology / StorageClass.allowedTopologies 内
     - 不满足 ⇒ 该节点被过滤 ⇒ Events: "node(s) had volume node affinity conflict"
③ 节点选定后 → provisioner 带上该节点拓扑去 CreateVolume
④ PV 创建并完成绑定 → kubelet 才开始 attach + mount
```

必答：这就是为什么 **延迟绑定 + 单可用区云盘** 是标准搭配；Immediate 模式下"PV 先落一 AZ、Pod 后落到另一 AZ"的经典事故就是这么来的。

A11. CSINode 对象为什么重要?

```yaml
apiVersion: storage.k8s.io/v1
kind: CSINode
metadata:
  name: node-1
spec:
  drivers:
    - name: ebs.csi.aws.com
      nodeID: i-0123456789
      topologyKeys: ["topology.ebs.csi.aws.com/zone"]
      allocatable:
        count: 39 # 该节点还能挂多少卷（版本相关）
```

作用：

- 记录**该节点上注册了哪些驱动**、nodeID、拓扑键；
- 调度器用它判断节点能否提供该 CSI 的卷；
- **节点上的 node-driver-registrar 挂了 ⇒ CSINode 不更新 ⇒ 新卷挂载全部失败**（已有挂载通常不受影响）；
- 节点 drain / 重建时若 CSINode 与真实能力不一致，会造成调度误判。

A12. 为什么 PVC / PV 会卡在 Terminating? （Finalizer 必考）

`StorageObjectInUseProtection` **准入插件** 会自动给对象加 finalizer：

- PVC：`kubernetes.io/pvc-protection`（正在被 Pod 使用时不允许删除）
- PV：`kubernetes.io/pv-protection`

因此：

现象	原因
PVC 一直 Terminating	还有 Pod 在用（包括已 Terminating 的 Pod）
PV 一直 Terminating	PVC 还在，或 storage 后端删除卡住
强删后残留	finalizer 被手动摘掉后，后端资产**不会自动清理** ⇒ 产生"孤儿磁盘"（费钱！）

```
# 正确排障
kubectl get pvc pvc-1 -n prod -o jsonpath='{.metadata.finalizers}'
kubectl get pods --all-namespaces -o json | jq '..|.volumes[]?.persistentVolumeClaim.claimName' | grep pvc-1
# 极端情况（工单级操作，注意后果）
kubectl patch pvc pvc-1 -n prod -p '{"metadata":{"finalizers":[]}}' --type=merge
```

A13. 卷快照是怎么实现的?

```
VolumeSnapshotClass（模板）
      +
VolumeSnapshot（用户申请）  ──→ VolumeSnapshotContent（实际绑定）
      │
      └─ snapshot-controller + external-snapshotter
            ├─ 调 CSI CreateSnapshot → 返回 snapshotID
            └─ 创建 VolumeSnapshotContent 并与 VolumeSnapshot 绑定
```

```yaml
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshotClass
metadata:
  name: csi-snapclass
driver: ebs.csi.aws.com
deletionPolicy: Delete # Delete | Retain
---
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata:
  name: db-snap-20240101
  namespace: prod
spec:
  volumeSnapshotClassName: csi-snapclass
  source:
    persistentVolumeClaimName: db-data
---
# 从快照恢复：PVC 的 dataSource 指向 VolumeSnapshot
apiVersion: v1
kind: PersistentVolumeClaim
spec:
  dataSource:
    name: db-snap-20240101
    kind: VolumeSnapshot
    apiGroup: snapshot.storage.k8s.io
  storageClassName: fast-ssd
  accessModes: ["ReadWriteOnce"]
  resources:
    requests:
      storage: 100Gi
```

必答：

- VolumeSnapshot 是**命名空间级**，VolumeSnapshotContent 是集群级；两者绑定语义与 PV/PVC 类似；
- `deletionPolicy: Retain` **能防止"删 PVC 连带把快照也删了"的误伤**（备份场景强烈建议）；
- 快照能力依赖驱动 `CREATE_DELETE_SNAPSHOT` capability，不是所有驱动都支持增量快照。

A14. 卷克隆（Clone）和快照恢复有什么区别?

维度	Clone（`dataSource.kind: PersistentVolumeClaim`）	快照恢复
数据源	另一个 PVC（**同一命名空间、同一 SC**）	VolumeSnapshot
CSI 调用	`CreateVolume` 带 `VolumeContentSource: VOLUME`	`CreateVolume` 带 `VolumeContentSource: SNAPSHOT`
时间点	**当前状态**（要考虑源卷正在写入 ⇒ 数据一致性风险）	快照时间点一致
用途	快速造一份同数据环境	恢复/回溯

必答：克隆在用数据库时通常要在应用层做 checkpoint/quiesce，否则可能得到一份"崩溃一致性"数据。

A15. PVC 扩容的正确姿势? 为什么扩了 Pod 里没变大?

前提条件：

1. `StorageClass.allowVolumeExpansion: true`；
2. 驱动支持 `EXPAND_VOLUME` / `NodeExpandVolume` capability。

流程：

```
改 PVC.spec.resources.requests.storage
   → external-resizer watch 到 → ControllerExpandVolume（扩后端磁盘）
   → kubelet 在下一次 reconcile 发现 PV.NewSize > 当前 → NodeExpandVolume（扩文件系统）
   → resize2fs / xfs_growfs
```

常见坑（必答）：

- **只能扩不能缩**（spec 改小会被拒绝）；
- 若卷**没有被 Pod 挂载**，某些文件系统无法离线扩容 ⇒ 需要重新挂载才生效（视 FS 与驱动能力）；
- `PVC.status.capacity` 与 `conditions: FileSystemResizePending` 能告诉你卡在哪一步： 条件是 `FileSystemResizePending` ⇒ 磁盘已扩，**文件系统还没扩**（通常因为没挂载）；
- 扩容后要确认 `df -h` 在容器视图里也变大了。

```bash
kubectl get pvc db -n prod -o jsonpath='{.status}'
kubectl describe pvc db -n prod | tail -20
```

A16. 卷健康监控（Volume Health）能做什么?

- kubelet 侧周期性 `NodeGetVolumeStats` ⇒ 暴露 `kubelet_volume_stats_*` 指标（**容量/inode 使用率的真相来源**）；
- CSI 侧 `VolumeCondition`（Abnormal）⇒ 通过 `external-health-monitor` 上报到 PVC 事件（该功能处于演进阶段，**请以所用版本为准**）；
- 应用层仍应自己监控 IO 延迟/错误，**不能只依赖 K8s 层。**

```
# 卷使用率（最基础的容量告警）
kubelet_volume_stats_used_bytes / kubelet_volume_stats_capacity_bytes > 0.85
# inode 使用率（被忽视的高频事故源）
kubelet_volume_stats_inodes_used / kubelet_volume_stats_inodes > 0.9
```

A17. emptyDir / hostPath / local PV 三者的区别?

类型	生命周期	调度感知	适用
emptyDir	与 Pod 同生共死	无	临时缓存、scratch 空间（`medium: Memory` 用内存，**会计入容器内存 limit**）
hostPath	数据留在节点上	**完全无感知**（Pod 漂到别的节点就"数据丢了"）	仅用于读取节点文件（日志采集、CNI），**禁止承载业务数据**
local PV（local volume）	跟随 PV（Retain/手工管理）	**有 nodeAffinity，调度器看得懂**	本地 SSD / 高性能缓存类有状态服务

```
# local PV 示例（由 local-static-provisioner 自动生成）
apiVersion: v1
kind: PersistentVolume
metadata:
  name: local-pv-nvme-1
spec:
  capacity: {storage: 1000Gi}
  volumeMode: Filesystem
  accessModes: ["ReadWriteOnce"]
  persistentVolumeReclaimPolicy: Retain
  storageClassName: local-nvme
  local:
    path: /mnt/disks/nvme1
  nodeAffinity:
    required:
      nodeSelectorTerms:
        - matchExpressions:
            - key: kubernetes.io/hostname
              operator: In
              values: ["node-7"]
```

必答：local PV 的高性能代价是 **Pod 被钉在该节点**（节点故障 ⟹ 数据暂时不可用），因此**必须配合应用层多副本或存储层副本。**

A18. 投影卷（projected）与 Secret/ConfigMap 挂载的热更新?

```
volumes:
  - name: all-in-one
    projected:
      defaultMode: 0400
      sources:
        - secret:
            name: db-cred
        - configMap:
            name: app-conf
        - downwardAPI:
            items:
              - path: labels
                fieldRef: {fieldPath: metadata.labels}
        - serviceAccountToken:
            path: token
            expirationSeconds: 3600
```

必答：

- projected 卷把多种来源合并到一个目录，且**更新是原子的**；
- ConfigMap/Secret 作为 volume 挂载会**定期更新**（非 subPath 时），但**环境变量注入不会更新**；
- **大量 Pod 挂载同一个 Secret ⟹ kubelet 会 watch 它 ⟹ apiserver 压力放大** ⟹ 大规模场景应使用 `immutable: true`（减少 watch/拉取成本）。

```
apiVersion: v1
kind: Secret
metadata:
  name: db-cred
immutable: true # 大规模建议：不可变 Secret 可显著降低 kubelet 侧 watch 成本
type: Opaque
```

A19. fsGroup 与权限：为什么 PVC 挂载后目录不可写？为什么首次挂载特别慢?

```
securityContext:
  fsGroup: 2000
  fsGroupChangePolicy: OnRootMismatch # ★ 关键优化项（版本相关，请核对）
```

- `fsGroup` 会让 kubelet 递归 **chown/chgrp + 设置 setgid**，让 Pod 有写权限；
- **痛点**：卷里文件非常多时，递归 chown 可能耗时**数十秒到数分钟**（这也是"Pod 首次挂大卷特别慢"的常见原因）；
- `fsGroupChangePolicy: OnRootMismatch` ⟹ 只有当**根目录权限不匹配时才做**，避免每次挂载都全量递归；
- 另一种做法：在 Pod 里用 `initContainer` 显式 `chown`，可控且可观测。

> 注意 fsGroup 配额（`supplementalGroups` + `seLinuxOptions`）也会影响。

A20. subPath 有什么坑?

```
volumeMounts:
  - name: data
    mountPath: /var/lib/mysql
    subPath: mysql # 只挂载卷里的某个子目录
```

坑	说明
热更新失效	subPath 挂载的 ConfigMap/Secret **不会热更新**（符号链接切换机制失效）
扩容/ cyanide 行为差异	有些行为与整卷挂载不一致（例如某些检查依赖卷根路径）
变量替换	`subPathExpr` 支持从 downward API 取值（`$(POD_NAME)`），可做"每 Pod 独立子目录"

A21. 挂载传播（mountPropagation）什么时候必须配?

值	行为	典型用途
`None`（默认）	完全私有	普通业务
`HostToContainer`	宿主机后续 mount 能传进容器	想看到节点新增的挂载
`Bidirectional`	容器里 mount 也能传回宿主	**CSI node driver、块设备管理类 DaemonSet 必须**

必答：忘记配 `Bidirectional` 是"CSI 节点插件在容器里看不到宿主 mount / 挂载不生效"的高频原因；因为它是特权能力，通常需要配合 privileged 或更细的能力授予（**注意安全风险，别全集群放开**）。

A22. StatefulSet 与 PVC 的关系? 磁盘为什么会泄漏?

```
apiVersion: apps/v1
kind: StatefulSet
spec:
  volumeClaimTemplates: # ① 每个副本自动创建 PVC
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        storageClassName: fast-ssd
        resources:
          requests: {storage: 100Gi}
```

命名规则：`<claim-template>-<statefulset>-<ordinal>`，例如 `data-mysql-0`。

必答的三个泄漏来源：

1. **删除 StatefulSet 不会删除 PVC**（设计如此，防撞误删）⇒ 需要人工清理或用 ownerReference 方案；
2. **缩容（scale down）也不会删 PVC**（保留数据以便扩容回来）⇒ 长期缩容会留一堆闲置盘；
3. `reclaimPolicy: Retain` **的 Dead PV** 没人管 ⇒ 孤儿磁盘持续计费。

治理：

- 建立定期巡检：找出"未被任何 PVC/Pod 引用的 PV"与"后端的孤儿磁盘"；
- 给 PVC/PV 打 owner/team/cost-center 标签，做成本归因；
- 删除流程必须进工单，并核对是否有最近的快照。

A23. CSI 驱动的"凭据"与轮换怎么做?

CreateVolume / NodePublish 常常需要凭据（通过 `csi.storage.k8s.io/node-publish-secret-name` 等参数引用 Secret）：

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: secure-rbd
provisioner: rbd.csi.ceph.com
parameters:
  clusterID: <ceph-fsid>
  pool: kubernetes
  csi.storage.k8s.io/provisioner-secret-name: csi-rbd-secret
  csi.storage.k8s.io/provisioner-secret-namespace: ceph-system
  csi.storage.k8s.io/node-stage-secret-name: csi-rbd-secret
  csi.storage.k8s.io/node-stage-secret-namespace: ceph-system
```

必答：

- 凭据泄漏风险 ⇒ 严格 RBAC，Secret 只给 CSI 相关 SA；
- **轮换要有"双凭据并存"窗口**，避免轮换瞬间所有挂载/供给失败；
- 凭据失效的典型表现：`Failed to provision volume ... AuthFailure` / 已有卷无法 mount。

A24. CSI driver 对象（`CSIDriver`）有什么作用?

```
apiVersion: storage.k8s.io/v1
kind: CSIDriver
metadata:
  name: ebs.csi.aws.com
spec:
  attachRequired: true # 是否需要 ControllerPublishVolume (attach)
  podInfoOnMount: true
  fsGroupPolicy: ReadWriteOnceWithFSType
  volumeLifecycleModes: ["Persistent"]
  storageCapacity: true # 是否上报 CSIStorageCapacity
  requiresRepublish: true # 是否需要周期性重新 publish（版本相关）
  tokenRequests: []
```

必答：`attachRequired` 决定 **是否走 attach 流程**（云盘 true；NFS/CephFS 通常 false）；`podInfoOnMount` 让驱动知道是哪个 Pod 在挂载（用于按 Pod 鉴权/审计）。

Part B • 常见存储方案 / 模式专项篇（18 题）

> 这一篇按"方案能力矩阵 + 各种模式的取舍"出牌。

B1. 先分清楚三类存储：块 / 文件 / 对象

类型	语义	K8s 里的消费方式	典型用例
块（Block）	裸磁盘/裸设备，自己格式化	PVC（RWO）为主，可 `volumeMode: Block`	数据库、Kafka、etcd
文件（File/NAS）	带文件系统的共享目录	PVC（**RWX**）	多 Pod 共享内容、CI 缓存、媒体处理
对象（Object）	HTTP S3/OSS 语义，非 POSIX	**应用 SDK 直连**（最正统），或经 CSI/FUSE 挂载（性能与语义有损）	图片/视频/备份归档/日志

必答：把对象存储当 PVC 挂载通常要点游览从来都不是最优解：POSIX 语义受限、ls 慢、追加写成本高。**选型铁律：能用 SDK 就别挂载。**

B2. 主流 CSI 驱动能力矩阵（背这张表足够）

驱动	类型	典型访问模式	AZ 约束	亮点 / 注意
AWS EBS CSI	块	RWO	**单可用区**（盘与 AZ 强绑定）	快照、扩容、加密；跨 AZ 会挂不上
AWS EFS CSI	文件	**RWX**	跨可用区（Regional）	多 Pod 共享；注意吞吐模式（bursting/provisioned/elastic）
GCE PD CSI	块	RWO	单 zone（另有 regional PD 选项）	hyperdisk 等新形态请以厂商文档为准
Azure Disk / Files	块 / 文件	RWO / RWX	Disk 受 zone 影响	Files 走 SMB/NFS 协议
vSphere CSI (CNS)	块（VMDK/FCD）	RWO	受 datastore/集群拓扑约束	与 vSphere 存储策略（SPBM）打通
Ceph CSI（RBD）	块	RWO	取决于 CRUSH/map 配置（常见按机架/主机）	强一致、性能好；运维复杂度集中在 Ceph 侧
Ceph CSI（CephFS）	文件	**RWX**	同上	多 Pod 共享；MDS 是容量/性能关键
Longhorn	块（iSCSI 数据面）	RWO	副本跨节点	轻量、易部署、适合边缘/中小；数据面成本在用户态 iSCSI
OpenEBS Mayastor	块（NVMe-oF）	RWO	取决于副本策略	面向 NVMe 高性能（容量陷阱：需要 HugePages/CPU 预留，请以版本为准）
OpenEBS cStor / Jiva	块	RWO	—	Jiva 已逐步退场（新版本请以官方说明为准），新部署建议 Mayastor/localPV
NFS subdir external provisioner	文件	**RWX**	无（但依赖 NFS server 可用性）	最简单；**官方 nfs-csi-driver 更推荐**，务必区分二者
JuiceFS CSI	文件（对象 + 元数据服务）	RWX	跨域	挂载进程在 Pod 侧，POSIX 语义较好；注意元数据性能
Portworx	块/文件（商业）	RWO/RWX	副本策略灵活	企业特性（加密、DR、auto-扩容），有 License
local-static-provisioner	本地盘	RWO	**节点强绑定**	最便宜的高性能；故障域最大
GlusterFS	文件	RWX	—	**社区已归档/不再推荐用于新集群**，面试时点明这一点很加分

B3. 什么时候该选"本地盘"而不是"网络分布式存储"？

维度	本地盘（local PV / 实例 SSD）	网络分布式（Ceph/Longhorn/云盘）
延迟/IOPS	**最好**	受网络影响
单盘容量上限	受机型限制	更灵活
节点故障影响	**数据暂时不可用**（需应用层多副本）	存储层自动切副本
调度灵活性	**Pod 被钉死在节点**	可迁移（取决于实现）
运维复杂度	低（但要管盘的健康/更换）	高（集群、副本、恢复、容量）
典型场景	Kafka/ES/OLAP 这类**自带多副本**的系统	数据库主从、通用有状态服务

标准答法：

> "选本地盘的前提是**应用层已经提供了冗余**（Kafka replication、MySQL 半同步、ES replica）。如果指望存储层做冗余，那就别用本地盘；反过来，如果应用层本来就有副本，用网络盘就是给同一份数据付两次复制成本。"

B4. 云盘最经典的坑：跨可用区挂载失败

现象：PVC 显示 Bound，Pod 一直 `ContainerCreating`，Events：`AttachVolume.Attach failed ... volume is in zone X, node is in zone Y`。

成因：云盘（EBS/云盘/Disk）是**可用区级资源**，只能挂给同 AZ 的虚拟机。

治理清单：

1. StorageClass 一定要 `volumeBindingMode: WaitForFirstConsumer`（**这是第一要务**）；
2. 需要保证副本分布 ⇒ 用 **TopologySpreadConstraints**（跨 zone）+ 允许等待也可以用 `allowedTopologies` 限定范围；
3. 恢复/迁移场景：**先给卷做快照再从快照创建新卷到目标 AZ**（这是标准迁移路径）；
4. 严禁跨 AZ 共享同一块的"土办法"（会数据损坏）。

B5. 需要 RWX（多节点读写）时怎么做?

方案	说明	注意
NFS CSI	最通用	NFS server 自身要高可用，且要注意并发一致性语义
CephFS	POSIX 语义较好	MDS 需要容量规划与高可用；配额要提前配
EFS / Azure Files / NAS	云托管文件存储	关注吞吐模式与突发额度
应用层上传对象存储	最利于扩展	语义要改造

必答警告:

- **绝大多数 RWX 文件系统都没有跨节点强一致锁保证**，同一文件的并发写要靠应用层协议（如数据库绝不能放 RWX 卷上，除非该存储明确支持并完成验证）；
- 关系型数据库、etcd 这类组件**只用 RWO/RWOP**。

B6. Rook-Ceph 面试会问什么？（分布式存储代表）

核心组件:

组件	作用	规模要点
MON	集群元数据脑核（维护 cluster map），**依赖 quorum（奇数）**	通常 3 或 5
MGR	对外展示指标/管理通道（dashboard/prometheus 模块）	至少 2（active/standby）
OSD	真正存数据的进程（一块盘一个 OSD）	数量决定容量/性能/故障域
MDS	CephFS 元数据服务	RWX 场景必需
RGW	对象网关	走 S3 协议场景

运维要点（必答）：

- **PG 状态是排障第一语言**：`active+clean` 正常；`degraded`（副本缺失）/ `incomplete`（PG 不完整）/ `inconsistent`（scrub 发现不一致）要立即处理；
- **副本 vs 纠删码**：副本简单但成本高（3x），EC 省空间但 CPU/恢复成本和读放大更高；
- **failure domain**：按 host/rack 分布，别让副本全在一台机器；
- **恢复与再平衡会引发 IO 风暴** ⇒ 要控制回填/恢复的并发与优先级（`osd_max_backfills`、`osd_recovery_*` 等参数，**请以所用版本 official doc 为准**）；
- **scrub/deep-scrub** 会消耗 IO，要错峰；
- 网络建议 **public network 与 cluster network 分离**（南北向客户端流量 vs 东西向复制流量）。

```
ceph -s # 一眼看 health / mon quorum / osd 数
ceph osd tree # 拓扑与权重
ceph pg stat / ceph pg dump_stuck
ceph health detail
```

B7. Longhorn 适合谁？它的短板在哪?

适合：中小集群/边缘/测试环境，追求"部署简单、能看到 GUI、社区活跃"，单盘容量需求不高。

短板（必答）：

- 数据面走 **iSCSI over TCP（用户态 tgt）**，高并发下的 CPU 开销与延迟明显高于 NVMe-oF/原生块；
- **大规模下会出现 iSCSI login/session 抖动**（成百上千卷时尤其要注意）；
- 副本重建会占满网络。\ ⇒ 结论：**不建议在大规模高 IOPS 的核心生产上作为主力**，更适合边缘与中等负载。

B8. OpenEBS 各模式怎么选?

模式	数据面	适用	备注
localPV-hostpath	本地目录	追求简单、可接受节点绑定	常用于快速搭建
localPV-device（LVM/ZFS）	本地设备	需要本地裸盘	TopoLVM 类方案也在此生态位
Mayastor	NVMe-oF（用户态 SPDK）	高性能本地网络存储	需要 CPU/HugePages 预留与节点能力校验（按版本确认）
Jiva / cStor	早期分布式引擎	**新建集群不再推荐**	请查看官方当前推荐路线

B9. 云盘的"性能档"怎么选？（比如 IOPS/吞吐突发）

要点:

- 云盘通常按"**容量越大、配额越高**"或"**可购买 IOPS/吞吐**"两种方式给配额（gp3/io2 之类的命名因厂商而异）；
- **突发型（bursting）盘有积分机制**：小容量盘扛不住持续高 IO，积分耗尽后 IOPS 直降到基线（典型"跑一阵子突然变慢"）；
- **治理**： 用 StorageClass parameters 固化 IOPS/吞吐档位，避免有人手抖建低配额盘； 监控卷的 IOPS/吞吐/延迟（云监控或节点层），对"接近配额"做告警； 压测要覆盖"稳态"而非仅看突发值。

```
# 卷接近 IOPS/吞吐配额（具体来源依赖云 exporter 或节点侧采集）
# 通用做法：采集 /var/lib/kubelet/pods 对应设备的 IOPS 并与云商配额对比做阈值告警
```

B10. 卷加密有几种做法?

做法	层	说明
云厂商静态加密	存储后端	最简单，通常默认开启，靠 KMS
CSI 层（LUKS/dm-crypt）	节点	云无关的通用做法，注意 key 管理与性能损耗
应用层加密	应用	最细粒度，成本最高
Secret 静态加密（etcd）	控制面	**这是保护 etcd 里的 Secret，与卷数据加密完全是两回事**（面试常混）

B11. 备份：Velero vs CSI 快照 vs 存储侧原生快照?

方案	粒度	恢复方式	适用
CSI 快照	单卷，速度快	快照→新卷→重建 PVC	快速回滚、同集群/同区域
Velero（支持 CSI 卷快照 + 文件系统备份两种模式）	命名空间/标签/整集群（含对象清单）	恢复到原集群或新集群	**异地容灾、集群迁移、误删恢复**
存储侧原生快照（集群自带的定时快照）	存储层	视产品	与 K8s 对象状态不一定一致，单独用风险大

必答：**Velero 的价值在于"对象清单 + 卷"一起备份**，纯 CSI 快照恢复时缺少 Deployment/StatefulSet/Service 等对象，两者应该配合而不是二选一。并且要定期做**恢复演练**（没有演练的备份等于没有备份）。

B12. VolumeGroupSnapshot 是干什么的？（多卷一致性）

一次给**多个卷**同时打快照，保证跨卷的时间一致性（常见于"日志卷+数据卷"或分片数据库）。**该能力为较新特性，请以所用版本的 Beta/GA 状态为准。**

B13. ReadWriteOncePod 的真实用途?

```
RWO：同一节点上可以有多个 Pod 挂载同一个 PVC
RWOP：整个集群仅限一个 Pod（kubelet 强制）
```

用途：防止"运维误操作把第二个 Pod 挂到同一个数据卷"导致文件损坏（比如手工 debug Pod 挂了生产盘）。**注意它不等于 fencing**——存储侧未必知道谁拿到了锁，只是 K8s 侧的单写保护。

B14. 有状态应用上线的标准清单?

```
1）Headless Service + StatefulSet（稳定网络身份）
2）volumeClaimTemplates + 合适的 StorageClass（带 allowVolumeExpansion）
3）PDB + TopologySpreadConstraints（跨可用区/机架）
4）terminationGracePeriodSeconds + preStop（优雅终止，避免脏数据）
5）探针：liveness 只查自身，readiness 区分"在同步/在 serving"
6）requests/limits 与 QoS：数据库类建议 Guaranteed
7）资源隔离：专用节点池（taint）避免吵闹邻居抢 IO
8）备份：定时快照 + 跨区域复制 + 定期恢复演练
9）监控：卷容量/inode/延迟/IOPS + 应用层 db metrics
10）升级策略：逐副本滚动 + 先扩勿删 + 明确回滚（含数据回滚）路径
```

B15. 每个节点能挂多少卷？谁在限制?

- 云商限制 **每台虚拟机可挂磁盘数**，K8s 会把它体现为节点的 **allocatable 资源**，形如 `attachable-volumes-aws-ebs: 39`（数字随实例类型而变）；
- 调度侧有相应的 **NodeVolumeLimits**（versions and csi-specific limits 以版本为准）做前置检查；
- **超限表现**：Pod Pending，Events 会明确写出该节点挂载数已达上限。

```
# 节点已挂载卷数 / 上限（依赖 kube-state-metrics / 云 exporter）
kube_node_status_allocatable{resource="attachable_volumes_aws_ebs"}
```

B16. CSI driver 自身要不要 HA?

部署	HA 做法
Controller（Deployment）	多副本 + **leader election**（sidecar 内部选主）+ PDB；跨 AZ
Node（DaemonSet）	每节点一份；**本身无 HA**，故障只能影响单节点 ⇒ 用节点池/灰度发布降低爆炸半径
livenessprobe	探活 socket，暴露健康状态，便于告警与自动重启

必答：Controller 挂 ⇒ **全集群新卷供给/attach/扩容全部停摆**（存量读写通常不影响）；Node 挂 ⇒ **单节点新挂载失败**（已挂载的卷通常不受影响）。

B17. CSI 驱动部署的实例配置（一个通用骨架）

```
# Controller（示意，务必以驱动官方 Chart/ manifests 为准）
apiVersion: apps/v1
kind: Deployment
metadata:
  name: csi-provisioner
  namespace: csi-system
spec:
  replicas: 2
  strategy: {type: RollingUpdate, rollingUpdate: {maxUnavailable: 1}}
  template:
    spec:
      containers:
        - name: csi-provisioner
          image: registry.k8s.io/sig-storage/csi-provisioner:v3.6.2
          args:
            - "--csi-address=$(ADDRESS)"
            - "--timeout=60s" # 建议：明确超时，避免无限等待
            - "--leader-election"
            - "--leader-election-namespace=csi-system"
            - "--extra-create-metadata"
          env:
            - {name: ADDRESS, value: /csi/csi.sock}
        - name: csi-attacher
          image: registry.k8s.io/sig-storage/csi-attacher:v4.4.2
          args: ["--csi-address=$(ADDRESS)", "--leader-election", "--timeout=60s"]
        - name: csi-resizer
          image: registry.k8s.io/sig-storage/csi-resizer:v1.9.2
          args: ["--csi-address=$(ADDRESS)", "--leader-election", "--timeout=60s"]
        - name: csi-snapshotter
          image: registry.k8s.io/sig-storage/csi-snapshotter:v6.3.2
          args: ["--csi-address=$(ADDRESS)", "--leader-election", "--timeout=60s"]
        - name: liveness-probe
          image: registry.k8s.io/sig-storage/livenessprobe:v2.11.0
          args: ["--csi-address=$(ADDRESS)"]
        - name: driver
          image: <厂商 CSI 镜像>
          args: ["--endpoint=$(CSI_ENDPOINT)", "--logtostderr", "-v=5"]
```

> 镜像 tag 请务必按自身版本矩阵选择（**sidecar 与 K8s 版本有兼容性要求**，这是升级前必须核对的东西）。

B18. 常见 CSI driver 故障的"现象→原因"速查

现象	常见原因
PVC 一直 Pending，`no persistent volumes available` / `ProvisioningFailed`	provisioner 没起来、StorageClass 名字拼错、后端配额不足、凭据失效
`FailedAttachVolume`	云 API 限流、卷在别的 AZ、已达节点挂载上限、上一次 detach 未完成
`FailedMount`	kubelet 侧 mount 失败（fs 损坏、mount 参数错、node plugin 崩）
`VolumeAttachment` 卡住	ControllerPublish 超时重试/VolumeAttachment 残留未清理
`node(s) had volume node affinity conflict`	WaitForFirstConsumer + AZ 拓扑不匹配
`Multi-Attach error`	上一个 Pod 还没完全释放卷（周期性驱逐/节点故障常见），或有第二个 Pod 非法挂载
卷挂载后只读	文件系统损坏进入 read-only、或两个 Pod 同时写
扩容后容器视图不变	NodeExpandVolume 未执行/卷未挂载

Part C • 大规模集群存储优化篇（14 题）

> 定义：**大规模 = ≥ 1000 节点 / PVC 数万 / 单集群有状态服务上百。**

C1. 大规模下最先出问题的四个地方

#	位置	症状	治理方向
1	**对象数量（PVC/PV/VolumeAttachment）**	apiserver/etcd 压力、list 慢	控制 PVC 数量、定期回收孤儿、审视 VolumeAttachment 残留
2	**调度侧的 VolumeBinding**	调度变慢、pending PVC 堆积	收敛挂载卷 Pod 的比例、减少等待绑定的 PVC
3	**Attach/Detach 串行 + 云 API 限流**	Pod 卡 ContainerCreating 十几分钟	分批创建、限流调优、避免大规模同时重建有状态服务
4	**节点挂载数上限与单/多 SAN storm**	挂载失败、iSCSI 会话抖动	节点池化、异构规格、控制单节点卷密度

C2. PVC/PV 数量对控制面的影响怎么量化?

```
# PVC/PV 总数增长趋势（依赖 kube-state-metrics）
count(kube_persistentvolumeclaim_info)
count(kube_persistentvolume_info)
# VolumeAttachment 残留（不该长期存在）
count(kube_volumeattachment_info)
# 处于 Pending / Lost 的 PVC（异常信号）
count(kube_persistentvolumeclaim_status_phase{phase="Pending"})
```

治理：

- 建立 **PVC 生命周期归属**（owner/team 标签），定期清理无主 PVC/PV；
- **孤儿磁盘巡检**：平台侧（PV 列表）与云/存储侧（实际卷列表）**对账**，这是最省钱的一步；
- 对"每批次重建大量有状态 Pod"的动作做**分批限制**（否则瞬间 VolumeAttachment 洪峰会打爆 controller 与云 API）。

C3. "Pod 挂着卷十几分钟起不来"怎么排查和根治?

定位顺序（必会）：

```
kubectl describe pod <pod> # 看 Events 的时间戳，判断卡在哪一步
# 00:00  Scheduled
# 00:00  SuccessfulAttachVolume  ← attach 完成（或这里一直是 FailedAttachVolume）
# 00:01  FailedMount / MountVolume.SetUp failed ← mount 阶段问题
# 00:15  MountVolume.SetUp succeeded
kubectl get volumeattachment | grep <pv-name> # 有没有残留
kubectl -n <csi-ns> logs <csi-controller> -c csi-attacher
ssh node: journalctl -u kubelet | grep -i mount
```

典型根因：

1. **上一次 detach 没完成**（节点宕机/强制删除 Pod）⇒ 新 Pod 一直等待，直到 detach reconcile 超时解除；
2. **云 API 限流**（大规模同时 create/attach）⇒ 需要给 controller 侧 qps/burst 调优 + 业务侧分批；
3. **文件系统修复/check**（例如卷被脏卸载后 fsck）⇒ 时间不可预测；
4. **fsGroup 递归 chown**（大卷）⇒ 用 `fsGroupChangePolicy: OnRootMismatch`；
5. **Node plugin 崩** ⇒ 该节点所有新挂载失效。

根治：

- 有状态服务的重建**必须分批**（Deployment/StatefulSet 的滚动参数 + PDB）；
- 明确 clean detach 流程：**先删除 Pod 并确认 VolumeAttachment 消失，再重建**；
- 监控"卷操作耗时"并建告警（见 E 篇）。

C4. 调度侧的 VolumeBinding 会成为瓶颈吗?

会。每个**处于等待绑定状态的 PVC** 都会让调度器在 Filter 阶段做拓扑匹配计算；大规模集群里"成千上万个未完成绑定的 PVC"会显著拉长调度周期。

治理：

- 用 `WaitForFirstConsumer` 但同时**避免积累大量无 Pod 消费的 PVC**（无用 PVC 直接删掉）；
- 尽量避免一个 Pod 挂**过多** PVC（每个都要参与调度计算）；
- 关注 `scheduler_framework_plugin_duration_seconds{plugin="VolumeBinding"}`（见调度篇 L1）。

C5. CSI Controller 挂了怎么办？怎么预防?

影响面	说明
新卷供给	全部 Pending
attach/detach	全部卡住（**会影响 Pod 重建**）
扩容	全部卡住
存量读写	**通常不受影响**（这是重要的冷静判断依据）

预防：

- Controller 多副本（一般 ≥2）+ leader election + PDB + 跨 AZ；
- livenessprobe + 自动重启 + 告警；
- **给 CSI Controller 单独稳定的 machine/cgroup 资源保障**（它被系统/OOM 干掉会很惨）；
- 变更（升级/配置）**必须灰度**：先在测试集群或单 StorageClass 上验证。

C6. 单节点挂载卷数量的治理

- 通过 Scheduling/allocatable 天然限制，但要**在容量规划时算进去**：`节点数 × 单节点可挂卷数 ≥ 期望 PV 总数`；
- 有状态服务的 **Pod 密度要低于无状态**（给 DaemonSet、临时 Pod、扩容留余量）；
- 倾向于"少量大卷"而不是"海量小卷"（对象数、iSCSI session、attach API 调用都会减少）；
- 对"每个 Pod 一个 PV 的分库分表"型场景要做好上限评估。

C7. 大规模 Secret/ConfigMap 挂载怎么优化?

- **不变即** `immutable: true`（kubelet 不再频繁 watch，显著降低 apiserver 压力）；
- 避免把大量数据塞进单个 Secret（会拖慢 list/watch 与 etcd）；
- 避免"成千上万个 Pod 挂载同一个 Secret"这种放大结构（改用投射卷/环境变量/启动时读取方案视场景而定）；
- 定期审计未被引用的 ConfigMap/Secret。

C8. fsGroup 与大卷挂载的性能问题

见 A19。**重复强调因为它是最容易被忽略的性能杀手**：

```
securityContext:
  fsGroup: 2000
  fsGroupChangePolicy: OnRootMismatch # 大卷场景强烈建议（版本相关，请核对）
```

验证方式：对比开启前后同一卷的**挂载耗时**（用 `kubelet_pod_start_duration_seconds` 或 storage operation 指标），而不是凭感觉。

C9. 卷容量平滑治理（Storage Capacity Tracking）

- CSI provisioner 可以上报 `CSIStorageCapacity` 对象，让调度器在"后端容量不足"时提前避开/做出更好决策（**特性的成熟度与开启方式请以版本为准**）；
- 即使不启用该特性，SRE 侧也应做 "**容量余量告警**"（见 E 篇），避免出现"PVC Pending 且原因是没空间"的被动局面。

C10. 有状态服务的存储 IO 隔离怎么做?

手段	说明	代价
专用节点池（taint + toleration）	最彻底，避免 IO 争抢	利用率下降
requests/limits + QoS Guaranteed	保证资源不被挤	需要准确的画像
`ephemeral-storage` limits	限制容器可写层	主要防写爆
cgroup IO 限制（`io.weight` / `blkio`）	**需要 cgroup v2 且场景验证**	配置复杂需压测
存储的 QoS/限流（云盘 IOPS 配额）	直接在存储层限制	受限于后端能力

必答：**数据库这种抖动敏感负载一定要做 IO 隔离**，否则"邻居跑个批处理就把 IOPS 吃满"会直接体现为业务 P99 恶化，而且很难定位。

C11. 有状态服务跨可用区/跨集群的容灾怎么做?

方案	RPO/RTO	复杂度	说明
应用层复制（主从/半同步/Raft）	取决于复制延迟	中	**最推荐**（数据语义正确）
存储层同步复制（跨 AZ 卷类型/存储集群多站点）	接近 0	高	依赖后端能力，成本高、写延迟上升
快照 + 异地 Snapshot copy	分钟~小时级	低	定期快照 + 跨区域复制，作为兜底
Velero 备份恢复	取决于备份频率	中	兜底 + 迁移

必答：不要把"备份"当成"容灾"。**任何方案都必须写明 RPO/RTO 并演练过。**

C12. 集群升级时，存储相关的变更顺序与风险

```
① 先升级/验证 CSI driver 与 sidecar 的版本兼容矩阵（这是最容易翻车的环节）
② Controller 优先（滚动、观察新卷供给能力）
③ Node 插件按节点池灰度（DaemonSet 滚动）
④ 有状态服务的节点：优先使用"不驱逐"的方式（或确保 detach/attach 流程顺畅，分批进行）
⑤ 升级前务必确认快照/备份可用
```

风险：

- CSI Node 插件与 kubelet skews会导致全部挂载失败；
- 节点 drain 时若有大量卷 detach，容易触发限流；
- 注意 sidecar 的参数在跨大版本时会有改名/废弃（如超时、选举相关参数），升级前必须逐项比对新旧 manifests 差异。

C13. 存储成本治理（面试会加分）

1. **孤儿资源对账**：PV ↔ 后端磁盘 双向核对；
2. **Retain 的 Dead PV 定期清理**（带审批）；
3. **快照保留策略**（太多旧快照往往比数据盘更贵）；
4. **性能档位治理**：把 burst 型盘升级/规划为 provisioned；不要把 gp2 之类的老一代默认盘常态化；
5. **本地盘 vs 网络盘**：按业务冗余能力选择，避免重复支付复制成本；
6. **标签与成本归因**：给 PVC/PV 打 team/app 标签，做 showback/chargeback。

C14. 给你一个 2000 节点集群，怎么设计存储基线？（综合题）

按顺序答：

1. **问 SLO**：数据重要性？RPO/RTO？是否需要跨可用区/跨城容灾？单卷最大容量与快照频率？合规要求（加密、保留年限）？
2. **选型**：通用有状态 → 网络块（云盘/Ceph RBD）；共享读 → 文件存储；自带副本的系统 → 考虑本地盘降本；对象类数据 → 走 SDK。
3. **StorageClass 分层**：`fast-rwo`（高 IOPS，生产库）、`standard-rwo`（通用）、`rwx-shared`（文件）、`local-nvme`（自带副本的中间件），并统一 `WaitForFirstConsumer` + `allowVolumeExpansion`。
4. **拓扑**：AZ 感知 + TopologySpreadConstraints，明确 AZ 级套餐的支持范围。
5. **CSI 部署**：Controller ≥2 副本跨 AZ + PDB + leader election；Node 插件优雅发布。
6. **容量与配额**： ResourceQuota（`requests.storage` + per-SC PVC 数量）+ LimitRange + StorageCapacity 监控。
7. **备份**：快照 + Velero + 定期恢复演练，明确 retention 与异地复制。
8. **可观测**：卷容量/inode、attach/mount 耗时、CSI sidecar 健康、云配额 IOPS（见 E 篇）。
9. **变更治理**：CSI 升级灰度 + 分批 + 回滚（含数据回滚）+ 成本巡检月度化。
10. **演练**：GameDay（见 F 篇）。

Part D · 反模式清单（可直接用于生产巡检）

#	反模式	证据（怎么发现）	等级	修复动作	验证方式
D1	云盘 StorageClass 用 `Immediate` 绑定	出现跨 AZ pending、`volume node affinity conflict`	🔴 高	改 `WaitForFirstConsumer`（老 PV 需重建）	跨 AZ 场景下 Pod 能正常绑定挂载
D2	数据库放到 RWX 共享文件系统	多 Pod 并发写同一数据目录	🔴 高	改 RWO/RWOP 块存储	单写约束验证通过
D3	无 storageclass 配额，谁都能建 10TiB 卷	账单异常	🟠 中	ResourceQuota（`requests.storage` + per-SC pvc 数）+ LimitRange	超额申请被拒绝且报错清晰
D4	`reclaimPolicy: Retain` 但无人巡检	Dead PV/Dead 磁盘堆积	🟠 中	月度孤儿资源对账（PV↔后端）+ 清理工单	对账脚本输出为 0
D5	手动删 finalizer 强删 PVC	后端磁盘变孤儿	🔴 高	先释放使用方再删；强删必须关联"磁盘回收"动作	对账无残留
D6	给容器配置 `fsGroup` 但不配 `OnRootMismatch`	大卷挂载耗时分钟级	🔴 高	加 `fsGroupChangePolicy: OnRootMismatch`	挂载耗时回到基线（实测）
D7	生产用 hostPath 承载业务数据	Pod 漂移后数据丢失	🔴 高	换 local PV 或网络存储	漂移演练数据不丢
D8	hostPath 卷可被任意创建（无策略约束）	存在逃逸/篡改节点风险	🔴 高	PSA/PSS `baseline` 以上 + 策略引擎禁 hostPath	策略拦截生效
D9	CSI Controller 单副本、无 PDB	Controller 挂 ⇒ 全集群供给/attach 停	🔴 高	多副本 + leader election + PDB + 跨 AZ	杀一个副本不影响供给
D10	大量 Pod 挂载同一 Secret 且非 immutable	apiserver Secret watch 放大	🟠 中	`immutable: true` + 收敛用量	apiserver Secret 请求 QPS 下降
D11	一次性批量重建成百有状态 Pod	云 API 限流、Attach 洪峰	🔴 高	分批 + PDB + 观察 VolumeAttachment	批次窗口内无 FailedAttach
D12	未监控 inode	inode 耗尽导致"有空间却写不进"	🟠 中	加 `kubelet_volume_stats_inodes_used` 告警	告警能在耗尽前触发
D13	只有快照没有恢复演练	出事才发现恢复不了	🔴 高	定期 restore 演练并记录 RTO	演练记录达标
D14	新集群还在用 GlusterFS / 已归档方案	依赖无人维护组件	🟠 中	迁移到 CephFS/云文件/NFS CSI	迁移收益（可维护性）书面化
D15	StorageClass 参数随意（无 IOPS 档位）	burst 积分耗尽后性能暴跌	🟠 中	在 SC 里固化性能档并监控配额使用率	稳态压测达标

Part E • 可观测性：SLO、指标与告警

E1. 建议的存储侧 SLO（初始建议值，必须实测校准）

SLI	建议目标（起点）	度量
卷供给成功率（PVC → Bound）	≥ 99.9%	PVC Phase 与 ProvisioningFailed 事件计数
新卷供给时长 P99（含后端创建）	≤ 30s	PV create 时间 − PVC create 时间（需自建观测或审计）
Attach + Mount 完成时长 P99	≤ 60s	挂载相关指标 + `FailedMount` 计数
卷容量使用率（告警阈值）	≤ 85%	`kubelet_volume_stats_used_bytes / capacity`
inode 使用率	≤ 90%	`kubelet_volume_stats_inodes_used / inodes`
快照成功率	≥ 99%	VolumeSnapshot ready 状态
备份 RTO / RPO	按演练实测填写	演练记录

E2. 指标清单（先 `curl -k https://127.0.0.1:10250/metrics | grep volume` 核对）

指标	含义	用途
`kubelet_volume_stats_capacity_bytes` / `_used_bytes`	卷容量	容量告警
`kubelet_volume_stats_inodes` / `_inodes_used` / `_inodes_free`	inode 用量	inode 告警
`storage_operation_duration_seconds{operation_name=...}`	kubelet 卷操作耗时（`volume_mount`、`volume_attach`、`verify_volumes_are_attached` 等）	**挂载慢定位首选**
`volume_manager_total_volumes`（可用性与状态明细依版本）	节点上卷数量	密度治理
kube-state-metrics：`kube_persistentvolumeclaim_status_phase` / `_resource_requests_storage_bytes` / `kube_persistentvolume_status_phase` / `kube_volumeattachment_*`	对象状态	规模与异常检测
CSI sidecar 自身 metrics（各 sidecar 端口/路径不同）	调用量与错误	CSI 健康

E3. Prometheus 告警规则（可直接粘贴，阈值需实测校准）

```
groups:
  - name: k8s-storage
    rules:
      - alert: PvcPendingTooLong
        expr: |
          kube_persistentvolumeclaim_status_phase{phase="Pending"} == 1
        for: 15m
        labels: {severity: warning}
        annotations:
          summary: "PVC {{ $labels.namespace }}/{{ $labels.persistentvolumeclaim }} 长时间未绑定"

      - alert: PvcLost
        expr: kube_persistentvolumeclaim_status_phase{phase="Lost"} == 1
        for: 5m
        labels: {severity: critical}
        annotations:
          summary: "PVC {{ $labels.namespace }}/{{ $labels.persistentvolumeclaim }} 状态为 Lost"

      - alert: VolumeNearlyFull
        expr: |
          kubelet_volume_stats_used_bytes / kubelet_volume_stats_capacity_bytes > 0.85
        for: 30m
        labels: {severity: warning}
        annotations:
          summary: "卷 {{ $labels.namespace }}/{{ $labels.persistentvolumeclaim }} 使用率 {{ $value | humanizePercentage }}"

      - alert: VolumeInodeNearlyFull
        expr: kubelet_volume_stats_inodes_used / kubelet_volume_stats_inodes > 0.9
        for: 30m
        labels: {severity: warning}
        annotations:
          summary: "卷 inode 使用率过高：{{ $labels.namespace }}/{{ $labels.persistentvolumeclaim }}"

      - alert: VolumeOperationSlow
        expr: |
          histogram_quantile(0.99,
            sum by (operation_name, le) (rate(storage_operation_duration_seconds_bucket[5m]))) > 30
        for: 10m
        labels: {severity: warning}
        annotations:
          summary: "卷操作 {{ $labels.operation_name }} P99 达 {{ $value }}s"

      - alert: VolumeAttachmentStuck
        expr: kube_volumeattachment_created > 0
        for: 60m
        labels: {severity: info}
        annotations:
          summary: "存在长期残留的 VolumeAttachment，检查是否有异常卷"

      - alert: CsiControllerDown
        expr: up{job=~"csi-.*-controller"} == 0
        for: 5m
        labels: {severity: critical}
        annotations:
          summary: "CSI Controller 不可用，新卷供给/attach/扩容都会停摆"

      - alert: VolumeProvisionFailed
        expr: |
          sum by (namespace, reason) (
            increase(kubernetes_events_total{involved_object_kind="PersistentVolumeClaim",
                     reason=~"ProvisioningFailed|ExternalProvisioningFailure.*"}[15m])) > 0
        for: 10m
        labels: {severity: critical}
        annotations:
          summary: "出现 {{ $labels.reason }}：命名空间 {{ $labels.namespace }} 的卷供给失败"
          description: "本规则依赖 event exporter 将事件转成指标，请先确认其在你的集群可用"
```

> **事件监控补充**：强烈建议部署 **kube-event-exporter / kubernetes-event-exporter**，把 `ProvisioningFailed`、`FailedAttachVolume`、`FailedMount` 这些**带明确失败原因的事件**转成指标与日志，这是存储排障里性价比最高的一步。

```
# 思路：把事件转为计数器后按 reason 聚合
sum by (reason, namespace) (increase(kubernetes_events_total{involved_object_kind="Pod",
       reason=~"FailedMount|FailedAttachVolume|FailedScheduling"}[15m]))
```

> 🔗 **关联阅读**：第三篇 · G2. Prometheus 告警规则（可直接粘贴，阈值需实测校准） ｜ 第七篇 · E3. Prometheus 告警规则（可直接粘贴，阈值需实测校准）

Part F · 演练（GameDay）计划表

场景	注入方式	期望表现	验证的 SLO
CSI Controller 全部挂掉	停 Controller 副本	存量读写不受影响；新卷 Pending 且告警及时	供给可用性 + MTTR
Node 插件崩（单节点）	停 DaemonSet Pod	单节点新挂载失败，已有卷可用；节点被隔离	爆炸半径可控
卷已满	写满一个卷	容量告警先触发；业务能优雅降级	容量治理
inode 耗尽	狂写小文件	inode 告警触发	inode 监控有效
detach 未完成	强制删节点	卷能在可接受时间内被强制回收（工单流程）	卷回收流程
快照失败	让后端快照 API 失败	备份失败告警触发，且有重试/告警闭环	快照成功率
恢复演练	用快照/Velero 恢复服务到隔离环境	数据一致、服务可起来、耗时达标	RTO/RPO
跨可用区重建	模拟 AZ 故障	有状态服务能在其它 AZ 恢复（应用层复制有效）	容灾目标
大规模重建	一次性重建 200 有状态 Pod	分批次 + 无限流（考察治理是否生效）	变更稳定性
孤儿资源巡检	对账 PV vs 后端磁盘	能发现并清理孤儿资产	成本治理

Part G • 10 张面试速记卡（临阵磨枪）

1. **四层抽象**：Pod 内 volume（跟 Pod）→ PVC（独立）→ PV（集群级）→ StorageClass（模板）；PV 与 PVC **1:1 独占绑定。**
2. **三种策略**：`reclaimPolicy`（Delete/Retain）、`volumeBindingMode`（**强烈建议 WaitForFirstConsumer**）、`allowVolumeExpansion`（只能扩不能缩）。
3. **访问模式是"声明不强制"**：RWO ≠ 只能一个 Pod（同节点多 Pod 也能挂）；要严格单写用 **RWOP**；数据库别放 RWX。
4. **attach vs stage vs publish**：`ControllerPublishVolume`（云盘挂给虚拟机）→ `NodeStageVolume`（格式化挂载到 staging，可被多 Pod 复用）→ `NodePublishVolume`（bind mount 到 Pod 目录）。对应事件：`FailedAttachVolume` / `FailedMount`。
5. **sidecar 四件套**：provisioner（增删改卷）、attacher（发布/解绑）、resizer（扩容）、snapshotter（快照）+ node-driver-registrar（注册）与 livenessprobe（探活）。
6. **拓扑决定生死**：云盘是 AZ 级资源 ⇒ `WaitForFirstConsumer` + `allowedTopologies`；报错关键字 `volume node affinity conflict`。
7. **卡 Terminating 看 finalizer**：`kubernetes.io/pvc-protection` / `pv-protection`；强删 finalizer 会留下**孤儿磁盘**（最贵的坑）。
8. **挂载慢的四大元凶**：fsGroup 递归 chown、上一次 detach 没完成、云 API 限流、Node plugin 崩。先看 `storage_operation_duration_seconds` 再细化。
9. **Controller 挂 vs Node 挂**：Controller 挂 ⇒ **全集群**新卷操作停（存量读写通常没事）；Node 挂 ⇒ **单节点**新挂载失败。
10. **StatefulSet 的 PVC 不会自动删**：删除/缩容都保留 ⇒ 磁盘泄漏的经典来源；必须做 monthly 对账 + 标签归因。

第九篇 · GPU 与大模型基础设施

> GPU 硬件 · 调度 · 训练网络 · 存储 · 推理

0. 使用说明

0.1 GPU/AI Infra 面试的考察层次

层级	考点	表现
L1 会用	会在 Pod 里写 `nvidia.com/gpu: 1`	只会跑起来
L2 懂机制	说得清 device plugin / CDI / DRA / runtime hook / MIG / 拓扑	能画 GPU 接入链路
L3 懂作业语义	理解 Gang/Coscheduling、DDP/TP/PP/ZeRO、checkpoint、RDMA 与 NCCL	能与算法同学对话
L4 有平台/事故经验	能讲"MFU 只有 30% 怎么查""训练任务 hang 在 NCCL init 怎么定位""checkpoint 写爆共享存储怎么办"、能做容量与成本设计	有指标、有公式、有灰度、有演练

0.2 术语速查（面试里混用这些词会立刻暴露）

缩写	含义
SM	Streaming Multiprocessor，GPU 的计算单元组
HBM	High Bandwidth Memory，GPU 显存
NVLink / NVSwitch	卡间高速互联 / 交换芯片
MIG	Multi-Instance GPU，硬件级切分
MPS	Multi-Process Service，多进程共享同一 CUDA context
MFU / HFU	Model / Hardware FLOPs Utilization，模型算力利用率
TP / PP / DP / SP / EP	Tensor / Pipeline / Data / Sequence / Expert 并行
ZeRO / FSDP	优化器/梯度/参数分片的显存优化技术
GDR / GDS	GPUDirect RDMA（网卡直达显存） / GPUDirect Storage（存储直达显存）
RDMA / RoCE / IB	远程直接内存访问 / 基于以太网的 RDMA / InfiniBand
PFC / ECN / DCQCN	无损以太网的流控与拥塞控制
XID	NVIDIA GPU 的错误码（_driver 层事件）
FA / Quantization	FlashAttention / 量化（W8A8、W4A16 等）
vLLM / TRT-LLM / SGLang / TGI	主流推理引擎
TTFT / TPOT / ITL	首 token 延迟 / 每 token 输出时间 / token 间隔

0.3 一张 AI 平台组件地图（先有大画面）

```
┌──────────────────────── 平台层（K8s 之上）────────────────────────┐
│ 作业编排：Training Operator(PyTorchJob) / MPI Operator / Kubeflow │
│ 队列与配额：Kueue (ClusterQueue/LocalQueue/ResourceFlavor) / Volcano │
│ 批调度：Volcano / scheduler-plugins Coscheduling / YuniKorn │
│ 推理编排：Deployment+LWS / KServe / vLLM Production Stack / AIBrix │
│ 网关路由：Ingress / Gateway API + Inference Extension（缓存/队列感知） │
├──────────────────────── 资源与设备层 ────────────────────────┤
│ NVIDIA GPU Operator（driver / toolkit / device plugin / DCGM / mig） │
│ NVIDIA Network Operator（MOFED 驱动 + 多网卡 CNI + RDMA 设备插件） │
│ AMD GPU Operator / ROCm · Intel Gaudi · Habana · AWS Neuron / TPU │
│ NFD / GPU Feature Discovery（给节点打拓扑与型号标签） │
│ DRA (Dynamic Resource Allocation) 与 CDI (Container Device Interface) │
├──────────────────────── 硬件拓扑层 ────────────────────────┤
│ 节点内：8 卡 HGX + NVSwitch / PCIe Gen5 MIG / NUMA │
│ 节点间：IB 或 RoCEv2 无损以太网 + rail-optimized 拓扑 + 高速交换机 │
│ 存储：并行文件系统（Lustre/GPFS/Weka） + 对象存储 + 本地 NVMe 缓存 │
└──────────────────────────────────────────────────────────┘
```

Part A • GPU 硬件与软件栈（14 题）

A1. GPU 跟 CPU 到底有什么本质区别?

维度	CPU	GPU
设计目标	低延迟、复杂控制流、通用	**高吞吐、大规模并行、简单控制流**
执行模型	少量强核心，乱序执行、分支预测	海量简单核心，**SIMT（单指令多线程）**
调度单位	线程/进程	**warp（32 线程一组）** → 送到 SM
缓存/内存	大 cache 换低延迟	小 cache + **极高带宽显存（HBM）**
内存带宽	数十~数百 GB/s	**TB/s 量级**
典型瓶颈	单核串行延迟	**访存带宽、占用率（occupancy）、并行度不足**

必答的结构:

```
GPU = 多个 SM（每个 SM 内有 CUDA Core / Tensor Core / 寄存器文件 / 共享内存）
   → 每个 SM 同时跑多个 warp
   → 寄存器文件大小决定 occupancy（能同时驻留多少 warp 来隐藏访存延迟）
   → Tensor Core 专门做矩阵乘（这就是 LLM 的算力来源）
   → HBM 提供带宽；PCIe/NVLink 提供互联带宽
```

面试题补刀：为什么大模型训练看的是 **TFLOPS + HBM 带宽 + 互联带宽** 三者，而不是单看 TFLOPS?

> 因为训练是"把巨大的矩阵搬运到计算单元里算完再搬回来"，**带宽不够时 Tensor Core 会空转。**这也是为什么 H100 相对 A100 的提升里，HBM3 带宽与 NVLink 带宽提升同样关键（具体数值请以厂商 Datasheet 为准）。

A2. 一张卡要看哪几个规格? （挑卡必答题）

指标	为什么重要	典型（公开资料中的量级，请复核）
显存容量（HBM）	决定"能不能放下模型/优化器/batch"	A100 40/80GB；H100/H200 80GB / 更高；MI300X 192GB 级别
显存带宽	决定访存密集算子的速度	A100 约 2 TB/s；H100 约 3.35 TB/s
算力（BF16/FP8 TFLOPS）	决定 GEMM 速度	不同代次差异巨大，**以 Datasheet 为准**
互联带宽（NVLink vs PCIe）	决定**卡间**集合通信成本	NVLink 远高于 PCIe；**跨节点则看 IB/RoCE**
功耗（TDP）	决定机房/机架密度与设计	主流训练卡 350W~700W 级
ECC / 可靠性特性	长时间训练的静默数据风险	数据中心卡通常支持 ECC

必答的两条"选型常识":

1. **游戏卡/消费级卡（如 RTX 40 系）不适合数据中心**：无 ECC、显存小、驱动与保修限制、无 NVLink、机架形态与供电不匹配——但在**极小预算的推理/实验**场景会被用到，面试要能说出取舍；
2. **同一名字可能有多个 SKU**：H100 有 SXM（NVSwitch 全互联，带宽高）与 PCIe（互联受限）两种形态，**采购时看互联，不只是看算力**；中国区特供型号（如 H800/A800 之类）的互联或算力有削减，务必核对官方规格。

A3. NVLink / NVSwitch / PCIe 与为什么要"8 卡一台"

```
同一个 NVSwitch 域（典型 8 卡 HGX）：
   GPU0..GPU7 之间经过 NVSwitch 高速互联，带宽远高于 PCIe
   ⇒ 张量并行（TP）这种"每层都要 AllReduce"的切法通常限制在单机内（NVLink domain）

跨机：
   走网卡 → IB / RoCEv2 → 交换机
   ⇒ 带宽通常显著低于机内 NVLink ⇒ **把通信量大的并行切法留在机内**
```

面试金句:

> "大模型并行策略的第一性原理是：**通信代价最大的切法放在带宽最高的那一层**（NVLink > PCIe > RDMA 网络 > 跨地域）。TP 放机内、PP/DP 跨机，就是这么来的。"

GB200 / NVL72 这类"整机架 NVLink 域"是趋势（更大的机内域意味着可以抬高 TP 度），具体规格请以厂商资料为准。

A4. GPU 里的显存都去哪儿了? （训练显存账，必考）

全参数微调（混合精度 + Adam）的经验公式:

```
每参数显存（bytes）≈
    2（bf16/fp16 权重）
  + 2（梯度，通常同 dtype）
  + 4（fp32 主权重 master weight）
  + 4（Adam 一阶动量 m）
  + 4（Adam 二阶动量 v）
= 16 bytes / parameter
（另有激活值 activation、临时 workspace、NCCL buffer、碎片）

⇒ 7B 模型约 112GB 起步（优化器状态部分），70B 约 1.1TB 量级 —— **这是估算，需用真实 profile 验证**
```

推理显存:

```
权重：bf16 = 2 bytes/param ⇒ 7B ≈ 14GB，70B ≈ 140GB
KV cache（每 token 每层）：2(K,V) × kv_heads × head_dim × dtype_bytes
      ⇒ 整序列：  × layers × seq_len × batch
例（某 70B 级 GQA 模型，layers=80、kv_heads=8、head_dim=128、bf16）：
      每 token ≈ 2×8×128×2 = 4096 B/层 ⇒ 整序列 ≈ 80层 × 4KB ≈ 320KB/token
      ⇒ 4k 上下文单序列约 1.2~1.3GB
```

隐形的占用（面试加分）：

- **激活值/recompute**：不做 activation checkpointing 时随 batch×seq 线性暴涨 ⇒ 用重计算换显存（以算力换显存）；
- **cuDNN/cuBLAS autotune workspace**：启动时/形状变化时会申请额外显存；
- **碎片**：PyTorch caching allocator 会造成"明明还有空闲却 OOM" ⇒ `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True` 等策略；
- **NCCL buffer**：集合通信自身的 buffer 也要占显存。

A5. 训练并行策略：DP / TP / PP / SP / EP 各自解决什么?

策略	切什么	通信类型	放哪一层	代价
DP（数据并行/DDP）	切 batch，每卡一份完整参数	**AllReduce 梯度**	跨机 OK	显存最浪费（每份都要优化器状态）
ZeRO-1/2/3 / FSDP	**把优化器状态/梯度/参数分片**	AllGather + ReduceScatter	跨机 OK	通信换显存
TP（张量并行）	切单个算子/权重矩阵	**每层多次 AllReduce/ReduceScatter**	**强烈建议机内 NVLink**	通信最密集，但显存最省
PP（流水并行）	切层（不同卡算不同层）	点到点传 activation（小）	跨机 OK	**流水线气泡（bubble）** ⇒ 靠 microbatch 摊销
SP / Ulysses / Ring Attention	切序列（长上下文）	AllToAll / Ring 通信	依赖拓扑	长上下文场景
EP（专家并行，MoE）	切专家	**AllToAll**（最吃网络）	依赖拓扑	AllToAll 对网络最敏感

必答的组合逻辑：

> 业内常见组合是 **DP × TP × PP**（三维并行）+ activation checkpointing + ZeRO/FSDP。做法是：**TP 限制在单机 NVLink 域内**（通常 8），**PP 跨节点**，**DP 最外层**；这样把最贵的通信压在最快的链路上。

A6. 常见的显存优化手段（为什么有时候 1 卡能跑，有时候不行）

手段	用"什么"换"什么"
Activation checkpointing / recompute	算力 → 显存
ZeRO / FSDP 分片	通信 → 显存
混合精度 bf16/fp16 + fp32 master weight	数值稳定性风险 → 显存与速度
8-bit 优化器 / Adafactor	优化器精度 → 显存
Gradient accumulation	时间 → 显存（等效 batch 不变）
LoRA / QLoRA（微调）	冻结主权重 → 显存大幅下降（训练侧可几十倍下降，取决于实现）
Offload（优化器到 CPU/NVMe）	PCIe/NVMe 带宽 → 显存（慢，慎用）
FlashAttention	显存与访存 → 速度双赢（IO-aware 实现）

A7. 推理侧的核心概念：prefill / decode / batching

```
prefill：把整段 prompt 一次性算完 ⇒ **算力密集**，决定 TTFT（首 token 时间）
decode ：逐 token 生成，每步读 KV cache ⇒ **带宽密集**，决定 TPOT/ITL
```

概念	作用
Continuous batching	请求逐个加入批次，避免静态 batch 的空等（vLLM 的核心之一）
PagedAttention / 分页 KV	把 KV cache 分页管理，**降低碎片、提升并发**
Prefix caching	相同前缀（system prompt、长文档）复用 KV ⇒ 大幅降 first-token 成本
抢占/换出（preemption \& swap/recompute）	显存不足时怎么处理在途请求
Chunked prefill / Prefill-Decode 分离	让两种不同特性的计算**分开部署/分开调度**（当下趋势）
投机解码（speculative decoding）	小模型起草 + 大模型验证，降延迟（牺牲少量算力）
量化（W8A8 / W4A16 / FP8 / AWQ / GPTQ）	省带宽省显存，换精度

必答的关键设计：**prefill 与 decode 的资源瓶颈不同**（算力 vs 带宽），因此大规模服务里会做 **分离部署（disaggregated）**，把两种阶段放在不同 GPU 池，并通过高速网络传 KV —— 这也是为什么"推理集群"的网络设计也开始重要。

A8. CUDA 软件栈：driver / toolkit / runtime 都是啥?

```
应用 → CUDA Runtime API（libcudart）→ CUDA Driver API（libcuda）→ 内核态驱动模块 → GPU
其中：
  - nvidia-driver Container（容器化驱动）：把驱动打包成容器，节点启动时由 GPU Operator 装载
  - CUDA Runtime/库（cuBLAS、cuDNN、NCCL）：传统做法是**打进镜像**
  - Forward Compatibility：`compat` 包允许容器内 toolkit 版本 > 宿主 driver 版本（有条件限制，请以官方说明为准）
```

必答的三条：

1. **容器里不需要装驱动，只需要 userspace CUDA 库**（由 nvidia-container-toolkit 从宿主注入驱动节点与库）；
2. **驱动版本 ↔ CUDA toolkit 版本有兼容矩阵**，混用会报 `CUDA driver version is insufficient for CUDA runtime version` 之类；GPU Operator 的价值之一就是统一管理这套矩阵；

3. **NCCL 是集合通信库**，训练和推理的多卡/多机都依赖它，它直接决定"能不能跑起来、跑多快"（见 C 篇）。

A9. GPU 是怎么被放进容器的？（运行时链路）

三条代际演进：

代际	机制	说明
老	**nvidia-docker2 / nvidia-container-runtime** hook	依赖 docker，逐步退场
主流	**nvidia-container-toolkit + libnvidia-container**，通过 **NRI / containerd** `runtime_class` **的 prestart hook**	containerd/CRI-O 下把 `/dev/nvidia*`、驱动库、CLI 注入容器，受环境变量控制（如 `NVIDIA_VISIBLE_DEVICES`）
未来	**CDI（Container Device Interface）**	标准化设备声明，运行时读 `/etc/cdi/*.json` 注入，**与 DRA 配合**是新一代路径

```
# CDI 规格文件示意（由 device plugin / GPU Operator 生成）
cat /etc/cdi/nvidia.yaml | head -40
```

必答：容器的 GPU 可见性由 `NVIDIA_VISIBLE_DEVICES` / `CUDA_VISIBLE_DEVICES` 控制；device plugin 通过 **CDI 或 hook** 把设备与驱动库塞进容器。面试能说出"CDI 是 DRA 之后的设备注入标准"会明显加分。

A10. MIG 为什么比"时间片共享"更高级？

维度	时间片共享（time-slicing）	MIG
隔离层级	仅调度时间片，**显存与错误域共享**	**硬件级切分**（SM + L2 + 显存控制器都分开）
QoS 保证	无	有（算力与显存都隔离）
显存	**不隔离**（一个进程 OOM 会拖累同卡其它进程）	每实例独立显存
灵活性	运行时即可用	需要设置 **MIG strategy（none/single/mixed）**，切换要排空节点
限制	无硬件要求	**仅部分型号支持**（如 A100/H100 系列），且**实例几何组合有限制**

```
# GPU Operator / device plugin 的 MIG 配置示意（请以所用版本文档为准）
config:
  name: mig-config
  default: all-balanced
```

常见的应对策略：

- **none**：整卡直通（训练场景默认，MIG 会损失部分算力与带宽效率）；
- **single/mixed**：按统一的 MIG 几何切分（推理场景）。

必答坑：一个节点上的 MIG 几何**必须统一规划**，混配置会导致可分配资源碎片化、调度困难；切换 MIG 策略通常要**排空节点**（这是重操作）。

A11. 时间片共享（time-slicing）什么时候用？有什么雷？

适合：开发/测试环境、小模型推理、利用率低但需要"能跑"的场景。

雷：

1. **无显存隔离**：一个 Pod 占满显存会让同卡 Pod 全部 OOM；
2. **上下文切换开销**：GPU 计算被切来切去，**延迟敏感推理会明显抖动**；
3. **K8s 侧无法知道"实际用了多少显存"** ⇒ 只能靠上层约束（requests 里写假的 gpu 数量）。

```
# device plugin 的 time-slicing 配置示意
apiVersion: v1
kind: ConfigMap
metadata:
  name: nvidia-device-plugin-config
data:
  config.yaml: |
    version: v1
    sharing:
      timeSlicing:
        resources:
          - name: nvidia.com/gpu
            replicas: 4 # 建议初值：按业务并发与延迟要求实测调整
```

> 想做到"显存也隔离"，原生 K8s 做不到 —— 需要 **HAMi / k8s-vGPU 这类第三方方案**（通过 admission + 运行时拦截做显存配额），或者使用 **MIG**。

A12. MPS 是什么？和 MIG/时间片有什么不同？

MPS（Multi-Process Service）让多个进程共享同一个 **CUDA context**，减少上下文开销、提高小 kernel 的利用率。

方案	隔离	适用场景
时间片	差（易互相影响）	轻量共享、测试
MPS	中（共享 context，一个进程崩溃可能影响同 context 其它进程）	多小任务、推理并发
MIG	强（硬件级）	需要 QoS 保证的场景

实践中：MPS + MIG 可以叠加使用（在 MIG 实例内再用 MPS），具体支持请以厂商文档为准。

A13. GPU 健康：XID 错误与常见故障信号

`XID` 是 NVIDIA 驱动上报的错误码（具体码值的含义以厂商文档为准），常见需要关注的类别：

类别	典型表现
显存类（如 ECC / DBE / row remap 相关码）	建议**排空节点并做 GPU reset**，反复出现需报修/换卡
NVLink / 互联类错误	机内集合通信失败、训练 hang ⇒ 通常是硬件或固件问题
GPU fallen off the bus（掉卡）	节点上看不到该卡 ⇒ 通常**必须重启/reset**
应用级非法访问/异常	多数是应用层 bug，但也可能是 ECC/固件

```
dmesg -T | grep -i -E "NVRM|Xid"
journalctl -k | grep -i xid
nvidia-smi -q | grep -i -A5 "ECC Errors"
nvidia-smi --query-gpu=index,serial,ecc.errors.uncorrected.aggregate.total --format=csv
nvidia-smi -q -d TEMPERATURE,POWER,UTILIZATION,MEMORY
```

平台侧建议（必答）：

1. 采集 **DCGM 指标 + XID 事件**，配置分级告警；
2. **自动排空**：检测到不可恢复错误 ⇒ cordon + drain + 触发维护流程；
3. **不要在"疑似坏卡"的节点上继续接收新任务**（避免浪费几个小时的训练时间）；
4. 硬件层还要关注：**温度/功耗墙（power throttle）、掉电、风扇、PCIe 降速（带宽从 x16 掉到 x8/x4）**。

A14. 常见"看起来像软件问题，其实是硬件"的现象

现象	可能的硬件/固件原因
训练速度忽然集体变慢	某卡 **thermal/power throttling**、PCIe 链路降级
NCCL hang / timeout	NVLink down、网卡丢包、 infiniband link flapping
反复 OOM 但账算得通	ECC 错误导致显存减少 / row remap
同一批任务里个别 rank 慢	该节点存在 **straggler**（硬盘/网卡/GPU 之一有问题）
偶发 `illegal memory access`	有可能是硬件，也可能是应用层越界 ⇒ **先换节点复现**定位归属

Part B • Kubernetes GPU 调度机制（16 题）

B1. 一个 Pod 想用 GPU，K8s 这条链路上到底发生了什么？

```
① 节点侧：GPU Operator 装载驱动 / nvidia-container-toolkit / CDI 配置
② NVIDIA device plugin 通过 /var/lib/kubelet/device-plugins/kubelet.sock 注册
③ 它周期性 ListAndWatch ⇒ kubelet 更新 Node allocatable：nvidia.com/gpu：8
④ 用户创建 Pod：resources.limits["nvidia.com/gpu"]：1
⑤ kubelet：调度器只是在 NodeResourcesFit 里做"数字减一"（它不懂 GPU 拓扑）
⑥ 节点上的 kubelet 通过 CRI 调 DevicePlugin.Allocate({"nvidia.com/gpu":["GPU-UUID-xxx"]})
⑦ device plugin 返回：要挂哪些设备节点、环境变量（如 NVIDIA_VISIBLE_DEVICES）
⑧ containerd + nvidia-container-toolkit（或 CDI）把 /dev/nvidia* + 驱动库注入容器
⑨ 容器里 nvidia-smi 就能看到这一张卡
```

必答的两个"不理解的事"：

1. **调度器不知道 GPU 的型号、拓扑、NVLink 关系**，只知道"还剩几张卡"；想要拓扑感知必须靠 **标签 + 亲和 + 自定义插件**；
2. **extended resource 只能是整数**，且 **requests 必须等于 limits（或只写 limits）** —— 写 requests=1/limits=2 会报错，也不支持"半张卡"（除非用 MIG 或第三方份额方案）。

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: train
spec:
  containers:
    - name: c
      image: pytorch/pytorch:latest
      command: ["python", "train.py"]
      resources:
        limits:
          nvidia.com/gpu: 1 # 只写 limits 即可（extended resource 惯例）
```

B2. device plugin 的能力边界在哪？（为什么它不够用）

能力	device plugin
上报数量	✅
选择具体设备	❌（kubelet 按顺序取，不过有 `GetPreferredAllocation` 接口可给出建议）
表达"型号/显存/拓扑/带宽"	❌（只能编码成资源名或依赖 NFD 标签）
表达"可共享/带属性"	❌
让**调度器**参与选择	❌（只有 kubelet 侧 Decide）

结论：device plugin 只是"节点侧的设备登记与分配"，**它不是调度系统**；大模型场景需要的"按拓扑、按份额、按型号分配"，它表达不了 ⇒ 催生了 **DRA**。

B3. DRA vs device plugin：新一代设备分配的差异？（必考）

维度	device plugin	DRA（Dynamic Resource Allocation）
资源模型	整数计数（nvidia.com/gpu: 1）	**结构化参数**：ResourceClaim + DeviceClass + ResourceSlice（带属性选择器）
谁选择设备	kubelet 侧	**调度器参与**（structured parameters 路径）
表达"我要 40G 显存且 NVLink 互联的 2 张卡"	做不到	**可以**（用选择器表达）
共享同一设备给多个 Pod	做不到	支持 shareable / 多次分配
网络附加设备	不适合	原生支持（network-attached devices）
成熟度	稳定通用	**仍在演进**（各版本 API 有差异，请以所用版本 Release Notes 为准）

```
# DRA 结构化资源（示意，请以所用版本的 API 为准）
apiVersion: resource.k8s.io/v1
kind: DeviceClass
metadata:
  name: gpu-h100
spec:
  selectors:
    - cel:
        expression: 'device.driver == "gpu.nvidia.com" && device.attributes["gpu.nvidia.com"].model == "H100"'
---
apiVersion: resource.k8s.io/v1
kind: ResourceClaim
metadata:
  name: two-linked-gpus
spec:
  devices:
    requests:
      - name: gpus
        deviceClassName: gpu-h100
        count: 2
---
apiVersion: v1
kind: Pod
spec:
  resourceClaims:
    - name: gpu
      resourceClaimName: two-linked-gpus
  containers:
    - name: c
      image: pytorch/pytorch:latest
      resources:
        claims: [{name: gpu}]
```

> **面试口径**：知道 DRA 在解决"**表达力 + 调度介入**"就够了；不要夸大到"生产已全面替代 device plugin"，当前大模型集群主流仍是 device plugin + 拓扑标签/NFD。

B4. NFD 与 GPU Feature Discovery：拓扑标签从哪来?

- **NFD（Node Feature Discovery）**：发现 CPU/内核/PCI 设备等特征打标签；
- **GPU Feature Discovery (GFD)**：专门发现 GPU 型号、显存、驱动/CUDA 版本、MIG 配置等，写标签（label 前缀请以所用版本为准）；
- 标签通常会被分类 many 个 label: e.g. `nvidia.com/gpu.product=NVIDIA-H100-80GB-HBM3`、`nvidia.com/gpu.count=8`、`nvidia.com/gpu.memory=81920`、`nvidia.com/gpu.driver-version=...`。

用途：让 Pod 能用 `nodeSelector/nodeAffinity` 选机型、避免把 H100 的任务调度到 A100 上。

```
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: nvidia.com/gpu.product
                operator: In
                values: ["NVIDIA-H100-80GB-HBM3"]
```

坑（必答）：GFD 只是"打标签"，**不知道 NVLink 拓扑的详细结构**；真正的"挑一组同 NVLink 域的卡"需要 topology manager / vendor 的拓扑感知插件 / 或者 job-level 自定义调度。

B5. 为什么大模型训练必须要 Gang Scheduling？（死锁题）

经典死锁场景：

```
集群剩 8 张卡分布在 2 台机器上（每台 4 张）
有两个 8 卡任务 A、B 先后启动：
  A 占了 4 张（machine1），B 占了 4 张（machine2）
  A 等剩下 4 张，B 也等剩下 4 张
  ⇒ 双方都持有部分资源互相等待 ⇒ 死锁，且资源被空占（GPU 不释放）
```

解法：All-or-Nothing（Gang Scheduling） —— 只有凑齐全部 minMember 才真正绑定，否则**谁都不给**。

实现方式（放在调度框架的哪个扩展点，是与《Scheduler 篇》的呼应）：

方案	机制
scheduler-plugins Coscheduling	**Permit** 扩展点：持有已通过 Filter/Score 的 Pod 等待配额；`PostFilter` 超时整组放弃；`QueueSort` 按 PodGroup 创建时间排序防饿死
Volcano	PodGroup CRD + 专属调度器 `volcano`，支持 Gang、公平份额、抢占、网络拓扑感知
YuniKorn	面向大数据/批处理的层级队列与公平性
Kueue	更偏**准入/配额队列**（见 B8），常与上面三者搭配

```yaml
# Volcano PodGroup + Job 示意
apiVersion: scheduling.volcano.sh/v1beta1
kind: PodGroup
metadata:
  name: training-job
spec:
  minMember: 16
  minResources:
    nvidia.com/gpu: "16"
  queue: default
---
apiVersion: batch.volcano.sh/v1alpha1
kind: Job
metadata:
  name: pytorch-job
spec:
  schedulerName: volcano
  minAvailable: 16
  tasks:
    - name: worker
      replicas: 8
      template:
        spec:
          containers:
            - name: pytorch
              image: pytorch/pytorch:latest
              resources:
                limits: {nvidia.com/gpu: 2}
```

必答的副作用（Permit 阶段的含义）：

> Pod 在 **Permit 等待期已经 assume 了资源**（《Scheduler 篇》Q2 的乐观绑定），所以"等待"是真的占着资源；因此必须配**超时**与**上限**，否则会变成资源泄漏。

B6. Training Operator / PyTorchJob 到底提供了什么?

组件	作用
Kubeflow Training Operator（CRD：`PyTorchJob`、`TFJob`、`MPIJob`…）	描述"训练作业"这个高层语义，自动创建 master/worker Pod，注入 `RANK/WORLD_SIZE/MASTER_ADDR/MASTER_PORT` 之类的 rendezvous 环境变量
MPI Operator	MPI 语义（如 Horovod 场景）
torchrun / torchelastic	容错与弹性 rendezvous（worker 挂了可重启/缩容）
LeaderWorkerSet（LWS）	**多节点的 leader/worker 组**（常见用于**多机推理**与大模型的 TP 组），保证组整体滚动更新与调度一致性

```yaml
# LWS 示意（API 版本请以所用集群为准）
apiVersion: leaderworkerset.x-k8s.io/v1
kind: LeaderWorkerSet
metadata:
  name: vllm
spec:
  replicas: 4 # 4 个"组"
  leaderWorkerTemplate:
    size: 2 # 每组 1 leader + 1 worker（如 TP=2）
    restartPolicy: RecreateGroupOnPodRestart
    leaderTemplate:
      spec:
        containers:
          - name: vllm
            image: vllm/vllm-openai:latest
            resources:
              limits: {nvidia.com/gpu: 1}
```

必答：LWS 解决的是"**一组 Pod 必须整体创建/整体更新/整体调度**"的问题，是多机推理（TP 跨机）编排的关键组件。

B7. 拓扑感知调度：怎么把通信压在最快的链路上?

调度能做的层次（从粗到细）：

层次	手段
节点层级（最常用）	`topologySpreadConstraints` 让同一作业的 Pod 尽量**集中在少数节点**（减少跨机）；或反向打散提高容灾
机型/域层级	nodeAffinity 限定同一机型、同一 rack/zone 标签
NUMA/PCIe 层级	kubelet `topologyManagerPolicy: single-numa-node`，让 GPU 与 CPU/NIC 在同一 NUMA（避免跨 socket PCIe 访问）
NVLink/交换机层级	依赖厂商/社区的拓扑感知插件或 job-level 自定义分配（部分由 Volcano / 厂商 operator 提供）

```yaml
spec:
  topologySpreadConstraints:
    - maxSkew: 1
      topologyKey: kubernetes.io/hostname
      whenUnsatisfiable: DoNotSchedule
      labelSelector:
        matchLabels: {job-name: train-job-123}
```

实践建议（需实测校准）：

- 训练作业：**倾向聚拢**（maxSkew 小 + 尽量少的主机数）⇒ 更少的跨机通信；
- 推理服务：**倾向打散**（跨可用区/机架）⇒ 更好的可用性。

B8. Kueue：为什么"队列/配额"在大模型集群里是必需品?

GPU 是稀缺且昂贵的共享资源，必须有"谁能用多少、谁先谁后"的规则。Kueue 的核心对象：

对象	作用
ResourceFlavor	描述一类资源（如 h100-80g、a100-40g、spot 卡池），可带 node label/taint 匹配
ClusterQueue	集群级配额与准入（quota、borrowing/lending、抢占策略）
LocalQueue	命名空间级入口，业务只看到它
AdmissionCheck	额外的准入检查（如等待外部 Provisioning 完成）
Cohort	一组 ClusterQueue 之间互相借用配额

```
apiVersion: kueue.x-k8s.io/v1beta1
kind: ResourceFlavor
metadata:
  name: h100-80g
spec:
  nodeLabels:
    nvidia.com/gpu.product: NVIDIA-H100-80GB-HBM3
---
apiVersion: kueue.x-k8s.io/v1beta1
kind: ClusterQueue
metadata:
  name: research-cq
spec:
  namespaceSelector: {}
  resourceGroups:
    - coveredResources: ["cpu", "memory", "nvidia.com/gpu"]
      flavors:
        - name: h100-80g
          resources:
            - name: "nvidia.com/gpu"
              nominalQuota: 64 # 建议按采购容量分配，需结合业务排期校准
---
apiVersion: kueue.x-k8s.io/v1beta1
kind: LocalQueue
metadata:
  name: research-lq
  namespace: research
spec:
  clusterQueue: research-cq
```

```
# 作业侧声明队列（Kueue 会先"准入"再真正调度，避免进入 scheduler 后白白 Pending）
metadata:
  labels:
    kueue.x-k8s.io/queue-name: research-lq
```

必答的价值：

1. **准入层排队** ⇒ 避免成千上万个不可调度的 Pod 堆积在调度队列里拖垮调度器（《Scheduler 篇》L3/L4 的治理）；
2. **配额与借用** ⇒ 多团队共享 GPU 池但保证底线；
3. **队列内优先级/抢占** ⇒ 让紧急任务可以先拿到资源。

B9. GPU 资源超卖 / 共享的正确姿势

从弱到强：

方案	隔离能力	适用
时间片共享（device plugin replicas）	差	开发/轻量推理
HAMi / k8s-vGPU 类（webhook + 运行时拦截显存/算力）	**中**（软限制，超额仍可能 OOM）	多租户推理、教学平台
MIG	**强**（硬件）	有 QoS 要求的推理场景
DRA 的份额模型	演进中	未来方向

```
# HAMi 风格的资源写法示意（请以所用项目文档为准）
resources:
  limits:
    nvidia.com/gpu: 1
    nvidia.com/gpumem: 8192 # 显存 MiB
    nvidia.com/gpucores: 30 # 算力百分比
```

必答风险：任何"软限制"方案都可能因为**上层声明与实际不符**导致同卡互相影响 ⇒ 必须配：显存使用监控 + OOM 关联到具体 Pod + 明确的赔偿/调度回退策略。

B10. GPU 节点的 bin-packing 与碎片

矛盾：

- 训练任务需要**整块**资源（8 卡甚至跨机）⇒ 需要**大块连续供给**；
- 推理/开发任务常常只要 1 卡 ⇒ 容易把 8 卡机切成碎片，导致训练排不上。

治理（高分答法）：

1. **节点池分层**：训练池（整卡、整机型）/ 推理池（可共享、可 MIG）/ 开发池（低优先级、可抢占）；
2. **打分策略分流**：训练池用 `MostAllocated`（尽快填满整机）或相反地留出整块，推理池按需；两种策略的目标不同，**必须分开 profile**；

3. **配额守底线**：用 Kueue/命名空间 ResourceQuota 保证训练不被推理吃光；
4. **MIG 只在专用池开**（MIG 一旦开启，该机就不能给训练用了）；
5. 监控"**分配率 vs 使用率**"双指标 —— 分配率高但使用率低 = 占着不用，这是最大的浪费源。

```
# 分配率（Requests ≠ 实际使用），配合 DCGM 的使用率对比
sum(kube_pod_container_resource_requests{resource=~"nvidia.com/gpu"}) by (node)
# 实际 GPU 利用率（需 dcgm-exporter）
DCGM_FI_PROF_GR_ENGINE_ACTIVE
```

B11. GPU 节点的自动扩缩（CA / Karpenter）

难点：GPU 节点**扩容慢**（实例创建 + 驱动初始化 + GPU Operator 就绪往往需要分钟级），而且**昂贵**。

实践要点：

- GPU 池**单独配置 ASG/MachineDeployment**，并用 taint 隔离，避免无 GPU 的 Pod 落到上面；
- 缩容要考虑**长训练任务**：必须 cordon + 等待任务结束，而不是硬性驱逐 ⇒ 通常需要上层作业系统配合（这也是很多团队选择 "常驻 GPU 池 + 队列" 而非按需伸缩的原因）；
- 推理侧更适合自动扩缩（无状态、可快速重建），但要解决**权重加载慢**问题（见 E 篇）。

B12. GPU 相关的 kubelet/system 层设置（与两篇联动）

项	建议
`topologyManagerPolicy`	若要 NUMA/PCIe 亲和，考虑 `single-numa-node` / `restricted`（可用性成本要压测）
`cpuManagerPolicy: static`	绑核场景才开，且必须配 `reservedSystemCPUs`
`reservedSystemCPUs`	给 GPU 节点的系统进程留 CPU，**尤其重要**（NCCL 通信需要 CPU 参与）
`memoryManagerPolicy`	与 NUMA 亲和配合，视版本支持使用
cgroup v2 / swap	GPU 节点通常**不建议依赖 swap**（Host-Device pinning 与延迟敏感）
Hugepages / hosts	RDMA/DPDK 类需要大页时才有必要配置

> 参考《Kubelet 篇》L10 的 NUMA/绑核陷阱，GPU 节点是这些坑最集中的地方。

B13. GPU Operator 到底管了什么？（部署必答）

一份典型的 GPU Operator 会包含这些组件（具体组件名请以所用版本为准）：

组件	作用
Driver DaemonSet（容器化驱动）	在节点上装载内核模块
Container Toolkit / CDI 配置	让容器能真正使用 GPU
Device Plugin DaemonSet	向 kubelet 上报 GPU
DCGM Exporter	暴露 GPU 指标
GPU Feature Discovery	打 GPU 相关标签
MIG Manager	统一管理 MIG 几何配置
Node Status Exporter / 健康检查相关组件	把 GPU 健康暴露到 Node condition
验证用的 validator Pod	部署后自检能否真跑 GPU 负载

面试提示：把 GPU Operator 理解成"**GPU 节点的标准化交付方式**"，它把驱动/工具链/设备发现/监控一次性收敛，避免手工装驱动导致的版本漂移（漂移是 GPU 集群第一大事故源）。

B14. GPU × 安全隔离：多租户能不能安全共享一张 GPU?

风险	说明
显存数据残留	容器退出后显存内容不一定清零 ⇒ 结合隔离方案与可信域管理
错误传播	time-slicing 下一个进程异常可能影响同卡其它进程
设备节点过大授权	`privileged` 或粗暴开放 `/dev/nvidia*` 会让容器能力过大
多租户是否伦理	建议：**一张物理卡在同一时间只服务同一租户**（或仅服务可信内部租户）；真正的多租户 LLM 服务应在**服务层**（推理框架）做隔离与配额，而不是共享 CUDA 进程

B15. 怎么判断"GPU 真的在被有效使用"？（利用率误区必考）

`nvidia-smi` 的 "GPU-Util 100%" 极具误导性：它只表示**有 kernel 在跑**，不代表 Tensor Core 在高效算有用的东西。更靠谱的是：

指标	含义
`DCGM_FI_PROF_GR_ENGINE_ACTIVE`	Graphics/Compute engine active（比 smi 更细的分帧统计）
`DCGM_FI_PROF_SM_ACTIVE` / `SM_OCCUPANCY`	SM 活跃比例与占用率
`DCGM_FI_PROF_PIPE_TENSOR_ACTIVE`	**Tensor Core 是否活跃**（LLM 训练关注它）
`DCGM_FI_DEV_FB_USED` / `_FB_FREE`	显存使用
`DCGM_FI_DEV_GPU_TEMP` / `POWER_USAGE` / `SM_CLOCK` / `MEM_CLOCK`	是否被限温/限功率/降频
`DCGM_FI_DEV_XID_ERRORS`	XID 错误事件
`DCGM_FI_DEV_ECC_*`	ECC 错误计数

```
# 全集群 GPU 平均算力引擎活跃度（有效利用率的粗口径）
avg(DCGM_FI_PROF_GR_ENGINE_ACTIVE)
# 有分配但几乎不用的"僵尸卡"（占着 GPU 不干活 — 最贵的浪费）
DCGM_FI_PROF_SM_ACTIVE < 5
# 降频/限功率检查（数值需以该型号的标称时钟为基准，并实测校准）
DCGM_FI_DEV_SM_CLOCK < 1300
DCGM_FI_DEV_POWER_USAGE / DCGM_FI_DEV_POWER_MGMT_LIMIT > 0.98
```

结论口径：

> "GPU 利用率要看三层：**① 有没有被分配（调度/配额）② 有没有 kernel 在跑（SM/Tensor active）③ 跑得是否有效（MFU）**。绝大多数集群的浪费发生在第 3 层：SM 很忙，但 MFU 只有 30%。"

B16. MFU 是什么？怎么算？（高阶必考题）

```
MFU = 实际达成的有效算力 / 峰值算力
    =（每步的有效 FLOPs × 每秒步数）/（GPU 数 × 单卡峰值 FLOPS）

其中每步有效 FLOPs（Transformer）常见估算：
    ≈ 6 × 参数量 × tokens （前向+反向的近似：前向 2N，反向 4N）
   若使用 activation checkpointing 则会额外增加一次前向（约 8N）
```

估算示例（所有数字均为"假设值"，用于演示口径；真实值必须用 Profiler 实测）：

```
假设：70B 参数、64 张卡、全局 batch = 262144 tokens/步、单步耗时 8.7s
  有效 FLOPs/步 ≈ 6 × 70e9 × 262144 ≈ 1.1e17 FLOPs
  达成算力 ≈ 1.1e17 / 8.7s ≈ 1.27e16 FLOPS ≈ 12.7 PFLOPS
  峰值算力 ≈ 64 × 989 TFLOPS（BF16 峰值量级，待以 Datasheet 复核）≈ 63.3 PFLOPS
  ⇒ MFU ≈ 12.7 / 63.3 ≈ 20%
```

这个公式最容易踩的三个坑（一定要在面试里主动说明）：

1. **tokens 的口径**：要用"每步的全局 tokens"，并且注意 **TP/PP 会让单卡重复/分担计算**，不能直接乘 GPU 数；
2. **是否含重计算**：开了 activation checkpointing 时系数从 ~6N 变成 ~8N；
3. **单位的幂次**：PFLOPS/TFLOPS 差 1000 倍，是这类题最常见的算术事故。

> 业界公开分享里，良好优化的 LLM 训练 MFU 常在 **30%~50% 区间**（具体务必以自家 workload 的 Profiler 数据为准）。**MFU 低于 30% 就该立项优化**，那是几百万上千万的算力差。

提升 MFU 的常见抓手：

1. 增大 batch（提升 GEMM 效率）⇒ 但要配合 LR schedule；
2. **算子融合 + FlashAttention**；
3. **减少重计算 vs 显存的权衡**；
4. **通信/计算 overlap**（ZeRO/FSDP 的 prefetch、NCCL 调优）；
5. **降低 bubble**（PP microbatch 数足够多）；
6. 输入 shape 尽量规整（避免 padding 浪费与 kernel autotune 抖动）。

Part C · 大模型训练网络（12 题）

> 训练集群里"网络不是配角，而是算力的一部分"。这一篇与《网络 / CNI 篇》交叉，但关注的是 **RDMA / 集合通信 / 拓扑**。

C1. 为什么大模型训练对网络的要求和 Web 服务完全不同?

维度	Web/微服务	大模型训练
流量模型	南北向为主、请求多而小	**东西向、极大规模、同步（集合通信）**
延迟敏感性	单请求 P99	**同步阻塞**：一次 AllReduce 慢 ⇒ 所有 rank 一起等
丢包容忍	靠重传，问题不大	**几乎不允许丢包**（RDMA 对丢包极敏感）
带宽量级	10~100G 常见	单节点 200~400G 起步，多轨聚合更高
关键指标	QPS/延迟	**有效带宽、通信占比、MFU**

必答的"同步放大效应"：

> 分布式训练是 **BSP（批量同步并行）** 式的：一次集合通信没完成，所有 GPU 都在空转。所以"网络慢 20%" 往往直接表现为"整体算力利用率掉一大截"，而不是某个请求慢。

C2. 集合通信量怎么估算？（会算这个非常加分）

Ring AllReduce 的通信量公式：

```
每个 GPU 要发送的数据量 ≈ 2 × (N-1)/N × D  (N = 参与的 GPU 数, D = 每个 GPU 持有的数据字节数)
≈ 2D (当 N 较大时)
```

举例（估算，需实测验证）：

```
70B 模型, bf16, DDP (不分片):
  单卡需同步的梯度 D ≈ 70B × 2 bytes = 140GB
  每步每卡通信量 ≈ 2 × 140GB = 280GB
  若跨机有效带宽 100 Gb/s ≈ 12.5 GB/s ⇒ 仅同步就需 ≈ 22s/步
  ⇒ 这就是为什么"朴素 DDP 跑大模型"根本不现实,
     必须用 ZeRO/FSDP 分片 + 把通信量大的切法压在 NVLink 上
```

结论：**并行策略的选择本质上是在"重排通信量分布"**，把大流量放在高带宽层（NVLink），把小流量放在低带宽层（网络）。

C3. NCCL 是什么？训练作业里最常调的环境变量有哪些?

NCCL 是 NVIDIA 的集合通信库，负责 AllReduce/AllGather/ReduceScatter/AllToAll 的高效实现（自动选择 ring/tree 等算法与路径：NVLink、PCIe、共享内存、IB/RoCE socket）。

排障与调优常用环境变量（请以所用 NCCL 版本文档为准）：

变量	用途
`NCCL_DEBUG=INFO`（或 `TRACE`）	**排障第一件事**，看它选了什么路径/算法
`NCCL_DEBUG_SUBSYS=INIT,ENV,GRAPH,TUNING`	缩小日志范围
`NCCL_SOCKET_IFNAME=eth0` / `NCCL_SOCKET_IFNAME=^docker,lo`	指定/排除用于控制面通信的接口（**选错接口是 hang 的头号原因**）
`NCCL_IB_HCA=mlx5_0,mlx5_1`	指定用哪些 RDMA 网卡
`NCCL_IB_GID_INDEX=3`	RoCEv2 场景常需要指定 GID index
`NCCL_NET_GDR_LEVEL` / `NCCL_P2P_LEVEL`	控制 GPUDirect RDMA / P2P 使用层级
`NCCL_ALGO=Ring / Tree`	强制算法（调参时用）
`NCCL_TOPO_FILE`	提供自定义拓扑描述（特殊机型）
`NCCL_ASYNC_ERROR_HANDLING`	错误处理行为（版本相关）
`NCCL_IB_TIMEOUT` / `NCCL_IB_RETRY_CNT`	RoCE 网络的超时重传参数

典型 hang 场景：多网卡机器（eth0 管理网 + 多个 RDMA 网卡 + CNI 的 veth）时，NCCL 可能选到错误的接口 ⇒ 表现为"卡在 init 或第一步不动"。

```
# 训练容器里第一件事
NCCL_DEBUG=INFO NCCL_DEBUG_SUBSYS=INIT,ENV,GRAPH python train.py 2>&1 | grep -i nccl | head -60
```

C4. 训练网络的两条技术路线：IB vs RoCE

维度	InfiniBand	RoCEv2（无损以太网）
生态	专网、专用交换机与 HCA，子网管理器（SM）	以太网交换机 + RDMA 网卡
无损	原生支持	**依赖 PFC + ECN + DCQCN 的精细调参**
运维	相对"开箱即用"但需专业技能	成本低、可复用以太网团队，但**调参不慎会出现 PFC 风暴/死锁**
规模	大规模成熟	大规模可行，但对交换机配置要求高

必答的 RoCE 风险：

- **PFC 风暴 / 死锁**：优先级流控扩散到整网 ⇒ 必须合理规划 PFC 作用范围（通常只在"服务器到 leaf" 段）、配置无损队列与缓冲区；
- **ECN/DCQCN 参数**需要与交换机侧协同；

- **MTU**：常用 9000（jumbo）降低 PPS，但**必须整条链路一致**；
- 与《网络篇》A11 呼应：**隧道封装会让 MTU 变小，而 RDMA 流量通常走**主机接口/ secondary network**，不要让它进隧道**。

C5. 训练集群的典型拓扑：rail-optimized 是什么?

```
常见 8 卡服务器 + 8 张 RDMA 网卡，每台服务器的第 i 张网卡（对应第 i 张 GPU）
连到同一台 leaf 交换机的同一"轨道"（rail i）：

  rail0: leaf-A  — node1:gpu0/nic0 — node2:gpu0/nic0 — ...
  rail1: leaf-B  — node1:gpu1/nic1 — node2:gpu1/nic1 — ...
  ...
```

好处：同号 GPU（做同样切分的那些 rank）之间的通信大多**只在同一 leaf 内一跳完成**，减少跨 leaf 流量与拥塞。

面试要点：

- 这种拓扑决定了作业的 **rank 到节点的映射策略**（把"通信最密集的 rank 对"放在同一 rail/同一 leaf）；
- 这也是"拓扑感知调度"在训练集群里的真实含义之一；
- 另一类常见拓扑是 **fat-tree/spine-leaf**（通用性好，但同号 GPU 不一定同 leaf）。

C6. K8s 里怎么给 Pod 提供 RDMA 网络？（与 CNI 篇联动）

方案	做法	适用
hostNetwork	训练 Pod 直接用宿主网络	最常见、性能最好、最简单（代价：端口冲突、网络策略失效）
Multus + macvlan/ipvlan + SR-IOV VF	给 Pod 额外一张"直通"的高性能网卡	需要网络隔离/多租户/不能 hostNetwork 的场景
NVIDIA Network Operator	统一管理 MOFED/DOCA 驱动、RDMA device plugin、secondary network CRD（如 MacvlanNetwork / IPoIBNetwork / HostDeviceNetwork）	生产标准化的做法
RDMA device plugin	上报 `rdma/hca_shared_devices` 等资源	配合上述方案

```yaml
# Multus + macvlan 给训练 Pod 加一张高速网卡（示意，请以所用版本为准）
apiVersion: k8s.cni.cncf.io/v1
kind: NetworkAttachmentDefinition
metadata:
  name: rdma-net
  namespace: default
spec:
  config: |
    {
      "cniVersion": "1.0.0",
      "type": "macvlan",
      "master": "ens5f0",
      "mode": "bridge",
      "ipam": { "type": "whereabouts", "range": "192.168.100.0/24", "exclude": ["192.168.100.1/32"] }
    }
---
# 引用
metadata:
  annotations:
    k8s.v1.cni.cncf.io/networks: rdma-net
```

必答的坑：

1. **IPAM 与 IP 规划**：whereabouts 需要真实网段规划，**IP 耗尽会让训练 Pod 起不来**（与《网络篇》L7 同源）；
2. **接口选择**：NCCL 必须选到这张高速网卡 ⇒ `NCCL_SOCKET_IFNAME` / `NCCL_IB_HCA` 要显式配；
3. **不要让它走隧道**：RDMA 流量不应经过 VXLAN/Geneve 封装（MTU 与性能都受损）；
4. **SR-IOV VF 与节点绑定**：VF 是硬件资源，迁移/重建会受限。

C7. GPUDirect RDMA（GDR）的价值与前提

作用：让网卡**直接读写 GPU 显存**，绕过"显存→主机内存→网卡"的多次拷贝 ⇒ 降低延迟、减少 CPU 占用、提升有效带宽。

前提：

- 驱动栈支持（如 nvidia-peermem / OFED 的 peer memory 相关模块，具体组件名以版本为准）；
- GPU 与 NIC 在**同一 PCIe 拓扑的合适位置**（通常要求同一 PCIe switch 或至少同一 NUMA，取决于平台）；
- 这也是为什么 `topologyManagerPolicy: single-numa-node` 在训练节点常被讨论（**是否必要取决于机型拓扑，需实测**）。

排障：`NCCL_DEBUG=INFO` 日志里可以看到是否用了 GDR 路径；若未启用通常提示 `NET/IB : GPU Direct RDMA Disabled` 之类（具体文案以版本为准）。

C8. 训练网络故障的现场定位顺序

```
① NCCL_DEBUG=INFO：看它选了哪些接口、是否用了 IB/GDR、报了什么错
② 确认节点 RDMA 设备存在：ibstat / ibv_devinfo / rdma link
③ 交换机侧：端口 up？速率/ MTU 是否一致？丢包/CRC 计数？PFC 暂停帧统计？
④ 端到端带宽实测：ib_write_bw / ib_send_bw（或 perftest 系列） ⇒ 与理论值对比
⑤ 单节点多卡（不走网络）是否正常 ⇒ 区分"机内问题" vs "网络问题"
⑥ NCCL 自带的测试：nccl-tests（all_reduce_perf）⇒ 直接给出 busbw
⑦ 抓包/计数器：ethtool -S、perfquery（IB）
```

```
# 常用基线测试（示意）
ibstat
ib_write_bw -d mlx5_0 -F --report_gbits -q 4 <server-ip>
all_reduce_perf -b 8M -e 8G -f 2 -g 1 # nccl-tests
```

必答：**nccl-tests 的 `busbw` 是训练网络的"黄金指标"**——上线新集群、改驱动、换交换机后都要跑一遍基线并归档（这属于"变更前后对比"的标准动作）。

C9. 大规模训练网络的"隐形杀手"

现象	根因
训练速度周期性抖动	网络拥塞 + 同步等待；或 checkpoint 写盘抢占带宽
只有部分 rank 慢（straggler）	某节点网卡/链路降级、PCIe 降速、GPU 降频
大规模（>256 卡）时 MFU 明显下降	跨 leaf 流量增多、拥塞、PP bubble
偶发 NCCL timeout	网络抖动/丢包 + 超时参数偏小
RoCE 集群整体变慢	PFC/ECN 参数不当、缓冲区/队列配置问题

治理思路：

- 建立 **基线测试 + 定期复测**（健康巡检的一部分）；
- 训练作业与数据 IO / checkpoint **错峰**或分流到不同网络平面；
- 对 straggler 做**自动识别与隔离**（见 F 篇）；
- 关键作业保留**"最小通信域"**的排布策略（尽量同 leaf/rail）。

C10. 推理侧网络：为什么"K8s Service + gRPC"会负载不均？（超经典）

问题：LLM 推理是长连接 + HTTP/2 多路复用。一个客户端到 Service 的**一条 TCP 连接**上跑很多 stream ⇒ 这条连接被分配到某个 Pod 后，**后续请求都落在同一个 Pod**，其它 Pod 闲着。

解法（按推荐度）：

方案	说明
限速单连接的并发 stream	让服务端发送 `SETTINGS_MAX_CONCURRENT_STREAMS=1` 之类配置 ⇒ 客户端被迫开新连接 ⇒ 重新负载均衡（**实践中最有效**）
客户端/网关侧按请求 LB	用支持 per-request 负载均衡的网关（Envoy/istio 的 `LEAST_REQUEST`），或在客户端做端点选择
Headless Service + 客户端 LB	应用自己解析 Endpoints 并建多条连接
合理的连接生命周期	设置 `maxAge` 让连接周期性重建（辅助手段）
Inference Gateway（Gateway API Inference Extension）	面向 LLM 的网关：感知模型/队列/缓存，做**有状态的端点选择**（见 E 篇）

C11. 推理服务的网络还要考虑什么？

项	说明
长超时	生成式请求动辄数十秒~数分钟，网关/Ingress 的 `proxy_read_timeout` / `timeout` 必须放大
流式响应（SSE）	反向代理必须**关闭响应缓冲**（如 `proxy_buffering off`），否则 token 变得不连续
请求排队	显存决定并发上限 ⇒ 需要在网关层做队列与优先级（而不是无限堆积）
模型预热与权重加载	冷启动几十秒到几分钟 ⇒ 需要预热池与就绪判定（见 E 篇）
跨可用区	推理多副本应跨区打散，但如果做了 Prefill/Decode 分离，跨区会带来 KV 传输延迟

C12. 网络侧的可观测与基线

```
# RDMA/网卡层（依赖 node-exporter 或厂商 exporter，指标名请核对）
rate(node_network_receive_bytes_total{device=~"ib.*|ens.*"}[5m])
rate(node_network_receive_drop_total[5m])
rate(node_network_receive_errs_total[5m])
# RDMA 特定计数（部分 exporter 提供）
rdma_rx_discards / rdma_out_of_sequence 之类（以实际 exporter 为准）
# GPU 侧"等待通信"的间接信号：SM 活跃度下降但任务在跑
DCGM_FI_PROF_SM_ACTIVE
```

```
# 网络基线告警（示例）
rate(node_network_receive_drop_total[5m]) > 0
rate(node_network_receive_errs_total[5m]) > 0
```

必答：**网络基线必须"用 nccl-tests 的 busbw 归档"**，而不是只看网卡利用率——因为"利用率为 0 但训练很慢"往往是 hang/同步等待，而不是带宽用满。

Part D · 大模型场景的存储（10 题）

> 与《存储 / CSI 篇》交叉，但这里聚焦：**训练数据吞吐、checkpoint、模型分发。**

D1. 训练对存储的要求：为什么"传统 PVC"常常不够?

阶段	需求特征
数据读取	海量小文件/打包大文件，**顺序吞吐**要求高；多 worker 并发读同一个数据集 ⇒ **元数据与聚合带宽**双压力
checkpoint 写入	**TB 级、突发、要求可靠**；写不完会拖慢训练
模型分发	权重几十 GB~数百 GB，需要**快速分发到大量节点**
推理加载	冷启动时从共享存储/本地缓存加载权重 ⇒ **影响扩容速度**

核心矛盾：**训练数据吞吐、checkpoint 带宽、模型分发速度**常常决定"GPU 有多少时间在等数据"。

D2. 训练数据的常见组织方式与坑

组织方式	优点	坑
大量小文件（图片/文本分文件）	直观	**元数据操作爆炸**，POSIX 文件系统会被打爆
打包格式（TFRecord / WebDataset / MDS / Parquet / Arrow）	顺序读、元数据友好	需要额外预处理
对象存储直接读	容量无限	**延迟高、不适合逐样本随机读**，需要批量预取

必答建议（需实测）：

- 训练数据**优先打包**；
- 多 epoch 训练时**把数据缓存到节点本地 NVMe**（见 D5 Fluid/缓存）；
- 用 `num_workers`、`prefetch_factor`、pin_memory 等 DataLoader 参数把 IO 与计算 overlap（**否则 GPU 会等数据**）。

D3. checkpoint：规模、频率与"最优间隔"

规模估算（示例公式，需用真实训练验证）：

```
checkpoint 大小 ≈ 参数量 × 每参数字节数
   全参数训练（含 fp32 优化器状态）≈ 16 bytes/param ⇒ 70B 约 1.1TB（量级）
   只存权重（bf16）  ≈ 2 bytes/param ⇒ 70B 约 140GB
```

频率怎么定（高阶题，能给出模型非常加分）：

```
若：M = 集群/任务的平均无故障时间（MTBF），δ = 一次 checkpoint 的耗时（或写放大成本）
Young/Daly 的一阶最优近似：
    最优间隔 L* ≈ sqrt(2 × δ × M)
（真实环境还有重启/恢复/队列等待等成本 ⇒ 这只是"量级参考"，务必结合自身数据校准）
```

实践：

- 写 checkpoint **不要直接堵住训练**：常用"**本地 NVMe 先落盘 → 异步上传到共享存储/对象存储**"；
- 保留策略要兼顾成本（旧 checkpoint 往往比数据盘贵）；
- **必须定期验证 checkpoint 可读**（很多团队踩过"存了 300 个 checkpoint 没有一个能恢复"）。

D4. 并行文件系统 vs 对象存储：怎么选?

维度	并行文件系统（Lustre/GPFS/Weka/...）	对象存储（S3/OSS/...）
语义	POSIX	对象 API
吞吐/延迟	**高吞吐、低延迟**，适合训练	高延迟、适合大块顺序读写
小文件	相对好	**差**
成本	高	低
容量	有限（需扩容）	近乎无限
典型用法	训练共享数据、checkpoint 热层	**数据湖、checkpoint 归档、模型仓库**

推荐组合：**对象存储做数据湖与归档 + 并行文件系统/本地 NVMe 做热数据**，通过缓存系统把热数据自动落到计算节点附近。

D5. Fluid / 数据缓存：为什么它是 K8s 原生的解法?

Fluid 提供 `Dataset` / `Runtime` 抽象，把 Alluxio/JindoFS 等缓存引擎编排到 K8s 里，并做**数据亲和性调度**（把 Pod 调度到已有缓存的节点）。

```yaml
apiVersion: data.fluid.io/v1alpha1
kind: Dataset
metadata:
  name: imagenet
spec:
  mounts:
    - mountPoint: s3://bucket/imagenet
      name: imagenet
  accessModes: ["ReadOnlyMany"]
---
apiVersion: data.fluid.io/v1alpha1
kind: AlluxioRuntime
metadata:
  name: imagenet
spec:
  replicas: 8
  tieredstore:
    levels:
      - mediumtype: SSD
        path: /var/lib/alluxio
        quota: 500Gi
```

价值（必答）：

1. **把"网络/远端存储带宽"换成"本地 NVMe 带宽"**，显著降低数据等待；
2. **缓存在节点上可复用**（多 epoch、多任务共享同一数据集）；
3. 配合调度亲和 ⇒ 减少"缓存未命中导致的慢节点"。

风险：缓存一致性（源数据更新后缓存失效策略）、缓存容量规划、缓存节点故障后的回源风暴。

D6. GPUDirect Storage（GDS）与为什么它重要

- GDS 让存储设备**直接 DMA 到 GPU 显存**，绕过主机内存中转；
- 对"超大 checkpoint / 超大样本加载"有收益；
- 需要驱动、文件系统、硬件链路的支持（**具体支持矩阵以厂商文档为准**），且配置复杂 ⇒ 只有当 IO 成为瓶颈时才值得投入。

D7. K8s 侧的卷使用模式

场景	推荐
只读模型权重、多 Pod 共享	`ReadOnlyMany` 共享文件卷，或每个节点本地缓存
checkpoint 写	共享文件卷（RWX 或专用写入卷）+ 本地 NVMe 缓冲
训练临时空间 / shm	`emptyDir`（必要时 `medium: Memory`，注意计入 memory limit，见《Linux 篇》A19）
数据集缓存	Fluid/本地 NVMe（hostPath/local PV 由平台统一管理）

```yaml
volumes:
  - name: shm
    emptyDir: {medium: Memory, sizeLimit: 8Gi} # 训练 DataLoader 多 worker 常见需求
  - name: model
    persistentVolumeClaim: {claimName: model-cache, readOnly: true}
```

必答坑：

- **大量卷挂载会让 Pod 启动变慢**（每个卷都要 attach/mount，《存储篇》A6/C3）；
- **RWX 文件系统的并发写语义**不适合做 checkpoint 的"多写"——checkpoint 的写入者应唯一。

D8. 模型分发：为什么不要"把权重打进镜像"

方案	问题/优势
权重打进镜像	镜像 10~100GB ⇒ 拉取慢、registry 压力大、版本管理混乱、**强烈不推荐**
共享文件卷 / 对象存储挂载	常见；但要解决并发拉取造成的带宽风暴
P2P/延迟加载（Dragonfly、Nydus/erofs、soci 等）	大规模分发友好
节点本地预缓存 + 预热 DaemonSet	推理弹性扩容的关键（见 E 篇）

```yaml
# 推理服务用 initContainer 预热权重到本地缓存（示意）
initContainers:
  - name: prefetch
    image: my-registry/model-fetcher:latest
    command: ["sh", "-c", "aws s3 sync s3://models/qwen-72b /cache/qwen-72b"]
    volumeMounts:
      - name: cache
        mountPath: /cache
```

D9. 存储侧的容量与性能度量

```
# 卷使用率与 inode（见《存储篇》E3）
kubelet_volume_stats_used_bytes / kubelet_volume_stats_capacity_bytes > 0.85
kubelet_volume_stats_inodes_used / kubelet_volume_stats_inodes > 0.9
# 节点磁盘吞吐打满（数据加载瓶颈的间接证据）
rate(node_disk_written_bytes_total[5m])
rate(node_disk_read_bytes_total[5m])
# IO 等待
rate(node_cpu_seconds_total{mode="iowait"}[5m])
```

训练侧更直接的判断：如果 GPU 利用率周期性掉到很低，且 `iowait` 或磁盘吞吐同时打满 ⇒ **数据/存储是瓶颈**，先做缓存与预取，而不是先调 NCCL。

D10. 存储方案的选型决策（综合）

```
1）数据规模与形态?
   - 海量小文件 → 打包 + 缓存
   - 大块顺序 → 对象存储直读 + 缓存
2）是否有 POSIX 强依赖?
   - 有（多数训练框架默认）→ 并行文件系统 或 缓存引擎提供的 POSIX
   - 无（自研数据管线）→ 对象存储 + 预取更便宜
3）checkpoint 频次与规模?
   - TB 级高频 → 本地 NVMe 写 + 异步上传；共享存储要能扛突发带宽
4）预算与运维能力?
   - 自建 Lustre/GPFS 复杂度高；托管并行文件/云文件存储省心但贵
5）是否需要跨地域/多集群?
   - 需要 → 对象存储做统一数据湖，计算侧各自缓存
```

Part E • 推理平台与弹性（10 题）

E1. 推理服务的形态：为什么它和 Web 服务差别很大？

维度	Web 服务	LLM 推理
单请求耗时	毫秒	**秒~分钟**
资源瓶颈	CPU/网络	**显存（KV cache）与带宽**
扩缩容速度	秒级	**分钟级**（权重加载慢）
弹性触发量	QPS/CPU	**排队长度、KV cache 使用率、TTFT/TPOT**
失败模式	5xx	超时、OOM（显存打满）、降级（拒绝/截断）
状态	无状态	**有 KV cache 状态（前缀缓存亲和）**

E2. 推理服务在 K8s 上的编排方式

方案	适用
Deployment + Service	单卡/单实例、无 TP
LeaderWorkerSet（LWS）	**多机 TP**（一组 Pod 整体调度/更新）
KServe / vLLM Production Stack / AIBrix / Dynamo	更高层的模型服务编排（路由、批处理、缓存、自动扩缩）
KubeRay	Ray 生态（vLLM 在 Ray 上的分布式部署）

```
# vLLM 部署要点示意
spec:
  containers:
    - name: vllm
      image: vllm/vllm-openai:latest
      args: ["--model", "/models/qwen-72b", "--tensor-parallel-size", "2", "--max-model-len", "32768"]
      resources:
        limits: {nvidia.com/gpu: 2}
      readinessProbe:
        httpGet: {path: /health, port: 8000}
        periodSeconds: 5
        failureThreshold: 60 # 权重加载慢 ⇒ 就绪探针必须放宽
      startupProbe:
        httpGet: {path: /health, port: 8000}
        failureThreshold: 120
        periodSeconds: 5
      terminationGracePeriodSeconds: 120 # 生成中的请求要能优雅结束
```

必答：推理服务的**探针必须考虑加载时间**（否则会陷入"启动即被杀"的循环，《Kubelet 篇》Q9/Q10），终止也要给足时间让在途生成结束。

E3. 推理的自动扩缩怎么做？

信号	说明
QPS / 并发请求数	基础
排队长度（queue depth）	最贴近用户体感
KV cache 使用率	决定还能不能再接请求（vLLM 类引擎会暴露）
TTFT / TPOT 的 P99	直接的 SLO 信号

```
# KEDA 基于自定义指标扩缩（示意）
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: vllm
spec:
  scaleTargetRef: {name: vllm}
  triggers:
    - type: prometheus
      metadata:
        serverAddress: http://prometheus:9090
        metricName: vllm_num_requests_waiting # 指标名以所用引擎为准
        threshold: "3"
  advanced:
    restoreToOriginalReplicaCount: false
```

冷启动对策（必答）：

1. **镜像与权重预热**：节点上预缓存权重（DaemonSet 预热 + 本地缓存卷）；
2. **常驻最小副本**（避免冷启动打到首个用户）；
3. **使用延迟加载/本地 NVMe 缓存**降低加载时间；
4. 扩缩策略**"快扩慢缩"**（缩容要保守，避免反复抖动）。

E4. 推理网关：为什么需要"模型感知"的路由？

普通的 L4/轮询路由对 LLM 很低效，因为：

- 请求差异巨大（prompt 长度、输出长度不同 ⇒ 成本不同）；
- **前缀缓存（prefix cache）命中率**取决于路由到哪个实例；
- 需要**队列与优先级**（显存决定并发上限，不能无限堆请求）；
- 需要按 LoRA/适配器、按模型版本路由。

社区方向：**Gateway API Inference Extension**（K8s 生态的推理网关扩展）提供"模型感知的端点选择 + 请求排队 + 关键性/优先级"，是近年面试的加分话题（**项目状态请以官方仓库为准**）。

E5. 高可用的多副本与打散

```
topologySpreadConstraints:
  - maxSkew: 1
    topologyKey: topology.kubernetes.io/zone
    whenUnsatisfiable: ScheduleAnyway
    labelSelector: {matchLabels: {app: vllm}}
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: vllm
spec:
  minAvailable: 2
  selector: {matchLabels: {app: vllm}}
```

必答：大模型推理的"**副本漂移很贵**"（加载慢），所以 **PDB 与优雅终止**尤其重要，节点维护/缩减时不能把多个副本同时干掉。

E6. 单卡放不下怎么办？（推理并行）

手段	说明
TP（张量并行）	一张卡放不下权重时切分；**强烈建议单机内（NVLink）**
PP（流水并行）	跨机推理可用（延迟增加）
量化（FP8/AWQ/GPTQ）	降显存与带宽 ⇒ 提高吞吐
Speculative decoding	降延迟（用算力换延迟）
Prefill/Decode 分离	两种阶段独立扩缩（趋势）

E7. 推理服务的 SLO 怎么定？（可落地）

SLI	建议目标（起点，需实测）
TTFT（首 token 延迟）P99	如 ≤ 2s（交互式）、≤ 10s（长 prompt 批处理）
TPOT / ITL（token 间隔）P99	如 ≤ 100ms/token（保证"看起来流畅"）
请求成功率	≥ 99.9%
拒绝率（显存/队列满）	≤ 0.1%
端到端完成时延 P99	按 prompt/输出长度分段定义（**不要用一个数概括**）

> **注意**：LLM 的延迟与输出长度强相关，SLO 必须**按"输入长度/输出长度分桶"**定义，否则平均值毫无意义。

E8. 批处理/离线推理怎么设计？

- 用 **Job + Kueue 排队**，用 GPU 池的"低优先级/可抢占"资源；
- 吞吐优先：大 batch、continuous batching、无严格延迟 SLO；
- 数据同样要缓存/预取，避免 IO 成为瓶颈。

E9. 推理与训练混部的风险

风险	缓解
训练的长任务占满 GPU 池	节点池分层 + Kueue 配额
推理延迟被训练的通信/IO 影响	网络平面/存储平面分流；NUMA 与网卡亲和
抢占导致推理中断	推理副本设更高优先级 + PDB；训练侧支持 checkpoint 恢复
显存/温度/功耗耦合	监控 + 节点池隔离

E10. GPU 池的"利用率治理"闭环（平台视角）

```
① 采集：DCGM 指标（SM/Tensor active、显存、温度、功耗、XID）+ 作业级标签
② 归因：按 team/model/job 聚合（标签必须落到 Pod 上）
③ 发现浪费：分配但未使用（僵尸卡）、MFU 低、排队久
④ 治理手段：队列配额、自动缩容空闲作业、抢占低优先级、开发池限时限量
⑤ 汇报：利用率看板 + 周报（showback），与预算挂钩
```

```
# 僵尸卡：分配出去但长时间几乎不干活（阈值需实测校准）
DCGM_FI_PROF_SM_ACTIVE < 5
# 按团队聚合的 GPU 小时（需要把 team 标签关联到 GPU 指标）
sum by (team) (DCGM_FI_PROF_GR_ENGINE_ACTIVE)
```

Part F • 可靠性、容错与成本（10 题）

F1. 训练任务的故障来源分布（先有概率直觉）

来源	典型
硬件	GPU XID/ECC/掉卡、NVLink down、网卡/光模块故障、内存 ECC、电源/散热
软件/框架	OOM、死锁、NCCL 初始化失败、数据样本异常（NaN/超长）、依赖版本冲突
平台	节点被驱逐、调度失败、镜像拉不动、存储/网络故障、证书过期
人为	参数写错、配额误配、误删卷/作业

关键认知：**训练时长越长，遭遇硬件故障的概率越高**（MTBF × 卡数 ⇒ 千卡训练几乎每天都会遇到点什么）。所以**容错不是可选项。**

F2. 训练容错的三层机制

```
① 自动重启（框架层）：torchelastic / Job 的 restartPolicy
      ⇒ 代价：从上一个 checkpoint 重跑（丢失间隔内的进度）
② Checkpoint + 恢复（框架/平台层）
      ⇒ 决定 RPO（丢多少进度）
③ 节点自愈（平台层）
      ⇒ 发现坏卡/坏节点 → cordon/drain → 维护流程 → 回池
```

必答的量化（见 D3 的最优间隔公式）：checkpoint 太频繁 ⇒ 写放大拖慢训练；太稀疏 ⇒ 故障损失大。**这是一道可以用公式回答的工程题。**

F3. 坏卡/坏节点怎么自动处理?

```
# 思路（具体工具请以集群所用方案为准，例如 node-maintenance / medik8s 类项目）
1. DCGM/事件检测到 XID 或 ECC 不可恢复错误
2. 触发告警 + 自动 cordon（停止接收新任务）
3. 若节点上无关键长任务 → drain（配合 PDB 与作业层容忍）
4. 触发 GPU reset / 重启驱动 / 重启节点
5. 自检通过（跑一遍 GPU 健康检查 + nccl-tests）→ uncordon 回池
```

必答：**不要自动 drain 正在跑长训练的节点**（会毁掉几天的算力）；正确做法是 **cordon + 等待作业结束或作业层决定。**

F4. 慢节点（straggler）怎么发现?

信号	来源
某 rank 的 step time 明显高于中位数	训练框架侧日志/指标
该节点的 GPU 时钟/温度/功耗异常	DCGM
该节点网卡丢包/降速	网络计数器
PSI / iowait 偏高	node exporter

处置：先 cordon 观察；若持续异常 ⇒ 从训练池摘除并走维修流程。**识别 straggler 是千卡训练的必修课**（它把一个整体的 MFU 拖下来）。

F5. 训练 hang 了怎么排查?

```
① 看是"卡在哪一步"：NCCL init? 第一步? 第 N 步（通常是数据/checkpoint）?
② NCCL_DEBUG=INFO + SUBSYS=INIT,ENV,GRAPH ⇒ 是否选错网卡、是否 GDR 未启用
③ py-spy dump / py-spy top -p <pid> ⇒ 看进程栈卡在哪（IO 还是通信）
④ nvidia-smi ⇒ 是否某卡显存暴涨/OOM、是否有进程残留
⑤ 网络：ibstat/perftest/nccl-tests 基线
⑥ 存储：df/iostat，checkpoint 目录是否写满/写慢
⑦ 若为同步等待：对比各 rank 的 step time（找 straggler）
```

```
py-spy dump --pid <pid>
py-spy top --pid <pid>
```

F6. 多租户 GPU 集群的配额与公平（与 B8 联动）

机制	作用
Kueue ClusterQueue / Cohort	配额、借用/归还、优先级与抢占
ResourceFlavor + taint	把不同机型/价格/可靠性的卡池区分开
Namespace ResourceQuota	`nvidia.com/gpu` 的硬上限
作业优先级	高优研究任务插队；低优开发任务可抢占

必答：**"抢占训练任务"代价极高**（丢进度）⇒ 通常只抢占**短任务/低优推理/开发任务**，对长训练用"排队 + 预约"而不是硬抢。

F7. Spot / 竞价实例能不能用来训练?

场景	结论
短任务（\<1h）、可重试	✅ 合适
长训练	⚠️ 可用但**必须有 checkpoint + 自动恢复**，且要接受中断重排
推理（在线）	❌ 一般不合适（除非多副本 + 快速重建 + 权重预热）

配套：节点池区分 spot/on-demand（用 taint 隔离），作业声明容忍哪种池（Kueue ResourceFlavor + nodeSelector）。

F8. GPU 成本怎么算与优化？（老板会问的题）

```
成本 = Σ(GPU 单价 × GPU 小时) + 网络/存储/机房
优化杠杆（按优先级）：
  1）利用率（分配率 × 使用率 × MFU）— 最大的一块
  2）机型选择（同样 MFU 下更便宜的卡 / 更合适的互联）
  3）采购与承诺（预留/承诺折扣） vs 按需 vs Spot
  4）并行策略与精度（FP8/量化显著提升吞吐）
  5）数据/存储开销（缓存、打包、checkpoint 保留策略）
  6）组织手段（配额、限时限量的开发池、showback/chargeback）
```

必答：**最大的浪费通常不是"买贵了"，而是"分了不用"和"用了但 MFU 低"** ⇒ 监控与归因机制比砍价更重要。

F9. 变更治理：GPU 集群哪些变更最危险？

变更	风险	做法
驱动/CUDA 版本	全集群不兼容、训练直接失败	节点池灰度（1 台 → 10% → 全量）+ 跑 nccl-tests/训练冒烟
GPU Operator / device plugin	设备不上报 ⇒ 全部新任务 Pending	先灰度一个池，保留可回滚版本
MIG 策略切换	需排空节点、资源形态改变	专用池 + 窗口期 + 通知 tenant
网络固件/交换机配置	训练性能断崖或 hang	变更前后跑 nccl-tests 基线对比
内核/cgroup	驱动装载失败	按《Linux 篇》C14 的节点池灰度

通用规则：**每次变更后跑"基线测试 + 冒烟训练（小步数）"，对比 busbw 与 step time**，通过才放量。

F10. 一份"GPU 节点上线 Checklist"

```
[ ] 驱动/CUDA/toolkit 版本与预期的矩阵一致
[ ] nvidia-smi 可见全部 8 张卡，ECC 开启，无 XID 历史
[ ] NFD/GFD 标签正确（型号、显存、驱动版本）
[ ] 拓扑符合预期（GPU↔NIC 的 NUMA/PCIe 关系）
[ ] RDMA 网卡 up、速率与 MTU 正确、nccl-tests busbw 达基线
[ ] device plugin 上报数量正确；MIG 配置符合池定位
[ ] DCGM exporter 正常，指标可查
[ ] kubelet 参数（reservedSystemCPUs / topology manager）已按池配置
[ ] 存储/缓存（本地 NVMe、共享卷）可读写并有容量告警
[ ] 加入正确的节点池（taint/label），并通过 Kueue/调度器的准入
[ ] 冒烟：跑一个小训练 + 一个小推理，确认端到端 OK
```

Part G · 反模式清单（可直接用于生产巡检）

#	反模式	证据	等级	修复	验证
G1	权重打进镜像（10GB+）	镜像列表体积异常、发布极慢	🔴 高	权重走共享存储/对象存储 + 预热	发布时间显著下降（实测）
G2	训练任务没有 Gang/Coscheduling	部分 Pod 起来互相等待、GPU 空占	🔴 高	Volcano/Kueue/coscheduling	死锁场景不再出现
G3	推理探针阈值沿用 Web 服务默认值	启动即被杀、滚动更新失败	🔴 高	startupProbe + 放宽 failureThreshold	冷启动场景能正常 Ready
G4	推理用 Service + gRPC 长连接无 per-request LB	副本负载严重不均	🔴 高	限制 MAX_CONCURRENT_STREAMS / 用推理网关	各副本请求量均衡（实测）
G5	NCCL 未指定网卡/接口	hang 在 init、跨机走错网卡	🔴 高	显式配置 `NCCL_*` 并固化到作业模板	nccl-tests 稳定通过
G6	无 checkpoint 就跑长训练	一次故障损失数天	🔴 高	checkpoint 策略 + 定期恢复演练	恢复演练成功且 RTO 达标
G7	把 MIG 开在训练池	训练拿不到完整卡、性能受损	🔴 高	池分层：训练整卡、推理可共享	训练作业调度成功率提升
G8	只看 `nvidia-smi` 的 GPU-Util	误判"很忙"实际 MFU 很低	🟠 中	DCGM SM/Tensor 指标 + MFU 计算	优化立项有数据支撑
G9	无 GPU 健康（XID/ECC）监控与自动排空	坏卡继续接任务，浪费算力	🔴 高	DCGM 告警 + 自动 cordon + 维护流程	注入 XID 演练能自动隔离
G10	训练与推理混部不分池	推理被训练挤兑、抢占损失大	🟠 中	节点池分层 + 配额	推理 SLO 稳定
G11	数据小文件直读共享存储	iowait 高、GPU 等数据	🟠 中	打包 + 本地缓存（Fluid）	GPU 空等时间下降
G12	checkpoint 直接写远端且不节流	写带宽打满，训练被拖慢	🟠 中	本地 NVMe 缓冲 + 异步上传	step time 抖动消失
G13	队列/配额缺失（谁先提交谁占满）	研究员抢卡、互相抱怨	🟠 中	Kueue/Volcano 队列与配额	配额守底线，借用可控
G14	无基线（nccl-tests / 训练冒烟）就做网络变更	变更后性能断崖才发现	🔴 高	变更前后的 busbw/step time 对比	基线归档可比对
G15	反向代理开启响应缓冲 + 短超时	SSE 流式输出被缓冲、请求被截断	🟠 中	关闭 buffering + 放大超时	流式输出连续

Part H • 可观测性：GPU 与大模型指标与告警

H1. 指标分层

层	指标来源	例子
GPU 设备	**dcgm-exporter**	`DCGM_FI_PROF_GR_ENGINE_ACTIVE`、`_SM_ACTIVE`、`_PIPE_TENSOR_ACTIVE`、`DCGM_FI_DEV_FB_USED/_FB_FREE`、`DCGM_FI_DEV_GPU_TEMP`、`DCGM_FI_DEV_POWER_USAGE`、`DCGM_FI_DEV_SM_CLOCK`、`DCGM_FI_DEV_XID_ERRORS`、`DCGM_FI_DEV_ECC_*`
分配/调度	kube-state-metrics + kubelet	`kube_pod_container_resource_requests{resource="nvidia.com/gpu"}`、`kube_pod_container_resource_limits`、`scheduler_pending_pods`、`kubeproxy_*`
网络	node exporter / 厂商 exporter / nccl-tests	网卡吞吐、丢包、错包、RDMA 计数、交换机计数
存储	`kubelet_volume_stats_*`、node exporter	容量、inode、磁盘吞吐、iowait
推理引擎	vLLM/TGI 自带 metrics	排队长度、运行中请求数、KV cache 使用率、TTFT/TPOT、吞吐(tokens/s)
训练框架	框架导出或日志	step time、loss、grad norm、samples/s、tokens/s
作业/队列	Kueue / Volcano	排队时长、准入数、抢占次数、配额使用率

> **原则**：GPU 指标必须能**关联到 team/model/job**（通过 Pod 标签 join），否则无法归因与治理。

H2. 告警规则（可直接粘贴，阈值需实测校准）

```
groups:
  - name: gpu-platform
    rules:
      - alert: GpuXidError
        expr: increase(DCGM_FI_DEV_XID_ERRORS[10m]) > 0
        for: 2m
        labels: {severity: critical}
        annotations:
          summary: "节点 {{ $labels.instance }} GPU {{ $labels.gpu }} 出现 XID 错误，建议排空检查"

      - alert: GpuEccError
        expr: increase(DCGM_FI_DEV_ECC_DBE_VOL_TOTAL[1h]) > 0
        for: 5m
        labels: {severity: critical}
        annotations:
          summary: "出现不可纠正 ECC 错误，需 GPU reset / 换卡"

      - alert: GpuMemoryPressure
        expr: DCGM_FI_DEV_FB_USED / (DCGM_FI_DEV_FB_USED + DCGM_FI_DEV_FB_FREE) > 0.95
        for: 15m
        labels: {severity: warning}

      - alert: GpuIdleButAllocated
        expr: DCGM_FI_PROF_SM_ACTIVE < 5
        for: 6h
        labels: {severity: info}
        annotations:
          summary: "GPU 长时间空闲但被占用，检查是否有僵尸任务"

      - alert: GpuThermalOrPowerThrottle
        expr: DCGM_FI_DEV_POWER_USAGE / DCGM_FI_DEV_POWER_MGMT_LIMIT > 0.98
        for: 30m
        labels: {severity: warning}
        annotations:
          summary: "GPU 触及功率上限，可能因温度/功耗导致降频"

      - alert: GpuClockThrottle
        expr: DCGM_FI_DEV_SM_CLOCK < 1000
        for: 30m
        labels: {severity: warning}
        annotations:
          summary: "GPU 时钟异常偏低，检查散热/供电/驱动"

      - alert: TrainingJobStalled
        expr: |
          # 需从框架侧导出 step time 或用自定义指标；示意：
          histogram_quantile(0.99, rate(train_step_duration_seconds_bucket[10m])) > 300
        for: 10m
        labels: {severity: warning}
        annotations:
          summary: "训练 step 耗时异常，可能是 straggler / 网络 / IO 问题"

      - alert: GpuQuotaQueueBacklog
        expr: kueue_pending_workloads > 50 # Kueue 指标名请以所用版本核对
        for: 30m
        labels: {severity: info}
        annotations:
          summary: "GPU 队列积压较多，评估是否需要扩容或调整配额"

      - alert: InferenceQueueBacklog
        expr: vllm_num_requests_waiting > 10 # 指标名以所用引擎为准
        for: 10m
        labels: {severity: warning}
        annotations:
          summary: "推理排队积压，需扩容或降级"

      - alert: InferenceKVCacheHigh
        expr: vllm_gpu_cache_usage_perc > 0.9 # 指标名以所用引擎为准
        for: 15m
        labels: {severity: warning}
        annotations:
          summary: "KV cache 使用率过高，可能出现抢占/拒绝请求"
```

> 同上篇规范：**指标名必须先在目标环境 `/metrics` 核对**（不同版本 dcgm-exporter 的字段命名有差异）。

H3. 建议的 SLO（初始建议值，需按业务与实测校准）

SLI	建议目标
GPU 有效利用率（分配且 SM active）	≥ 70%（平台目标，需归因口径）
训练作业 MFU	≥ 30%（低于即立项优化）
作业排队时长 P90	≤ 数小时（按队列承诺）
训练任务非计划中断率	≤ 1 次 / 每 100 GPU·天（按自身基线定）
推理 TTFT P99 / TPOT P99	按分桶定义（见 E7）
推理请求成功率	≥ 99.9%
坏卡自动隔离时长	≤ 15min（从告警到 cordon）

Part I · 演练（GameDay）计划表

场景	注入方式	期望表现	验证的 SLO
坏卡（XID/ECC）	注入 XID 事件或人为标记	告警 → 自动 cordon → 维护流程；不再接新任务	坏卡隔离时长
GPU 掉卡	模拟 `fallen off bus`	节点被隔离，作业按策略失败/重排	容错与 RTO
训练 hang	让 NCCL 走错接口	能快速定位（NCCL_DEBUG + py-spy），超时后作业重启	MTTR
straggler	人为降频/限速某节点	能被识别并隔离	训练稳定性
网络降级	限速某条链路/降 MTU	busbw 基线下降可被发现；训练 step time 上升可归因	网络基线有效性
checkpoint 失败	让存储不可写	训练不静默失败；有告警；可恢复	容错可靠性
checkpoint 恢复演练	从 checkpoint 重启训练	数据一致、loss 曲线连续、耗时达标	RPO/RTO
推理冷启动	从 0 扩容	权重预热有效，TTFT 达标	弹性与 TTFT
推理负载不均	单连接压测	网关/连接策略生效，副本均衡	负载均衡有效性
推理显存打满	发送超长请求	引擎按策略拒绝/抢占，不整体崩溃	稳定性
配额挤兑	多团队同时提交	Kueue 队列按优先级与配额准入，无饿死	公平性
驱动升级	灰度一个池	冒烟与 nccl 基线通过才放量	变更安全性

Part J • 面试速记卡（15 条）+ 一道综合设计题

J1. 15 张速记卡

1. **容器用 GPU 的链路**：驱动容器 → toolkit/CDI → device plugin 上报 → limits `nvidia.com/gpu: N` → kubelet Allocate → 注入 `/dev/nvidia*` 与库。
2. **扩展资源的铁律**：只能是整数、requests 必须等于 limits；调度器**只做数字减法**，不懂型号/拓扑。
3. **device plugin 的短板**：不会选设备、不能表达属性、调度器不参与 ⇒ 这就是 **DRA** 的动机。
4. **DRA 三件套**：DeviceClass（模板）+ ResourceClaim（申请）+ ResourceSlice（驱动上报的能力与属性），**调度器参与选择。**
5. **Gang Scheduling 是刚需**：否则出现"各占一半互相等待"的死锁；实现靠 **Permit 扩展点**（占位等待 = 真占资源，必须配超时）。
6. **并行策略第一性原理**：通信最贵的切法放最快链路 —— **TP 在机内 NVLink，PP/DP 跨机。**
7. **显存公式**：训练（混合精度 + Adam）≈ **16 bytes/参数**；推理权重 bf16 ≈ 2 bytes/参数；KV cache 按 `2 × kv_heads × head_dim × layers × seq × dtype` 估。
8. **MFU**：`6N × tokens / step_time / (卡数 × 峰值)`；**低于 30% 就该立项**；口径（tokens、重计算、单位）是这题最容易翻车的地方。
9. **GPU 利用率三层**：有没有被分配 → 有没有 kernel 在跑（SM/Tensor active）→ 跑得是否有效（MFU）。只看 `nvidia-smi` 会误判。
10. **XID/ECC 是坏卡信号**：必须监控 + 自动 cordon；**不要自动 drain 正在跑长训练的节点。**
11. **NCCL 排障三板斧**：`NCCL_DEBUG=INFO`、指定 `NCCL_SOCKET_IFNAME` / `NCCL_IB_HCA`、`nccl-tests` 的 busbw 基线。
12. **RDMA 网络在 K8s 里**：hostNetwork（简单常用）或 Multus+SR-IOV/macvlan（隔离）；**别让 RDMA 流量进隧道。**
13. **checkpoint 最优间隔**：`sqrt(2 × δ × M)` 量级（δ=写一次成本，M=MTBF）；长训练必须先落本地 NVMe 再异步上传。
14. **推理三件套**：探针要放宽（加载慢）+ 优雅终止要加长 + **Service+gRPC 要处理连接级负载不均**（限制 MAX_CONCURRENT_STREAMS 或推理网关）。
15. **成本杠杆顺序**：利用率（分配率×使用率×MFU） > 机型/采购方式 > 精度与批处理优化 > 存储与数据开销。

J2. 综合设计题："给你一个 512 卡（64 台 8 卡机）集群，支撑训练 + 推理，怎么设计？"

按这个顺序答（面试官会认为你真做过）：

① 先问清需求（SLO 与约束）

- 训练：模型规模（7B/70B/…）、并行策略、期望 MFU、作业并发数、单作业最长时长、RPO（能接受丢多少进度）；
- 推理：模型、QPS 与并发、TTFT/TPOT 目标、是否需要多机 TP、是否流式；
- 组织：团队数、配额规则、优先级、是否有开发池；
- 预算：机型、云/自建、预留 vs 按需 vs Spot。

② 硬件与拓扑

- 机型统一（避免异构导致调度与性能不一致）；
- 机内 8 卡 NVLink 域 ⇒ **TP ≤ 8（机内）**；
- 跨机：**IB 或 RoCEv2** + rail-optimized 拓扑；每节点 ≥1~2 张高速网卡（按卡数配比）；
- 存储：并行文件系统/共享存储 + 本地 NVMe（缓存与 checkpoint 缓冲）+ 对象存储（数据湖与归档）。

③ 平台层（K8s）

- GPU Operator 统一交付驱动/工具链/device plugin/DCGM/GFD；Network Operator 管 RDMA 与多网卡；
- 节点池分层：**训练池（整卡、禁 MIG）/ 推理池（可共享或 MIG）/ 开发池（低优、可抢占、限时限量）**；
- 调度：**Kueue（队列/配额/准入）+ Volcano 或 coscheduling（Gang）**；
- 拓扑：`topologySpreadConstraints`（训练聚拢、推理打散）+ nodeAffinity 限定机型 + 必要时 single-numa-node。

④ 作业层

- 训练：Training Operator / MPIJob + torchelastic + checkpoint 策略（本地 NVMe → 异步上传 → 保留策略）+ 恢复演练；
- 推理：LWS（多机 TP）+ vLLM 类引擎 + 推理网关（队列与缓存感知）+ KEDA/Prometheus 指标扩缩 + 权重预热。

⑤ 可观测与治理

- DCGM 指标 + XID/ECC 告警 + 自动 cordon；
- 训练：step time、loss、MFU、straggler 检测；
- 推理：TTFT/TPOT 分桶、排队长度、KV cache 使用率；
- 网络：nccl-tests busbw 基线归档；存储：容量/inode/吞吐；
- 归因：GPU 指标必须能 join 到 team/model/job。

⑥ 变更与演练

- 驱动/网络/调度组件变更 → 节点池灰度 + 基线对比；
- GameDay：坏卡、训练 hang、straggler、checkpoint 恢复、推理冷启动、配额挤兑。

⑦ 成本闭环

- 利用率看板 + showback/chargeback；
- 开发池限时限量；僵尸卡自动回收；Spot 池只给可中断任务。

> **收尾加分句**："这个设计里最容易失败的地方不是硬件，而是**治理**——没有队列与配额，512 卡会在两周内变成'先到先得 + 大量占着不用'。"

第十篇 • Istio 服务网格

> Envoy · xDS · 流量治理 · 安全 · 可观测性

0. 使用说明

0.1 Istio 面试的考察层次

层级	考点	表现
L1 会用	会写 VirtualService / DestinationRule / Gateway	只会抄 YAML
L2 懂原理	说得清 istiod、xDS、Envoy 拦截、mTLS 身份、证书下发	能画架构与流量路径
L3 懂落地	会做金丝雀、mTLS 迁移、多集群、限流熔断、可观测	有迁移步骤与灰度方案
L4 有规模与事故经验	能讲"控制面推送风暴""指标基数爆炸""注入后启动变慢/探针失败""503 排查"	有指标、有裁剪手段、有回滚

> **答题模板**：先说**流量怎么走**（谁拦截 → 谁决策 → 谁转发），再说**控制面怎么下发**（xDS 与身份），最后给**证据**（CR 字段 / Envoy 配置 / 指标名 / 诊断命令）。

0.2 术语速查

缩写/名词	含义
istiod	合并后的控制面：Pilot（配置转换与 xDS 推送）+ CA（证书签发）
Envoy / istio-proxy	数据面代理（sidecar 或 gateway 形态）
pilot-agent	sidecar 容器里管理 Envoy 生命周期与 bootstrap 的小进程
xDS	Envoy 动态配置协议：LDS/RDS/CDS/EDS/SDS + ADS 聚合
HBONE	Ambient 模式下基于 HTTP/2 的隧道协议（默认端口 15008 相关）
ztunnel / waypoint	Ambient 模式的节点级 L4 代理 / 按需 L7 代理
PeerAuthentication	mTLS 模式（STRICT / PERMISSIVE / DISABLE）
AuthorizationPolicy	L7 授权（ALLOW / DENY / CUSTOM）
SPIFFE	工作负载身份格式：`spiffe://<trust-domain>/ns/<ns>/sa/<sa>`
ServiceEntry / WorkloadEntry	把网格外服务或非 K8s 负载纳入网格
Telemetry API	声明式配置指标/日志/追踪（取代大量 EnvoyFilter 用法）

0.3 一张"包路径"总览（先建立画面）

```
① 入站：Client → LB → Ingress Gateway(Envoy) → [HBONE/mTLS] → Sidecar(Envoy inbound) → App
② 出站：App → Sidecar 拦截(iptables/TPROXY) → 策略/路由 → [mTLS] → 对端 Sidecar → 对端 App
③ 控制面：istiod 监听 K8s/CRD → 生成 Envoy 配置 → 通过 xDS(ADS, 15012) 推送给所有 proxy
④ 身份：istiod CA 签发证书 → 通过 SDS 下发 → 双向 mTLS 建立，身份 = SPIFFE ID
```

Part A • 原理篇（26 题）

A1. 服务网格到底解决什么问题？Istio 不做哪些事？

解决（把"服务治理"从业务代码里下沉到基础设施）：

能力	说明
流量治理	路由、分流、灰度、镜像、故障注入、重试/超时、熔断
安全	工作负载身份（SPIFFE）、**mTLS 透明加密**、L7 授权（JWT/属性）
可观测	统一的 RED 指标、访问日志、链路追踪（含 mTLS/响应标志等上下文）
出口/边界治理	Egress 统一出口、审计与策略

不做（高频送分点）：

- 不负责 **Pod 之间的连通性**（那是 CNI 的事，见《网络篇》）；
- 不负责 **服务发现本身**（它消费 K8s Service/EndpointSlice，或用 ServiceEntry 扩展）；
- 不实现 **NetworkPolicy**（L3/4 网络策略仍由 CNI 执行，两者互补）；
- 不是 **API 网关的全部**（Gateway API / Ingress 仍负责南北向，网格管东西向 + 部分南北向）；
- 也不替代 **业务层的幂等/事务**（重试只能治理瞬时故障）。

一句话定位：**Istio = 以身份为中心的东西向流量与策略平面。**

A2. Istio 架构怎么演进到现在这样？

阶段	形态
早期 1.0~1.4	**微服务化控制面**：Pilot（流量）、Mixer（策略与遥测、**性能瓶颈**）、Citadel（CA）、Galley（配置校验）多进程
1.5 起	**合并为单体 istiod**：Mixer 逻辑外置（遥测交给 Envoy 原生统计 + Telemetry API），控制面大幅简化
1.6~1.19	持续强化：VM 支持、多集群、Telemetry API、Gateway API 支持等
近版本	引入 **Ambient Mesh（sidecarless）**：ztunnel + waypoint；同时 **Kubernetes Gateway API** 成为推荐的入口配置方式（**具体 GA 状态以版本为准**）

必答的"为什么"：Mixer 被移除是因为它是**每次调用都要打点上报的中心化组件**，造成延迟与可用性耦合——现在遥测主要由 **Envoy 侧统计 + Telemetry API 配置**完成。

A3. Envoy 数据面：关键对象与进程模型

Envoy 概念	作用
Listener	监听端口；**filter chain** 决定怎么处理（HTTP 连接管理器、TCP 代理等）
Route（RDS）	域名/路径匹配 → 目标 cluster
Cluster（CDS）	逻辑上游（对应 K8s Service 或 subset）
Endpoint（EDS）	cluster 的真实后端地址（来自 EndpointSlice 或 ServiceEntry）
Secret（SDS）	证书与密钥（**按需下发，不落盘**）
Filter	处理管道（HTTP filter：路由、认证、限流、遥测）

进程模型：Envoy 是**多线程 + 事件驱动**，主线程负责配置与管理，worker 线程处理连接；通过 **热重启（hot restart）** 与 **--drain-time** 实现无损重启。

sidecar 容器里其实有两个进程：

- `pilot-agent`：引导与管理 Envoy（生成 bootstrap、健康探测 15021、metrics 聚合 15020）；
- `envoy`：真正的数据面。

```
# 排障常用
kubectl exec <pod> -c istio-proxy -- curl -s localhost:15000/config_dump | less
kubectl exec <pod> -c istio-proxy -- curl -s localhost:15000/clusters
kubectl exec <pod> -c istio-proxy -- curl -s localhost:15000/listeners
kubectl exec <pod> -c istio-proxy -- pilot-agent request GET /debug/config_dump
```

常用端口（请与所用版本核对）：

端口	用途
15001	Envoy **出站**重定向监听
15006	Envoy **入站**重定向监听
15000	Envoy admin 接口（config_dump、stats、日志级别）
15020	pilot-agent 的聚合监控端口（merged metrics）
15021	sidecar 就绪健康 `/healthz/ready`
15010 / 15012	istiod 的 xDS（明文 / mTLS）
15014	istiod 自身的监控端口
15008	Ambient（HBONE 隧道）相关端口

A4. xDS 与控制面推送模型（必考）

```
istiod 监听：Service / EndpointSlice / VirtualService / DestinationRule / Gateway / AuthZ ...
   → 组装成 Envoy 配置的"模型"
   → 通过 ADS（gRPC 双向流，通常 15012 端口）推送给每个 proxy
   → proxy 应用配置并 ACK/NACK
```

要点：

- **ADS**：把各类 xDS 聚合在一条流上，保证**更新顺序一致**（避免"路由指向了还没就绪的 cluster"这种中间态）；
- **增量推送**：配置变化只推送受影响的部分，但**影响面分析做得不好就会变成"推送风暴"**（见 C1）；
- **NACK**：proxy 拒绝错误配置 ⇒ 会保留旧配置，但**istiod 侧要能看到 NACK 并告警**；
- **debounce / 节流**：Endpoint 抖动会被合并，避免频繁推送（相关环境变量/参数以版本为准）。

```
# istiod 侧推送与连接（指标名以版本为准）
pilot_xds_pushes
pilot_xds_push_errors
pilot_xds_eds_instances # EDS 规模
pilot_proxy_convergence_time_bucket
# NACK 往往是"配置错误"的第一信号
pilot_xds_cds_reject / pilot_xds_lds_reject 之类（以实际指标名为准）
```

A5. sidecar 是怎么被注入的?

```
① 给 namespace 打 label：istio-injection=enabled  （或为 Pod 打 sidecar.istio.io/inject=true）
② 创建 Pod 时，API Server 回调 istiod 的 mutating webhook（sidecar-injector）
③ webhook 把 istio-init（或 CNI 模式）与 istio-proxy 两个容器注入 Pod spec
④ istio-init：写 iptables 规则把流量重定向到 15001/15006（**需要 NET_ADMIN**）
   或用 istio-cni 插件：由节点 CNI 链路完成重定向（**免 NET_ADMIN**，更安全）
⑤ istio-proxy 启动、连 istiod、拉配置、就绪（15021）后应用容器才被认为可服务
```

关键注解（高频）：

```
metadata:
  annotations:
    sidecar.istio.io/inject: "true"
    traffic.sidecar.istio.io/excludeOutboundPorts: "3306" # 出站不走代理
    traffic.sidecar.istio.io/excludeInboundPorts: "22"
    traffic.sidecar.istio.io/includeInboundPorts: "8080"
    proxy.istio.io/config: '{"holdApplicationUntilProxyStarts": true}'
    sidecar.istio.io/proxyCPU: "500m"
    sidecar.istio.io/proxyMemoryLimit: "512Mi"
```

必答的两条：

1. `holdApplicationUntilProxyStarts` 解决"应用比 sidecar 先起来 ⇒ 早期请求丢失/探针失败"；
2. **istio-cni** 的价值是**去掉 NET_ADMIN 特权**，符合安全基线（代价是节点多一个 CNI 插件依赖）。

A6. 流量拦截：iptables 是怎么"偷走"流量的?

- 出站：Pod 内所有到其它服务的 TCP 被 **REDIRECT/TPROXY** 到 **15001**；Envoy 根据原始目的（通过 SO_ORIGINAL_DST / TPROXY）决定路由；
- 入站：到业务端口的流量被重定向到 **15006**；Envoy 处理后转发给本机应用端口；
- 排除：自身 Envoy 的流量、健康检查、以及 excludePorts 配置。

常见坑（必答）：

- 应用**需要拿到客户端真实 IP** ⇒ 需要正确传递（Envoy 会写入 `X-Forwarded-For`、`X-Envoy-External-Address`）；
- **非 TCP 协议/特殊端口**（如某些数据库协议）拦截后可能出问题 ⇒ 用 excludePorts；
- **hostNetwork: true 的 Pod** 注入通常不适用（流量不进 Pod netns）；
- 与 CNI 插件的兼容性（链式 CNI 顺序、istio-cni 与 Calico/Cilium 的协作要验证）。

A7. 为什么默认"全量下发"在大规模集群会炸？怎么收敛?

istiod 默认把**集群内所有服务与配置**下发给每个 sidecar（这样任何服务都能被解析与路由）。结果是：**配置体积随服务数线性增长**，服务上千后 Envoy 内存与 istiod 推送成本都会失控。

收敛三件套（必答，且是大规模第一优化）：

```
# ① Sidecar CR：限制该命名空间内 sidecar 的可见范围
apiVersion: networking.istio.io/v1
kind: Sidecar
metadata:
  name: default
  namespace: prod
spec:
  egress:
    - hosts:
        - "./*" # 本命名空间
        - "istio-system/*" # 控制面
        - "infra/*" # 依赖的基础设施命名空间
```

```
# ② istiod 的 discoverySelectors：只发现指定命名空间（MeshConfig 侧，版本相关）
meshConfig:
  discoverySelectors:
    - matchLabels:
        istio-discovery: enabled
```

```
# ③ 给 CR 用 exportTo 限制可见范围
spec:
  exportTo: ["."] # 只在本命名空间可见
```

> 面试时说出"**默认全量下发是 Istio 在大规模下的头号成本来源，收敛靠 Sidecar CR + discoverySelectors + exportTo**"就是标准答案。

A8. 流量管理 API 各自管什么？（最容易混的一组）

CRD	作用	常见坑
Gateway	定义**网格边缘**（Ingress/Egress）的监听：端口、协议、TLS、hosts	它不是路由；只负责"开门"
VirtualService	**路由规则**：把请求匹配后指向某个 destination（可加权/匹配/重写/重试）	匹配有**优先级顺序**；多 VS 对同一 host 会合并，行为易混乱
DestinationRule	**目标策略**：subset 定义、负载均衡、连接池、异常点检测、TLS 设置	没有 DR 就没有 subset；TLS 设置与 PeerAuthentication 冲突会 503
ServiceEntry	把**网格外**服务加入服务发现（外部 API、VM 服务）	没注册则访问被拦或走 Passthrough，策略不生效
WorkloadEntry / WorkloadGroup	非 K8s 工作负载（VM）注册与分组	需要配合 ServiceEntry 使用
Sidecar（资源）	收敛 sidecar 可见性（见 A7）	名字与其它概念同名，容易混
EnvoyFilter	直接改 Envoy 配置	**最后手段**，升级易碎，优先用 Telemetry API / WasmPlugin

必答的"host 匹配"坑：

- VS 的 `hosts` 要写**服务在网格内的名字**（K8s Service 名或 FQDN），写错不会报错但路由不生效；
- gateway 场景要写外部域名，且 `gateways` 字段要指定 gateway 名；
- 短名与 FQDN 的解析差异，是"为什么本地能通、从网关进不来"的常见原因。

A9. 路由匹配的优先级规则（必考细节）

```
按以下顺序，先匹配者胜：
  1. 更具体的 URI（精确 > 前缀 > 正则）
  2. 更具体的 method / headers / queryParams 等匹配条件
  3. 最后落到默认的 route（无 match 的规则）
```

坑：把"兜底路由"写在最前面 ⇒ 后面所有规则失效。**建议总是把最具体的规则放前面，兜底放最后**，并用 `delegate` 拆分大型路由表。

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts: [reviews]
  http:
    - match:
        - headers:
            end-user: {exact: jason} # ① 更具体
      route:
        - destination: {host: reviews, subset: v2}
    - route: # ② 兜底放最后
        - destination: {host: reviews, subset: v1}
```

A10. DestinationRule：subset 从哪来？负载均衡与熔断怎么配？

```
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: reviews
spec:
  host: reviews
  trafficPolicy:
    loadBalancer:
      simple: LEAST_REQUEST # ROUND_ROBIN/LEAST_CONN/LEAST_REQUEST/RANDOM
      # 一致性哈希（会话保持）：
      # consistentHash: {httpHeaderName: x-user}
    connectionPool:
      tcp: {maxConnections: 100, connectTimeout: 5s}
      http: {h2UpgradePolicy: UPGRADE, http2MaxRequests: 1000, maxRequestsPerConnection: 100}
    outlierDetection:
      consecutive5xxErrors: 5 # 连续 5 次 5xx ⇒ 摘除
      interval: 10s
      baseEjectionTime: 30s
      maxEjectionPercent: 50
  subsets:
    - name: v1
      labels: {version: v1}
    - name: v2
      labels: {version: v2}
```

必答：

- **subset 的 labels 匹配的是 endpoint 的标签**（来自 Pod labels），不是 Service selector；

- `outlierDetection` 是**被动健康检查**（Envoy 侧，基于实际请求结果），与 K8s 的 readiness 互补；
- 熔断是**每个上游（cluster）维度**的，不是全局的；
- `trafficPolicy.tls` 与 `PeerAuthentication` 不一致是 **503 的高频原因**（见 B8）。

A11. 弹性四件套：超时、重试、熔断、故障注入

```
http:
  - route:
      - destination: {host: ratings}
    timeout: 3s
    retries:
      attempts: 3
      perTryTimeout: 1s
      retryOn: 5xx,connect-failure,refused-stream,retriable-4xx
      retryBudget: # 限制重试占比，防重试风暴
        percent: 20
        minRetryConcurrency: 10
```

能力	注意
超时	Envoy 默认无超时，必须显式配（**这是很多人不知道的默认值**）
重试	只对幂等操作使用；**必须配 retryBudget**，否则雪崩放大；POST 慎用
熔断/异常点检测	见 A10
故障注入	`fault.delay/abort`，只用于测试环境
流量镜像	`mirror` + `mirrorPercentage`，请求发给影子服务且**忽略响应**，用于上线前验证

A12. mTLS 与身份：Istio 安全的三块基石

```
① 身份：每个工作负载有 SPIFFE ID = spiffe://<trust-domain>/ns/<ns>/sa/<sa>
        （来自 Pod 的 ServiceAccount，由 istiod CA 签发证书）
② 传输：mTLS（PeerAuthentication 控制模式）
③ 授权：AuthorizationPolicy 基于身份/属性做 ALLOW/DENY
```

```
# 网格级：宽松模式（迁移期）→ 严格模式
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: default
  namespace: istio-system # 网格级
spec:
  mtls:
    mode: STRICT # STRICT | PERMISSIVE | DISABLE
```

```
# 授权：只允许特定 SA 访问
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: allow-web-to-api
  namespace: prod
spec:
  selector:
    matchLabels: {app: api}
  action: ALLOW
  rules:
    - from:
        - source: {principals: ["cluster.local/ns/prod/sa/web"]}
      to:
        - operation: {methods: ["GET", "POST"], ports: ["8080"]}
```

```
# JWT 认证（RequestAuthentication + 授权要求）
apiVersion: security.istio.io/v1
kind: RequestAuthentication
metadata:
  name: jwt
  namespace: prod
spec:
  selector: {matchLabels: {app: api}}
  jwtRules:
    - issuer: "https://issuer.example.com"
      jwksUri: "https://issuer.example.com/.well-known/jwks.json"
      audiences: ["my-api"]
```

必答坑：

- `RequestAuthentication` **只验证 JWT，不强制要求**；要拒绝无 token 请求必须再配 `AuthorizationPolicy` 要求 `requestPrincipals`；
- `AuthorizationPolicy` 的 `DENY` 优先于 `ALLOW`；没有匹配任何 ALLOW 时**默认是拒绝还是允许**取决于是否存在该 selector 的 ALLOW 策略（**这是最容易答错的地方：一旦有 ALLOW 策略选中该工作负载，未匹配的请求即被拒绝**）；
- mTLS 模式冲突（服务端 STRICT、客户端 DISABLE）⇒ **503 upstream connect error。**

A13. 证书是怎么签发和轮换的?

```
proxy 启动 → 通过 SDS 向 istiod 请求证书（带 SA token 做身份认证）
  → istiod CA（或插件 CA：cert-manager/Vault/SPIRE/自定义 root）签发
  → 证书短期有效（默认 TTL 以版本为准，常见为 24 小时量级）
  → 到期前自动轮换（无需重启 Pod）
```

必答：

- 短期证书 + 自动轮换是设计核心（**泄露风险窗口小**）；
- 生产上通常用**自定义根 CA / 外部 CA 插件**以满足合规（**插拔方式以版本文档为准**）；
- **控制面不可用时**，已有 sidecar 仍能用现有证书通信，但**新 Pod 拿不到证书 ⇒ 起不来** ⇒ istiod 必须 HA（见 C11）。

A14. 入口与出口：Gateway / Ingress / Egress 的关系

组件	角色
Ingress Gateway	南北向入口（Deployment + Envoy，通常配 LB Service），由 `Gateway` + `VirtualService` 配置
Kubernetes Ingress	也能被 Istio 支持（通过 IngressClass），但**社区更推荐 Gateway API / Istio Gateway**
Egress Gateway	统一出口：**固定出口 IP + 集中审计/策略 + TLS 发起**，用于合规场景
East-West Gateway	多集群之间的跨集群流量入口（见 A18）

```
apiVersion: networking.istio.io/v1
kind: Gateway
metadata:
  name: web-gw
  namespace: istio-system
spec:
  selector:
    istio: ingressgateway
  servers:
    - port: {number: 80, name: http, protocol: HTTP}
      hosts: ["*.example.com"]
    - port: {number: 443, name: https, protocol: HTTPS}
      hosts: ["*.example.com"]
      tls: {mode: SIMPLE, credentialName: example-tls} # 引用 K8s Secret (SDS)
```

必答：`credentialName` 让 gateway 通过 SDS 动态取证书，**无需挂载 Secret 卷、无需重启**（这是常见加分点）。

A15. Ambient Mesh 是什么? 为什么它被视为下一代形态?

组成	位置	职责
ztunnel	每个节点一个（DaemonSet）	**L4**：mTLS、认证、L4 授权、遥测（不含 L7 解析）
waypoint	按需，通常按服务账号部署	**L7**：HTTP 路由、鉴权、L7 遥测与策略
HBONE	隧道	基于 HTTP/2 的封装（默认端口 15008 相关），承载 mTLS 与元数据

价值（必答）：

1. **不再给每个 Pod 注入 sidecar** ⇒ 省内存/CPU、Pod 启动更快、**应用与代理解耦**（升级网格不需要重启业务 Pod）；
2. **渐进采用**：无 sidecar 的 Pod 与有 sidecar 的 Pod 可以互通，可按需给服务打开 waypoint 拿 L7 能力；
3. 更适合"只想拿 mTLS 与可观测、不需要复杂 L7 策略"的场景。

限制与风险（必须说）：

- 功能覆盖与 sidecar 模式不完全等价（**具体差异以所用版本官方矩阵为准**）；
- hostNetwork、某些 CNI/内核依赖、以及部分高级特性需要逐项验证；
- 排障工具链从"进 sidecar 看 config_dump"变成"看 ztunnel/waypoint"，团队技能要更新。

A16. Sidecar vs Ambient vs "不用网格"怎么选?

维度	Sidecar	Ambient	无网格（只用 CNI + 库）
每 Pod 开销	一个 Envoy（内存/CPU 随配置增长）	无（节点级共享）	无
升级网格	需滚动重启业务 Pod	**通常不需要重启业务**	—
L7 能力	全	按需（waypoint）	靠 SDK
隔离性	强（每 Pod 独立代理）	节点级共享（多租户需评估）	应用内
成熟度	最成熟	较新（**版本相关**）	—
适用	需要完整 L7 策略/可观测的核心服务	想先拿 mTLS 与基础遥测、追求低开销	简单场景

标准答法：

> "不要为了上网格而上网格。如果诉求只是 mTLS + 基础可观测，Ambient 的性价比明显更高；如果需要复杂的 L7 路由/鉴权/限流，sidecar（或带 waypoint）更稳。**两者可以在同一网格共存**，这就是渐进迁移的路径。"

A17. DNS 与无 VIP 服务：为什么 Istio 需要 DNS 代理?

问题：K8s 的 **Headless Service**（ClusterIP=None）没有 VIP，返回的是 Pod IP 列表。Envoy 若只按 VIP 转发，就处理不了这类服务；此外**外部服务（ServiceEntry）也没有 VIP**。

解法：Envoy 开启 **DNS 代理/捕获**，sidecar 自己解析 DNS 并把解析结果纳入路由与负载均衡。

相关坑（必答）：

- StatefulSet + Headless 的服务必须确认 DNS 捕获生效，否则访问失败或负载不均；
- 与 `ndots`/search 域的组合会放大 DNS 查询（与《网络篇》A7 同源）；
- 解析失败的排障：看 Envoy 的 `dns` 相关统计与 `upstream` 状态。

A18. 多集群的三种拓扑（与"为什么复杂"）

拓扑	说明	适用
Multi-primary（多主）	每个集群有自己的 istiod，都连接所有集群的 API Server	高可用最好，运维最复杂
Primary-remote（主-远）	一个主集群的 istiod 管理远端集群的数据面	常见、简化控制面
External control plane	控制面在专用集群（甚至托管）	厂商/托管场景

关键点：

- **东西向网关**用于跨集群流量（网络不扁平时必须）；
- **共享/独立信任域**：跨集群 mTLS 需要统一或联邦的信任配置；
- **网络要求**：扁平网络（Pod IP 可直连）最简单；非扁平则依赖网关与隧道；
- **服务发现**：远端集群的 Service 如何被发现（远端 secret/配置，**机制以版本为准**）。

A19. VM / 非 K8s 负载怎么接入网格?

- **WorkloadGroup**：描述一组非 K8s 工作负载（模板）；
- **WorkloadEntry**：注册具体实例（IP/端口/labels），常与 **ServiceEntry** 配合，让它们像网格内服务一样被发现与路由；
- 也有 **Workload Onboarding** 类方案用于自动化接入（**成熟度以版本为准**）。

```
apiVersion: networking.istio.io/v1
kind: WorkloadEntry
metadata:
  name: vm-db-1
  namespace: prod
spec:
  address: 10.10.1.5
  labels: {app: db, version: v1}
  serviceAccount: db
```

A20. 可观测性：Istio 提供什么？成本在哪?

类型	内容
指标	`istio_requests_total`、`istio_request_duration_milliseconds`、`istio_request_bytes`、`istio_tcp_*` 等（由 Envoy 统计并通过 15020 暴露）
访问日志	默认可开启（文本或 JSON），含身份、响应标志、上游等上下文
追踪	传播 W3C `traceparent` / B3，采样率可配，导出到 OTel/Jaeger 等
拓扑/可视化	Kiali 等

成本（必答）：

- **指标基数爆炸**：`istio_requests_total` 带 source/destination 的工作负载、命名空间、principal、响应码、响应标志等多个标签 ⇒ 服务数平方级增长的组合；
- 访问日志全量写盘/采集很贵；
- **解法**：用 **Telemetry API 裁剪维度**、调整采样、Prometheus 侧 relabel 丢弃高基数标签。

```
# Telemetry API：去掉高基数维度（示意，字段以版本为准）
apiVersion: telemetry.istio.io/v1
kind: Telemetry
metadata:
  name: mesh-default
  namespace: istio-system
spec:
  metrics:
    - providers: [{name: prometheus}]
      overrides:
        - match: {metric: REQUEST_COUNT, mode: CLIENT_AND_SERVER}
          tagOverrides:
            request_host: {operation: REMOVE}
            source_principal: {operation: REMOVE}
```

A21. 与 K8s 原生能力如何分工?

能力	K8s 原生	Istio
服务发现	Service + EndpointSlice	消费它，并可用 ServiceEntry 扩展

能力	K8s 原生	Istio
负载均衡	kube-proxy（L4，per-connection）	Envoy（L7，per-request，带重试/超时/异常点）
入口	Ingress / Gateway API	Ingress Gateway（也支持 Gateway API）
网络策略	**NetworkPolicy（L3/4，CNI 实现）**	AuthorizationPolicy（L7，身份与属性）
mTLS	无	有
可观测	需自行埋点	自动 RED 指标 + 追踪上下文

必答：**NetworkPolicy 与 AuthorizationPolicy 不冲突、应叠加**——前者管"能不能连到端口"，后者管"以什么身份、操作什么资源"。

A22. 协议与 SNI：TLS 直通 vs 终止

模式	行为	适用
PASSTHROUGH	Envoy 不解密，按 SNI 转发	外部 HTTPS、客户端证书认证
SIMPLE / MUTUAL	在网关终止 TLS（可选再 mTLS 到后端）	常规 Web
AUTO_PASSTHROUGH	多集群/跨网格场景	东西向

坑：

- 不配置 SNI 的 TLS 流量在 PASSTHROUGH 下无法路由；
- 端口协议识别错（HTTP 被当成 TCP）⇒ L7 能力全部失效（见 A23）。

A23. 端口命名/协议识别：为什么"服务看起来通但 L7 规则不生效"？

Envoy 需要知道端口上的协议才能应用 HTTP 路由与策略。识别顺序大致是：

```
① 显式声明（Service port 的 name = http-<suffix> / grpc-... 或 appProtocol）
② 否则做协议嗅探（有代价且不总是准）
③ 仍无法识别 ⇒ 按 TCP 处理（只有 L4 能力）
```

必答：

- 现代写法推荐用 `appProtocol`（Service port 字段）显式声明；
- 名字写错（如 `http2` 写成 `http2-xxx` 之外的形式）会导致协议识别失败；
- TCP 服务（MySQL、Redis）不要强行按 HTTP 处理。

A24. istiod 自身的高可用与依赖

- istiod 是**无状态化依赖**（配置来自 K8s API）⇒ 可多副本；
- 但**证书签发/部分协调逻辑有选主**（以版本实现为准）⇒ 通常 2~3 副本；
- **istiod 挂了的影响**：已有流量不受影响（数据面已持有配置与证书），但**新 Pod 起不来、配置变更不生效** ⇒ 必须 HA + PDB + 资源保障。

A25. 版本兼容与升级模型

- **控制面与数据面版本差**有官方支持范围（**常见是相差不超过一个 minor 版本，请以所用版本官方支持矩阵核对**）；
- **金丝雀升级**：安装新版本控制面时带 **revision**，通过 label/tag 让部分命名空间逐步切换数据面：

```
istioctl install --revision=1-24-0
istioctl tag set prod-canary --revision 1-24-0
kubectl label ns prod istio.io/rev=prod-canary --overwrite
kubectl rollout restart deploy -n prod # 让 sidecar 换版本
# 验证通过后，把默认 tag 指向新 revision，再逐命名空间滚动
```

必答：升级后要验证：① istiod 无 NACK；② 关键服务流量与 mTLS 正常；③ 指标与追踪不丢。

A26. 网格里最常见的"服务不通"原因排序

排序	原因	快速验证
1	**端口协议识别错**（当 TCP 处理）	`config_dump` 看 listener 类型
2	**mTLS 模式冲突**（STRICT 侧收到明文）	日志 `upstream connect error / 503 UF,URX`
3	**VirtualService host 写错** / 未绑定 Gateway	看 Envoy route 是否包含目标
4	**外部服务未注册 ServiceEntry**	出站被拒或走 Passthrough
5	**Sidecar 未注入 / 注入失败**	`kubectl get pod -o jsonpath` 看容器数
6	**AuthorizationPolicy 未放行**（尤其新加 ALLOW 后默认拒绝）	临时改 action 或删策略验证
7	**DNS/Headless 问题**	看 DNS 统计与解析结果
8	**熔断/异常点把上游摘了**	看 outlier 统计与 `outlier_detection` 状态

```
# 万能三连：看 proxy 状态、看配置同步、看统计
istioctl proxy-status
istioctl analyze -n <ns>
istioctl proxy-config route <pod> -o json
kubectl exec <pod> -c istio-proxy -- curl -s localhost:15000/stats | grep -i "upstream_rq\|outlier"
```

Part B • 方案 / 模式专项篇（14 题）

B1. 注入方式对比：什么时候用哪种?

方式	说明	适用
namespace label 自动注入	`istio-injection=enabled`	常规
Pod annotation 覆盖	`sidecar.istio.io/inject: "true"` 覆盖 ns 设置	例外场景/灰度
istio-cni	由 CNI 链路做重定向（**免 NET_ADMIN**）	有安全基线要求（禁特权/禁 NET_ADMIN）
Ambient（无 sidecar）	namespace 打 ambient 标签 + 按需 waypoint	想要低开销与免重启升级
手动注入	`istioctl kube-inject`	调试/特殊控制器

```
# 常用开关（顺序：ns label 是基础，pod annotation 优先）
kubectl label ns prod istio-injection=enabled
kubectl label ns legacy istio-injection=disabled
# Ambient（标签名以版本为准）
kubectl label ns prod istio.io/dataplane-mode=ambient
```

B2. 金丝雀发布的三种做法?

```
# ① 原生 VirtualService 权重
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata: {name: reviews, namespace: prod}
spec:
  hosts: [reviews]
  http:
    - route:
        - destination: {host: reviews, subset: v1}
          weight: 90
        - destination: {host: reviews, subset: v2}
          weight: 10
```

方式	特点
手动改权重	简单，但没有自动化判断
Flagger / Argo Rollouts	基于指标自动推进/回滚（**生产推荐**）
基于请求头/用户	用于"内部先试"（见 A9 示例）

必答：金丝雀要配**一致哈希或会话保持**（避免同一用户在 v1/v2 之间跳），并且要有**自动回滚阈值**。

B3. 流量镜像（Shadow）怎么用?

```
http:
  - route:
      - destination: {host: reviews, subset: v1}
    mirror:
      host: reviews
      subset: v2
    mirrorPercentage: {value: 10.0}
```

必答：镜像请求的**响应被丢弃**，所以它适合验证新版本是否能处理真实流量；但**有写操作的服务要谨慎**（可能造成重复写入）⇒ 通常要求影子服务"只接受不落库"或使用独立存储。

B4. 熔断与限流：为什么"本地限流"不精确?

类型	机制	精度
本地限流（Envoy local rate limit / token bucket）	每个 proxy 各自计数	实际配额 ≈ 单实例配额 × 副本数，**不精确**
全局限流（Rate Limit Service）	外部 RLS + Redis 计数	精确，但多一跳且是**新的依赖与故障点**
熔断（outlierDetection / 连接池）	按上游健康度摘除	保护调用方

实践建议：本地限流用于"防打爆自己"的兜底；全局限流用于租户配额；**两者都要有"限流生效"的可观测**（否则出问题不知道是被限流还是服务挂了）。

B5. JWT + 授权的完整闭环（常见漏配）

```
# ① 认证：验证 JWT（不强制要求）
apiVersion: security.istio.io/v1
kind: RequestAuthentication
metadata: {name: jwt, namespace: prod}
spec:
  selector: {matchLabels: {app: api}}
  jwtRules:
    - issuer: "https://issuer.example.com"
      jwksUri: "https://issuer.example.com/.well-known/jwks.json"
---
# ② 授权：强制必须带有效 JWT（关键！只配 ① 不会拒绝无 token 请求）
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata: {name: require-jwt, namespace: prod}
spec:
  selector: {matchLabels: {app: api}}
  action: ALLOW
  rules:
    - from:
        - source: {requestPrincipals: ["https://issuer.example.com/*"]}
```

B6. mTLS 从 PERMISSIVE 迁移到 STRICT 的标准步骤（落地大题）

```
① 全网格设 PERMISSIVE（同时接受明文与 mTLS）
② 用指标确认"明文流量已经消失":
     istio_requests_total 的 connection_security_policy="mutual_tls" 占比 → 100%
③ 按命名空间逐个切 STRICT（先非关键、再核心）
④ 每次切换后观察 503/失败率
⑤ 全网格 STRICT + 保留"例外"（如需要明文健康检查的来源）
```

```
# ② 的证据：非 mTLS 请求占比
sum(rate(istio_requests_total{connection_security_policy!="mutual_tls"}[5m]))
  / clamp_min(sum(rate(istio_requests_total[5m])), 0.001)
```

必答的坑：

- 某些**外部健康检查/监控**走明文 ⇒ 切 STRICT 后失败 ⇒ 需要 `PeerAuthentication` 的 `portLevelMtls` 例外，或让探测端也进网格；
- **非网格客户端**（集群外直连 Pod IP）在 STRICT 下会被拒；
- DR 的 `trafficPolicy.tls.mode` 与 PeerAuthentication 冲突时会 503。

B7. Egress 治理：三种策略怎么选?

方案	说明
PASSTHROUGH（默认）	允许所有出站，但**无法审计与策略**
ServiceEntry + 策略	把外部服务纳入网格：可命名、可策略、可遥测
Egress Gateway	统一出口 IP、集中 TLS/审计/限流（合规场景必需）

```
apiVersion: networking.istio.io/v1
kind: ServiceEntry
metadata: {name: external-api, namespace: prod}
spec:
  hosts: ["api.external.com"]
  location: MESH_EXTERNAL
  ports: [{number: 443, name: https, protocol: HTTPS}]
  resolution: DNS
```

配套：`outboundTrafficPolicy.mode = REGISTRY_ONLY`（只允许已注册服务出站）⇒ 防止"偷偷访问外部"，但需要先把所有合法外部依赖注册全（**这是最常踩的坑**）。

B8. Gateway API vs Istio API：未来怎么选?

维度	Istio API（Gateway + VirtualService）	Kubernetes Gateway API（Gateway + HTTPRoute）
定位	Istio 专有	**K8s 标准**，跨实现可移植
表达能力	很强（含 subset、DestinationRule）	标准化，部分高级能力靠实现扩展
团队分工	单一团队	**角色划分**（基础设施/集群运维/应用开发）
演进	仍支持	**社区推荐方向**（支持程度以版本为准）

建议：新集群入口优先 **Gateway API**，东西向与高级流量治理仍用 VirtualService/DestinationRule；迁移要按服务逐个验证（**不要一次性切换**）。

B9. 网格内的"入口"选型：Ingress Gateway vs Ingress Controller?

方案	适用
Istio Ingress Gateway	已经用网格、希望入口与网格策略/遥测一致
独立 Ingress/Nginx + 网格	入口团队与网格团队分离；或入口需要大量 Nginx 特性
Gateway API 实现	想统一南北向标准

必答：入口是**单点故障源**，不管选哪个都要：多副本跨区 + PDB + 优雅终止 + 容量压测 + 独立告警。

B10. Sidecar 资源与启动顺序调优

```
metadata:
  annotations:
    proxy.istio.io/config: |
      holdApplicationUntilProxyStarts: true
      concurrency: 2 # Envoy worker 数（按 CPU 限额实测调整）
      proxyStatsMatcher: # 裁剪统计，降低 CPU/内存
        inclusionRegexps: [".*upstream_rq_total.*", ".*circuit_breakers.*"]
    sidecar.istio.io/proxyCPU: "500m"
    sidecar.istio.io/proxyCPULimit: "2"
    sidecar.istio.io/proxyMemory: "256Mi"
    sidecar.istio.io/proxyMemoryLimit: "1Gi"
```

必答：

- `holdApplicationUntilProxyStarts` 让**应用容器等 sidecar 就绪**，避免早期请求丢失；
- Envoy 的 **CPU/内存随"可见服务数"增长** ⇒ 资源调优必须配合 A7 的配置收敛，否则加资源只是治标；
- `proxyStatsMatcher` 裁剪统计能显著降低 Envoy 内存（**大规模必备**）。

B11. 与 spring/SDK 治理共存时的注意点

- **重试叠加**：SDK 重试 × 网格重试 ⇒ 放大倍数相乘 ⇒ 必须统一约定（通常保留网格重试，SDK 关掉或只保留少量）；
- **超时叠加**：外层超时 \< 内层重试总时长 ⇒ 白重试；
- **服务发现冲突**：SDK 直连 IP + 网格路由 ⇒ 绕过策略与遥测 ⇒ 应统一走 Service 名。

B12. 网格与 CNI 的配合（与网络篇联动）

- Istio 依赖 CNI 提供 **Pod IP 可达**；Cilium 的 eBPF/kube-proxy replacement 与 sidecar 的 iptables 拦截可以共存，但**必须逐项验证**；
- **Ambient/ztunnel 与 CNI 的耦合更紧**（依赖节点级转发），选型时要看官方兼容矩阵；
- NetworkPolicy 需要放行：**istiod（15010/15012/15014/15020）、DNS、以及应用端口——忘了放行 istiod 是新 namespace 启用网格后 Pod 起不来的头号原因。**

B13. 常见"网格引入后变慢/失败"的现象归因

现象	原因
Pod 启动变慢	sidecar 拉配置 + `holdApplicationUntilProxyStarts`；或 istiod 压力大
探针失败	探针流量被拦截/未放行；或 sidecar 未就绪
P99 增加	每跳多两次用户态代理；连接池不足；HTTP/1.1 未升级 HTTP/2
偶发 503	mTLS 冲突、熔断摘除、Endpoint 变更瞬时空窗
内存暴涨	配置全量下发 + 统计未裁剪
控制面 NACK	配置错误（DR/VS 冲突）

B14. 网格的"渐进落地路线"（架构题）

```
阶段 1：只做可观测（注入 + 遥测，不开 mTLS 严格模式）
阶段 2：mTLS PERMISSIVE → 逐 ns STRICT（拿到身份）
阶段 3：超时/重试/熔断统一到网格（去掉 SDK 重复实现）
阶段 4：金丝雀与流量镜像常态化（配合 Flagger/Argo）
阶段 5：授权（AuthorizationPolicy）按服务逐步开启
阶段 6：按需引入 Ambient 降低开销 / 或按需 waypoint
```

> **关键原则**：**每一步都要有可回滚开关**（label 控制注入、CR 控制策略），不要一次性全开。

Part C • 大规模集群网格优化篇（12 题）

C1. 控制面推送风暴：为什么会"一个变更打满 istiod"

机制：Service/Endpoint/VS/DR 变更 ⇒ istiod 重新计算 ⇒ 向**所有相关 proxy** 推送。服务数与 sidecar 数都很大时，一次"热门服务"变更可能触发成千上万次推送 ⇒ CPU 打满、推送延迟、配置滞后。

治理：

1. **Sidecar CR 收敛可见性**（最关键，见 A7）；
2. `discoverySelectors` 缩小发现范围；
3. `exportTo` 限制 CR 可见；
4. 控制 Endpoint 抖动频率（Pod 频繁重建会让 EDS 不断变更）；
5. 关注推送相关指标与 NACK（见 C10 告警）。

```
# 直观判断配置规模
istioctl proxy-config cluster <pod> -o json | jq 'length'
istioctl proxy-status # 看有多少 proxy 处于 Stale/Synced
```

C2. 数据面内存与 CPU 的主要来源

来源	治理
可见服务/配置数量	Sidecar CR + discoverySelectors
统计指标基数	`proxyStatsMatcher` 裁剪
连接数（每个上游连接池）	调整连接池上限、开启 HTTP/2 复用
访问日志	按需开启 + 采样/裁剪字段
Envoy worker 数	按 CPU 限额设置 `concurrency`

C3. 指标基数爆炸：怎么把 Prometheus 救回来

```
# 先看基数（最直接的证据）
topk(10, count by (__name__)({__name__=~"istio_.*"}))
# 看时间序列总量增长
sum(scrape_samples_scraped{job=~".*istio.*"})
```

治理：

- **Telemetry API 删除高基数维度**（`source_principal`、`request_host`、`response_flags` 等按需保留）；
- Prometheus 侧 `metric_relabel_configs` drop 高基数标签；
- 关闭不必要的 CLIENT 侧指标（保留 SERVER 侧即可覆盖 RED）；
- 按命名空间分级（核心服务保留明细，其它聚合）。

C4. Endpoint 规模与 EDS 更新

- 大服务（数千 endpoint）的 EDS 更新体积大且频繁；
- EndpointSlice 已在 K8s 侧降低变更成本（见《控制面篇》A6），网格侧要避免"每次 Pod 重建都引发大面积推送"；
- 治理：稳定 Pod（减少不必要的重建）、合理分批、观察 `pilot_xds_*` 推送量。

C5. 网格对延迟的影响怎么量化?

```
单跳成本 ≈ 出站 sidecar 处理 + 入站 sidecar 处理（两次用户态转发 + 可能的 mTLS 加解密）
量化方法（务必实测，不要用别人的数字）：
  A/B 对比：同一服务在"网格内 vs 直连 PodIP"的 P50/P99
  指标：istio_request_duration_milliseconds（SERVER 侧）+ 业务侧 P99
  控制变量：同样 QPS、同样副本、同样连接池
```

结论口径：网格带来的是"**可控的常数级延迟成本 + 可观测与治理收益**"，是否划算由业务 SLO 决定；**延迟极敏感的核心链路要单独压测。**

C6. Ambient 在大规模下的收益与限制

收益：省掉每 Pod sidecar ⇒ 内存/CPU 总量下降明显；升级网格**不需要滚动重启业务 Pod**（这是运维上最大的收益）。

限制：L7 能力依赖 waypoint（要按需规划 waypoint 数量与容量）；功能等价性与版本成熟度需逐项验证；**多租户共享 ztunnel 的隔离性要评估。**

C7. istiod 的 HA 与容量

项	做法
副本	通常 2~3，跨可用区
PDB	保证至少 1 可用
资源	按"Sidecar 数 × 配置规模"实测给 requests/limits
依赖	apiserver 的 watch 压力（可用 discoverySelectors 降低）
监控	`up{job="istiod"}`、推送错误、NACK、proxy Stale 数量、内存增长

C8. Gateway 的容量与高可用

- **入口/出口 Gateway 是"集中式 Envoy"** ⇒ 必须独立压测与容量规划；
- 多副本跨区 + PDB + 优雅终止（`terminationGracePeriodSeconds` 要够长，Envoy 需要 drain）；
- HPA 基于 CPU/QPS 扩容，并预留"突发"余量；
- **优雅终止是网关事故高发点**：drain 时间不足会导致连接重置（与《Kubelet 篇》Q11 同源）。

C9. 多集群的运维成本与是否值得

收益	成本
容灾、就近访问、隔离	控制面部署复杂度、跨集群服务发现、信任域、网关与网络要求

建议：**先明确是否需要跨集群。**很多场景用"多集群 + 上层 DNS/全局负载均衡切换"即可，不必做全互通的单一网格。

C10. 网格的最小告警集

```
# ① istiod 存活与推送
up{job="istiod"} == 0
rate(pilot_xds_push_errors[5m]) > 0
# ② proxy 不同步（Stale）比例（istioctl proxy-status 的指标化版本，指标名以版本为准）
# ③ 5xx 与 mTLS 混合状态
sum(rate(istio_requests_total{response_code=~"5.."}[5m]))
  / clamp_min(sum(rate(istio_requests_total[5m])), 0.001) > 0.01
# ④ 明文流量（迁移 STRICT 前必须归零）
sum(rate(istio_requests_total{connection_security_policy!="mutual_tls"}[5m])) > 0
# ⑤ 熔断/异常点生效（被摘除的上游）
sum(rate(envoy_cluster_outlier_detection_ejections_active[5m])) > 0 # 指标名以版本为准
# ⑥ 网关容量
sum(rate(istio_requests_total{reporter="source"}[5m])) by (destination_service_name)
```

C11. 网格变更的灰度与回滚（生产治理）

变更	灰度	回滚触发	回滚方式
控制面升级	新 revision → 单 ns 切换 → 逐步全量	NACK 上升、503 上升、istiod 内存异常	把 ns label 切回旧 revision + 滚动重启
mTLS 收紧	单 ns → 多 ns	非 mTLS 流量未归零 / 失败率上升	改回 PERMISSIVE（**改 CR 即生效，最快**）
授权策略	先 DENY 少量、再 ALLOW 细化	健康检查失败、业务报错	删除/放宽策略
Telemetry 裁剪	单 ns	指标缺失	恢复覆盖配置
Ambient 启用	单 ns/单服务	连通性或功能缺失	去掉 ns 标签 + 重建 Pod

> **黄金规则**：**所有网格策略都应该"可一键回退"**——优先用 CR/label 表达，而不是改镜像或改注入模板。

C12. 给你一个 1000 服务的新集群，怎么设计网格？（综合题）

1. **先问 SLO 与诉求**：要 mTLS？要 L7 授权？要统一金丝雀？能接受多少延迟与资源开销？
2. **形态选择**：先 Ambient 还是直接 sidecar（按 L7 需求与版本成熟度定）；
3. **边界**：入口用 Gateway API + Ingress Gateway；出口按合规决定是否 Egress Gateway；
4. **规模治理**：**Sidecar CR + discoverySelectors + exportTo** 必须在首批服务接入时就落地（事后补很痛）；
5. **可观测**：Telemetry 裁剪 + 采样 + 指标 relabel + 访问日志按需；
6. **安全**：mTLS 迁移路线（PERMISSIVE→STRICT）、授权按服务推进、外部 CA 接入；
7. **HA**：istiod 多副本跨区 + PDB；Gateway 多副本 + 优雅终止 + 压测；
8. **变更治理**：revision 升级、label 灰度、一键回退；
9. **演练**：istiod 挂、Gateway 挂、误配策略、证书过期、推送风暴。

Part D · 反模式清单（可直接用于生产巡检）

#	反模式	证据	等级	修复	验证
D1	大规模下不做配置收敛（全量下发）	Envoy 内存高、istiod 推送量大	🔴 高	Sidecar CR + discoverySelectors + exportTo	cluster 数量与推送量下降（实测）
D2	直接全网格 STRICT	明文探测/外部客户端 503	🔴 高	按 B6 分阶段迁移，先证明明文流量归零	`connection_security_policy` 全为 mutual_tls
D3	只配 RequestAuthentication 以为"加了鉴权"	无 token 请求仍被放行	🔴 高	必须配 AuthorizationPolicy 要求 requestPrincipals	无 token 请求返回 403
D4	大量使用 EnvoyFilter	升级后被破坏	🔴 高	优先 Telemetry API / 原生 CR；EnvoyFilter 作为最后手段并做升级验证	升级演练通过
D5	未放行 istiod 端口的 NetworkPolicy	新 ns 启用网格后 Pod 全挂	🔴 高	放通 15010/15012/15014/15020 与 DNS	新 ns 注入后 Pod 正常启动
D6	指标全量开启不做裁剪	Prometheus 基数爆炸、抓取超时	🔴 高	Telemetry 裁剪 + relabel	时间序列数量与抓取耗时回归
D7	重试不设 budget、POST 也重试	雪崩放大	🔴 高	retryBudget + 仅幂等接口重试	故障注入演练不放大流量
D8	未配超时（依赖 Envoy 默认）	请求无限悬挂	🟠 中	显式配 timeout/perTryTimeout	超时行为符合 SLO
D9	启用 REGISTRY_ONLY 但未注册全部外部依赖	外部调用突然全失败	🔴 高	先梳理依赖并建 ServiceEntry，再收紧	依赖清单对账完成
D10	sidecar 资源一刀切（默认不改）	大 ns 的 sidecar OOM/CPU 打满	🟠 中	按可见规模分级设置资源 + 裁剪统计	内存回到基线
D11	网关 drain 时间不足	发布/缩容时连接重置	🟠 中	放大 grace + 配置 drain 相关参数（版本相关）	滚动网关期间无连接重置
D12	网格与 SDK 重试/超时叠加	放大倍数相乘	🟠 中	统一约定，关闭一侧	压测确认放大倍数
D13	端口协议未显式声明	L7 规则"配了不生效"	🟠 中	用 `appProtocol` /端口命名	config_dump 中 listener 为 HTTP 类型
D14	istiod 单副本/无 PDB	istiod 维护期新 Pod 起不来	🔴 高	多副本跨区 + PDB + 资源保障	杀一个副本不影响新 Pod 启动
D15	一次性把全部服务接入网格	问题集中爆发且难回退	🟠 中	按 ns/服务分批，label 可回退	每批次有验证窗口与回滚记录

Part E • 可观测性：指标、SLO 与告警

E1. 核心指标清单（请先 `kubectl exec ... -c istio-proxy -- curl :15020/stats/prometheus` 核对）

指标	含义	用途
`istio_requests_total`	请求数（含 reporter/source/destination/响应码/响应标志/mTLS 状态等标签）	RED 的 Rate/Errors
`istio_request_duration_milliseconds`	请求延迟直方图	RED 的 Duration
`istio_request_bytes` / `istio_response_bytes`	流量字节	容量
`istio_tcp_*`	TCP 层指标	非 HTTP 服务
`pilot_xds_pushes` / `_push_errors` / NACK 系列	控制面推送	控制面健康
`istio_agent_...` / `envoy_*`	sidecar/Envoy 自身状态	数据面健康
控制面/数据面进程指标	`process_*`、`container_memory_*`	容量

E2. 建议的 SLO（初始建议值，需实测校准）

SLI	建议目标
网格内服务成功率（非 5xx）	≥ 99.9%
网格引入的延迟增量 P99	明确预算（如 ≤ 10ms/跳，**必须 A/B 实测**）
mTLS 覆盖率（迁移完成后）	100% `mutual_tls`
istiod 可用性	≥ 99.95%（多副本 + PDB）
配置同步延迟（变更 → proxy 生效）P99	≤ 30s（需实测）
控制面 NACK	= 0（>0 即告警）

E3. Prometheus 告警规则（可直接粘贴，阈值需实测校准）

```
groups:
  - name: istio
    rules:
      - alert: IstiodDown
        expr: up{job="istiod"} == 0
        for: 2m
        labels: {severity: critical}
        annotations: {summary: "istiod 不可用：新 Pod 无法获取配置与证书"}

      - alert: IstioXdsPushErrors
        expr: rate(pilot_xds_push_errors[5m]) > 0
        for: 10m
        labels: {severity: warning}
        annotations: {summary: "istiod 推送出现错误，检查配置合法性"}

      - alert: IstioMeshErrorRateHigh
        expr: |
          sum(rate(istio_requests_total{response_code=~"5.."}[5m]))
            / clamp_min(sum(rate(istio_requests_total[5m])), 0.001) > 0.01
        for: 10m
        labels: {severity: critical}
        annotations: {summary: "网格内 5xx 比例 {{ $value | humanizePercentage }}"}

      - alert: IstioPlaintextTraffic
        expr: sum(rate(istio_requests_total{connection_security_policy!="mutual_tls"}[5m])) > 0
        for: 30m
        labels: {severity: info}
        annotations: {summary: "仍存在明文流量，STRICT 迁移前置检查"}

      - alert: IstioResponseFlagAbnormal
        expr: |
          sum(rate(istio_requests_total{response_flags!="-"}[5m])) > 0
        for: 10m
        labels: {severity: warning}
        annotations: {summary: "出现响应标志异常（UF/URX/DC 等），检查上游连接与熔断"}

      - alert: IstioScrapeSlow
        expr: |
          histogram_quantile(0.99,
            sum by (le) (rate(scrape_duration_seconds_bucket{job=~".*istio.*"}[5m]))) > 5
        for: 15m
        labels: {severity: warning}
        annotations: {summary: "istio 相关抓取耗时过高，通常意味着指标基数需要裁剪"}

      - alert: IstioGatewayHighLatency
        expr: |
          histogram_quantile(0.99,
            sum by (le) (rate(istio_request_duration_milliseconds_bucket{reporter="destination"}[5m]))) > 1000
        for: 10m
        labels: {severity: warning}
        annotations: {summary: "网格请求 P99 超过 1s"}
```

> 抓取耗时是最实用的"基数爆炸"领先指标：抓取变慢通常先于 Prometheus 内存问题出现。
> 🔗 **关联阅读**：第三篇 · G2. Prometheus 告警规则（可直接粘贴，阈值需实测校准） ｜ 第五篇 · D3. Prometheus 告警规则（可直接粘贴，阈值请实测校准）

E4. 排障工具箱（命令速查）

```
istioctl analyze -n <ns> # 配置静态检查（上线前必跑）
istioctl proxy-status # 各 proxy 是否同步（Stale 要查）
istioctl proxy-config cluster <pod> -o json # 上游
istioctl proxy-config route <pod> -o json # 路由是否包含目标
istioctl proxy-config listener <pod> # 端口与协议
istioctl proxy-config endpoint <pod> --port 8080
kubectl exec <pod> -c istio-proxy -- curl -s localhost:15000/config_dump
kubectl exec <pod> -c istio-proxy -- curl -s localhost:15000/stats | grep -E "upstream_rq|outlier|ssl"
kubectl logs <pod> -c istio-proxy --tail=200
kubectl logs -n istio-system deploy/istiod --tail=200
```

Part F · 演练（GameDay）计划表

场景	注入方式	期望表现	验证的 SLO
istiod 全部挂掉	缩到 0 副本	存量通信不受影响；**新 Pod 起不来**且告警及时	控制面可用性 + MTTR
istiod 单副本挂	杀一个 Pod	无感知；新 Pod 仍可获取配置	HA 有效性
网关副本滚动	滚动更新 gateway	无连接重置（drain 生效）	优雅终止
误配策略	下发错误 AuthorizationPolicy	立刻看到失败 + 可秒级删除恢复	变更安全
mTLS 收紧	某 ns 切 STRICT	明文流量先归零；切换后无 503	迁移流程
配置推送风暴	批量重建大量 Pod	推送指标可控，无 NACK	控制面容量
指标基数爆炸	放开全部维度	告警/抓取耗时可发现	可观测治理
证书异常	阻断证书轮换	新 Pod 失败可定位到 SDS	证书链路
Envoy OOM	压测大配置 ns	内存增长有告警与兜底	数据面容量
网格升级	新 revision + 单 ns 切换	逐 ns 验证通过后再放量	升级安全性

Part G • 10 张面试速记卡

1. **Istio = 身份为中心的东西向流量与策略平面**；不负责 Pod 连通性（CNI）、不负责服务发现本身、不替代 NetworkPolicy。
2. **架构**：istiod（Pilot + CA）+ Envoy 数据面；Mixer 已移除，遥测靠 Envoy 统计 + Telemetry API。
3. **端口**：15001 出站 / 15006 入站 / 15000 admin / 15020 聚合监控 / 15021 就绪 / 15012 xDS(mTLS) / 15008 HBONE(Ambient)。
4. **注入**：mutating webhook 注入 `istio-init`（iptables，需 NET_ADMIN）或 **istio-cni**（免特权）+ `istio-proxy`；`holdApplicationUntilProxyStarts` 保证顺序。
5. **大规模头号成本 = 全量配置下发** ⇒ **Sidecar CR + discoverySelectors + exportTo** 三件套收敛。
6. **CR 分工**：Gateway 开门、VirtualService 路由、DestinationRule 目标策略与 subset、ServiceEntry 网格外服务、Sidecar 收敛可见性、**EnvoyFilter 是最后手段**。
7. **安全三件套**：SPIFFE 身份 + PeerAuthentication(mTLS) + AuthorizationPolicy；**RequestAuthentication 只验证不强制**，必须配授权才拒绝无 token。
8. **mTLS 迁移**：PERMISSIVE → 证明明文归零 → 逐 ns STRICT；保留 portLevel 例外应对明文探测。
9. **Ambient**：ztunnel(L4 节点级) + waypoint(L7 按需) + HBONE；最大收益是**升级网格不用重启业务 Pod**，限制是功能等价性与成熟度要逐项验证。
10. **可观测成本**：`istio_requests_total` 基数易爆炸 ⇒ Telemetry 裁剪 + 采样 + Prometheus relabel；**所有网格策略都要能一键回退**（CR/label 表达优先）。

第十一篇 · 排障实战 60 例

> 现象 → 证据 → 止血 → 根治 → 防复发

0. 怎么用这本手册

0.1 万能三步（任何故障，先做这三步再往下翻）

```
第 1 步（30 秒）：看"对象自己怎么说"
    kubectl get pod/svc/pvc/node -o wide
    kubectl describe <obj> # Events 是最高性价比的证据
    kubectl logs <pod> --previous # CrashLoop 场景必看

第 2 步（3 分钟）：判断"卡在哪一层"
    调度层（Pending / FailedScheduling）→ 第 1 章
    容器层（ContainerCreating / CrashLoop / OOM）→ 第 2 章
    节点层（NotReady / PLEG / 驱逐）→ 第 3 章
    网络层（DNS / 不通 / 超时）→ 第 4 章
    控制面（kubectl 卡 / 写入失败）→ 第 5 章
    存储层（PVC Pending / FailedMount）→ 第 6 章
    GPU 层（训练 hang / MFU 低 / 推理抖动）→ 第 7 章

第 3 步（10 分钟）：取"权威证据"而不是猜
    调度：scheduler_pending_pods / schedule_attempts_total / 事件
    容器：crictl ps、容器退出码、cgroup memory.events、cpu.stat
    节点：kubelet 日志、PLEG relist 指标、dmesg
    网络：nsenter/NodeLocal/conntrack 计数、CoreDNS 指标
    控制面：apiserver_request_duration_seconds、etcd fsync/leader 指标
    存储：storage_operation_duration_seconds、kubelet_volume_stats_*
    GPU：DCGM 指标、XID、NCCL_DEBUG、nccl-tests busbw
```

0.2 止血的优先级（先别忙着"根治"）

① 止损：停止继续产生损失（停止发布、回滚、隔离坏节点、摘流量）
② 恢复：让业务可用（扩容、切流、重启、重试、从备份/快照恢复）
③ 保留现场：日志/快照/事件/指标截图 — 这一步最容易被忘，但决定你能不能找到根因
④ 再谈根治：改配置、改代码、改架构

0.3 现象关键词速查表

关键词/现象	案例
Pending 没有任何 Event	P01
insufficient cpu/memory	P02
untolerated taint	P03
node affinity / selector 不匹配	P04
anti-affinity / topology spread 不满足	P05
exceeded quota（403 forbidden）	P06
unbound PersistentVolumeClaims	P07
preemption / 抢占 victims	P08
ContainerCreating 卡住	P09 / P10 / P11
ImagePullBackOff	P12
CrashLoopBackOff	P13
OOMKilled / 137	P14
优雅终止超时 / 143	P15
探针失败、不 Ready	P16
发布窗口 502	P17
Terminating 卡死	P18
kill -TERM 1 没反应 / 僵尸	P19
No space left on device（但空间够）	P20
too many open files	P21
时区错	P22
read-only file system	P23
permission denied（挂载卷）	P24
CPU 不高但 RT 抖动（throttling）	P25
Node NotReady	P26
PLEG is not healthy	P27
kubelet 证书过期	P28
MemoryPressure / DiskPressure 驱逐	P29
节点宕机后 Pod 迟迟不重建	P30
节点磁盘满	P31
inode 耗尽	P32
kubelet 挂 / lease 争抢	P33

关键词/现象	案例
时钟漂移	P34
Pod 全部 Unknown	P35
DNS 间歇 5 秒超时	P36
DNS 慢 / QPS 高	P37
CoreDNS SERVFAIL / 不可用	P38
Service 有 endpoints 却不通	P39
conntrack table full	P40
kube-proxy 规则同步慢	P41
IP 耗尽 / IPAM	P42
大包失败 / MTU	P43
跨可用区不通 / 延迟高	P44
加 NetworkPolicy 后 DNS 挂	P45
源 IP 丢失 / LB 不均	P46
kubectl 卡 / apiserver 慢	P47
写操作全失败（etcd NOSPACE）	P48
etcd 慢 / 频繁切主	P49
409 Conflict	P50
too old resource version	P51
webhook 阻断全集群	P52
APF 拒绝请求	P53
控制面证书过期	P54
PVC 一直 Pending	P55
FailedAttachVolume / FailedMount	P56
卷挂载十几分钟	P57
PV/PVC 卡 Terminating / 孤儿盘	P58
训练 hang 在 NCCL	P58
MFU 低 / 训练变慢 / straggler / 坏卡	P59
推理冷启动慢 / 副本负载不均 / KV cache 打满	P60

第 1 章 • 调度与准入（P01–P08）

P01 • Pod 一直 Pending，且完全没有 Event

项	内容
现象	`kubectl describe pod` 的 Events 是空的，Pod 停在 Pending
30 秒验证	`kubectl get pod -o yaml ``| grep schedulerName`，再查是否真有该调度器在跑：`kubectl -n kube-system get lease <name>`
常见根因	① `spec.schedulerName` 拼错或无对应调度器运行（**最高频**）；② 调度器进程挂了 / lease 争抢失败；③ Pod 未进入任何调度器的 informer（选择器/字段问题）
止血	有对应调度器就重启它；调度器名写错只能**删除重建 Pod**（`schedulerName` 不可变）
根治	用 admission/校验 webhook 校验 schedulerName 白名单；CI 模板固定调度器名；调度器多副本 + lease 告警
防复发	巡检脚本：Pending 且 0 Event 的 Pod 列表；`absent(up{job="kube-scheduler"})` 告警
关联	【Scheduler 篇 · L5 排障路径】【Scheduler 篇 · 多 profile】

P02 • `0/N nodes are available: N Insufficient cpu/memory`

项	内容
现象	FailedScheduling，资源不足
30 秒验证	`kubectl describe node ``| grep -A8 Allocatable`；对比 `kubectl top node`
常见根因	① requests 虚高（申请远大于实际用量）；② 节点规格/数量不足；③ 大规格 Pod（如 32C）在碎片化节点上放不下；④ DaemonSet/系统预留吃掉一部分
止血	扩容节点池；临时降低非关键副本数；把大 Pod 调度到专用大机型池
根治	requests 校准（用实测量画像驱动）；装箱策略（离线用 MostAllocated）；节点池分层；Descheduler 整理碎片
防复发	资源分配率告警 + 单规格可调度性巡检
关联	【Scheduler 篇 · 打分与装箱】【Kubelet 篇 · 预留】

P03 • `node(s) had untolerated taint`

项	内容
现象	FailedScheduling，污点不容忍
30 秒验证	`kubectl get node -o json ``| jq '.items[].spec.taints'`；`kubectl describe node ``| grep -i taint`
常见根因	① 专用节点池（GPU/在线/离线）没配 tolerations；② 节点被自动打了压力污点（MemoryPressure 等）；③ 节点 cordon（`node.kubernetes.io/unschedulable`）；④ 云/CNI 初始化污点未移除
止血	给 Pod 加 tolerations，或把 Pod 调度到正确池；若是压力污点 ⇒ 先解决节点压力
根治	节点池 + tolerations 纳入模板管理；对压力污点建告警（这是"节点快不行了"的信号）
防复发	`kube_node_status_condition{condition=~"MemoryPressure`
关联	【Scheduler 篇 · 污点容忍】【Kubelet 篇 · 驱逐】

P04 • `didn't match Pod's node affinity/selector`

项	内容
现象	FailedScheduling，亲和/标签不匹配
30 秒验证	`kubectl get node --show-labels ``| grep <key>`；核对 Pod 的 `nodeSelector/nodeAffinity`
常见根因	① 标签拼写/值不一致；② 目标节点池已被缩掉或改了标签；③ 用了 NFD/GFD 标签但节点上没打；④ 硬性 `requiredDuringScheduling` 太严格
止血	修正标签或改为软亲和（`preferredDuringScheduling`）让服务先起来
根治	标签治理（NFD/节点池模板统一）；关键亲和条件要有多可用区兜底
防复发	对"因亲和无法满足而 Pending 的 Pod 数"做监控
关联	【Scheduler 篇 · 亲和三兄弟】【GPU 篇 · NFD/GFD】

P05 • 反亲和 / TopologySpread 不满足导致调度不出

项	内容
现象	FailedScheduling：`didn't match pod anti-affinity rules`，或副本数上不去
30 秒验证	统计每个节点上该应用的副本分布：`kubectl get pod -l app=X -o wide`
常见根因	① 打散维度太严（hostname 级 + 副本数 > 节点数）；② `whenUnsatisfiable: DoNotSchedule` 且域不足；③ 与资源不足叠加
止血	放宽为 `ScheduleAnyway`；扩容节点；临时减少副本
根治	用 `topologySpreadConstraints` 替代 podAntiAffinity；`maxSkew` + `minDomains` 设计合理；打散维度与节点数匹配
防复发	对"因 spread 无法调度"的计数与副本就绪率告警
关联	【Scheduler 篇 · 打散用 TopologySpread】【Scheduler 篇 · 大规模亲和成本】

P06 • `forbidden: exceeded quota`

项	内容
现象	Pod 创建直接被拒，甚至不会进入调度队列
30 秒验证	`kubectl get resourcequota -n X -o yaml`；看 `status.used`
常见根因	① 命名空间配额打满（cpu/memory/pods/gpu/storage）；② 上层控制器不断重试导致大量失败事件；③ LimitRange 缺失导致 requests 被默认放大
止血	调整配额或清理无用对象（旧 ReplicaSet、失败 Job、孤儿 PVC）
根治	配额 + LimitRange 默认值 + 容量规划；对 GPU 用 Kueue 做队列与配额
防复发	配额使用率告警（≥80%）；失败事件监控
关联	【控制面篇 · ResourceQuota controller】【GPU 篇 · Kueue】

P07 • `pod has unbound PersistentVolumeClaims` / `volume node affinity conflict`

项	内容
现象	Pod Pending，提示卷未绑定或卷拓扑不匹配
30 秒验证	`kubectl get pvc -n X`； `kubectl describe pvc` （看事件与 selected-node）
常见根因	① StorageClass 用 `Immediate` ⇒ PV 落在别的可用区；② `allowedTopologies` 过严；③ provisioner/后端配额不足；④ CSI 驱动 CSINode 未注册
止血	若是 AZ 问题：删除 PVC 让它在正确 AZ 重新供给；或把 Pod 调度到 PV 所在 AZ
根治	**云盘类 StorageClass 一律** `WaitForFirstConsumer`；拓扑与副本分布一起设计
防复发	PVC Pending 时长告警；StorageClass 基线校验
关联	【存储篇 · WaitForFirstConsumer 与拓扑】【调度器篇 · VolumeBinding】

P08 • 抢占频繁 / 在线服务被挤掉

项	内容
现象	在线 Pod 被驱逐后重建，Events 显示 Preempted；或 `scheduler_preemption_victims` 增长
30 秒验证	`kubectl get priorityclass`；查看被驱逐 Pod 的 priority 与抢占者
常见根因	① 高优先级任务过多且允许抢占；② 资源不足才"需要抢"；③ 默认 PriorityClass 抢占策略过激
止血	暂停批处理发布；给在线服务配 PDB
根治	**默认 PriorityClass 设** `preemptionPolicy: Never`；在线/离线节点池隔离；离线用队列配额而不是抢占
防复发	`scheduler_preemption_attempts_total` 告警（>0 就应 Review）
关联	【Scheduler 篇 · 抢占全流程】【Scheduler 篇 · 反模式 C3】

第 2 章 • Pod 与容器（P09–P25）

P09 • ContainerCreating 卡住：镜像拉取慢 / 失败

项	内容
现象	Pod 长时间 ContainerCreating，Events 有 Pulling / Failed to pull image
30 秒验证	`kubectl describe pod`（看 Events 的 image 与 reason）；节点上 `crictl images ``| grep <img>`
常见根因	① 镜像大 + 串行拉取（`serializeImagePulls` 默认 true）；② 仓库限流/认证失败；③ 跨区域拉取；④ 节点首次拉取无缓存
止血	临时降低并发发布批次；确认凭据；必要时手动 `crictl pull`
根治	`serializeImagePulls: false` + 仓库限流参数；P2P/预热；镜像瘦身；就近仓库
防复发	`kubelet_runtime_operations_duration_seconds{operation_type="pull_image"}` P99 告警
关联	【Kubelet 篇 · L3 镜像拉取】

P10 • ContainerCreating 卡住：Sandbox / CNI 失败

项	内容
现象	`FailedCreatePodSandBox`，Pod 拿不到 IP
30 秒验证	`kubectl describe pod`； `kubectl -n kube-system get pod -l k8s-app=calico-node` 等；节点上 `nsenter -t <pid> -n ip a`
常见根因	① CNI DaemonSet 在该节点未就绪；② IPAM 地址耗尽；③ CNI 配置错误（`Failed to find plugin ... in /opt/cni/bin`）；④ 节点刚扩容、网络未初始化
止血	重启 CNI Pod；检查 IP 池；必要时 cordon 该节点
根治	CNI 健康与就绪纳入节点上线校验；IP 容量规划；新节点用 `network-unavailable` 污点防呆
防复发	`kube_node_status_condition{condition="NetworkUnavailable"}` 告警 + IP 池水位告警
关联	【网络篇 · CNI/IPAM】【网络篇 · L7】

P11 • ContainerCreating 卡住：卷挂载（FailedMount / FailedAttach）

> 与 P56/P57 同源，区别：这里从"Pod 视角"快速判断归属。

项	内容
现象	Events 出现 `FailedAttachVolume`（控制面/云 API）或 `FailedMount`（kubelet/CSI 节点侧）
30 秒验证	看事件字段名即可分界：`FailedAttach*` = attach 层；`FailedMount` = mount 层
止血	attach 层：检查云配额/限流、AZ 一致性；mount 层：检查 node plugin 与文件系统
根治	按【存储篇 B18】的现象→原因表处理；限制单节点卷数；分批重建
防复发	`storage_operation_duration_seconds` 与事件转指标告警
关联	【存储篇 · 故障速查】【存储篇 · C3】

P12 • ImagePullBackOff

项	内容
现象	Pod 反复重试拉镜像
30 秒验证	`kubectl describe pod` 看具体 reason：`toomanyrequests`（限流）、`manifest unknown`（tag 不存在）、`no basic auth credentials`（凭据）
常见根因	① imagePullSecrets 缺失/失效；② 私有仓库证书/域名不通；③ tag 写错；④ 仓库限流
止血	修正 secret 或 tag；必要时用已缓存节点
根治	secret 统一注入（ServiceAccount imagePullSecrets）；镜像仓库鉴权与配额治理；关键镜像预热
防复发	事件转指标：ImagePullBackOff 计数告警
关联	【Kubelet 篇 · 镜像 GC 与拉取】

P13 • CrashLoopBackOff

项	内容
现象	容器反复重启，退避 10s→20s→…→300s
30 秒验证	`kubectl logs <pod> --previous`； `kubectl get pod -o jsonpath='{.status.containerStatuses[0].lastState.terminated.exitCode}'`
常见根因	① 应用异常退出（配置/依赖/证书）；② 探针过严（还没起来就被杀）；③ OOM（137，见 P14）；④ 依赖未就绪
止血	回滚版本；改探针；临时加 initContainer 等待依赖
根治	加 `startupProbe`；探针只查自身活性；依赖就绪检查前置；退出码语义纳入监控
防复发	重启次数告警 + 按 Deployment 聚合
关联	【Kubelet 篇 · Q9 退避】【Kubelet 篇 · Q10 探针】

P14 • OOMKilled（退出码 137）

项	内容
现象	容器被杀，`reason=OOMKilled`
30 秒验证	`kubectl describe pod`；容器内 `cat /sys/fs/cgroup/memory.events`（看 oom 计数）；宿主 `dmesg -T | grep -i "out of memory"`
常见根因	① memory limit 小于真实峰值；② **page cache 计入 cgroup**（读写文件暴涨）；③ 节点内存压力导致全局 OOM 选中；④ 应用按宿主内存算参数（JVM 堆）
止血	提高 limit；临时降低并发/批大小
根治	用实测峰值（含 cache）设 limit；容器感知 cgroup；QoS=Guaranteed；给大量 IO 临时目录单独卷
防复发	`kube_pod_container_status_terminated_reason{reason="OOMKilled"}` 告警；working_set/limit 水位告警
关联	【Linux 篇 · A10 memory cgroup】【Kubelet 篇 · QoS】

P15 • 优雅终止超时（143 / 30 秒后被 SIGKILL）

项	内容
现象	Pod 删除慢，容器退出码 143；日志显示被硬杀
30 秒验证	`kubectl get pod -o yaml`（`terminationGracePeriodSeconds`）；看应用是否处理 SIGTERM
常见根因	① 应用不处理 SIGTERM；② `preStop` 执行时间过长或卡住；③ grace 太短；④ 卷卸载/storage 慢
止血	增大 `terminationGracePeriodSeconds`；修复 preStop
根治	应用处理 SIGTERM；preStop 用于等待 endpoint 传播；配 PDB 与滚动参数
防复发	发布窗口错误率/502 监控；优雅终止时长指标
关联	【Kubelet 篇 · Q11 优雅终止】【Linux 篇 · A3 PID1】

P16 • 探针失败：不 Ready / 被反复重启

项	内容
现象	Pod 不进入 Ready；或 liveness 失败导致重启
30 秒验证	`kubectl describe pod` 看 Readiness/Liveness 失败信息；`kubectl get endpoints`
常见根因	① 探针路径/端口错；② 阈值太严（`failureThreshold × periodSeconds` 小于应用启动时间）；③ liveness 检查了外部依赖（DB/下游）⇒ 依赖抖动引发雪崩重启
止血	放宽阈值；把依赖检查从 liveness 挪到 readiness
根治	liveness 只查自身活性；慢启动用 startupProbe；配 `minReadySeconds`
防复发	探针失败率与重启率告警
关联	【Kubelet 篇 · Q10 探针铁律】

P17 • 滚动发布 / 缩容瞬间出现 502

项	内容
现象	发布窗口错误率尖峰，连接重置
30 秒验证	看发布期间 Endpoint 变化时间 vs Pod 终止时间；抓一条失败请求看打到了谁
常见根因	① 进程已停但 endpoints/LB/Ingress 传播未完成；② preStop 与 grace 太短；③ 长连接未优雅关闭；④ 客户端无重试
止血	回滚或暂停发布；增大 grace
根治	preStop sleep + 放宽 grace + 应用处理 SIGTERM；`maxUnavailable: 0` + PDB；网关侧做优雅摘流
防复发	发布期间 SLO 守护（自动暂停发布）
关联	【Kubelet 篇 · Q11】【网络篇 · 优雅下线】

P18 • Pod 卡在 Terminating

项	内容
现象	Pod 一直 Terminating，删不掉
30 秒验证	`kubectl get pod -o yaml` 看 `metadata.finalizers` 与 `deletionTimestamp`；容器是否还在运行
常见根因	① **Finalizer 未清除**（控制器挂了/自定义 finalizer）；② preStop 卡住；③ 卷无法卸载（存储 hang）；④ 容器进程 D 状态
止血	恢复对应控制器；修复卡住的 preStop；若卷 hang ⇒ 先 cordon 节点评估
根治	自定义 finalizer 必须有超时/自愈；卷卸载失败要有告警；避免依赖阻塞式 preStop
防复发	"Terminating 超过 5 分钟"的 Pod 巡检
关联	【Linux 篇 · B10 D 状态】【存储篇 · C3】

P19 • 容器内 `kill -TERM 1` 无反应 / 僵尸进程堆积

项	内容
现象	发信号没效果；`ps` 里出现 `<defunct>`
30 秒验证	`kubectl exec` 看 PID 1 是什么（`ps -p 1 -o comm=`）；`ps -ef | grep defunct | wc -l`
常见根因	① PID 1 是 shell（信号不转发、不 reap 僵尸）；② 应用未注册 handler 且是 PID 1（终止信号被内核忽略）
止血	重启 Pod；必要时用 `kubectl exec` 内部 kill 具体子进程

项	内容
根治	**exec 形式入口**或 `dumb-init/tini`；镜像规范检查
防复发	镜像 CI 校验入口形式；僵尸数量监控
关联	【Linux 篇 · A3 PID namespace】

P20 • `No space left on device` 但 `df -h` 有空间

项	内容
现象	写入失败，但容量看起来够
30 秒验证	`df -i`（inode）； `df -h /dev/shm`（共享内存，默认常很小）
常见根因	① **inode 耗尽**（overlay 层 + 海量小文件）；② `/dev/shm` 太小（PyTorch/Chrome/Postgres 常见）；③ 删除的文件仍被进程持有（`lsof ``| grep deleted`）
止血	清理文件；重启持有 fd 的进程；临时扩 shm
根治	emptyDir(Memory) 挂载 `/dev/shm` 并同步提高内存 limit；日志轮转与清理；inode 监控
防复发	`node_filesystem_files_free / node_filesystem_files < 0.1` 告警
关联	【Linux 篇 · A18 overlayfs / A19 shm】

P21 • `too many open files`

项	内容
现象	应用报错；或 kubelet/agent 报同样错误导致挂载失败
30 秒验证	`cat /proc/<pid>/limits ``| grep "open files"`； `ls /proc/<pid>/fd ``| wc -l`； `cat /proc/sys/fs/file-nr`
常见根因	① 进程 fd 泄漏；② 系统 `fs.file-max` 太小；③ **inotify watches/instances 耗尽**（常被误报成这个）
止血	重启泄漏进程；临时调大限制
根治	修复泄漏；调 `fs.file-max` 与 inotify 参数；Secret 用 `immutable: true` 减少 watch
防复发	fd 使用率告警；inotify 用量纳入监控
关联	【Linux 篇 · A20 ulimit / C5 inotify】

P22 • 容器时区/时间不对

项	内容
现象	日志时间差 8 小时；定时任务跑错点
30 秒验证	`kubectl exec -- date`；看 `/etc/localtime` 是否挂载；宿主 `timedatectl`
常见根因	① 镜像未设 TZ 且未挂 `/etc/localtime`；② 节点时钟漂移（chrony 未同步）
止血	挂载 hostsPath localtime 或设置 TZ 环境变量
根治	基础镜像统一时区；节点 NTP/chrony 标准化并监控
防复发	`abs(node_timex_offset_seconds) > 0.05` 告警
关联	【Linux 篇 · A25 时钟】

P23 • `Read-only file system`（容器内写不了）

项	内容
现象	应用写临时文件失败
30 秒验证	`kubectl get pod -o yaml ``| grep readOnlyRootFilesystem`；看挂载类型
常见根因	① 安全加固开启 `readOnlyRootFilesystem: true`；② 文件系统进入只读（磁盘错误）；③ 挂载卷为只读
止血	给需要写的目录挂 emptyDir（**不要为了省事关掉 readOnlyRootFilesystem**）
根治	明确"临时目录/缓存目录"清单并挂载 writable 卷；应用配置指向这些目录
防复发	镜像规范 + 安全基线校验
关联	【Linux 篇 · B3 加固】【Linux 篇 · B4 #15】

P24 • 挂载卷后 `permission denied`

项	内容
现象	容器无法写入挂载的卷
30 秒验证	`kubectl exec -- ls -ld /data`；宿主上看卷目录属主；**注意不要直接 chmod 777**
常见根因	① `fsGroup` 未配；② fsGroup 递归 chown 未完成/未配 `OnRootMismatch`；③ SELinux 标签不对；④ NFS/文件存储导出选项不匹配
止血	配 `securityContext.fsGroup`；核对导出选项
根治	统一 fsGroup 策略 + `fsGroupChangePolicy: OnRootMismatch`；SELinux 正确 relabel
防复发	挂载耗时监控（大卷 chown 会拖慢启动）
关联	【存储篇 · A19 fsGroup】【Linux 篇 · A17 SELinux】

P25 • CPU 使用率不高但 RT/P99 抖动（CFS 限流）

项	内容
现象	平均 CPU 只有 30%~50%，但延迟毛刺明显
30 秒验证	cgroup：`cat /sys/fs/cgroup/.../cpu.stat`（`nr_throttled` / `throttled_usec`）；Prometheus：`container_cpu_cfs_throttled_periods_total / container_cpu_cfs_periods_total`
常见根因	CPU limit 设置过小（在 100ms 周期里早早用完配额）；突发流量；GC/批处理叠加
止血	临时放大 limit
根治	延迟敏感服务**去掉 CPU limit 或放宽到 request 的 2~4 倍**；用节点配额与隔离兜底
防复发	throttled 占比 >10% 告警（**这是最容易漏的指标**）
关联	【Linux 篇 · A8 CFS throttling】

第 3 章 • 节点（P26–P35）

P26 • Node NotReady

项	内容
现象	节点 Ready=False/Unknown，Pod 开始被驱逐或重排
30 秒验证	`kubectl describe node`（Conditions 与 Events）；节点上 `systemctl status kubelet`； `journalctl -u kubelet -n 200`
常见根因	① kubelet 挂了/无法连 apiserver（证书、网络、LB）；② **PLEG 不健康**（见 P27）；③ 节点资源耗尽（内存/磁盘/PID）；④ 时钟漂移导致证书无效；⑤ 真的宕机/网络断开
止血	若节点可恢复：修 kubelet/磁盘/时钟；不可恢复：cordon + 等待 Pod 重排（**默认约 5~6 分钟**，见 P30）
根治	心跳与 lease 监控；资源预留与驱逐阈值；证书/时钟治理；关键服务跨区打散
防复发	`kube_node_status_condition{condition="Ready",status="false"}` 告警 + kubelet `up` 告警
关联	【Kubelet 篇 · Q16 时间账】【控制面篇 · C4】

P27 • `PLEG is not healthy: pleg was last seen active 3m ago`

项	内容
现象	节点上所有 Pod 集体异常（ContainerCreating/Unknown），节点 NotReady
30 秒验证	`journalctl -u kubelet ``| grep -i pleg`；指标 `kubelet_pleg_relist_duration_seconds`； `crictl ps ``| wc -l`
常见根因	① 容器密度过高导致 relist 慢；② runtime（containerd）卡顿（磁盘 IO、shim 泄漏）；③ 节点 CPU 被抢（kubelet 自己没保障）
止血	cordon 节点；必要时重启 kubelet/containerd（**会中断该节点新操作**）；严重时重启节点
根治	降 Pod 密度；kube/system reserved 与 cgroup 真正生效；治理 runtime；较新版本评估 Evented PLEG
防复发	relist P99 > 5s 告警（**早于 NotReady 触发**）
关联	【Kubelet 篇 · Q4 PLEG / L2】

P28 • kubelet 证书过期 / 节点无法注册

项	内容
现象	kubelet 日志大量 TLS/x509 错误；节点不注册或 status 不更新
30 秒验证	`journalctl -u kubelet ``| grep -i "certificate``|x509"`； `openssl x509 -in /var/lib/kubelet/pki/kubelet-client-current.pem -noout -dates`
常见根因	① 客户端证书轮换未生效；② `serverTLSBootstrap` 开启但 CSR 无人批准；③ 节点时钟漂移；④ CA 轮换后未同步
止血	手动批准 CSR 或重新 bootstrap 节点（必要时 drain 后重建）
根治	启用 `rotateCertificates`；确保 CSR 审批通道可用；时钟同步；证书有效期告警
防复发	证书剩余有效期 \< 30 天告警（**这条告警能救一次 P0**）
关联	【Kubelet 篇 · Q20 证书】【控制面篇 · B9】

P29 • MemoryPressure / DiskPressure 导致 Pod 被驱逐

项	内容
现象	节点 condition 出现压力，Pod 状态 Evicted
30 秒验证	`kubectl describe node ``| grep -i pressure`； `kubectl get pod -A ``| grep Evicted`； `dmesg` / 磁盘 `df -h`、`df -i`
常见根因	① requests 虚低导致严重超卖；② 日志/临时文件暴涨；③ 镜像堆积；④ `evictionHard.memory.available` 默认 100Mi 触发太晚
止血	清理日志/镜像；扩容节点；把离线任务挪走
根治	requests 校准；预留 + cgroup 生效；调大阈值 + 软驱逐 + 优雅期；日志轮转与 GC
防复发	压力 condition 告警 + Evicted 增长告警 + 磁盘/inode 水位告警
关联	【Kubelet 篇 · Q12 驱逐 / L5 风暴】【Linux 篇 · C8 磁盘】

P30 • 节点宕机后，Pod 迟迟不重建（约 5~6 分钟）

项	内容
现象	节点挂了，业务副本很久才在其它节点起来
30 秒验证	算一遍：lease 10s → `node-monitor-grace-period` 50s → taint → Pod 默认 `tolerationSeconds=300`
常见根因	**这是设计行为，不是故障**；若有状态服务被误驱逐则是容忍配置过短
止血	想快：显式设更短的 tolerationSeconds（无状态 30~60s）
根治	无状态服务快恢复、有状态服务放宽容忍；配合 PDB 与副本跨区分布
防复发	明确写入容量方案："节点故障恢复时间 ≈ 5~6 分钟（默认）"，并据此设定副本数与超时
关联	【Kubelet 篇 · Q16】【控制面篇 · C4】【Scheduler 篇 · 排障】

P31 • 节点磁盘被打满

项	内容
现象	DiskPressure、Pod 被逐、kubelet/containerd 异常

项	内容
30 秒验证	`df -h`、`du -sh /var/lib/containerd /var/lib/kubelet /var/log/pods`；`lsof | grep deleted`
常见根因	① 容器日志无轮转；② 镜像堆积（GC 阈值/频率）；③ 已删除文件被进程持有；④ 业务写 emptyDir/宿主目录
止血	触发镜像 GC；清理日志；重启持有 fd 的进程
根治	日志轮转参数 + 采集侧限流；imagefs 与 nodefs 分盘；ephemeral-storage limits
防复发	分盘容量告警（imagefs/nodefs 分别）+ inode 告警
关联	【Kubelet 篇 · Q18 GC】【Linux 篇 · A18 overlayfs】

P32 • inode 耗尽

项	内容
现象	报 No space left on device，但容量充足；新建文件失败
30 秒验证	`df -i`；`find /var/lib/containerd -xdev -type f | wc -l`
常见根因	镜像层多 + 海量小文件；overlay 白出/合并导致 inode 增长
止血	清理镜像与无用层；迁移部分 Pod
根治	镜像瘦身与层数收敛；小文件打包；inode 容量纳入机型选型
防复发	inode 使用率告警
关联	【Linux 篇 · A18】【存储篇 · D12】

P33 • kubelet 进程挂掉 / 调度器 lease 争抢失败导致调度暂停

项	内容
现象	新 Pod 全 Pending；或节点状态停止更新
30 秒验证	`kubectl -n kube-system get lease kube-scheduler kube-controller-manager`；进程状态 `systemctl status kubelet`
常见根因	① 进程 OOM/被驱逐；② leader election 抖动（控制面压力、网络）；③ 配置错误导致启动失败
止血	重启进程；确认只有一个 leader；必要时固定到健康实例
根治	给控制面组件保障资源与 APF 优先级；lease 变更告警；配置变更走灰度
防复发	`up{job="kube-scheduler"}`、lease 更新停滞告警、pending_pods 堆积告警
关联	【Scheduler 篇 · Q16 HA】【控制面篇 · C3】

P34 • 节点时钟漂移

项	内容
现象	证书校验失败、日志错序、Lease 异常、分布式组件报错
30 秒验证	`timedatectl`；`chronyc sources` / `ntpq -p`；`node_timex_offset_seconds`
常见根因	chrony/ntp 未配置或不可达；虚拟机时钟源不稳定；容器共享宿主时钟
止血	手动同步（谨慎，避免时间跳变影响有状态服务）；必要时重启 chrony
根治	节点镜像固化 NTP 配置；监控偏移；避免使用会导致跳变的同步方式
防复发	`abs(node_timex_offset_seconds) > 0.05` 告警
关联	【Linux 篇 · A25】【控制面篇 · 证书】

P35 • 节点上所有 Pod 变成 Unknown

项	内容
现象	Pod status.phase=Unknown，节点可能 NotReady
30 秒验证	Node Ready 状态 + kubelet 是否上报；`kubectl get pod -o wide` 看是否集中在一个节点
常见根因	节点失联（kubelet 挂/网络断/PLEG 崩），apiserver 保留 Pod 直到驱逐完成
止血	恢复节点；若节点不可恢复 ⇒ 走"强制移除 Pod"流程（**有状态服务必须先确认卷已释放**）
根治	见 P26/P27；有状态服务要配 PDB 与卷释放流程
防复发	节点健康 + Pod 未知状态巡检
关联	【Kubelet 篇 · Q16】【存储篇 · Multi-Attach】

第 4 章 • 网络（P36–P46）

P36 • DNS 间歇性 5 秒超时（经典）

项	内容
现象	偶发请求卡 5 秒后失败，概率低但持续存在
30 秒验证	CoreDNS 侧是否收到请求（对比发起侧与服务端计数）；节点 `conntrack -S` 看 `insert_failed`
常见根因	**内核 conntrack + UDP NAT 竞态** ⇒ 包被丢弃；UDP 不重传 + resolver 超时 5s；search 域多、并发查询放大冲突概率
止血	应用侧改用 TCP 或加重试；临时降低并发 DNS 查询
根治	**NodeLocal DNSCache**（本地 link-local 缓存，绕开 conntrack 竞态）；FQDN 写法 + 降 ndots；必要时升级内核
防复发	CoreDNS SERVFAIL 与 P99 告警；NodeLocal 缓存命中率看板
关联	【网络篇 · A8 5 秒超时】【网络篇 · C3 DNS 优化】

P37 • DNS 慢 / CoreDNS QPS 奇高

项	内容
现象	解析延迟上升；CoreDNS CPU 高
30 秒验证	看 Pod 内 `/etc/resolv.conf`（`ndots:5` 与 search 域）；`coredns_dns_requests_total`
常见根因	`ndots:5` 导致外部域名被放大 4~10 倍查询；副本数不足；缓存未命中率高
止血	CoreDNS 扩容
根治	NodeLocal DNSCache（**同时解决延迟与 QPS**）；外部域名写 FQDN；按需调 ndots；autoscaler
防复发	QPS/延迟/缓存命中率告警
关联	【网络篇 · A7 / C3】

P38 • CoreDNS 不可用 / SERVFAIL 飙升

项	内容
现象	全集群解析失败
30 秒验证	`kubectl -n kube-system get pod -l k8s-app=kube-dns`；从 Pod 内 `nslookup kubernetes.default`
常见根因	① CoreDNS Pod 被驱逐/资源不足；② 上游 forward 不可达；③ 配置错误（Corefile）；④ 节点网络问题
止血	恢复 CoreDNS 副本；临时切换上游
根治	多副本跨区 + PDB + 资源保障；NodeLocal 缓存兜底；配置变更灰度
防复发	`up{job="coredns"}`、SERVFAIL 率、解析成功率合成探测
关联	【网络篇 · C3】【网络篇 · E3】

P39 • Service 有 endpoints，但访问不通

项	内容
现象	`kubectl get endpointslices` 有后端，但 curl ClusterIP 失败
30 秒验证	① 直接 curl PodIP（排除 Service 层）② 另一节点 curl PodIP（排除 CNI 跨节点）③ 看规则：`iptables-save ``| grep <clusterIP>` 或 `cilium service list`
常见根因	① kube-proxy 未同步（进程挂/规则不同步）；② CNI 跨节点不通；③ NetworkPolicy 拦截；④ 目标端口/协议错；⑤ hairpin 未开（访问自身 Service）
止血	重启 kube-proxy；临时绕过（用 PodIP）
根治	按分层逐段验证；监控规则同步与网络策略命中
防复发	`kubeproxy_sync_proxy_rules_last_timestamp_seconds` 停滞告警 + 业务合成探测
关联	【网络篇 · B17 排障套路】【控制面篇 · D5 kube-proxy】

P40 • `nf_conntrack: table full, dropping packet`

项	内容
现象	新建连接失败/超时，短连接高峰明显
30 秒验证	`cat /proc/sys/net/netfilter/nf_conntrack_count` vs `_max`；`conntrack -S` 看 `insert_failed` / `drop`
常见根因	① 表太小（默认值常不够）；② 短连接并发高 + timeout 过长；③ 大量 SNAT（NodePort/masquerade）
止血	调大 `nf_conntrack_max`（注意 buckets 与内存）；缩短 TIME_WAIT/CLOSE_WAIT timeout
根治	按并发实测规划容量；高并发短连接考虑 eBPF socket-LB 路径（绕开 conntrack）
防复发	使用率 > 75% 告警 + insert_failed > 0 告警
关联	【网络篇 · A18 / C5】【Linux 篇 · C2】

P41 • kube-proxy 规则同步慢（Service 变更后不生效）

项	内容
现象	Service/Endpoint 改了，但访问行为没变；节点 CPU 尖峰

项	内容
30 秒验证	`kubeproxy_sync_proxy_rules_duration_seconds`； `iptables-save ``| wc -l`； `up{job="kube-proxy"}`
常见根因	① iptables 模式规则数线性膨胀（Service/Endpoint 多）；② `minSyncPeriod` 设置不合理；③ kube-proxy 卡住/挂了
止血	重启 kube-proxy（已有规则不受影响）
根治	切 IPVS 或 Cilium socket-LB；收敛 Service 数量
防复发	同步耗时 P99 告警 + 最后同步时间停滞告警
关联	【控制面篇 · D1–D7】【网络篇 · C2】

P42 • Pod 拿不到 IP / IP 池耗尽

项	内容
现象	`FailedCreatePodSandBox: failed to allocate for range`；或云 CNI 报 VPC IP 不足
30 秒验证	CNI 侧地址池使用情况；节点 Pod 数 vs maxPods；云侧子网剩余 IP
常见根因	① Pod CIDR/子网规划不足；② IPAM 泄漏（Pod 删除未释放）；③ 滚动更新期间新旧并存导致峰值翻倍
止血	回收泄漏 IP；临时减少副本或扩容网段（云上可加 CIDR/前缀委派）
根治	容量规划（预期 Pod 数 × 2）；定期 IPAM 对账；开启前缀委派等提升密度的能力
防复发	地址池水位告警； `kubelet_running_pods / maxPods` 告警
关联	【网络篇 · L7】【Kubelet 篇 · maxPods】

P43 • 大包失败、TLS 握手卡住（MTU 黑洞）

项	内容
现象	ping 通、小请求正常，大文件/大 body 传输卡死
30 秒验证	从 Pod 内 `ping -M do -s <size>` 逐步找临界值；对比宿主与 Pod 的 MTU
常见根因	① 隧道封装（VXLAN/IPIP/Geneve）后 MTU 未同步下调；② PMTUD 被 ICMP 过滤阻断；③ 配置漂移（部分节点不一致）
止血	统一 MTU；开启 MSS clamp
根治	MTU 写入节点池基线；放通必要的 ICMP；变更后复测大包
防复发	大包连通性巡检
关联	【网络篇 · A11 / C10】

P44 • 跨可用区 Pod 不通 / 延迟明显更高

项	内容
现象	同区正常，跨区失败或延迟高
30 秒验证	分别 curl 同区与跨区 PodIP；看路由与 CNI 模式（Overlay vs 路由）
常见根因	① 纯路由模式（host-gw/BGP）跨三层不通；② 安全组/ACL 未放通 Pod CIDR；③ 跨区流量成本与延迟（架构问题）
止血	改为 Overlay 或 CrossSubnet 模式；放通安全组
根治	按底层网络能力选型；用拓扑感知路由降低跨区流量；副本按区打散
防复发	跨区流量占比监控 + 连通性合成探测
关联	【网络篇 · A10 / C9】【网络篇 · B2】

P45 • 加了 default-deny NetworkPolicy 后 DNS 全挂

项	内容
现象	策略上线后所有域名解析失败、健康检查挂
30 秒验证	删除策略是否恢复；看策略是否放行 kube-system 的 53/UDP+TCP
常见根因	default-deny 未放行 DNS 出口与关键探测来源；或 CNI 根本不支持 NetworkPolicy（策略"看起来生效"其实没生效）
止血	立即删除策略
根治	分层策略：先 default-deny，再显式放行 DNS/探测/依赖；确认 CNI 支持策略
防复发	策略变更必须先在灰度 Namespace 验证
关联	【网络篇 · A9】【网络篇 · B14】

P46 • 拿不到客户端源 IP / LB 流量不均

项	内容
现象	应用看到的是节点 IP 或 Pod IP；或流量集中到少数 Pod
30 秒验证	看 Service 的 `externalTrafficPolicy`；看 LB 后端健康检查状态
常见根因	① `externalTrafficPolicy: Cluster` 会做额外 SNAT ⇒ 丢源 IP；② `Local` 模式下副本未按区打散 ⇒ 部分区无后端；③ gRPC/HTTP2 长连接导致连接级不均衡
止血	按需切换 `Local` 并补齐多区副本
根治	源 IP：Local + 多副本打散，或网关注入 header/PROXY protocol；均衡：限制 `MAX_CONCURRENT_STREAMS` 或用 per-request 网关
防复发	各副本请求量/连接数看板

项	内容
关联	【网络篇 · A19】【GPU 篇 · C10 / E4】

第 5 章 • 控制面（P47–P53）

P47 • kubectl 卡住 / API 延迟高 / 偶发 5xx

项	内容
现象	所有操作变慢；控制器大量重试；偶发 timeout
30 秒验证	`apiserver_request_duration_seconds` 按 verb/resource 分解； `apiserver_current_inflight_requests`；客户端侧 `rest_client_request_duration_seconds`
常见根因	① **etcd 慢**（最常见真因）；② 某客户端大量 list（看 `sum by (user,resource)`）；③ APF 排队；④ 准入 webhook 慢；⑤ LB 把流量导到坏副本
止血	找出吵闹邻居并限流；摘掉坏 apiserver；必要时临时提高 inflight
根治	治理大 list（informer + selector + 分页）；APF 给关键组件留额度；webhook 短超时与作用域；etcd 磁盘与容量
防复发	延迟 P99 + 5xx 比例 + APF 拒绝 + inflight 告警
关联	【控制面篇 · A14 / E1】【控制面篇 · E2】

P48 • 写操作全部失败：`mvcc: database space exceeded`（etcd NOSPACE）

项	内容
现象	创建/删除对象失败，但读通常还能进行
30 秒验证	`etcdctl endpoint status --write-out=table --cluster`； `etcdctl alarm list`
常见根因	配额满（未压缩/未 defrag）⇒ 进入只读；Event 或高频更新放大
止血	依次执行： `compact <rev>` → `defrag --cluster`（逐成员）→ `alarm disarm`
根治	自动压缩 + `event-ttl` + 独立 events etcd；调大配额；容量与增长监控
防复发	`etcd_db_total_size_in_bytes / quota > 0.8` 告警
关联	【控制面篇 · B3 / E6】

P49 • etcd 慢 / 频繁切主

项	内容
现象	全集群 API 延迟升高；日志 `apply request took too long`、`heartbeat took too long`
30 秒验证	`etcd_disk_wal_fsync_duration_seconds` P99（建议 \<10ms）、`etcd_disk_backend_commit_duration_seconds`（\<25ms）；`etcd_network_peer_round_trip_time_seconds`； `etcd_server_leader_changes_seen_total`
常见根因	① 磁盘慢/共享盘/节流盘；② 网络 RTT 高（跨地域部署是硬伤）；③ 负载过高（大对象、频繁写）
止血	降低写入量（限流吵闹客户端、清理无用对象）；必要时切换 leader 到健康成员
根治	独立 NVMe；同城低延迟组网；对象体积治理；成员数 3/5 且奇数
防复发	fsync/commit P99 告警 + leader 变更次数告警
关联	【控制面篇 · B4 / B6】

> 🔗 **关联阅读**：第三篇 · B4. etcd 的头号性能指标是什么?

P50 • 409 Conflict（更新冲突）

项	内容
现象	控制器/CI 更新对象时报冲突
30 秒验证	看是否携带 `metadata.resourceVersion`；是否有多个 writer
常见根因	乐观并发：**带上过期的 resourceVersion 更新**；或多个控制器同时写同一对象
止血	重试（重新 Get 再 Update）
根治	客户端使用 client-go 的重试语义；避免多写者；用 patch/apply 而非全量 update
防复发	控制器 409 计数监控
关联	【控制面篇 · A6 乐观锁 / C1 控制器范式】

P51 • `too old resource version` / watch 频繁断开（410）

项	内容
现象	客户端报错 410 Gone，informer 不断 relist
30 秒验证	客户端日志；etcd 压缩情况（revision 保留策略）
常见根因	① watch 的 resourceVersion 已被压缩；② 客户端断线过久后从旧版本续接；③ 压缩保留设置过短
止血	客户端全量 relist（正常行为）；缩短重连间隔
根治	客户端正确处理 410；开启 watch bookmark；压缩保留策略与客户端行为匹配
防复发	客户端 relist 频率与 410 计数监控
关联	【控制面篇 · A6】【控制面篇 · E2】

P52 • 准入 webhook 挂掉导致全集群变更失败

项	内容
现象	所有 create/update 被拒，报 webhook 连接失败
30 秒验证	`kubectl get validatingwebhookconfigurations,mutatingwebhookconfigurations`；看 `failurePolicy` 与后端 Pod
常见根因	① `failurePolicy: Fail` + 后端不可用；② webhook 作用于所有 namespace（含 kube-system）；③ 超时过长拖垮 apiserver
止血	**立即删除/禁用该 webhook 配置**（这是最快的恢复手段）
根治	`namespaceSelector` 排除系统 ns；多副本跨故障域；短超时；后端健康告警
防复发	webhook 耗时与错误告警；变更前的故障演练
关联	【控制面篇 · A4 / F2】

P53 • APF 拒绝请求 / 关键组件被饿死

项	内容
现象	部分请求被拒（429 或排队超时）；kubelet/scheduler 心跳异常
30 秒验证	`apiserver_flowcontrol_rejected_requests_total` 按 priority_level；`apiserver_flowcontrol_current_inqueue_requests`
常见根因	① 未配 APF，总量限流被吵闹邻居占满；② FlowSchema 配错，把关键流量分到低优先级；③ 队列长度不足
止血	修正/删除错误的 FlowSchema；给关键组件临时保障
根治	为 leader-election、kubelet、scheduler、探针建专用优先级；对全集群 list 类请求降优先级
防复发	APF 拒绝率与队列深度告警
关联	【控制面篇 · A7 / E4】

第 6 章 • 存储（P54–P57）

P54 • PVC 一直 Pending / `ProvisioningFailed`

项	内容
现象	PVC 不绑定，Pod 起不来
30 秒验证	`kubectl describe pvc`（事件与 SC 名）； `kubectl -n <csi-ns> logs <csi-provisioner>`； `kubectl get sc` 确认 provisioner 名一致
常见根因	① StorageClass 名拼错或不存在；② provisioner 未运行/选主失败；③ 后端配额/凭据不足；④ 拓扑不满足（AZ）
止血	修正 SC；恢复 provisioner；补配额
根治	CSI Controller 多副本 + leader election + PDB；SC 模板化；后端容量告警
防复发	PVC Pending 时长告警；事件转指标（ProvisioningFailed）
关联	【存储篇 · A6 / B18 / C5】

P55 • `FailedAttachVolume` 与 `FailedMount`

项	内容
现象	Pod 卡 ContainerCreating，卷不可用
30 秒验证	**看事件字段名即可分界**：`FailedAttach*` = attach 层（控制面/云 API）； `FailedMount` = kubelet/CSI 节点侧
常见根因	attach：云配额/限流、AZ 不一致、节点挂载数上限、上次 detach 未完成；mount：node plugin 崩、文件系统损坏、mount 参数错
止血	attach 层：核对 AZ 与配额、等待/清理残留 VolumeAttachment；mount 层：重启 node plugin、检查文件系统
根治	WaitForFirstConsumer + 拓扑；分批重建有状态服务；node plugin 优雅发布
防复发	`storage_operation_duration_seconds` 告警 + 事件转指标
关联	【存储篇 · B18 / C3】

P56 • 卷挂载耗时十几分钟

项	内容
现象	Pod 起来极慢，Events 时间戳显示 mount 阶段耗时巨大
30 秒验证	看 Events 时间戳差； `storage_operation_duration_seconds{operation_name=~"volume_mount``|volume_attach"}`；节点 `journalctl -u kubelet`
常见根因	① **fsGroup 递归 chown**（大卷文件多）；② 上一次 detach 未完成；③ 云 API 限流；④ 文件系统检查/修复；⑤ node plugin 崩
止血	等待或强制清理残留；分批重建
根治	`fsGroupChangePolicy: OnRootMismatch`；规范 detach 流程（先确认 VolumeAttachment 消失）；限流调优 + 分批
防复发	挂载耗时 P99 告警
关联	【存储篇 · A19 fsGroup / C3】【Kubelet 篇 · 卷管理】

P57 • PV/PVC 卡 Terminating / 孤儿磁盘（含扩容不生效）

项	内容
现象	删不掉；或扩容后容器内视图没变；或账单里躺着没人认的磁盘
30 秒验证	`kubectl get pvc -o jsonpath='{.metadata.finalizers}'`； `kubectl get pv -o wide` 与后端磁盘列表对账； 扩容场景看 PVC `status.conditions` 是否 `FileSystemResizePending`
常见根因	① finalizer（`pvc-protection` / `pv-protection`）未释放；② **摘掉 finalizer 强删后后端资产变孤儿**；③ 扩容只完成磁盘未完成文件系统（卷未挂载）； ④ StatefulSet 删除/缩容不删 PVC
止血	释放使用方后删除；扩容场景重新挂载触发 NodeExpand；孤儿盘：对账后走工单清理
根治	PV ↔ 后端磁盘定期双向对账； `reclaimPolicy: Retain` 要有巡检；StatefulSet 生命周期流程化；标签与成本归因
防复发	孤儿资源月度巡检；Dead PV 告警
关联	【存储篇 · A12 Finalizer / A15 扩容 / A22 泄漏 / C13】

第 7 章 • GPU 与大模型（P58–P60）

P58 • 训练 hang 在 NCCL 初始化 / 第一步

项	内容
现象	任务启动后卡住不动，无报错或只报 NCCL timeout
30 秒验证	打开 `NCCL_DEBUG=INFO NCCL_DEBUG_SUBSYS=INIT,ENV,GRAPH`；确认 `NCCL_SOCKET_IFNAME` / `NCCL_IB_HCA` 是否选对网卡；节点 `ibstat` / `rdma link`
常见根因	① **选错网卡**（多网卡机器最常见）；② RDMA 链路/MTU/GID 配置问题；③ 未启用 GDR 但配置期望它；④ rank 数量与 rendezvous 不一致
止血	显式指定正确的网卡与环境变量；必要时先退回 hostNetwork/单节点验证
根治	把 NCCL 环境变量固化到作业模板；网络基线（nccl-tests busbw）上线前验收；CNI 方案固定（勿让 RDMA 走隧道）
防复发	上线/变更后必跑 nccl-tests 基线；训练 step time 异常告警
关联	【GPU 篇 · C3 / C6 / C8】

P59 • 训练变慢 / MFU 低 / 出现 straggler / 坏卡

项	内容
现象	整体吞吐下降；个别 rank 明显慢；或节点报 XID/ECC
30 秒验证	DCGM：`DCGM_FI_PROF_SM_ACTIVE` / `_PIPE_TENSOR_ACTIVE`、温度/功耗/SM clock、`DCGM_FI_DEV_XID_ERRORS`；网络：丢包与 nccl-tests；对比各 rank 的 step time
常见根因	① 降频/限功率/温度墙；② NVLink/网卡链路降级；③ straggler 节点；④ 数据/存储 IO 拖累；⑤ 并行策略或 batch 配置不佳
止血	cordon 可疑节点；把受影响作业重排；必要时重启驱动/节点
根治	坏卡自动检测 + 隔离 + 维护流程；网络与 IO 基线；并行策略与算子优化（FlashAttention、重计算权衡、通信 overlap）
防复发	XID/ECC 告警；SM/Tensor active 低水位告警；step time 分位数告警
关联	【GPU 篇 · A13 / B15 / C9 / F1–F4】

P60 • 推理服务：冷启动太慢 / 副本负载不均 / KV cache 打满

项	内容
现象	扩容后很久才可用；某些副本被打爆；请求被拒或延迟飙升
30 秒验证	看 Pod 就绪时长（权重加载）；按副本统计请求数/连接数；推理引擎指标（排队长度、KV cache 使用率）
常见根因	① **权重加载慢 + 探针阈值太严**（启动即被杀/迟迟不 Ready）；② gRPC/HTTP2 **连接级负载不均**；③ KV cache 打满导致拒绝/抢占；④ 网关超时与响应缓冲（SSE 被缓存）
止血	放大 startupProbe 阈值与 grace；减少单连接并发（限制 `MAX_CONCURRENT_STREAMS`）或换 per-request 网关；限流降级
根治	权重预热（本地缓存/预取）+ 常驻最小副本；推理网关做模型/队列/缓存感知路由；网关禁用响应缓冲并放大超时；按 KV cache 与排队长度扩缩
防复发	TTFT/TPOT 分桶 SLO；各副本请求量看板；排队长度与 KV cache 告警
关联	【GPU 篇 · C10 / C11 / E2–E5】

附录 A · 出事第一时间要抓的"证据清单"

```
# ① 对象侧（30 秒内）
kubectl get pod/pvc/node -o wide
kubectl describe pod <pod> # Events 是最高性价比证据
kubectl logs <pod> --previous # CrashLoop 必看
kubectl get events -n <ns> --sort-by=.lastTimestamp | tail -40

# ② 节点侧
systemctl status kubelet && journalctl -u kubelet -n 300
dmesg -T | tail -50 # OOM / XID / 硬件
crictl ps -a | head -30
cat /sys/fs/cgroup/<path>/memory.events # cgroup OOM 计数
cat /sys/fs/cgroup/<path>/cpu.stat # throttling

# ③ 网络侧
nsenter -t <pid> -n ip a / ss -lntp / tcpdump
cat /proc/sys/net/netfilter/nf_conntrack_count
kubectl -n kube-system logs -l k8s-app=kube-dns

# ④ 控制面
kubectl -n kube-system get lease
kubectl get --raw /metrics | grep apiserver_request_duration
etcdctl endpoint status --write-out=table --cluster ; etcdctl alarm list

# ⑤ 存储侧
kubectl get pvc,pv,volumeattachment -A
kubectl describe pvc <pvc>
kubectl -n <csi-ns> logs <csi-provisioner> <csi-attacher>

# ⑥ GPU 侧
nvidia-smi
nvidia-smi -q -d TEMPERATURE,POWER,UTILIZATION,MEMORY
dmesg -T | grep -i -E "NVRM|Xid"
NCCL_DEBUG=INFO ... （重跑一次最小复现）
```

保留现场清单：Events 截图/导出、Pod YAML（含 finalizers/status）、容器日志（含上一实例）、节点 dmesg、监控面板时间窗、etcd 快照（控制面类故障）。

附录 B · 最容易被误判的 10 件事

#	误判	真相
1	"Pending 就是资源不够"	也可能是 **schedulerName 写错 / 调度器没跑**（完全没有 Event）
2	"GPU 利用率 100% 说明跑满了"	可能是空转；要看 **SM/Tensor active + MFU**
3	"CPU 使用率不高说明没瓶颈"	可能正被 **CFS throttling** 卡住
4	"没超内存怎么会被 OOM"	**page cache 计入 cgroup memory**
5	"节点挂了 Pod 会马上重建"	默认约 **5~6 分钟**（grace 50s + toleration 300s）
6	"删了容器磁盘就该释放"	可能有进程持有已删除文件（`lsof ``| grep deleted`），或 **inode 没释放**
7	"No space left 就是容量满了"	可能是 **inode 耗尽** 或 `/dev/shm` 太小
8	"NetworkPolicy 配了就一定生效"	**Flannel 不支持**；CNI 不实现就完全不生效
9	"PV/PVC 删不掉是 bug"	是 **finalizer** 在保护；强删会留下孤儿磁盘
10	"apiserver 慢就是 apiserver 的问题"	绝大多数是 **etcd 磁盘慢** 或某个客户端在疯狂 list

附录 C · 复盘模板（一页纸）

```
1. 影响：时间窗、受影响服务/租户、SLO 消耗、错误预算消耗比例
2. 时间线：发现 → 定位 → 止血 → 恢复（关键动作与耗时）
3. 根因：直接原因 / 根本原因 / 为什么监控没提前发现
4. 为什么发生：流程（变更？容量？设计）+ 技术（代码/配置/依赖）
5. 改进项：
   - 检测（新增/优化告警，含阈值与验证方式）
   - 预防（配置校验、CI 门禁、策略、容量规划）
   - 恢复（手册/脚本/演练，明确 RTO）
   - 防扩散（灰度、限流、隔离、回滚）
6. 每项改进：负责人 + 截止时间 + 验收方式
7. 演练计划：如何验证改进真的有效（GameDay 场景）
```

附录 D · 最小可用告警集（先有这 20 条再谈治理）

领域	告警
节点	Node NotReady、Node Pressure（Mem/Disk/PID）、kubelet `up`、PLEG relist P99、证书剩余有效期
Pod/容器	Pending 超时、CrashLoop/重启次数、OOMKilled、Evicted、Terminating 超时、CPU throttled 占比
调度	`scheduler_pending_pods` 堆积、调度失败率、抢占次数、无 Event 的 Pending Pod
网络	CoreDNS 延迟/ SERVFAIL/ `up`、conntrack 使用率与 insert_failed、kube-proxy 同步停滞、CNI NetworkUnavailable、IP 池水位
控制面	apiserver 延迟 P99 / 5xx / `up`、etcd fsync+commit P99、etcd 容量比、etcd leader 变更、APF 拒绝
存储	PVC Pending、卷容量/inode 水位、卷操作耗时 P99、VolumeAttachment 残留
GPU	XID/ECC、SM active 低水位、显存水位、推理排队长度与 KV cache 使用率

> **原则**：告警要有明确"下一步动作"。没有行动项的告警等于噪音，最终会被静音——**静音的告警比没有告警更危险。**